

Platinum Administrative Guide

This guide is delivered subject to the following conditions and restrictions:

This guide contains proprietary information belonging to ELMO Motion Control Ltd. Such information is supplied solely for the purpose of assisting users of ELMO's servo drive(s) in their installation.

The text and graphics included in this manual are for the purpose of illustration and reference only. The specifications on which they are based are subject to change without notice.

ELMO Motion Control and the ELMO Motion Control logo are trademarks of ELMO Motion Control Ltd.

Information in this document is subject to change without notice.



ELMO Motion Control and the ELMO Motion Control logo are registered trademarks of ELMO Motion Control Ltd.



EtherCAT® and Safety over EtherCAT® are registered trademarks and patented technologies, licensed by Beckhoff Automation GmbH, Germany.



EtherCAT® Conformance tested is a registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.



CANopen compliant. CANopen® is a registered trademark and patented technology, licensed by CAN in Automation (CiA) GmbH, Kontumazgarten 3, DE-90429 Nuremberg, Germany.

Document no. MAN-P-ADMINGUIDE (Ver. 1.002)

Copyright © 2022

ELMO Motion Control Ltd.

All rights reserved.

Revision History

Version	Date	Details
Ver. 1.000	Feb 2022	Initial Release
Ver. 1.001	June 2022	Updated
Ver. 1.002	Aug 2022	Updated

Chapter 1:	Introduction	16
1.1	Scope	16
1.1.1	Mission	16
1.1.2	Platinum Documentation Concept Requirements	16
1.1.2.1	Requirements	16
1.1.2.2	Detailed Design	17
1.1.2.3	Module Tests	17
1.1.2.4	System Tests	17
1.2	Terms and Abbreviations	18
1.3	Relevant Elmo Documentation	21
1.4	Standards	21
Chapter 2:	The Platinum Drive	22
2.1	Firmware Architecture & Update	22
2.1.1	General Requirements	22
2.1.2	Memory Organization	22
2.1.3	Firmware Images	22
2.1.4	Firmware Status	22
2.1.5	The Firmware Version	22
2.1.6	Updating Firmware Version	22
2.2	Drive Identification and Personality	23
2.3	Drive Units, Data Type & Conversions	23
2.4	The Multi Axes Concept	23
2.5	Communication & Network	23
2.5.1	Functional Communication	23
2.5.2	Fieldbus Communication	23
Chapter 3:	Communication and Interpreter	24
3.1	Introduction	24
3.2	Description of Attributes	24
3.3	Platinum Line Communications	26
3.4	Drive Communication & Interpreter	27
3.4.1	Assign and Retrieve Parameters	28
3.4.2	Interpreter Echo	28
3.4.3	Command Format: Scalar vs Array	29
3.4.4	Command & Data Types	29
3.4.5	Bit-field Format	30
3.4.6	Hexadecimal Format	30
3.4.7	Mathematic and Logical Operation	30
3.4.8	Streaming Commands	33
3.4.9	User Program Global Variables	34
3.4.10	Commands Prefix	34
3.4.11	Elmo Commands Alias Objects	35

3.4.12	The Multi-axes	35
3.4.13	BIT Status Indications	36
3.4.14	Status LED Blinks	36
Chapter 4:	DS402 Motion Profile	37
4.1	Introduction	37
4.1.1	Scope	37
4.1.2	Introduction	37
4.2	Device Control Objects	39
4.2.1	General	39
4.2.2	Remote and Local modes	39
4.2.3	State Machine	40
4.2.4	Drive States	42
4.2.5	State Transitions of the Drive	43
4.2.6	Illegal Transition	45
4.3	Deceleration Option Objects	46
4.4	Modes of Operation Objects	47
4.5	Factors Objects	47
4.5.1	General	47
4.5.2	Relationship between Physical and Internal Units	48
4.5.3	Position Units	48
4.5.4	Velocity Units	49
4.5.5	Acceleration Units	49
4.5.6	Jerk Units	49
4.5.7	Restrictions	49
4.6	Position Motion Function Objects	50
4.6.1	General	50
4.6.2	Functional description	50
4.7	Profile Position Mode	52
4.7.1	General	52
4.7.2	General Structure	53
4.7.3	Controlword of the Profile Position Mode	53
4.7.4	Statusword of the Profile position mode	55
4.7.5	Functional Description	56
4.7.6	Single Set-Point	58
4.7.7	Buffered Set-Point	59
4.7.8	Buffered Set-Point, non-blended motion	60
4.7.9	Buffered Set-Point, Blended	63
4.8	Interpolated Position Mode	64
4.9	Cyclic Synchronous Position Mode	65
4.9.1	General	65
4.9.2	Functional Description	65
4.9.3	<i>Controlword</i> of Cyclic Synchronous Position Mode	66
4.9.4	Statusword of Cyclic Synchronous Position Mode	67

4.10	Cyclic Synchronous Velocity Mode	68
4.10.1	General	68
4.10.2	Functional Description	69
4.10.3	Controlword of Cyclic Synchronous Velocity Mode	69
4.10.4	Statusword of Cyclic Synchronous Velocity Mode	70
4.11	Cyclic Synchronous Torque Mode	72
4.11.1	General	72
4.11.2	Functional Description	72
4.11.3	Controlword of Cyclic Synchronous Torque Mode	73
4.11.4	Statusword of Cyclic Synchronous Torque Mode	73
4.12	Profile Velocity	75
4.12.1	General	75
4.12.2	Functional Description	76
4.12.3	Controlword of the Profile velocity mode	76
4.12.4	Statusword of the Profile velocity mode	77
4.13	Profile Torque Mode	78
4.13.1	General	78
4.13.2	Functional Description	78
4.13.3	Controlword of the Profile Torque Mode	79
4.13.4	Statusword of the Profiled Torque Mode	80
4.14	Profile Torque Mode with Commutation Angle	81
Chapter 5:	The Object Dictionary	82
5.1	Objects Properties	84
5.2	Product Code & Revision Number	99
5.3	Interpreter Prefixes	99
5.4	Communication Objects	100
5.4.1	Object 0x1000: Device type	100
5.4.2	Object 0x100A: Manufacturer software version	100
5.4.3	Object 0x100B: Bootloader version	101
5.4.4	Object 0x1001: Error register	101
5.4.5	Object 0x1002: Manufacturer status register	102
5.4.6	Object 0x1003: Pre-defined error field	103
5.4.7	Object 0x1006: Communication cycle period	104
5.4.8	Object 0x1008: Manufacturer device name	104
5.4.9	Object 0x1009: Manufacturer hardware version	105
5.4.10	Object 0x100A: Manufacturer software version	105
5.4.11	Object 0x100B: Bootloader version	106
5.4.12	Object 0x1010: Store parameters	106
5.4.12.1	Example of write SDO to object 0x1010 sub index 1	107
5.4.13	Object 0x1011: Restore default parameters	108
5.4.13.1	Example of write SDO to object 0x1011 sub index 1	108
5.4.14	Object 0x1013: High Resolution Time Stamp	109
5.4.15	Object 0x1016: Consumer heartbeat time	109
5.4.16	Object 0x1017: Producer heartbeat time	110

5.4.17	Object 0x1018: Identity Object	111
5.4.18	Object 0x1023: OS Interpreter Command	112
5.4.19	Object 0x1024: OS command mode	114
5.4.20	Object 0x1029: Error behavior object	114
5.4.21	Object 0x10F1: Error Settings Object	115
5.5	rPDO	118
5.5.1	Objects 0x1600 – 0x1603, 0x1701 - 0x1704: rPDO mapping parameter	118
5.5.2	Objects 0x1400 - 0x1403: rPDO communication parameter	119
5.6	tPDO	122
5.6.1	Objects 0x1A00-0x1A03, 0x1B01-0x1B04: Transmit PDO mapping parameter	122
5.6.2	Objects 0x1800 - 0x1803: Transmit PDO communication parameter	124
5.6.2.1	Description of tPDO transmission type, sub index 2	126
5.6.2.2	Inhibit Time	127
5.6.2.3	Event Timer	127
5.7	ECAT Related Objects	128
5.7.1	Objects 0x1C00: Sync manager communication type	128
5.7.2	Objects 0x1C12: rPDO assign (Sync Manager Channel 2)	128
5.7.3	Objects 0x1C13: tPDO assign (Sync Manager Channel 3)	129
5.7.4	Objects 0x1C32, 0x1C33: Sync Manager 2, 3 parameters	130
5.8	Elmo Specific Objects	138
5.8.1	Object 0x200A: Version object	138
5.8.2	Object 0x2012: Binary interpreter input	139
5.8.3	Object 0x2013: Binary interpreter Receive	139
5.8.4	Object 0x2030: Recorder data	140
5.8.4.1	Uploading Recorded Data	141
5.8.4.2	Example of uploading recorded data	141
5.8.4.3	Uploading the Velocity Command	141
5.8.5	Object 0x2031: Recorder header	142
5.8.6	Object 0x2035: Upload data parameters	143
5.8.7	Object 0x2036: Upload data	144
5.8.8	Object 0x2045: SDO block upload delay	144
5.8.9	Object 0x2046: ECAT DC Inhibit Time	145
5.8.10	Object 0x2051: CAN download data	145
5.8.11	Object 0x2065 Vector Mode Configuration Object	146
5.8.12	Object 0x2082: CAN controller status	150
5.8.13	Object 0x2090: CAN FW download	150
5.8.14	Object 0x20FE: Output Modification Mask	151
5.8.15	Object 0x2202: Extended Inputs	152
5.8.16	Object 0x2205: Analog Input Object	154
5.8.17	Object 0x2206: 5V DC supply	156
5.8.18	Object 0x2207: Shunt Count	156
5.8.19	Object 0x2210: Counter Object	157
5.8.20	Object 0x22A1: Extended Outputs	158
5.8.21	Object 0x22F0: User Integer Array	160
5.8.22	Object 0x22F1: User Float Array	161
5.8.23	Object 0x22F3: SIL R1 Array	162

5.8.24	Object 0x22F4: SIL R2 Array	163
5.8.25	Object 0x2300/0x2301: Error Logger Primary/Secondary Safety CPU	164
5.8.25.1	Error Representation	164
5.8.25.2	The Error Module Groups	164
5.8.26	Object 0x2303: Safety Monitoring Error Object	165
5.8.26.1	Example for Safe Function	166
5.8.26.2	Example for Logic Function Violation	166
5.8.27	Object 0x2304: ErrAck Logger Clear	167
5.8.28	Object 0x2F00: User Integer Array	168
5.8.29	Object 0x2F01: User Float Array	169
5.8.30	Object 0x2F20: tPDO asynchronous events	169
5.8.31	Object 0x2F21: Emergency Event mask	172
5.8.32	Object 0x2F30: Socket's sensor internal temperature	174
5.8.33	Object 0x2F31: Socket's sensor external temperature	175
5.8.34	Object 0x2FE2: Time Capture Tag	177
5.8.35	Object 0x2FE3: Time Event Counter	177
5.8.36	Object 0x2FE4 – 0x2FE6: Socket position 1 - Socket position 3	178
5.8.37	Object 0x2FE8 – 0x2FEA: Socket velocity 1 - Socket velocity 3	180
5.8.38	Object 0x2FEC: Analog Sensor Amplitude	181
5.8.39	Object 0x3000: Elmo Commands	182
5.8.40	Object 0x3605: General Reference	183
5.8.41	Object 0x3606: Home On Touch Probe	184
5.8.42	Object 0x3607: Elmo Status Register Array	185
5.8.43	Object 0x360D: Vector Mode Table	186
5.8.44	Object 0x3610: Drive Temperature in °C	187
5.8.45	Object 0x3617: Power calculation	188
5.8.46	Object 0x3618: Max Thermal Limit - Percentage	189
5.8.47	Object 0x3620: Home On Block Limit Parameters	189
5.8.48	Object 0x3625: Gantry Yaw Offset	192
5.8.49	Object 0x3630: Safety Position Calibration Start	193
5.8.50	Object 0x3640: Sample VQ	193
5.8.51	Object 0x3675 – Extrapolation Timeout	194
5.8.52	Object 0x36E0: Gain Scheduling Manual Index	194
5.8.53	Object 0x36E4: Additional Position Actual value	196
5.8.54	Object 0x36E5: Additional Velocity Actual Value	197
5.8.55	Object 0x36FD Digital Inputs	198
5.9	DS-402	200
5.9.1	Object 0x6007: Abort Connection Option Code	200
5.9.2	Object 0x603F: Error code	201
5.9.3	Object 0x6040: Controlword	202
5.9.3.1	Bits 0 – 3 and 7	202
5.9.3.2	Bits 4, 5, 6 and 9	203
5.9.3.3	Bit 8	204
5.9.3.4	Bits 10 - 15	204
5.9.4	Object 0x6041: Statusword	205
5.9.4.1	Bits 0 - 3, 5 and 6	206
5.9.4.2	Bit 4: Voltage Enabled	206

5.9.4.3	Bit 5: Quick Stop	206
5.9.4.1	Bit 7: Warning	206
5.9.4.2	Bit 8	206
5.9.4.3	Bit 9: Remote	206
5.9.4.4	Bit 10: Target Reached	207
5.9.4.5	Bit 11: Internal Limit Active	207
5.9.4.6	Bits 12 and 13	207
5.9.4.7	Bits 14 and 15	207
5.9.5	Object 0x605A: Quick Stop Option Code	208
5.9.6	Object 0x605B: Shutdown Option Code	209
5.9.7	Object 0x605C: Disable operation option code	210
5.9.8	Object 0x605D: Halt option code	211
5.9.9	Object 0x605E: Fault Reaction Option Code	212
5.9.10	Object 0x6060: Modes of operation	213
5.9.11	Object 0x6061: Modes of Operation Display	214
5.9.12	Object 0x6062: Position Demand Value	215
5.9.13	Object 0x6063: Position Actual Internal Value	216
5.9.14	Object 0x6064: Position Actual Value	216
5.9.15	Object 0x6065: Following Error Window	217
5.9.16	Object 0x6066: Following Error Time Out	218
5.9.17	Object 0x6067: Position Window	218
5.9.18	Object 0x6068: Position Window Time	219
5.9.19	Object 0x6069: Velocity Sensor Actual Value	220
5.9.20	Object 0x606A: Sensor Selection Code	220
5.9.21	Object 0x606B: Velocity Demand Value	221
5.9.22	Object 0x606C: Velocity Actual Value	222
5.9.23	Object 0x606D: Velocity Window	222
5.9.24	Object 0x606E: Velocity Window Time	223
5.9.25	Object 0x606F: Velocity Threshold	224
5.9.26	Object 0x6070: Velocity Threshold Time	224
5.9.27	Object 0x6071: Target Torque	225
5.9.27.1	Example	225
5.9.28	Object 0x6072: Maximum Torque	226
5.9.29	Object 0x6073: Maximum Current	227
5.9.30	Object 0x6074: Torque Demand Value	228
5.9.31	Object 0x6075: Motor Rated Current	228
5.9.32	Object 0x6076: Motor Rate Torque	229
5.9.32.1	Motor Torque Constant – KT	229
5.9.33	Object 0x6077: Torque Actual Value	230
5.9.34	Object 0x6078: Current Actual Value	232
5.9.35	Object 0x6079: DC Link Circuit Voltage	232
5.9.36	Object 0x607A: Target Position	233
5.9.37	Object 0x607B: Position Range Limit	234
5.9.38	Object 0x607C: Home Offset	235
5.9.39	Object 0x607D: Software Position Limit	236
5.9.40	Object 0x607E: Polarity	238
5.9.41	Object 0x607F: Max Profile Velocity	239

5.9.42	Object 0x6080: Maximum Motor Speed	239
5.9.43	Object 0x6081: Profile Velocity	240
5.9.44	Object 0x6082: End Velocity	241
5.9.45	Object 0x6083: Profile Acceleration	242
5.9.46	Object 0x6084: Profile Deceleration	242
5.9.47	Object 0x6085: Quick Stop Deceleration	243
5.9.48	Object 0x6086: Motion Profile Type	244
5.9.49	Object 0x6087: Torque Slope	245
5.9.50	Object 0x608F: Position Encoder Resolution	245
5.9.51	Object 0x6090: Velocity Encoder Resolution	246
5.9.52	Object 0x6091: Gear Ratio	248
5.9.53	Object 0x6092: Feed Constant	250
5.9.54	Object 0x6096: Velocity Factor	251
5.9.55	Object 0x6097: Acceleration Factor	253
5.9.56	Object 0x6098: Homing Method	254
5.9.57	Object 0x6099: Homing Speeds	254
5.9.58	Object 0x609A: Homing Acceleration	256
5.9.59	Object 0x60B0: Position Offset	257
5.9.60	Object 0x60B1: Velocity Offset	257
5.9.61	Object 0x60B2: Torque Offset	258
5.9.62	Object 0x60B8: Touch Probe Function	259
5.9.63	Object 0x60B9: Touch Probe Status	261
5.9.64	Object 0x60BA: Touch Probe 1 Positive Edge	262
5.9.65	Object 0x60BB: Touch Probe 1 Negative Edge	263
5.9.66	Object 0x60BC: Touch Probe 2 Positive Edge	263
5.9.67	Object 0x60BD: Touch Probe 2 Negative Edge	264
5.9.68	Object 0x60C5: Max Acceleration	265
5.9.69	Object 0x60C6: Max Deceleration	265
5.9.70	Object 0x60D0: Touch Probe Source	266
5.9.71	Object 0x60D1: Touch Probe Time Stamp 1 Positive Value	268
5.9.72	Object 0x60D2: Touch Probe Time Stamp 1 Negative Value	268
5.9.73	Object 0x60D3: Touch Probe Time Stamp 2 Positive Value	269
5.9.74	Object 0x60D4: Touch Probe Time Stamp 2 Negative Value	269
5.9.75	Object 0x60D5: Touch Probe 1 Positive Edge Counter	270
5.9.76	Object 0x60D6: Touch Probe 1 Negative Edge Counter	270
5.9.77	Object 0x60D7: Touch Probe 2 Positive Edge Counter	271
5.9.78	Object 0x60D8: Touch Probe 2 Negative Edge Counter	272
5.9.79	Object 0x60E0: Positive Torque Limit Value	272
5.9.80	Object 0x60E1: Negative Torque Limit Value	273
5.9.81	Object 0x60E3: Supported Homing Methods	274
5.9.82	Object 0x60EA: Commutation Angle	275
5.9.83	Object 0x60F2: Position Option Code	276
5.9.84	Object 0x60F4: Following Error Actual Value	278
5.9.85	Object 0x60FA: Control Effort	278
5.9.86	Object 0x60FC: Position Demand Internal Value - Increments	279
5.9.87	Object 0x60FD: Digital Inputs	279
5.9.88	Object 0x60FE: Digital Outputs	280
5.9.89	Object 0x60FF: Target Velocity	281

5.9.90	Object 0x6502: Supported Drive Modes	282
5.10	SRA – Safety Related Application Objects	283
5.11	Elmo Manufacturer Specific SRA Objects	284
5.11.1	Object 0x2E00: Safety Mode	284
5.11.2	Object 0x2E01: Safety Manager Parameters	284
5.11.3	Object 0x2E02: Drive safe parameters CRC	286
5.11.4	Object 0x2E03: Extended Safety Options	287
5.11.5	Objects 0x2E10/0x2E11 Safety I/O Parameters	288
5.11.5.1	Object 0x2E10[i]	288
5.11.5.2	Object 0x2E11[i]	291
5.11.6	Objects 0x2E12/0x2E13: Safety Input Indication (Preliminary)	294
5.11.7	Object 0x2E20: Safety Option	294
5.11.8	Object 0x2E30: Safety Address	296
5.11.9	Object 0x3771: Pole/Sensor Resolution Factor	297
5.11.10	Object 0x3772: SLP Digital Input/S For Position Absolute Sensing	298
5.11.11	Object 0x3773: Absolute Reference Position	299
5.11.12	Object 0x3774: Absolute Reference Position Window	299
5.11.13	Object 0x377B: Safety Output Number used to Operate Brake	301
5.11.14	Object 0x377C: Safety Input Number for Brake Manual Release	301
5.11.15	Object 0x3780: General Sensor 1 Parameters	302
5.11.15.1	SSI Sensor	302
5.11.15.2	BISS Sensor	303
5.11.15.3	ENDAT Sensor	304
5.11.15.4	SIN COS Sensor	304
5.11.15.5	Resolver Sensor	305
5.11.15.6	Quad Sensor	306
5.11.16	Object 0x3781: General Sensor 2 Params	307
5.11.16.1	SSI Sensor	307
5.11.16.2	Biss Sensor	308
5.11.16.3	ENDAT Sensor	309
5.11.16.4	SINCOS Sensor	309
5.11.16.5	Resolver Sensor	310
5.11.16.6	Quad Sensor	311
5.11.17	Object 0x3783: Torque Sensor Configuration Options	312
5.11.18	Object 0x3784: Torque Sensor General Parameters	313
5.11.19	Object 0x3785: Primary Torque Sensor Parameters	314
5.11.20	Object 0x3787: Secondary Torque Sensor Parameters	316
5.11.21	Object 0x3788: SLT Fault Reaction Parameters	318
5.11.22	Object 0x3790: Feedbacks Options	322
5.11.22.1	Lower Byte	322
5.11.22.2	Upper Byte	322
5.11.23	Object 0x3791: Position Comparison Tolerance	323
5.11.24	Object 0x3795: Safe Sensor Position Value	324
5.11.25	Object 0x379A: Time to Software STO	325
5.11.26	Object 0x37A0: Request for Operational Halt or Fault Stopping Profile	326

5.11.27	Object 0x37B0: SBT Command	327
5.11.28	Object 0x37B1: SBT SOS Service	327
5.11.29	Object 0x37B2: SBT Ramp Time Delay	328
5.11.30	Object 0x37B3: SBT Test Duration Time	328
5.11.31	Object 0x37B4: SBT Torque Limit	329
5.11.32	Object 0x37B5: SXT Status	329
5.11.33	Object 0x37B6: SXT Torque Maximum Positive Limit	330
5.11.34	Object 0x37B7: SXT Torque Maximum Negative Limit	330
5.11.35	Object 0x37B8: SXT Error Reaction	331
5.11.36	Object 0x37B9: SAT Command	331
5.11.37	Object 0x37BA: SAT Torque Limit	332
5.11.38	Object 0x37BB: SAT Error Reaction	332
5.11.39	Object 0x37BC: SAT Status	333
5.12	ETG-6100 Safety Drive Profile Objects	334
5.12.1	Object 0x6611: Safe Position	334
5.12.2	Object 0x6613: Safe Velocity	334
5.12.3	Object 0x6616: Safe Torque	335
5.12.4	Object 0x6630: Restart Acknowledge Control	335
5.12.5	Object 0x6631: Restart Acknowledge Status	336
5.12.6	Object 0x6632: Error Acknowledge Control	336
5.12.7	Object 0x6633: Error Acknowledge Status	337
5.12.8	Object 0x6640: STO Command	337
5.12.9	Object 0x6641: STO Restart Acknowledge Behavior	338
5.12.10	Object 0x6643: Safety STO Activate SBC	338
5.12.11	Object 0x6644: STO Status	339
5.12.12	Object 0x6650: SS1 Command	339
5.12.13	Object 0x6651: Safety SS1 Time to STO	340
5.12.14	Object 0x6653: Safety SS1 Velocity Zero Window	340
5.12.15	Object 0x6654: Safety SS1 Time for Velocity Zero	341
5.12.16	Object 0x6656: Safety SS1 Deceleration Limit Window	341
5.12.17	Object 0x6657: Safety SS1 Time Delay for Deceleration Monitoring	342
5.12.18	Object 0x6658: Safety SS1 Activate SBC	342
5.12.19	Object 0x665F: SS1 Status	343
5.12.20	Object 0x6660: SBC Command	343
5.12.21	Object 0x6661: SBC Brake Time Delay	344
5.12.22	Object 0x6667: SBC Status	344
5.12.23	Object 0x6668: SOS Command	345
5.12.24	Object 0x666A: Safety SOS Position Zero Window	345
5.12.25	Object 0x666C: Safety SOS Velocity Zero Window	346
5.12.26	Object 0x666F: SOS Status	346
5.12.27	Object 0x6670: SS2 Command	347
5.12.28	Object 0x6671: SS2 Time to SOS	347
5.12.29	Object 0x6672: SS2 Time for Velocity Zero	348
5.12.30	Object 0x6674: Safety SS2 Deceleration Limit	348
5.12.31	Object 0x6675: Safety SS2 Time Delay for Deceleration Monitoring	349
5.12.32	Object 0x6676: Safety SS2 Restart Acknowledge	349
5.12.33	Object 0x6677: SS2 Error Reaction	350
5.12.34	Object 0x667F: SS2 Status	350

5.12.35	Object 0x6680: SSR Command	351
5.12.36	Object 0x6681: SSR Time to Velocity Monitoring	351
5.12.37	Object 0x6683: SSR Velocity Upper Limit	352
5.12.38	Object 0x6685: SSR Velocity Lower Limit	352
5.12.39	Object 0x6686: SSR Time for Velocity In Limit	353
5.12.40	Object 0x6688: SSR Deceleration Limit	353
5.12.41	Object 0x6689: SSR Time Delay for Deceleration Monitoring	354
5.12.42	Object 0x668A: SSR Error Reaction	354
5.12.43	Object 0x668F: SSR Status	355
5.12.44	Object 0x6690: SLS Command	355
5.12.45	Object 0x6691: SLS Time to Velocity Monitoring	356
5.12.46	Object 0x6693: SLS Velocity Limit	356
5.12.47	Object 0x6694: SLS Time for Velocity In Limit	357
5.12.48	Object 0x6695: SLS Time Delay for Deceleration Monitoring	357
5.12.49	Object 0x6697: SLS Deceleration Limit	358
5.12.50	Object 0x6698: SLS Error Reaction	358
5.12.51	Object 0x669F: SLS Status	359
5.12.52	Object 0x66A0: SLP Command	359
5.12.53	Object 0x66A2: SLP Position Upper Limit	360
5.12.54	Object 0x66A4: SLP Position Lower Limit	360
5.12.55	Object 0x66A5: SLP Error Reaction	361
5.12.56	Object 0x66A7: SLP Status	361
5.12.57	Object 0x66A8: SMS Status	362
5.12.58	Object 0x66AA: SMS Velocity Maximum Positive	362
5.12.59	Object 0x66AC: SMS Velocity Maximum Negative	363
5.12.60	Object 0x66AD: SMS Error Reaction	363
5.12.61	Object 0x66B0: SLT Command	364
5.12.62	Object 0x66B1: SLT Torque Limit Positive Direction	364
5.12.63	Object 0x66B2: SLT Torque Limit Negative Direction	365
5.12.64	Object 0x66B3: SLT Error Reaction	365
5.12.65	Object 0x66B7: SLT Status	366
5.12.66	Object 0x66B8: SLI Command	366
5.12.67	Object 0x66BA: SLI Position Relative Upper Limit	367
5.12.68	Object 0x66BC: SLI Position Relative Lower Limit	367
5.12.69	Object 0x66BD: SLI Error Reaction	368
5.12.70	Object 0x66BF: SLI Status	368
5.12.71	Object 0x66D0: SDI Positive Direction Command	369
5.12.72	Object 0x66D1: SDI Negative Direction Command	369
5.12.73	Object 0x66D3: SDI Position Zero Window	370
5.12.74	Object 0x66D5: SDI Velocity Zero Window	370
5.12.75	Object 0x66DE: SDI Positive Direction Status	371
5.12.76	Object 0x66DF: SDI Negative Direction Status	371
5.12.77	Object 0x66E0: SSM Status	372
5.12.78	Object 0x66E2: SSM Velocity Upper Limit	372
5.12.79	Object 0x66E4: SSM Velocity Lower Limit	373
5.12.80	Object 0x66E8: SCA Status	373
5.12.81	Object 0x66EA: SCA Position Upper Limit	374
5.12.82	Object 0x66EC: SCA Position Lower Limit	374

5.12.83	Object 0x6760: FSoE Slave Frame Elements	375
5.12.84	Object 0x6770: FSoE Master Frame Elements	376
5.12.85	Object 0x6790: Connection Module Information	378
5.12.86	Object 0x6791: Slave Connection Parameters	381
5.12.87	Object 0x67A0: Slave Connection Diagnosis – FsoE	384
5.13	MDP – Modular Device Profile	386
5.13.1	Object 0xF000: Modular Device Profile Object	386
5.13.2	Object 0xF030: Configuration Device Profile	387
5.13.3	Object 0xF050: Detected Module List	388
5.13.4	Object 0xF5FF: Configuration Management	389
5.13.5	Object 0xF6F0: Input Latch Local Timestamp	390
5.13.5.1	Notes	390
5.13.6	Object 0xF6F2: Input Latch ESC Timestamp	391
5.13.6.1	Notes	391
5.13.7	Object 0xF980: Device Safety Address	392
5.13.8	Object 0xF9F0: Manufacturer Serial Number	393
5.13.8.1	Notes	393
5.13.9	Object 0xF9F1: CDP Functional Generation Number	393
5.13.10	Object 0xF9F2: SDP Functional Generation Number	394
5.13.11	Object 0xF9F3: Vendor Name	395
5.13.12	Object 0xF9F4: Semiconductor SDP Device Name	395
5.13.13	Object 0xF9F5: Output Identifier	396
5.13.14	Object 0xF9F6: Time Since Power On	397
5.13.15	Object 0xF9F8: Firmware Update Functional Generation Number	398
5.13.16	Object 0xFBF0: Device Reset Command	398
5.13.16.1	Notes	400
5.13.17	Object 0xFBF2: Store Parameters Command	400
5.13.17.1	Notes	402
5.13.18	Object 0xFBF3: Calculate Checksum Command	403
5.13.18.1	Notes	404
5.13.19	Object 0xFBF4: Load Parameter Command	405
5.13.19.1	Notes	406
Chapter 6:	User Programming Language	407
6.1	Scope	407
6.2	General	407
6.3	Structure and Timing	407
6.4	Keywords	408
6.5	How to Write a Program	409
6.6	User Functions	410
6.7	Variables and Types	412
6.7.1	Variable Names	412
6.7.2	Global Variables	413
6.8	Program Development and Execution	414

6.8.1	Editing a Program	414
6.8.2	Compiling a Program	414
6.9	Error Codes	415
6.10	The Preprocessor	428
6.11	Compiler Directives	428
6.11.1	#define	428
6.11.2	#undef	429
6.12	Evaluating Expressions Used in Compiler Directives	431
6.13	Program Flow Commands	432
6.13.1	if...elseif...else...end	432
6.13.2	goto	433
6.13.3	switch...case...otherwise...end	433
6.13.4	continue	435
6.13.5	break	436
6.13.6	exit	436
6.13.7	try...catch	436
6.14	Loops	438
6.14.1	while	438
6.14.2	for	438
6.14.3	until	439
6.15	Program Delay Command	440
6.15.1	wait	440
6.16	User Program Multi thread	441
6.16.1	Debugging and breakpoints in multi thread	442
6.17	Auto-Routines	443
6.17.1	AUTOEXEC	444
6.17.2	AUTO_ER	444
6.17.3	AUTO_STOP	445
6.17.4	AUTO_BG	445
6.17.5	AUTO_RLS	446
6.17.6	AUTO_FLS	446
6.17.7	AUTO_ENA	447
6.17.8	AUTO_I1 to AUTO_I6	447
6.17.9	AUTO_HM	449
6.17.10	AUTO_PERR	449
6.18	User Program Stability Protection (AUTO_PERR, try...catch)	451
6.19	Reordering Global Variables	451
6.20	User Program Restrictions and Limitations	452
6.21	Examples	453
Chapter 7:	Drive Errors and Warnings	457
7.1	BIT – Built In Tests	457

7.2	Drive Continuing Status	457
7.2.1	Optional Status	458
7.3	Error Codes Originating from ELMO Commands	458
7.4	Emergency (EMCY)	459
7.4.1	Emergency Codes	459
7.5	Drive Non-volatile Parameters	466
7.6	Safety Errors	467
7.6.1	Diagnostics Errors	467
7.6.1.1	Diagnostic Error List	468
7.6.2	Monitoring Violation	471
7.6.2.1	Monitoring Error List	473
7.6.2.2	General Details for Reading Monitoring Errors	474
7.7	Sensors Errors and Warnings	476
7.7.1	Sensor ID=1: Encoder Quad	476
7.7.2	Sensor ID=3: Analog Sin/Cos	476
7.7.3	Sensor ID=4: Halls Only	477
7.7.4	Sensor ID=5: BiSS	477
7.7.5	Sensor ID=6: Panasonic	477
7.7.6	Sensor ID=7: Panasonic Incremental	479
7.7.7	Sensor ID=8: Sine Virtual Sensor	480
7.7.8	Sensor ID=9: EnDAT	480
7.7.8.1	W#[4] Sensor Errors Detailed Description	482
7.7.8.2	W#[5] sensor warnings detailed description	483
7.7.9	Sensor ID=11: Pulse & Direction	483
7.7.10	Sensor ID=15: Copy Main Profile	484
7.7.11	Sensor ID=16: Analog Input	484
7.7.12	Sensor ID=18: SSI	484
7.7.13	Sensor ID=19: Yaskawa	485
7.7.14	Sensor ID=20: Gantry Sensor	486
7.7.15	Sensor ID=21: Serial Exclusive	486
7.7.15.1	Master In Three Slaves Configuration	486
7.7.15.2	Master in single slave configuration (Gantry)	488
7.7.15.3	Slave	488
7.7.16	Sensor ID=22: Resolver	489
7.7.17	Sensor ID=25: Nikon	490
7.7.17.1	W#[4] sensor errors detailed description	491
7.7.18	Sensor ID=28: Hiperface	492
7.7.19	Sensor ID=30: Velocity Observer IR	493
7.7.20	Sensor ID=34: Motor Simulation	493
7.7.21	Sensor ID=36: BEMF Observer	493
7.7.22	Sensor ID=38: Generic Serial IMU/Gyro	494
7.7.23	Sensor ID=39: Mitsubishi	494
7.7.24	Sensor ID=40: Acurolink	495
7.7.25	Sensor ID=41: Openlink	497

Chapter 1: Introduction

This Administration Guide is created to explain and illustrate the Commands and Objects detailed in the Platinum Line Command Reference Guide. It consists of a considerable number of chapters which are continuously updated. The manual is gradually developing in stages, building chapter by chapter, reviewed carefully for relevance of its material.

The following safety symbols are used in this and all Elmo Motion Control manuals:



Warning:

This information is needed to avoid a safety hazard, which might cause bodily injury or death because of incorrect operation.



Caution:

This information is necessary to prevent bodily injury, damage to the product or to other equipment.



Important:

Identifies information that is critical for successful application and understanding of the product.



Note: Information critical to the understanding and/or operating the feature.



Tip: Information that helps understanding a feature, is good practice or a possible different way of action.

1.1 Scope

The following describes the document scope.

1.1.1 Mission

This document defines the requirements for the Platinum drive and presents the documentation structure of each of the functionalities. It includes top level Requirements, each which has a specified requirement and design per the relevant module document.

Throughout this document, we differentiate between a Platinum drive and the Platinum Quartet which are both physically different and operate differently with respect to their capability. Refer to the section 1.2 Terms and Abbreviations for a definition of the difference between a Platinum drive and Platinum Quartet drive.

1.1.2 Platinum Documentation Concept Requirements

The following document presents the documentation structure of each of the functionalities.

1.1.2.1 Requirements

The requirements document consists of the top-level requirements. It must include all the functionalities and top level required information. The drive functionalities are based on the requirements document.

1.1.2.2 Detailed Design

This document details the design of every function. The design document may include detailed requirements sections that elaborate the top-level requirements. The outcome of this document is a clear view of how the functionality should be programmed and all the interfaces to other relevant functions. Every function in the Detailed Design document must include a reference to the **Requirements** document.

The document should include (one or more of the following) detailed explanation about the function, flowcharts, pseudo code, user API, algorithm, notifications, and all other information that the designer sees which can result in a simple programming. Adding the motivation for the selected design is highly recommended.

1.1.2.3 Module Tests

This document shall include the insertion tests (known also as unit tests) that cover all the tests that the programmer must do to assure that the module works as designed. These tests are “White Box” tests meaning that the tester is aware of the code, algorithm, and procedure, and actually has the ability to test the specific functionality of each branch and term in the code.

The Module test will have a reference to the function in the **Detailed Design** document. Module tests must include “Good” tests that result in a positive flow and “Bad” tests that result in errors making sure that the expected error is informed and that the function behavior is as expected in erroneous situations. Note that in Module tests the code can and should be modified for the sake of the test BUT the tested function\module should not. Tests are planned and performed by the engineer that programmed the functionality.

1.1.2.4 System Tests

This document includes tests that are required from QA as part of the system tests. These are “Black Box” tests which mean that the QA engineer is not aware of any implementation or design methods. The tests should be treated as reference inputs and expected results and should include failures path tests as well as success path tests. Each test shall have reference to the **Requirement** document.

1.2 Terms and Abbreviations

The following abbreviations are used in this document:

Term	Explanation
abs/rel	Absolute and relative, which are indications of how to treat the position reference command in relation to the actual location.
CAL	CAN application layer.
CAN client or master	A host — typically a PC — or other control equipment that supervises the nodes of a network.
CAN server or CAN slave	A node in the CAN network that can give service under control of the CAN master.
CMS	CAN message specification.
COB	Communication object; a CAN message.
COB-ID	A binary bit-field that includes the ID of the server with which the master talks, and the type of COB.
csp	Cyclic Synchronous Position mode (available in ECAT only)
csv	Cyclic Synchronous Velocity mode (available in ECAT only)
cst	Cyclic Synchronous Torque mode (available in ECAT only)
cstca	Cyclic Synchronous Torque with angle commutation (available in ECAT only)
CW	DS-402 ControlWord, Object 0x6040
CW	Controlword, object 0x6040
DIN	Digital input
DOUT	Digital output
Download	Transfer of data from the host to the drive
DSP	Digital Signal Processor
EAS	An Elmo Application Studio application used for controller setup, application downloading, and monitoring.
ECAT	EtherCAT protocol\fieldbus.
EDS	Electronic data sheet. The list of CAN objects supported by a device, in a form suitable for standard configuration software. The EDS is used by external CAN configurators.
FSOE	Fail Safe Over EtherCAT
Hexadecimal	Numbers marked with either <i>h</i> (such as 1000h) or <i>0x</i> (such as 0x1000) refer to a hexadecimal value. Objects and numbers may appear in either form in different CAN documents.
hm	Homing mode
ID	Identifier: the name by which a CAN device is addressed.

Term	Explanation
IDE	Integrated development environment
ip	Interpolated position mode (available in CANopen only)
Load position	What the position sensor measures, expressed in position units (in contrast to position sensor increments).
LSB	Least Significant Bit (or Byte)
LSS	Layer Setting Service: methods for configuring the ID and baud rate of a slave, using the standard DSP-305
MSB	Most Significant Bit (or Byte)
NMT	Network Management
NONE	Wherever NONE is defined as the Default Value for an Object or parameter, no default value is relevant.
Non-volatile	The object data may be saved to the flash memory of a device using the SV command, or by setting object 0x1010 (sub1).
Object	A CAN message with a meaningful functionality and/or data. Objects are referenced according to addresses in the object dictionary. DO: Data object.
OD	Object dictionary, which is the full set of objects supported by the node. It is the interface between the application and communication (see "Object" below)
PDO	Process data object. A CAN message type, which eliminates the need for allocation the data payload for object addressing by pre-agreement concerning the message contents (PDO mapping)
Platinum Drive	A single drive with advanced hardware and software functions, capability of supporting two axis
Platinum Quartet	A drive with the capability of supporting four axes as single or vector axes
PLC	Programmable controller. A PLC can serve as a CAN master for all digital servo drives.
Position sensor increments	Units measured by the load position sensor. The speed is derived from the position sensor.
pp	Profiled position mode
pv	Profiled velocity mode
Receive	In this manual, "received" data is sent from the control equipment to the servo drive.
Reference	Motion parameters can be specified in terms of meters/second for speed, or encoder counts for position.
rfg	The reference generator, which generates the trajectory for velocity mode only.
SW	Statusword, object 0x6041
tq	Profiled torque mode

Term	Explanation
Transmit	In this manual, “transmitted” data is sent from the servo drive to the other equipment.
Upload	Transfer of data from the drive to the host

The following table lists the shortened terms used in this manual:

Prefix/Suffix	Definition
UU	User defined Units
Cnt/sec	counts per second
MDP	Modular Device Profile
Sub	Sub Index
RxMap	Mappable to rPDO
TxMap	Mappable to tPDO

Table 1: Shortened Terms

1.3 Relevant Elmo Documentation

This manual is part of the Elmo Platinum documentation set, which also includes:

- The Platinum Safety Drive Manual provides full instructions for all Platinum servo drives and individual Installation Guides.
- The EASII Inline Help includes explanations of all the software tools that are a part of Elmo's Application Software environment.
- The Platinum Command Reference Manual describes each software command used to manipulate the motion controller.
This is the main source of detailed explanations of all Platinum commands mentioned in this manual.
- Chapter 6: User Programming Language in this manual describes the comprehensive software used with Platinum digital servo drives
- The CANopen DS301 implementation guide, is a chapter of the Elmo Platinum digital servo drive Administrative Guide.

1.4 Standards

xxx	yyy

Table 2: Standards

Chapter 2: The Platinum Drive

2.1 Firmware Architecture & Update

This chapter includes general firmware architecture and requirements from the image and the program flow. The concept here is to treat the drive as a single entity regardless of the processors and different option FW (firmware) structure.

2.1.1 General Requirements

Platinum Drive firmware consists of the following:

- A single file allowing a single download procedure via dedicated commands
- Includes a bootloader section and firmware section. There will be no operating system.
- Is constructed from a single file and will include a major FW version
- The FW section shall include at least 3 interrupts, the real time, the High speed and the Safety and background cycle where nondeterministic functions can be executed.
- The background cycle and the CPU load time shall be measurable via non-user Elmo command. This allows the designer to see the load of the processor.
- Drive interrupts shall be synchronized with each other and to an external event.

2.1.2 Memory Organization

There is an option to address the memory via Elmo command. This is required for debugging purposes also in the field. This option will not be revealed outside the R&D.

2.1.3 Firmware Images

The firmware image includes all the parts of the firmware for updating the version. This means that a single update of firmware procedure shall update all the subcomponent of the drive.

The drive shall have a redundant FW image allowing the user to fall back to the previous version for any reason. The fallback shall be allowed if the last FW update failed.

2.1.4 Firmware Status

The user shall be able to detect the status of the firmware right after power up and at any time, and shall be able to recover from any problematic situation after firmware update.

2.1.5 The Firmware Version

The FW version shall distinguish between versions regardless of the component that was modified. There can be other subversion distinguishing between any of the sub-components.

2.1.6 Updating Firmware Version

FW update can be done in all communication channels.

2.2 Drive Identification and Personality

This section includes the requirements for the Drive Identification which is the section that identifies the specific hardware characteristics of the drive such as drive name (VR), Voltage (BV), Current (MC), critical default and limits, temperature limits, etc.

The DI is used by the drive FW and should save code space and logic. The Personality is the database that includes relevant data embedded in the firmware and can be used by the host to improve the user experience and the interface.

The chapter deals with how to detect a drive in the network with quick introduction to ESI files.

It includes details about drives with and without ECAT switches and how to read and handle these inputs.

2.3 Drive Units, Data Type & Conversions

This section defines the requirements for the physical parameter's units (e.g. velocity, acceleration) and the data types that can be used. It also includes conversion which (and where) is required for any calculation between user units and the physical \ internal units.

2.4 The Multi Axes Concept

This section defines the requirements for multi axes handling, number of axes and the method by which multi axes is treated in the drive level. The requirements in this chapter cover the multi axes handling in all the different methods which can be used to address an axis.

2.5 Communication & Network

This section includes the top-level requirements of all communication channels and all the methods by which a host can address the drive. The chapter also defines the timing requirements according to the communication type and with respect to the control and profiler.

2.5.1 Functional Communication

Deals with RS232, USB, UDP, TCP/IP.

2.5.2 Fieldbus Communication

Deals with ECAT & CAN PDO, SDO, Sync managers, ID, state machine.

Chapter 3: Communication and Interpreter

3.1 Introduction

This chapter describes the Regular IO, Safe IO, and other communication Objects and operational modes of the Elmo DSP-based motion controller implementation of the DS-301 and DS-402 protocols. These Objects are the aliases of the commands, the Elmo Legacy Commands, referenced in the Platinum Command Reference document - **MAN-P-CommandRef**. The Interpretation of the commands apply to the following communication channels:

- RS232
- USB
- Ethernet: UDP or TCP\IP
- EtherCAT: EoE

For further details of the command's interpreter, refer to section Drive Communication and Interpreter in the Platinum Command Reference.

Most of the platinum functionality is standard, according to CiA documents DS-301, DSP 402 (proprietary), and the CiA OS interpreter. This chapter contains data which is relevant to the operation of the Elmo drive.

The most important part of a device profile is the **Object Dictionary** description. The object dictionary is essentially a grouping of objects accessible via the network in an ordered pre-defined fashion. The DS-401 and DS-402 standard objects of single-axis drives are all in the index range of 0x1000 to 0x67ff and possible sub-index range of 1 to 254.

3.2 Description of Attributes



Note:

In some situations where indexes(bits) are not tabulated, the indexes are to be considered as Reserved, and may be set to 0.

The following defines the indexed headings for all objects:

Attribute	Description
Name	Name of the object
Object code	Variable (VAR): the entry is a variable with single entry Array (ARRAY): the object includes sub-indices with same variables type distinguished by sub index value only. Record (RECORD): the object includes sub-indices with various variable types with unique name of each entry.
Data type	The base data type: BOOLEAN: True \ falls single bit BYTE: unsigned byte, Octet 8 bits integer INT16 \ UINT16: signed or unsigned 16 bits integer DINT \ UDINT \ SIGNED32 \ UNSIGNED32: signed or unsigned 32 bits SIGNED64 \ UNSIGNED64: 64 bits integer

Attribute	Description
	REAL32: float 32 bits REAL64: double 64 bit OCTET_STRING(x): string size x VISIBLE_STRING: null terminated ASCII-based string
Protocol	EtherCAT CAN Not all objects are supported by both protocols
Multi-axes	Yes: if the object can be referred based on the axis with typical offset of 0x800 per axis. No: the object is not supported for multi axis references.
Sub-index	The number of available sub index for this object
Description	The sub index object description
Access	CONST: constant value RO: Read Only WO: Write Only RW: Read/Write
PDO mapping	Mapping options for this object: rPDO: available for receive PDO only tPDO: available for transmit PDO only NO: mapping is not relevant YES: receive & transmit PDO are available
Non-volatile	Saving availability of the object via SV command or object 0x1010 or object 0xFBFB2. No: the object is not saved to the flash Yes: the object is saved to the flash.
Value range	Possible range of the object. Sometimes not all values within the range are available. An adequate error will be populated if value is not available.
Default value	The factory reset value. Factory reset is available via RS command or object 0xFBFB0.

3.3 Platinum Line Communications

Platinum Line digital servo drives can simultaneously communicate using both EtherCAT and USB communication lines, which are always open for communication. The communication parameters are set using the **PP** command.

The following table compares the main features of both communication modes, as implemented with Elmo Platinum Line digital servo drives:

Features	EtherCAT
Baud rate	50,000 - 1,000,000
Interpreter method	Binary or ASCII
Fast referencing	Yes, for PVT, PT and ECAM (To be edited and applied)
Network of servo drives	Yes
Multiple servo drive synchronization	Yes
Standardization	Compliant with EtherCAT and CiA standard
Special equipment required	CAN communication interface (available as an add-on ISA or PCMCIA card for PCs) with appropriate software
Ease of use	Basic capabilities included in Elmo EAS program and the GMAS or PMAS master.

EtherCAT communication achieves higher rates and is able to support the following advanced functions:

- High speed online reference generation, required for supporting complex motions
- Binary interpretation, which gives a simple method of communicating the drive commands and parameters via Elmo interpreter.
- Servo network applications

To benefit from EtherCAT communication and the DS-301/DS-402 standards, the user must have a good understanding of the basic programming and timing issues of an EtherCAT network.

3.4 Drive Communication & Interpreter

Interpretation of commands as described in this document is derived from the following communication channels:

- RS232
- USB
- Ethernet: UDP or TCP\IP
- EtherCAT: EoE

These channels are considered as functional channels where the interpretation and activation of the command have no time restriction and are performed in the background task of the drive. The drive supports real time channels via fieldbus communication that include EtherCAT CoE and CANopen. The method of addressing these channels is not in the scope of this document.

The commands described in this document can be also used via the User Program of the drive. The User program allows the user to download any desired code to the drive and run the code via a command or automatically after power up.

The drive interpreter evaluates a string which is a sequence of characters, terminated by semicolon ';'. The interpreter will process the command once the terminator is detected.



Note:

For the terminator the 'line feed' (ASCII 10) and the 'carriage return' (ASCII 13) can also be used. However, this is less recommended due to the extra functionality of these chars which is not always welcome.

The interpreter will reply to the host with ';' on a success interpolation or with '<CHAR>?;' the '?' indicates an error, the <CHAR> includes the error in binary format. The actual value of the error can then be retrieved via **EC** command. List of the errors are indicated in the Platinum *Command reference: **MAN-P-CommandRef***.

It should be noted that:

- Echo ignores white characters such as <Space> & <Tab>.
- In Ethernet based communication the echo terminator (';') can arrive in the next packet after the echo was sent.

The Platinum drive maintains the legacy format of the 2 letters command with enhancement that the format of [letter][number] can also be used. The numbers are from 1 to 9 and the letters from A to Z. The first command is **A1** and the last command is **ZZ**. The interpreter also uses a prefix to the command. The prefix is described in the following section.

The maximum length of the line can be 1024 characters.

3.4.1 Assign and Retrieve Parameters

Assigning a parameter is done with the '=':

```
UI[1]=1000;
```

```
AC=UI[1] * 5 ;
```

Retrieving a value is done by a direct request as follows:

```
UI[1];
```

```
AC;
```

The return of the parameter will be according to the data type of the parameter. Data types are described in the attribute of each parameter in this document.

The interpreter can process basic mathematics operation allowing the user to manipulate commands, for example:

Command Line	Result	Remarks
3+4	7	
UI[1]=10; UI[1]-3	7	UI[1] is set to 10, and 3 is then subtracted.
(3.2+4)/2	3.600000	

3.4.2 Interpreter Echo

The characters that are sent to the drive are echoed back in any of the channels. **EO** command can be used to deactivate the echo.

When echo is activated the interpreter reply will be added in consecutive to the echo for example:

Assign command: the value '5' into the command "UI" , sub-index '23' :

```
'UI[23]=5;'
```

The reply with echo: 'UI[23]=5;;'

The reply w/o echo: ';' ;

Retrieved command:

Host sends: 'UI[23] ;'

The reply with echo: 'UI[23];5;'

The reply w/o echo: '5;'

Error with Echo

In case of echo with error the reply is as follows:

Host sends: 'UI[50]=7;'

Reply with echo: 'UI[50]=7;3?;'



Note:

'3' is the binary ascii character of 3 (ETX)). Send **EC** to get the '3' which is the error or Bad Index (as 50 is not a legal index).

3.4.3 Command Format: Scalar vs Array

Two types of commands are used:

Scalar commands: **AC**, **BG** etc.

Array commands: **CA[1]**, **KP[3]** etc.

The Platinum drive handles all commands as an array where sub-index 0 describes the total number of elements in the array and is the highest sub index supported.

For compatibility the legacy scalar commands can also be addressed as array with first index for example:

VR & **VR[1]** result in the same reply from the drive.

3.4.4 Command & Data Types

The term "command" includes the following interpretation options:

Commands	Write <i>only</i> commands that perform an action such as " BG " to start motion or " ST " to stop motion.
Parameters	Commands that include data and have the following attribute • read only (e.g. "VR" for firmware version), • write only (e.g. TW[16] to manually control the drive LEDs) or • read/write (e.g. UI[1] general user parameter that can be freely used) that sets or gets a parameter from the drive.

The parameters can be volatile (e.g. "**PA**" for Position Absolute reference) or non-volatile ("**AC**" profile acceleration).

Parameters have a defined data type that is described in this document. The commands have no data type.

The following data types are supported:

Type	Bytes	Range
char	1	-128 ... +127
Unsigned char	1	0 ... 255
short	2	-32768 ... +32767
unsigned short	2	0 ... 65535
long	4	-2,147,483,648 ... +2,147,483,647
unsigned long	4	0 ... 4,294,967,295
float	4	-1e37 ... 1e37
double	8	-1.0e307 ... +1.0e307
long long	8	-9,223,372,036,854,775,807 ... 9223372036854775807

3.4.5 Bit-field Format

Parameters can include bit-field format. The bit-field means that each bit or group of bits within the same parameters have an individual meaning (indication). In this format there is no meaning to the decimal presentation of the number. Typically, these types of variables are defined as unsigned type. It is useful to receive the values using the hex decimal format to simplify the visual interpretation. The attribute of the command in this document indicates a bit-field type.

3.4.6 Hexadecimal Format

For integer values, typically bit-field type, it is useful to request the drive to transmit the value in a hexadecimal format. Setting **HX=1** will convert any integer to Hex value with the 0x prefix. Setting an integer parameter can be done in decimal format or by using the prefix '0x' for a hexadecimal format. There is no need to set **HX=1** for the "set" of a parameter.

Note that the interpreter replies for mathematical operation when Hex format is active is always in long long format. See the example below.

The following examples are a reply from the drive for a "get" request:

Command	Dec (HX=0)	Hex (HX=1)
SR	2105351	0x00202007
UI[1]=0x1234	4660	0x1234
UI[1]+0xA	4670	0x000000000000123E (math operation is long long)

3.4.7 Mathematic and Logical Operation

The following math and logic operations are available including a simple example:

Operator/ Description	Symbol	Examples
Arithmetic addition	+	4+5=9 3.45+2.78=6.23
Arithmetic subtraction	-	3.45-2.78=0.67
Arithmetic multiplication	*	UI[1]=UI[1]*2 multiple UI[1] by 2 5*4=20 1.5*2=3.0
Arithmetic division	/	20/4=5 3/1.5=2.0
Remainder after the division of two integers	%	20%4=0 23%4=3
Bitwise NOT	~	~3 is 0x00000000ffffffc, (if HX=1, note: -4.)
Bitwise OR		UI[1]=0x2 0x5 is equivalent to UI[1]=7.

Operator/ Description	Symbol	Examples
Bitwise AND	&	UI[1] =0x7&0x3 is equivalent to UI[1] =3
Logical equality	==	If UI[1] = 3 and UI[2] = 3, then UI[1]==UI[2] yields 1 If UI[1] = 3 and UI[2] = 5, then UI[1] == UI[2] yields 0
Logical inequality	!=	If UI[1] = 3 and UI[2] = 3, then UI[1]!= UI[2] yields 0 If UI[1] = 3 and UI[2] = 5, then UI[1]!= UI[2] yields 1
Logical greater than	>	If UI[1] = 3 and UI[2] = 3, then UI[1]>UI[2] yields 0 If UI[1] = 3 and UI[2] = 2, then UI[1]>UI[2] yields 1 If UI[1] = 1 and UI[2] = 2, then UI[1]>UI[2] yields 0
Logical greater than or equal to	>=	If UI[1] = 3 and UI[2] = 3, then UI[1]>=UI[2] yields 1 If UI[1] = 3 and UI[2] = 2, then UI[1]>=UI[2] yields 1 If UI[1] = 1 and UI[2] = 2, then UI[1]>=UI[2] yields 0
Logical less than	<	If UI[1] = 3 and UI[2] = 3, then UI[1]<UI[2] yields 0 If UI[1] = 3 and UI[2] = 2, then UI[1]<UI[2] yields 0 If UI[1] = 1 and UI[2] = 2, then UI[1]<UI[2] yields 1
Logical less than or equal to	<=	If UI[1] = 3 and UI[2] = 3, then UI[1]<=UI[2] yields 1 If UI[1] = 3 and UI[2] = 2, then UI[1]<=UI[2] yields 0 If UI[1] = 1 and UI[2] = 2, then UI[1]<=UI[2] yields 1
Logical AND: The result is 1 if both arguments are non-zero, and 0 if one is zero.*	&&	1&&5 yields 1 0&&2 yields 0
Logical OR The result is 1 if one argument is non-zero, and 0 if both are zero.*		1 0 yields 1 0 0 yields 0
Logical NOT: The result is 1 if the argument is zero; otherwise, it is 0.*	!	!4 yields 0 !0 yields 1
Unary minus: The result is negative if the argument is positive, and vice versa.*	-	-4.5 yields -4.5 -4 yields -4 (-4) yields -4 -5+5 yields 0
Bitwise left shift: Shifts the first operand to the left by the number of	<<	8<<2 yields 32

Operator/ Description	Symbol	Examples
positions specified in the second operand.*		
Bitwise right shift: Shifts the first operand to the right by number of positions specified in the second operand.*	>>	8>>2 yields 2

* The arguments are truncated to integers before evaluation.



Note:

When a float is used, then the result will be classified as float.

Division of integer will be classified as integer (base operation).

Scientific notation can be used for float presentation and settings:

Example 1.e1 is 10.0 and 1e-2 is 0.01

The **DP** command can be used to determine the presented number precision. By default, the value is -1 using the max available range. The value of float number will round to the nearest integer – see example below.

Note that internally the value is not modified, and the **DP** only effects the presentation.

Examples:

Command	DP=-1	DP=2	DP=3
1.1239E4 * 0.0001	1.123900e0	1.12	1.124 (near integer)

The drive supports build in function operation as follows – note function names are case-sensitive.

System functions are mainly used via user program but can also be called via the interpreter. To get the system functions via interpreter need to set bit 8 in **AF[2]** command (**AF[2] |= 256 ;**):

Function	Description	Returns
sin	UF[1]=sin(x) x [rad] For example, sin (3.1415 / 2) is 1.00 Complex expression can be used: sin(2*3.1415+(3.1415/2) which is sin (2 * π + π/2) is 1.000	double
cos	UF[1]=cos(x) x [rad] For example, the angle 60° is approx. 1.0472 radians (60 * (2 * π / 360)). The result of cos(1.0472) is 0.500	double
sqrt	y=sqrt(x) Square root of the argument is negative result in “nan” For example, the results for sqrt(16) is 4.00, the result of sqrt(-81) is “nan”.	double

Function	Description	Returns
fix	$y = \text{fix}(x)$ Truncates the input argument to an integer. For example, the result of $\text{fix}(3.8)$ is 3, and the result of $\text{fix}(-3.8)$ is -3. Note: If the input argument exceeds the long value range, it will be limited to the maximum long value (for positive numbers) or the minimum long value (for negative numbers).	long long
rnd	$y = \text{rnd}(x)$ Truncates the input argument to the nearest integer. For example, the result of $\text{rnd}(3.8)$ is 4, the result of $\text{rnd}(-3.8)$ is -4, and the result of $\text{rnd}(3.4)$ is 3. Note: If the input argument exceeds the long value range, it will be limited to the maximum long value (for positive numbers) or the minimum long value (for negative numbers).	long long
sign	$y = \text{sign}(x)$ Returns the sign of the input argument: -1 for negative numbers, 1 for positive numbers, 0 for a zero. For example, the result of $\text{sign}(-3.8)$ is -1, the result of $\text{sign}(3.8)$ is 1, and the result of $\text{sign}(0)$ is 0.	long long
real	$y = \text{real}(x)$ Converts an integer to a float. If the argument is a floating-point number, the function does nothing: The result of $5/2$ is 2, the result of $\text{real}(5)/2$ is 2.5, and The result of $5/\text{real}(2)$ is 2.5.	double

3.4.8 Streaming Commands

Stream commands are commands that are used to upload and download buffers from the drive. The list of commands is:

VR[1], DL, UL, BH, RH etc.

These commands either transmits back or receive a relatively big data. For example, the **BH** is used to upload recorder data stream from the drive. The **DL** is used for example to download a User Program or Parameter xml file to the drive. The handling of these commands is done via a specific stream buffer and typically the amount of data to be transmitted or received is part of the relevant command protocol.

3.4.9 User Program Global Variables

The Platinum drive can run a User Program allowing the user to activate the drive as a standalone entity via an embedded software that the user can program in an advanced programming language and download to the drive.

The User Program includes global and local variables which the user can define and use along the program. All global variables defined in the user program can also be addressed via the Interpreter.

Example for user program via the EAS “Drive Programming” window:

```
1 |  
2 //-----  
3 // Global Variables  
4 long PXv ;  
5 float CA_[2]  
6 //-----  
7 // Main Function (also the default entry point)  
8 function main()  
9  
10 global long PXv ;  
11 global float CA_[2];  
12 short ACII;  
13  
14 CA_[1]=10;  
15 ACII = 2 ;  
16 PXv = ACII+CA_[1]  
17  
18 return  
19
```

The **PXv** and **CA_[1]** are global variables, scalar and array which after compilation can be read and set via the interpreter.

```
:> Terminal  
Drive04>PXv  
PXv=0  
Drive04>PXv=125  
Drive04>PXv  
PXv=125  
Drive04>CA_[1]  
CA_[1]=0  
Drive04>CA_[1]=55  
Drive04>CA_[1]  
CA_[1]=55  
Drive04>
```



Note:

The interpreter can analyze the global variables even when the program is not running. Build and download suffices.

3.4.10 Commands Prefix

Enhancement for the interpretation is done via command prefix allowing to extend the functionality of the command. The prefixes are a set of characters that are sent as part of the command as follows:

[Prefix][.][Command].

The rules of the assignment or retrieve commands with a prefix are the same as a command w/o a prefix. The interpreter analyzes the prefix and performs the functionality accordingly.

The Prefix are:

- **AX** to address a specific axis

- **GR** to address a predefined group
- **OJ** to address an object in decimal format
- **OH** to address an object in Hex-Binary format.

Refer to the section [Interpreter Prefixes](#).

3.4.11 Elmo Commands Alias Objects

Objects are commands that are used in fieldbus protocol and are interpreted via an Object dictionary. Each object obtains the relevant functionality from the dictionary. Each object can have a sub-index of up to 254 entries. Wherever the object is from an array (or struct) type, sub index 0 will inform the number of the last index in the array.

For example, the command to set the protocol profile acceleration is done via object 0x6083 sub index 0 (0x6083.0).

The drive uses set of objects to allow the fieldbus to address Elmo's legacy commands. The set of objects that is used for this purpose starts from object 0x3000 until 0x35FF. The protocol stack converts the object 0x3000 to the A1 command and the 0x3001 to A2 command etc. In this example if the host want to address **UI[1]** it will send 0x32E2.1 and for **UI[10]** it will send 0x32E2.10. The alias objects are axis dependent with offset of 0x800 between axes. For example, the alias object for the DC command of axis 1 is 0x3078, the DC of axis 2 is 0x3878 and axis 3 is 0x4078.

All the objects can be addressed via the drive interpreter as well. The method of doing that is via the **OJ** prefix and the **OH** prefix. Details about the **OJ\OH** format are in the chapter "Interpreter Prefix"

The alias object for each command is written in the command description.

3.4.12 The Multi-axes

The Platinum drive include multi axes functionality. Depending on the product, several axes can be supported at the same time. To address the relevant axis, the prefix **AX** is used. Details about the AX prefix are in the "Interpreter Prefix" chapter.

The axis always treats the basic format as if the first axis was addressed.

For example:

PX; retrieves the main position of the first axis. This is the same as writing AX1.PX; or AX1.PX[1];

Addressing the position of axis 2 the AX2 prefix MUST be used and in this example: AX2.PX;

To address the object of the second axis an offset of 0x800 is used. For example:

As mentioned above the **UI** command alias object is 0x32E2. The **UI** of the second axis is 0x800 from the first axis hence: 0x32E2 + 0x800 = 0x3AE2 To set **UI[3]=10** for axis 2 the following options can be used:

AX2.UI[10]=10;

OJ.32E2[10] = 10;

Not all the parameters are axis related. Consult the relevant command attribute for commands which are related to the device. Note that in this situation addressing the command via different **AX** will result the same value for example the **AN[6]** retrieves the instantaneous bus voltage. The drive includes a single voltage entry and retrieving the value from **AX1** or **AX2** results in the same value.

3.4.13 BIT Status Indications

The drive includes a built-In Test (BIT) each time it starts. The **BZ[1]** command includes the result of the BIT. It is advisable to retrieve the value of the **BZ[1]** each power up.

3.4.14 Status LED Blinks

The Status LED blinks in the following scenarios:

- FW download is performed
- Drive FW is burnt
- BIT error (**BZ[1]**)

The following table details the status LED states:

LED \ Time (mSeconds)	Meaning
Green constant	Power stage ready for motor enable (Refer to Note below)
Red Constant	Power state error: over\under voltage, over temperature, sensor error, etc.
Alternate: Red 600, Off 200c	Drive in Safety error (BZ[2]\BZ[3])
Alternate: Red 200, Off: 200	Parameter process failed during power up (CD command)
Alternate: Red 200, Green 200 Red 600, Green 200	Flashing GREEN/RED/OFF during burn, frequency depends on the stage of burning/validation and the CPLD/FPGA that is been burned



Note: Green LED is presented by default and when no motor is defined

Chapter 4: DS402 Motion Profile

4.1 Introduction

4.1.1 Scope

This chapter describes the objects and operational modes of the Elmo DSP-based motion controller implementation of the CiA DSP402 protocol.

The Elmo drive includes a digital and power section. When the drive comes to motion, the digital section includes the profiler where a requested target (set-point) has been processed to result in the desired trajectory and the control loops where a control algorithm (PI/P) ensures that the physical load follows the desired trajectory within the specified limits.

The DS402 protocol includes the definition for controlling the servo State Machine that enables\disables\stops the servo as well as fault reset and behavior and it defines how to control the motion profiler behavior via both sets of parameters and controlling the mode of operation. The DS402 also determines how to set the desired target also named set-point, required speed, acceleration, limits and handling of violations. The DS402 however does not deal with control parameters such as PI/P, scheduling, filters and feedforward.

The Elmo controller provides a number of different options for setting commands and parameters, such as via the proprietary binary interpreter, OS interpreter, RS-232 interpreter and user programs. When the user works with DS402, all relevant motion commands must be given through this method only.

Other command sources are useful for purposes not covered by the DS402 protocol. Examples include:

- ECAM modes
- Output compare modes
- Monitoring servo status with digital output settings etc.

4.1.2 Introduction

The CiA DS402 *CANopen Device Profile for Drives and Motion Control* is used to provide drives as well as controller (PLC, CANopen masters) in a CAN network with an understandable and consistent behavior. The profile is built on top of a CAN communication level, called CANopen, which is defined in DS-301 protocol and describes the basic communication mechanisms common to all devices in the CAN network.

Note: DS402 was designed for CANopen communication and was well adopted by the EtherCAT communication protocols from Beckhoff. The CAN Over EtherCAT (CoE) is designed to meet the requirements of the DS402 operating via an EtherCAT network. *The ETG6010 CiA402 ImplDirective* introduces defaults and inputs that are aligned with the EtherCAT definition. Unless mentioned specifically, the described operation or object complements both communication channels. For more details about EtherCAT refer to the Chapter *EtherCAT Implementation*.

The drive receives messages from the motion controller (host) such as Elmo PMAS. These messages include configuration parameters, operational commands and status requests. The drive transmits the following back to the host; status messages, required feedbacks and fault indications over the network.

Typical operation via the CANopen network is that the host configures the drive via Service Data objects (SDO messages) and with the requested real time parameters. After the configuration is confirmed, the host transfers the drive status to operational state (OP state) where the preconfigured Process Data Objects (PDO)

are used for the real time control. The PDOs increase the communication throughput allowing the motion controller to set the next set-point (motion target), modify the drive status and receive feedback information such as actual position, velocity and current in highly efficient way.

Note: The DS402 defines sets of commands and operations, named “objects”, to up to 8 axes on a single device. The Platinum object and object dictionary (see below) include a sets of commands supporting up to 6 axes. Each axis has an offset of 0x800 e.g., the “Operation Mode” object for axis 1 is 0x6060 while the Operation Mode of axis 2 is 0x6860 and the 6th axis is 0x8860. In This document we will refer to the drive as a single axis nevertheless the rule for the multi axis applies. In some situations, objects are relevant to the device and not to the axis e.g., Software version: 0x10A0.0, in this case the 0x800 offset is not relevant.

The motion controller controls the drive using the Controlword (object 0x6040) and receives the status of the drive via Statusword (object 0x6041). It can select the motion mode of the drive via the Mode Of Operation object (0x6060). The optional modes are Profile Position, Profile Velocity, Profile torque, Interpolated Position , Homing mode and in EtherCAT based drives: Cyclic Synchronous Position, velocity or torque modes.

As mentioned all the commands to the drive are indicated via Objects and the set of objects are gathered in the database named the **Object Dictionary**. For more information about the objects and the Object Dictionary range refer to “ *The Object Dictionary*” chapter..

4.2 Device Control Objects

4.2.1 General

The following objects are presented in this chapter:

Object	Definition
0x6040	<i>Controlword</i>
0x6041	<i>Statusword</i>

The Device Control function block controls all functions of the device, categorized as:

- Device control of the state machine (only in Remote mode, see details later)
- Operation mode functions
- The status of the device is indicated by the *Statusword*.

4.2.2 Remote and Local modes

The drive can be controlled in Remote (the *Remote* indication in the *Statusword* bit 9 is set to '1') or Local modes (bit 9 is set to '0').

The desired mode is selected by **AF[1]=1 (Remote)**, **AF[1]=0 (Local)** commands (Refer to Platinum drive Command Reference). By default, the drive is in Remote mode.

- In Remote mode the state machine is controlled by an external field bus host (EtherCAT\CANopen) via **Object 0x6040**. In this mode the servo cannot be enabled via the legacy servo enable command **MO=1**, but it can be disabled by command **MO=0**. In that case the drive switches automatically to Local mode (read access to **AF[1]** command returns 0).
- In Local mode Controlword is ignored. The drive is controlled by Elmo legacy commands **MO** and **BG** (refer to Platinum drive Command Reference). The objects exclusive 0x6040 can be accessed via SDO.

4.2.3 State Machine

When the drive is in Remote mode, the write access to the *Controlword* is always allowed. In addition, the state machine is controlled by internal signals such as faults and modes of operation.

Figure 1 illustrates the Device Control function.

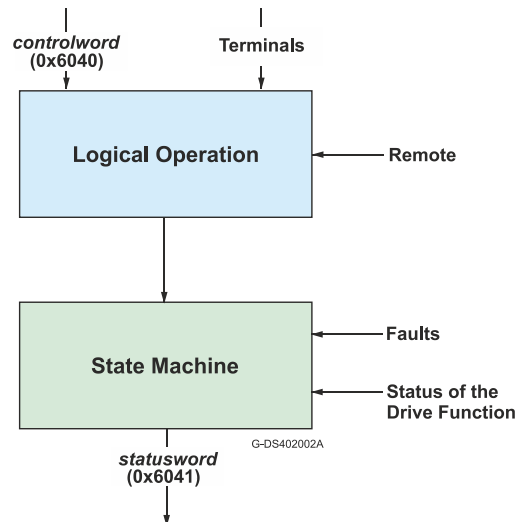


Table 3: Remote Mode

The state machine describes the device status and the possible control sequence of the drive. A single state represents a special internal or external behavior. The state of the drive also determines which commands are accepted. For example, a point-to-point motion can be started only when the drive is in OPERATION ENABLED state.

States may be changed using the *Controlword* and/or according to internal events. The current state can be read using the *Statusword*. Refer to the chapter "The Object Dictionary" for detailed descriptions of the objects 0x6040 and 0x6041.

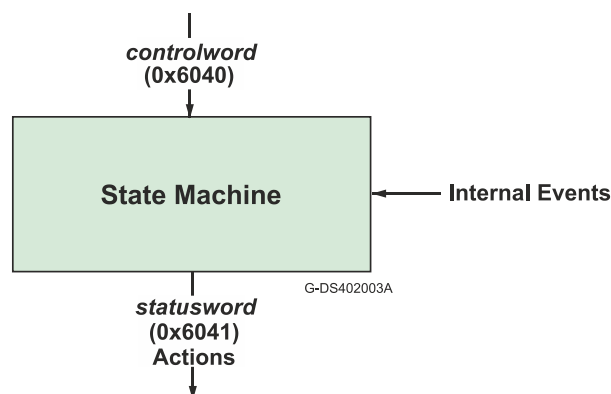


Table 4: State Machine in System Context

The device states and possible control sequence of the drive are described by the state machine, as depicted in Figure 3 below:

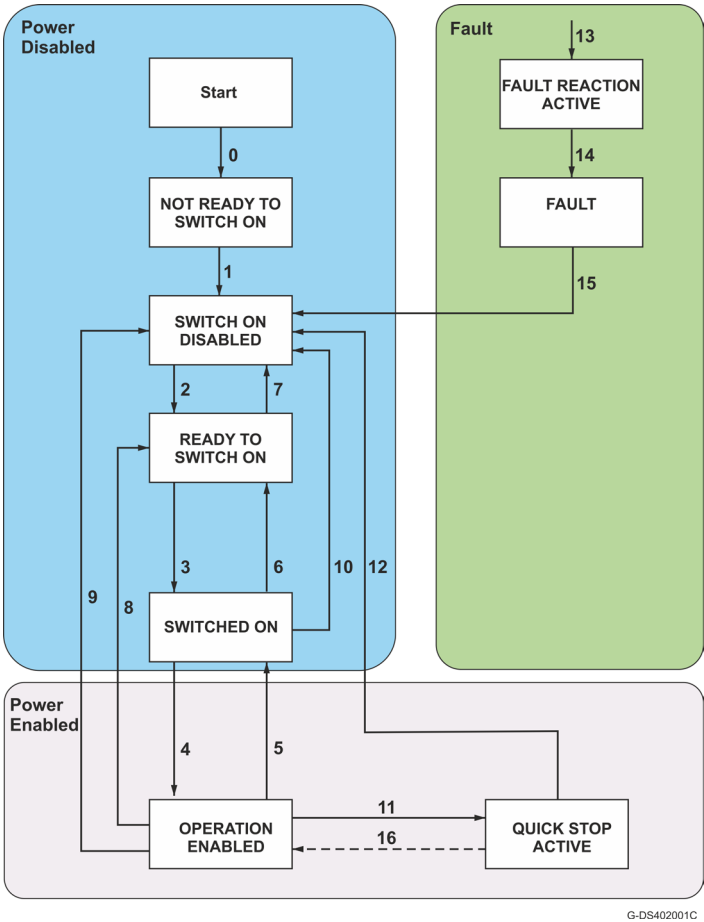


Table 5: State Machine Block Diagram

4.2.4 Drive States

The drive states are as follows:

Drive State	Process	Description
NOT READY TO SWITCH ON	Low-level power (24V) has been applied to the drive. The drive is being initialized and is running the self-test. A brake output, if present, is applied in this state.	The drive function is disabled. This state is an internal state in which communication is enabled only at the end. The user can neither retrieve nor monitor this state.
SWITCH ON DISABLED	Drive initialization is complete. The drive parameters have been set up.	SWITCH ON DISABLED is the minimum state to which a user may switch. The drive function is disabled. In this state, there is no fault indication even if high power is applied. The application must be responsible for handling the state transition.
READY TO SWITCH ON	High voltage may be applied to the drive. The drive function is disabled.	
SWITCHED ON	High voltage has been applied to the drive. The power amplifier is ready.	The drive function is disabled. In this state, there is no fault indication if the drive high voltage has not been applied.
OPERATION ENABLED	No faults have been detected. The drive function is enabled and power is applied to the motor.	This state is the normal operation of the drive In this state, a brake is automatically released according to the brake parameter (BP[N]) timing.
QUICK STOP ACTIVE	The quick stop function is being executed. The drive function is enabled and power is applied to the motor.	In this state, according to the quick stop option code, the drive stops the motion with a quick stop deceleration or profiler deceleration and either stays in quick stop state or switches to SWITCH ON DISABLED state. In addition, there is the option to go to SWITCH ON DISABLED state immediately without stopping. The term <i>drive stops</i> means that the profiler completed the deceleration trajectory and not that the motor is stationary.

Drive State	Process	Description
FAULT REACTION ACTIVE	A fault has occurred and the fault reaction function is being executed. Following that, the drive automatically switches to FAULT state.	In this state, according to the fault reaction option code, the drive stops the motion with quick stop deceleration or profiler deceleration, and the drive function is disabled. In addition, there is the option to immediately disable a drive function without stopping it.
FAULT	A fault has occurred in the drive.	In this state the high voltage switch-on/-off depends on the application. The drive function is disabled.

4.2.5 State Transitions of the Drive

State transitions are a result of internal events in the drive or commands from the host through the *Controlword*.

When a command that causes a change of state is received, it is processed completely and the new state is attained before the next command is processed.

The drive performs transitions 0 and 1 after initiation, either at power up or in case of CAN communication at NMT node reset. This state remains until there is a change due to a received host command.

The FAULT OCCURRED indication implies that a fault has occurred in the drive during OPERATION ENABLED state. This results in a transition to the FAULT REACTION ACTIVE state, during which the device executes a motor disable function. After executing this fault reaction, the device switches to the FAULT state. The transition from FAULT state to SWITCH ON DISABLED requires a Fault Reset command. The transition will happen even if the fault still exists.

If an error occurs in OPERATION ENABLED state, an emergency message (if not masked via object 0x2F21) is sent with the fault reason. The last 16 fault messages are latched and are retrieved by via object 0x1003, refer to the chapter "The Object dictionary".

State Transition	Mode	Event	Action
0	From START to NOT READY TO SWITCH ON	Reset	The drive self-tests and/or self-initializes.
1	From NOT READY TO SWITCH ON to SWITCH ON DISABLED	The drive has completed self-test and/or initialized successfully.	Activate communication.
2	From SWITCH ON DISABLED to READY TO SWITCH ON	Shutdown command received from host.	None
3	From READY TO SWITCH ON to SWITCHED ON	Switch On command received from host.	The power section is switched on if it is not already on.

State Transition	Mode	Event	Action
4	From SWITCHED ON to OPERATION ENABLED	Enable Operation command received from host.	The drive function is enabled
3 + 4	From READY TO SWITCH ON to OPERATION ENABLED	Enable Operation command received from host.	The drive function is enabled
5	From OPERATION ENABLED to SWITCHED ON	Disable Operation command received from host.	The action is defined by the object 0x605C "Disable operation option code"
6	From SWITCHED ON to READY TO SWITCH ON	Shutdown command received from host.	None
7	From READY TO SWITCH ON to SWITCH ON DISABLED	Quick Stop or Disable Voltage commands received from host.	None.
8	From OPERATION ENABLED to READY TO SWITCH ON	Shutdown command received from host.	The action is defined by the object 0x605B "Shutdown option code"
9	From OPERATION ENABLED to SWITCH ON DISABLED	Disable Voltage command received from host.	The power section is switched off immediately, and the motor is free to rotate if no brake is applied.
10	From SWITCHED ON to SWITCH ON DISABLED	Disable Voltage or Quick Stop command received from host.	None.
11	From OPERATION ENABLED to QUICK STOP ACTIVE	- Quick Stop command received from host (in all modes) - A communication error which is defined by the Abort Option code causing an auto stated machine to transition from Operation Enable to Quick Stop (Object 0x6007) - Extrapolation time exceeded the value defined by the object 0x3675 in time depended modes (ip, csp, csv, cst)	The quick stop function is executed. The action is defined by the object 0x605A "Quick stop option code". If the drive is in the time depended modes (ip, csp, csv, cst) the interpolation stops and the relevant bits in the Statusword are set. In hm mode, the home error bit is set.
12	From QUICK STOP ACTIVE to SWITCH ON DISABLED	This transition is performed automatically when deceleration is completed if the object 0x605A "Quick stop option code" is less	The profile generator finished the deceleration and the motor is disabled.

State Transition	Mode	Event	Action
		than 4, or command "Disable voltage" was received	
13	From OPERATION ENABLED to FAULT REACTION ACTIVE	A fault has occurred in the drive.	Execute appropriate fault reaction. The action is defined by the object 0x605E "Fault reaction option code".
14	From FAULT REACTION ACTIVE to FAULT	The fault reaction is completed.	The drive function is disabled.
15	From FAULT to SWITCH ON DISABLED	Fault Reset command received from host.	The fault state is reset. After resetting the Fault state via the <i>Controlword</i> Fault reset bit (bit 7), the Fault Reset bit of the <i>Controlword</i> must be cleared by the host. The drive does not monitor this bit in other states. If this bit is not cleared from a previous fault state, when the next fault occurs, the drive may automatically transit to switch on disabled state without any indications or warning.
14 + 15	From FAULT REACTION ACTIVE to SWITCH ON DISABLED	Disable Voltage commands received from host.	The drive function is disabled.
16	From QUICK STOP ACTIVE to OPERATION ENABLED	Enable Operation command received from host. This transition is possible if the quick stop option code (Object 0x605A) is set to 5, 6 or 7.	The drive function is enabled.

4.2.6 Illegal Transition

After initiation of a drive by either power up on or NMT node reset, the drive automatically performs transitions 0 and 1 to the **SWITCH ON DISABLED** state. The *Controlword* can then be used to initiate any of the transitions defined previously. If a transition is illegal (such as requesting a **QUICK STOP** in a **FAULT** state), it is ignored. No indication is provided by the drive in such occasions.

4.3 Deceleration Option Objects

Deceleration option objects determine the action to be performed by the drive when receiving commands Quick Stop, Shutdown, Disable operation, Halt or Fault or Abort Connection condition were detected.

The possible required actions are:

Action	Description
Slow down ramp	Profile deceleration (object 0x6084)
Quick stop ramp	Quick stop deceleration (object 0x6085)
Disable the drive function	Disable the servo, similar to MO=0 command, go to SWITCH ON DISABLED state

The objects are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition	Description
0x605A	Quick stop option code	This Object determines the action to take if the Quick Stop function is executed. Quick Stop is determined in two cases: - Request from the master to switch from Operation enable to Quick Stop state - A communication error which is defined by the Abort Option code causing an auto stated machine to transition from Operation Enable to Quick Stop (Object 0x6007) - Extrapolation time exceeded the value defined by the object 0x3675 in time depended modes (ip, csp, csv, cst, cstca)
0x605B	Shutdown option code	This object determines which action to take in the following transition: From OPERATION ENABLED to READY TO SWITCH ON.
0x605C	Disable operation option mode	This Object determines which action to take in the following transition: from OPERATION ENABLED to SWITCHED ON.
0x605D	Halt option code	This Object determines which action to take if bit 8 (Halt) in the <i>Controlword</i> is active.
0x605E	Fault reaction option code	This Object indicates what motion will be performed in case of a drive fault.
0x6007	Abort Connection Option Code	This object defines the motor control behavior when the following events occurred: - CAN bus-off - CAN heartbeat - CAN NMT stopped state entered via NMT command

Object	Definition	Description
		- ECAT Watchdog event - ECAT state machine switch from Operational state to Pre-Operational, Safe Operational, or Initial state while the servo was enabled.

4.4 Modes of Operation Objects

The objects are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition
0x6060	Modes of operation
0x6061	Modes of operation display
0x6502	Supported drive modes

The supported motion modes are described in object 0x6060.

The user can switch from any mode to any other mode.

By default, Modes of operation is initialized to *no mode* (value 0). Modes can be set in any state, including OPERATION ENABLED.

The actual mode is reflected via **Object 0x6061**.

When the drive is in Operation Enable, the motor can be controlled via the set point and the *Controlword* state, depending on the motion mode.

The *Statusword* contains *mode dependent* bits. When switching modes, the *mode dependent* bits in the *Controlword* and *Statusword* must be monitored.

4.5 Factors Objects

4.5.1 General

The chapter describe the conversion between physical units and internal units.

NOTE: the EAS provides an intuitive GUI for setting the factors.

Physical dimensions and sizes need to be converted into the device internal units, requiring a number of different factors. This chapter describes how these factors influence the system, how they are calculated and which data is needed to build them.

The objects are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition
0x608F	Position encoder resolution
0x6090	Velocity encoder resolution
0x6091	Gear Ratio

Object	Definition
0x6092	Feed Constant
0x6096	Velocity factor
0x6097	Acceleration factor

4.5.2 Relationship between Physical and Internal Units

The factors defined in the factor group determine a relationship between the Elmo drive internal units and the application physical units. Objects, which values are not dependent on the factor group have fixed units specified with the Objects. Figure 4 explains the scaling concept.

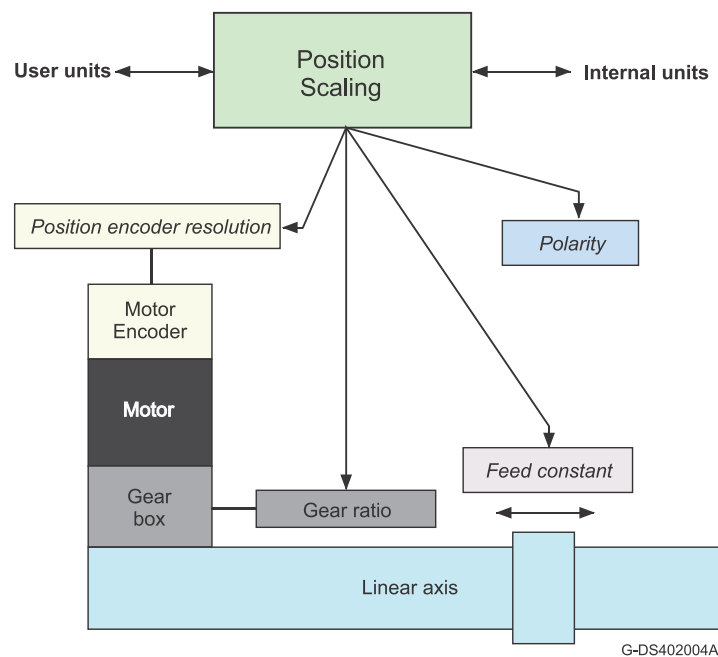


Table 6: Position Scale concept

4.5.3 Position Units

Position value in user units can be calculated from position in internal units by the formula:

$$PosUU = \frac{PosIU * FC}{PosEncRes * G}$$

Where:

PosUU	Position value in user units, Object 0x6064
PosIU	Position in internal units (encoder counts), Object 0x6063
FC	Feed constant, Object 0x6092
PosEncRes	Position encoder resolution, Object 0x608F
G	Gear Ratio, Object 0x6091

Position in internal units can be calculated from position value in user units by the formula:

$$PosIU = \frac{PosUU * PosEncRes * G}{FC}$$

4.5.4 Velocity Units

Velocity value in user units can be calculated from velocity in internal units by the formula:

$$VelUU = \frac{VelIU * FC}{VelEncRes * G} * VFac$$

Where:

VelUU	Velocity value in user units, Object 0x606C
VelIU	Velocity in internal units (encoder counts/sec), Object 0x6069
VelEncRes	Velocity encoder resolution, Object 0x6090
VelFac	Velocity factor, Object 0x6096 , =0x6096.1/0x6096.2

Velocity in internal units (encoder counts/sec) can be derived from the same position encoder, when sensor selection code , **Object 0x606A** is zero or from separate encoder, when sensor selection code , **Object 0x606A** is 1.

4.5.5 Acceleration Units

Acceleration value in user units can be calculated from velocity in user units by the formula:

$$AccUU = \frac{VelUU}{Sec} * AccFac$$

Where:

AccUU	Acceleration value in user units
AccFac	Acceleration factor, Object 0x6097 , =0x6097.1/0x6097.2

4.5.6 Jerk Units

Jerk is not supported by the. Refer to SF command for alternative smoothing of motion via Elmo proprietary object.

4.5.7 Restrictions

- Factors cannot be set while the drive is in OPERATION ENABLED state.
- Divisors cannot be set to 0. An abort message with abort code 0609 0030 is transmitted.
- Values are truncated to the nearest integer.

4.6 Position Motion Function Objects

4.6.1 General

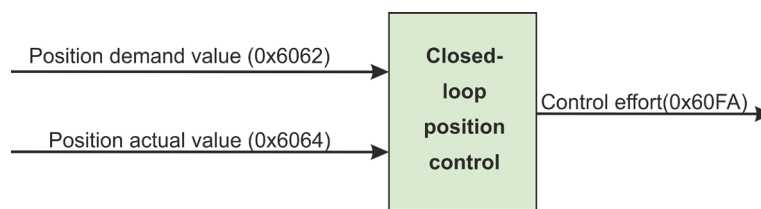
The objects determine the parameters required for closed-loop position control.

The objects are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition
0x6062	Position demand value
0x6063	Position actual internal value
0x6064	Position actual value
0x6065	Following error window
0x6066	Following error time out
0x6067	Position window
0x6068	Position window time out
0x60F4	Following error actual value
0x60FA	Control effort
0x60FC	Position demand internal value
0x60F2	Positioning option code

4.6.2 Functional description

The control loop is fed with the position demand value 0x6062 as one of the outputs of the trajectory generator and with the output of the position detection unit (position actual value, 0x6064) as input parameters. The behavior of the control is influenced by the control parameters. Position control parameters (PI/P) may be set using the EAS during setup. The inputs and output of closed loop position control function is presented in Figure 5.



G-DS402035B

Table 7: Position Control Function

To ensure that the physical limits of a drive are not exceeded, an absolute limit function is implemented for the position control effort. The Elmo drive implements a cascaded control loop in which the position control effort is a velocity demand value for the velocity control loop. For further information about tuning the position loop and using the EAS, refer to the EAS User Manual.

The following terms are used in this section:

Following error	A <i>position actual value</i> outside the allowed range of the <i>following error window</i> around a <i>position demand value</i> for longer than the <i>following error timeout</i> results in setting bit 13, <i>following error</i>, in the <i>Statusword</i> to 1 This is shown in detail in Figure 6. Depending on the supported modes of operation csp, ip, pp, hm and on the capabilities of different categories of drives, only some of the mentioned input parameters may be necessary.
Position reached	This function, shown in Figure 7, provides the option of defining a position range around a <i>position demand value</i> to be regarded as valid. If a drive position is within this area for a specified time — the <i>position window time</i> — the related control bit 10 target reached in the <i>Statusword</i> is set to 1 in operating modes ip, pp.

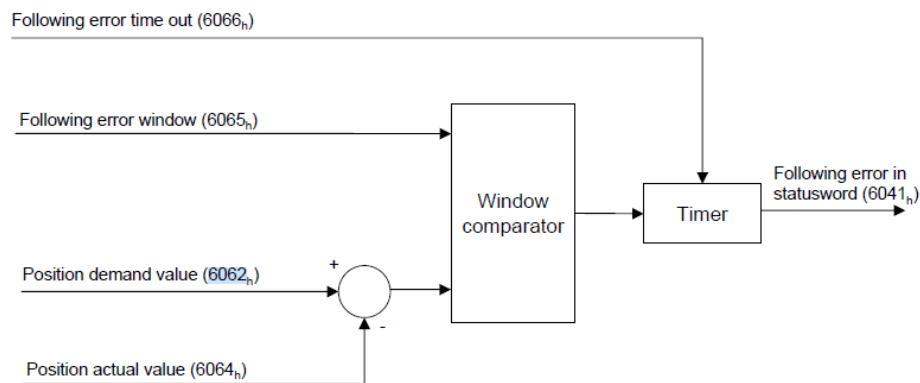
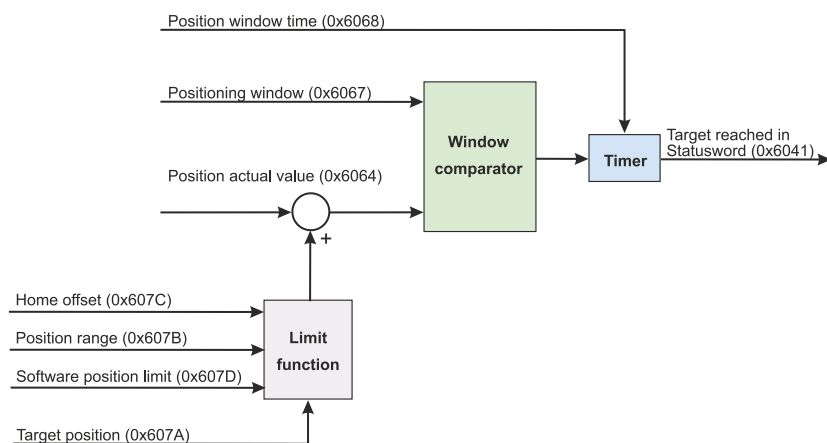


Table 8: Following error



G-DS402036A

Table 9: Position Reached Function

4.7 Profile Position Mode

4.7.1 General

This chapter describes Profile Position mode functionality.

The objects related to Profile Position mode are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition
0x607A	Target position
0x607B	Position range limit
0x607D	Software position limit
0x607E	Polarity
0x607F	Maximum profile velocity
0x6081	Profiled velocity
0x6082	End velocity
0x6083	Profile acceleration
0x6084	Profile deceleration
0x6085	Quick stop deceleration
0x6086	Motion profile type
0x60C5	Maximum acceleration
0x60C6	Maximum deceleration
0x60F2	Positioning Option Code

Following describes how to set a point-to-point (PTP) movement under a profiled position where a *target position* is applied to the trajectory generator. It generates a *position demand value* to the control loop. The trajectory generator input includes profiled velocity, acceleration, deceleration, and selection of motion type, motion polarity and stopping option.

4.7.2 General Structure

The general structure of the Profile Position Mode is presented in Figure 8

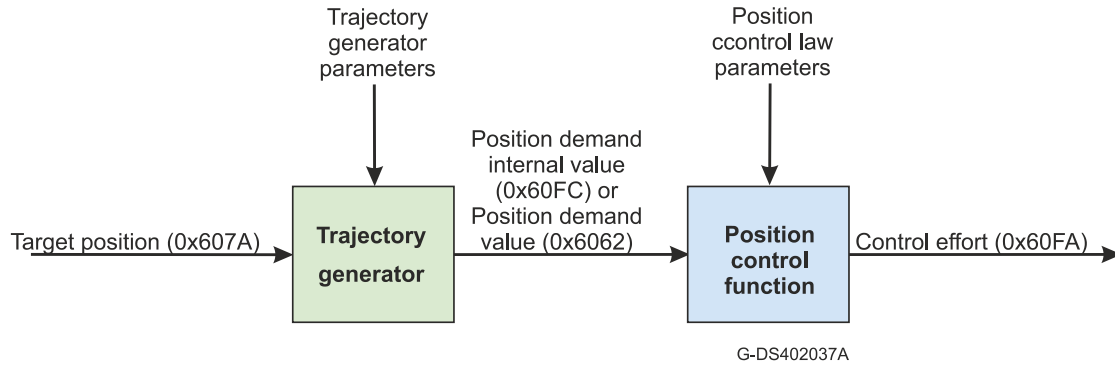


Table 10: Profile Position Mode General Structure

Figure 9 presents a detailed structure of a trajectory generator (profiler).

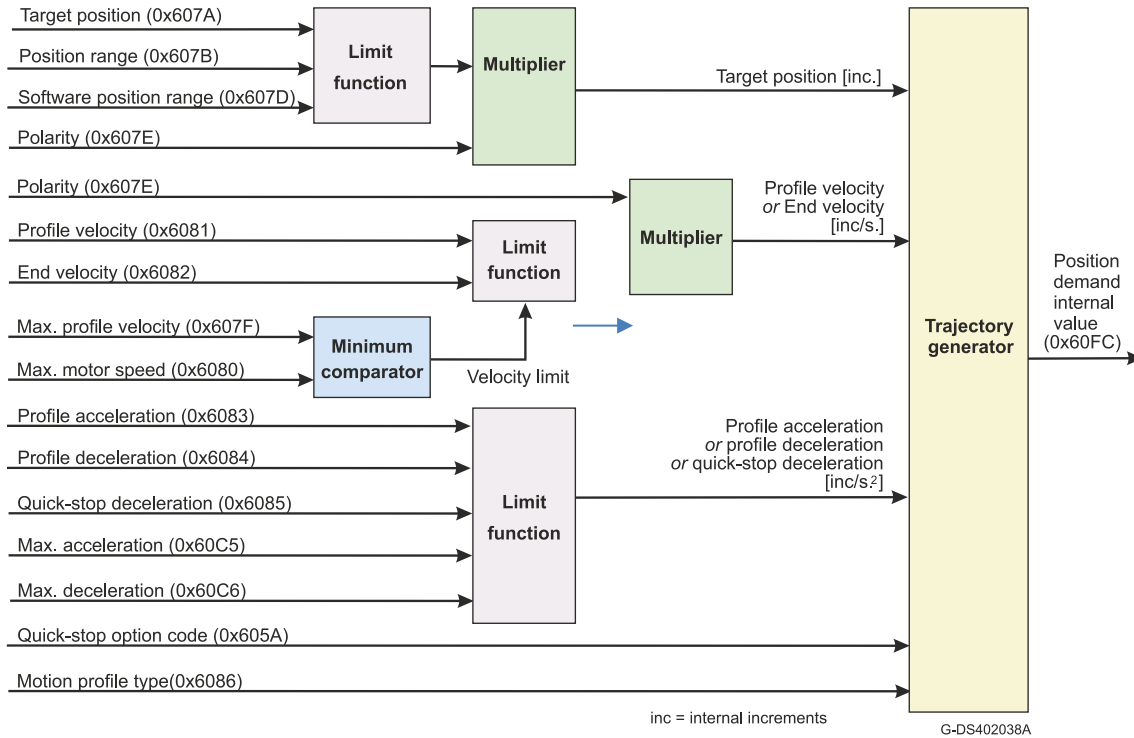


Table 11: Profiler Position Mode, Profiler Structure

4.7.3 Controlword of the Profile Position Mode

Bit	Function
0	Switch on (Note 1)
1	Enable voltage (Note 1)
2	Quick stop (Note 1)
3	Enable operation, set motor on (Note 1)
4	New set-point
5	Change set-point immediately

Bit	Function
6	Absolute\Relative movement 0: Target position is absolute value 1: Target position is relative value depending on object 0x60F2
7	Fault reset (Note 1)
8	Halt: 0: Position function shall be continued or executed 1: The drive halts according to Halt Option Code (0x605D)
9	Change on set-point (Blended)
10 - 15	Reserved

Note 1: see the description in Object 0x6040 *Controlword*

The relations between the motion mode bits of the *Controlword* is described below. Further details refer to functional descriptions.

Motion Type	Bit Name and Values				
	Bit 9. Blended	Bit 8. Halt	Bit 6. Relative	Bit 5. Change set point immediately	Bit 4. New set point
New set point is buffered until previous set point is reached. After which the new set point is executed	0	0	X	0	0->1
Motion stopped according to halt option code (0x605D).	X	1	X	X	X
The new set point is buffered until Halt bit will be reset and previous set point is reached. After which the new set point is executed	0	1	X	0	0->1
The previous set point is aborted. New set point is buffered until Halt bit will be reset. After which the new set point is executed	0	1	X	1	0->1
The new set point is executed immediately. The previous set point is aborted.	X	0	X	1	0->1
Blended motion. The actual performed positioning task is continued without attempting to stop on target position and is blended to the newly commanded task (considering potentially	1	0	X	0	0->1

Motion Type	Bit Name and Values				
	Bit 9. Blended	Bit 8. Halt	Bit 6. Relative	Bit 5. Change set point immediately	Bit 4. New set point
changed profile velocity and accelerations etc.) when target position is touched					
Set point is processed as absolute position	X	0	0	X	0->1
Set-point is processed as relative positioning in accordance with Positioning option code (0x60F2).	X	0	1	X	0->1

- **The Quick stop bit (CW bit 2)**

When Quick stop bit is set to 0 (refer to sub-chapter "Object 0x6040 Controlword"), current Set Point is aborted and the drive performs the action defined by Quick stop option code, object 0x605A.

A new set point is accepted under QUICK STOP ACTIVE state, when changing CW bit 4 from 0 to 1. In that case, when the drive performs transition 16 (QUICK STOP ACTIVE to OPERATION ENABLED), the new set-point will be processed regardless of CW bit 4, i.e. OPERATION ENABLED state can be set by CW=0x2F, CW=0x3F or CW=0x0F.

4.7.4 Statusword of the Profile position mode

Bit	Function
0	Ready to switch on (Note 1)
1	Switched on (Note 1)
2	Operation enabled (Note 1)
3	Fault (Note 1)
4	Voltage enabled (Note 1)
5	Quick stop (Note 1)
6	Switch on disabled (Note 1)
7	Warning (Note 1)
8	Manufacturer specific, reserved, always 0
9	Remote (Note 1)
10	Target reached, see following table
11	Internal limit active (Note 1)
12	Set-point acknowledge, see following table
13	Following error, see following table

Bit	Function
14 - 15	Manufacturer specific, reserved, always 0

Note 1: See the description in Object 0x6041 *Statusword*

Name	Value	Description
SW bit 10, Target reached	0	Halt = 0: <i>Target position</i> not reached. Halt = 1: Axis decelerates.
	1	Halt = 0: <i>Target position</i> reached (Note 1). Halt = 1: Velocity of axis is 0.
SW bit 12, Set-point acknowledge	0	Set-point buffer is available, previous set-point already processed
	1	Previous set-point still processed; set-point buffer is not available.
SW bit 13, Following error	0	No following error
	1	Following error (Note 2)

Note 1: If the actual value of the position encoder is within the Position window during position window time, this target position is regarded as reached. See chapter " Object dictionary" and the description of the objects 0x6066 " Position window ", 0x6067 " Position window time "

Note 2: See chapter " Position Control Function Objects" and the description of the objects 0x6065 "Following error window", 0x6066 "Following error time out"

4.7.5 Functional Description

The set-points setting is controlled by the timing of the new set-point bit and the *change set-point immediately* bit in the *Controlword* as well as the set-point acknowledge bit in the *Statusword*.

Single set-point (<i>change set-point immediately</i> is set to 1)	The drive immediately processes the new received <i>target position</i> that results in a new motion . When reaching the <i>target position</i> of processing set-point, the velocity gets to zero and target reached bit is set to 1. The drive is ready to receive a new set point. For details refer to section "Single Set-Point".
Set of set-points (<i>change set-point immediately</i> is set to 0)	Buffering of set-points is available. The drive has four set-point buffers. There are two options: <i>Controlword</i> bit 9 (blended) is set to 1 - velocity of the drive is not normally reduced to zero after achieving a set-point position. <i>Controlword</i> bit 9 (blended) is set to 0 - velocity of the drive is reduced to zero after achieving a set-point position. In both cases <i>target reached</i> bit is set when the last set point is reached. For details refer to sections "Buffered set-point non-blended" and "Buffered set-point blended"

The new set-point , change set immediately bits in the *Controlword*, and set-point acknowledge bit in *Statusword* define a handshake request-response mechanism.

The sequence of a setting and activating new set-point is:

1. The host sends the trajectory data.
2. The host validate the data and initiates the motion by setting the *new set-point* bit (CW.4).
3. The drive acknowledges reception and buffering of the new data by setting *set-point acknowledge* (SW.12) bit.
4. The motion begins.
5. The host resets the *new set-point* bit (CW.4). If the drive can accept more set points, the *set-point acknowledge* bit resets (SW.12). Note, that this operation can be forced automatically via object 0x60F2.
6. Unless interrupted by a *change set-point immediately* (CW.5) bit, the next trajectory, if acknowledged, is executed. The *target reached* bit in the Statusword (SW.10) will be set only after all set-points were processed.

Figure 10 below represents using the *new set-point* , *change set-point immediately* and *set-point acknowledge* bits for request-response mechanism.

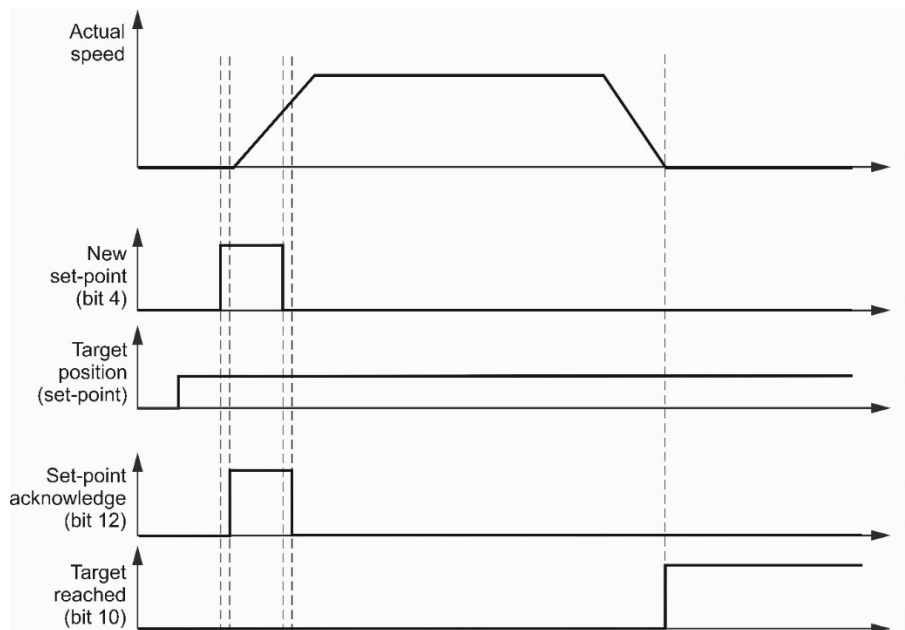


Table 12: Request-response mechanism

The detailed descriptions of the drive's behavior in the cases of:

- Single set-point
- Buffered set-point, non-blended motion
- Buffered set-point, blended motion

are described in the sections 4.7.6 below onwards.

4.7.6 Single Set-Point

The example of two single set-point point (*change set immediately* bit of *Controlword* is 1) is represented in Figure 11. Both set-points are acknowledged immediately when receiving set-point position (bit 12 of *Statusword* reset) and the new motion starts with its set point data. The first set-point is discarded when receiving the new one without setting target reached bit in *Statusword*. This bit is set when the target position of the second set-point is reached.

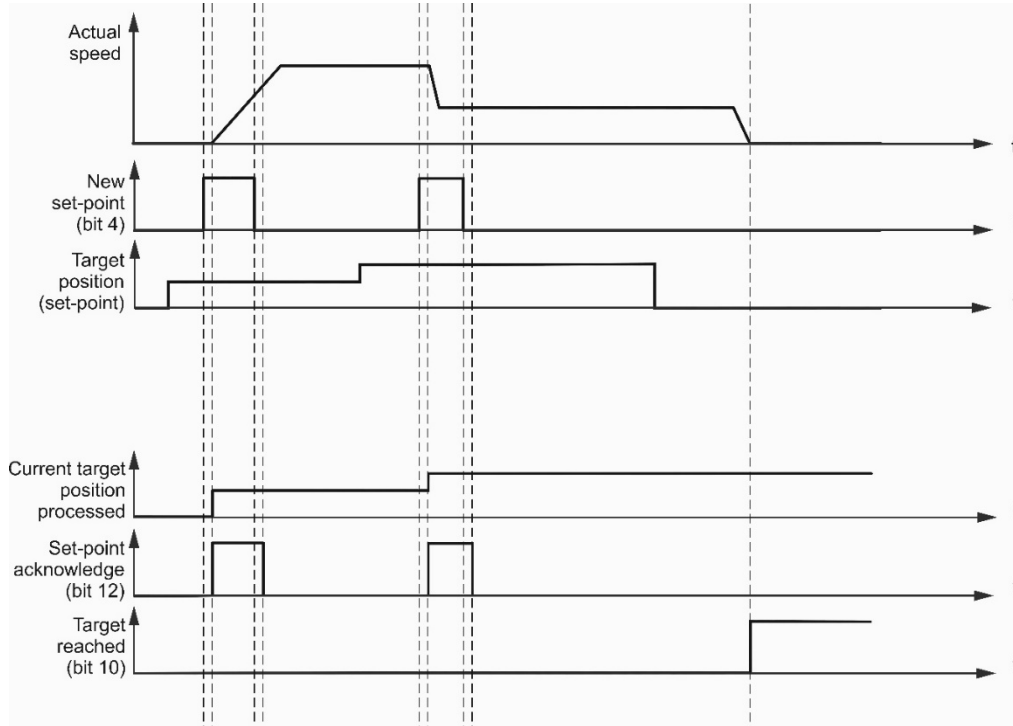


Table 13: Profile position mode of two single set points

The following script simulates two single set points,

Note, the values are in Hexadecimal

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
00	01	00							Start message
601	22	60	60	00	01	00	00	00	Set PP mode
601	22	83	60	00	40	42	0F	00	Set Profile acceleration 1e6
601	22	84	60	00	40	42	0F	00	Set Profile deceleration 1e6
601	22	40	60	00	06	00	00	00	Ready to Switch On
601	22	40	60	00	07	00	00	00	Switch On
301	50	78	00	00	00	00	00	00	Set PX=0
601	22	40	60	00	0F	00	00	00	Enable motor
601	22	7A	60	00	30	75	00	00	1 st set-point, set target position 30000
601	22	81	60	00	50	C3	00	00	1 st set-point, set speed 50000

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
601	22	40	60	00	3F	00	00	00	1 st set-point is absolute, immediate, non-blended motion
601	22	40	60	00	0F	00	00	00	1 st set-point clear
									Delay 200 ms
601	22	7A	60	00	A0	86	01	00	2 nd set-point, set target position 100000
601	22	81	60	00	B8	0B	00	00	2 nd set-point, set speed 3000
601	22	40	60	00	3F	00	00	00	2 nd set-point is absolute, immediate, non-blended motion
601	22	40	60	00	0F	00	00	00	1 st set-point clear

4.7.7 Buffered Set-Point

If the *change set immediately* bit of the *Controlword* is set to 0, the buffering of set-points is available. The drive has four buffers the set-points. When the first set-point is processed, the drive can receive additionally three set-points that will be processed when the preceding was reached.

Note that the target reached bit will be set only after the last set-point was reached. The example of four set-points handling is presented in Figure 12. *Set-point acknowledge* bit is set to 1 when *new set-point* is buffered and it is reset when *new set point* bit is reset and if one buffer is still available. Target reached bit is set when the last set point is reached.

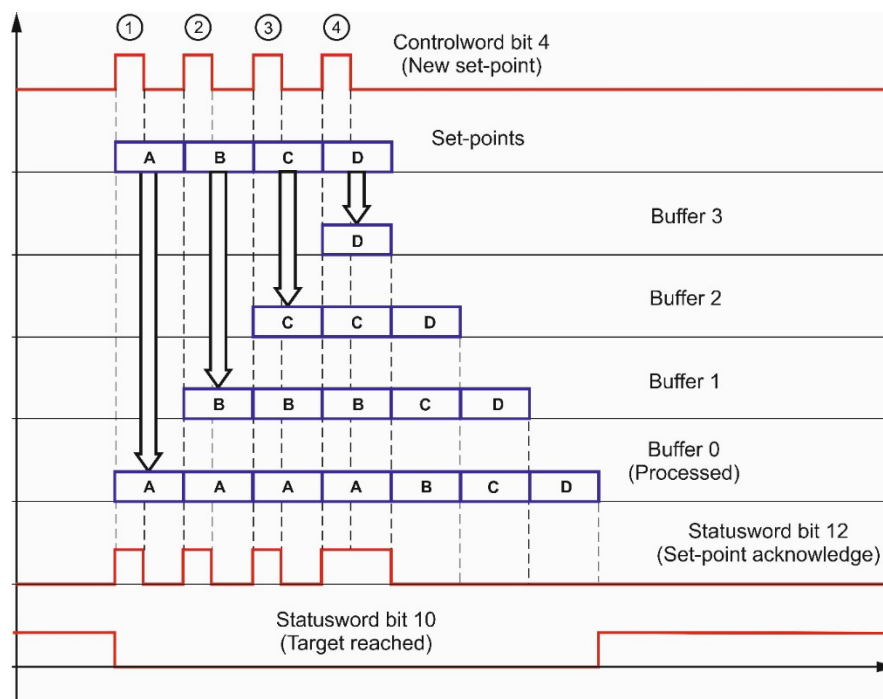


Table 14: Set-point handling for four set-points

4.7.8 Buffered Set-Point, non-blended motion

If the *change set immediately* bit of the *Controlword* is set to 0, one set-point is still in progress and a new one is activated with the blended bit reset, then the next set point will be processed after the previous one was completed.

Note, that in this case the previous motion is not discarded. The drive reaches target position and comes to a complete stop without setting the target reached bit. After which a new motion is started with the next buffered set-point. The target reached bit is set at the end of the new profile. This example is represented in Figure 13.

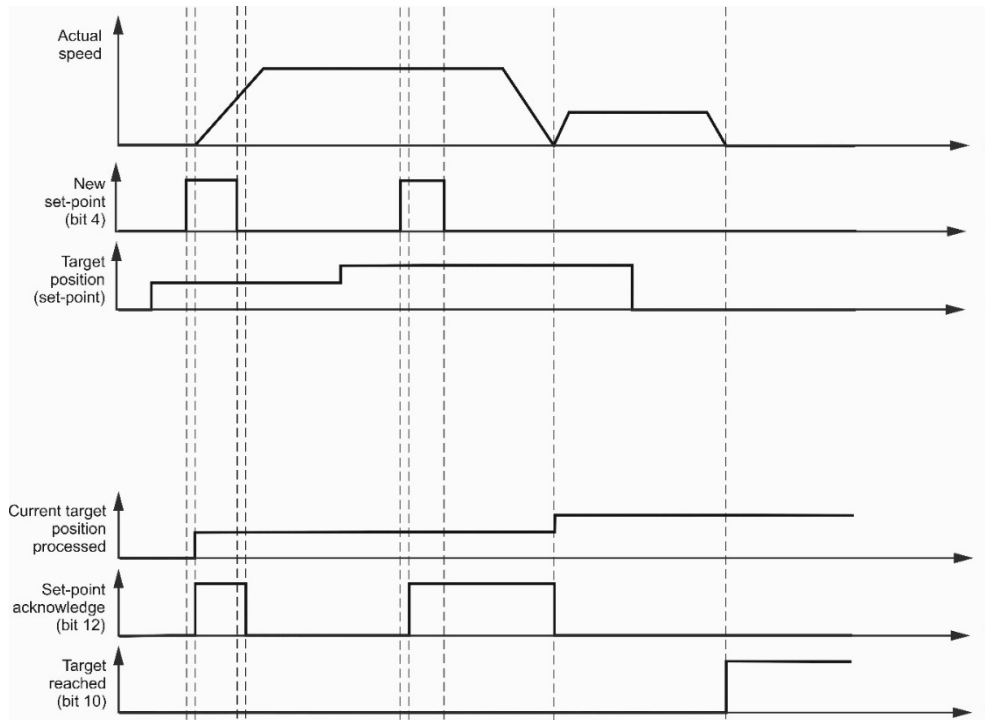


Table 15: Profile Position mode buffered set points, non-blended motion

The following script emulates the use of non-blended set of set points.

Note, the values are in Hexadecimal

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
00	01	00							Start message
601	22	60	60	00	01	00	00	00	Set PP mode
601	22	83	60	00	40	42	0F	00	Set Profile acceleration 1e6
601	22	84	60	00	40	42	0F	00	Set Profile deceleration 1e6
601	22	40	60	00	06	00	00	00	Ready to Switch On
601	22	40	60	00	07	00	00	00	Switch On
301	50	78	00	00	00	00	00	00	Set PX=0
601	22	40	60	00	0F	00	00	00	Enable motor
601	22	7A	60	00	30	75	00	00	1 st set-point, set target position 30000

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
601	22	81	60	00	50	C3	00	00	1 st set-point, set speed 50000
601	22	40	60	00	1F	00	00	00	1 st set-point is absolute, non-immediate, non-blended motion
601	22	40	60	00	0F	00	00	00	1 st set-point clear
									Delay 200 ms
601	22	7A	60	00	A0	86	01	00	2 nd set-point, set target position 100000
601	22	81	60	00	10	27	00	00	2 nd set-point, set speed 10000
601	22	40	60	00	1F	00	00	00	2 nd set-point is absolute, non-immediate, non-blended motion
601	22	40	60	00	0F	00	00	00	1 st set-point clear

The following example shown in Figure 14 is a set of five set-points for non-blended buffered mode where the 5th set-point is ignored by the drive.

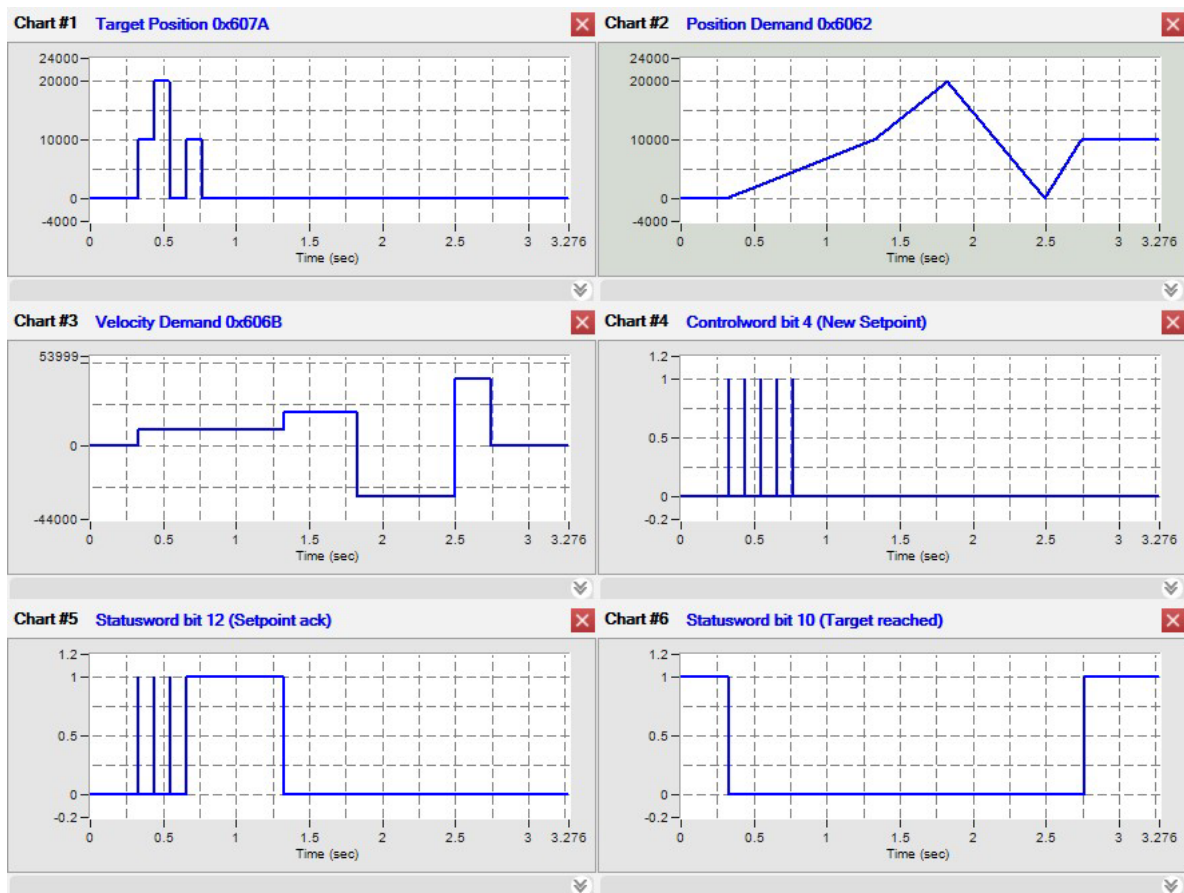


Table 16: Five set-points (the 5th ignored)

Set points 2,3,4 are sent to drive while the 1-st one is processed. After the acknowledgement of the drive, set-points 2,3,4 are waiting in buffer. The 5th is ignored because there is no space in the buffer. At the end of the

profile, and after set-point 4 was processed and reached, the axis gets to position defined by 4th set-point and *target reached* bit is set.

In the example the 4th set-point position is 10,000 while the 5th is 0. The Position Demand shows that when the *target reached* bit was set, the position is 10,000. The position 0, defined by set-point 5 is ignored.

When receiving 1th, 2nd and 3rd set-points, the *Set-point acknowledge* bit is set. In these set-points, when the host resets *new set-point* bit, the *set-point acknowledge* bit is reset to 0. This is done since there is a space for next set-point in the buffer. But when receiving the 4th set-point, *Set-point acknowledge* bit is set and then remains in high state since there is no space in the buffer. The *set-point acknowledge* bit is cleared when the 1st set point process is finished and the place in the set-point buffer is released.

The following is the script used to execute the example above.

Note, the values are in Hexadecimal

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
00	01	00							Start message
601	22	40	60	00	80	00	00	00	Reset any fault
601	22	40	60	00	06	00	00	00	Ready to Switch On
									Delay 2000 ms
601	22	60	60	00	01	00	00	00	Set PP mode
301	50	78	00	00	00	00	00	00	Set PX=0
									Delay 100 ms
601	22	40	60	00	07	00	00	00	Switch On
601	22	40	60	00	0F	00	00	00	Enable motor
									Delay 1000 ms
601	22	7A	60	00	10	27	00	00	1 st set-point, set target position 10000
601	22	81	60	00	10	27	00	00	1 st set-point, set speed 10000
601	22	40	60	00	1F	00	00	00	1 st set-point is absolute, non-immediate
601	22	40	60	00	0F	00	00	00	1 st set-point clear
601	22	7A	60	00	20	4E	00	00	2 nd set-point, set target position 20000
601	22	81	60	00	20	E4	00	00	2 nd set-point, set speed 20000
601	22	40	60	00	1F	00	00	00	2 nd set-point is absolute, non-immediate
601	22	40	60	00	0F	00	00	00	2 nd set-point clear
601	22	7A	60	00	00	00	00	00	3 rd set-point, set target position 0
601	22	81	60	00	30	75	00	00	3 rd set-point, set speed 30000

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
601	22	40	60	00	1F	00	00	00	3 rd set-point is absolute, non-immediate
601	22	40	60	00	0F	00	00	00	3 rd set-point clear
601	22	7A	60	00	10	27	00	00	4 th set-point, set target position 10000
601	22	81	60	00	40	9C	00	00	4 th set-point, set speed 40000
601	22	40	60	00	1F	00	00	00	4 th set-point is absolute, non-immediate
601	22	40	60	00	0F	00	00	00	4 th set-point clear
601	22	7A	60	00	00	00	00	00	5 th set-point, set target position 0
601	22	81	60	00	50	C3	00	00	5 th set-point, set speed 50000
601	22	40	60	00	1F	00	00	00	5 th set-point is absolute, non-immediate
601	22	40	60	00	0F	00	00	00	5 th set-point clear
									Delay 100 ms

4.7.9 Buffered Set-Point, Blended

If the *change set-point immediately* bit of the *Controlword* is set to 0, one set-point is still in progress and a new one is activated with *blended* bit set, the previous motion is not discarded and must be completed.

The drive reaches target position with defined profile speed without decreasing it to zero and without setting the target reached bit. After that, a new motion is started with its target position and speed. The *target reached* bit is set on the end of the new profile. The example is represented in Figure 15.

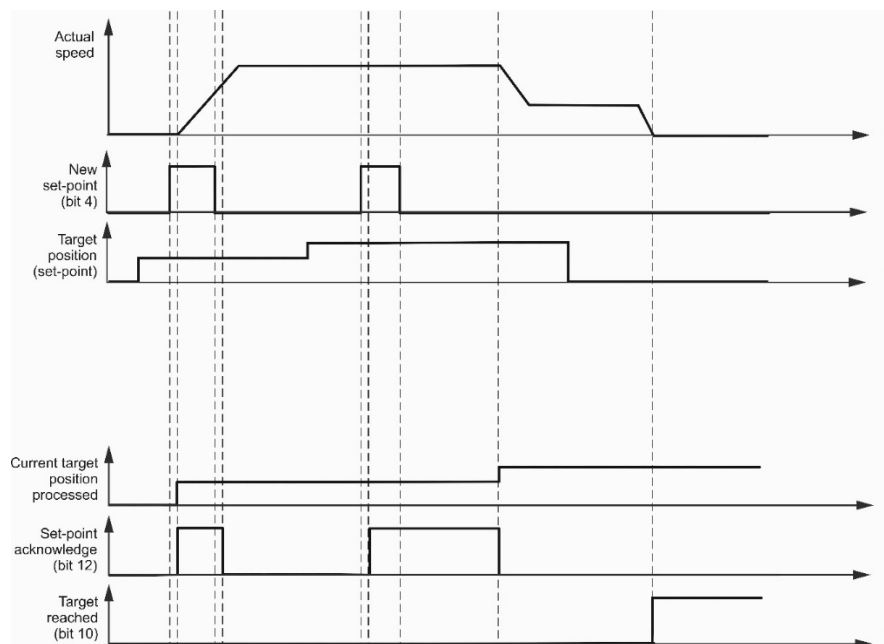


Table 17: Profile Position mode, buffered set points, blended motion

Following is the script that is used in the CAN Analyzer to execute the example described motion.

Note, the values are in Hexadecimal

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Description
00	01	00							Start message
601	22	60	60	00	01	00	00	00	Set PP mode
601	22	83	60	00	40	42	0F	00	Set Profile acceleration 1e6
601	22	84	60	00	40	42	0F	00	Set Profile deceleration 1e6
601	22	40	60	00	06	00	00	00	Ready to Switch On
601	22	40	60	00	07	00	00	00	Switch On
301	50	78	00	00	00	00	00	00	Set PX=0
601	22	40	60	00	0F	00	00	00	Enable motor
601	22	7A	60	00	30	75	00	00	1 st set-point, set target position 30000
601	22	81	60	00	50	C3	00	00	1 st set-point, set speed 50000
601	22	40	60	00	1F	02	00	00	1 st set-point is absolute, non-immediate, blended motion
601	22	40	60	00	0F	00	00	00	1 st set-point clear
									Delay 200 ms
601	22	7A	60	00	A0	86	01	00	2 nd set-point, set target position 100000
601	22	81	60	00	20	4E	00	00	2 nd set-point, set speed 20000
601	22	40	60	00	1F	02	00	00	2 nd set-point is absolute, non-immediate, blended motion
601	22	40	60	00	0F	00	00	00	1 st set-point clear

4.8 Interpolated Position Mode

Interpolated position mode is not supported by Platinum drive

4.9 Cyclic Synchronous Position Mode

4.9.1 General

This section describes Cyclic Synchronous Position mode functionality.

The objects related to the mode are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition
0x60B0	Position Offset
0x60B1	Velocity Offset
0x60B2	Torque Offset
0x607A	Target position
0x6062	Position demand value
0x60F4	Following error actual value
0x6064	Position actual value
0x606C	Velocity actual value
0x6077	Torque actual value
0x3675	Extrapolation timeout

This mode is only available for ECAT. The overall structure for this mode is shown in Figure 16. In this mode, the trajectory generator is located in the control device, not in the ELMO drive, providing a target position to the drive, which performs position control, velocity control and torque control. Optionally, additive velocity and torque values can be provided by the control system in order to allow for velocity and/or torque feedforward.

Measured by sensors, the ELMO drive may provide actual values for position, velocity and torque to the control device. The behavior of the control function is influenced by control parameters like limit functions, which are externally applicable.

4.9.2 Functional Description

Figure 16 shows the inputs and outputs of the drive control function. The input values (from the control function point of view) are the *target position* and optionally a *position offset* (to be added to the target position to allow two instances to set up the position) as well as an optional *velocity offset* and an optional *torque offset* used for feedforward control. Especially in cascaded control structures, where a position control is followed by a velocity or torque control, the output of the position control loop is used as an input for a further calculation in the drive device. Limit functions may be used to restrict the range of values to avoid unintended positions.

The drive device monitors the following error. Other features specified in this mode are limitation of motor speed and a quick stop function for emergency reasons. The torque may be limited as well.

The interpolation time period defines the time period between two updates of the target position and/or additive position and is used for inter-cycle interpolation.

The target position is interpreted as absolute value. The position actual value is used as mandatory output to the control device. Further outputs may be the velocity actual value, torque actual value and the velocity sensor actual value.

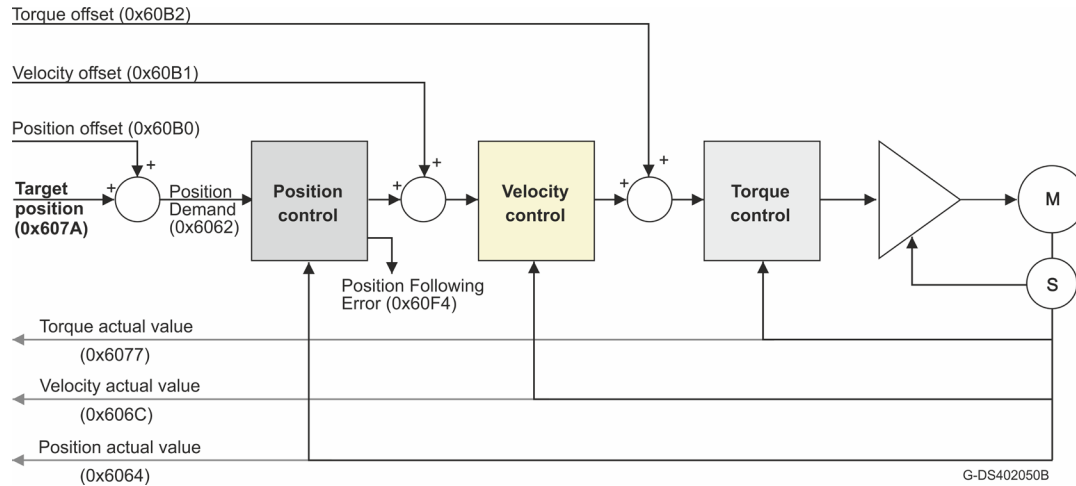


Table 18: Cyclic synchronous position control function

If necessary, all values can be transformed from user-defined units to internal units such as increments with factors. A target position value or position offset outside the allowed range of the following error window around a position demand value for longer than the following error time out results in setting bit 13 (following error) in the *Statusword* to 1.

4.9.3 Controlword of Cyclic Synchronous Position Mode

Bit	Function
0	Switch on (Note 1)
1	Enable voltage (Note 1)
2	Quick stop (Note 1)
3	Enable operation (set motor on) (Note 1)
4 - 6	Reserved, 0
7	Fault reset (Note 1)
8	Halt: 0: Position function shall be continued or executed 1: The drive stops the interpolation and stops the motor according to <i>halt options code</i> , see the description in object 0x605D.
9 - 15	Reserved, 0

Note 1: see the description in object 0x6040 *Controlword*

If the motor is stopped due to an internal fault or *Controlword* command, the interpolation is disabled. Interpolation can be enabled again only after the device enters the OPERATION_ENABLE mode.

If no target 0x607A is received within the extrapolation timeout (Object 0x3675), the motor stops according to *quick stop options code*, see the description in object 0x605A.

4.9.4 Statusword of Cyclic Synchronous Position Mode

Bit	Function
0	Ready to switch on (Note 1)
1	Switched on (Note 1)
2	Operation enabled (Note 1)
3	Fault (Note 1)
4	Voltage enabled (Note 1)
5	Quick stop (Note 1)
6	Switch on disabled (Note 1)
7	Warning (Note 1)
8	Manufacturer specific, reserved, always 0
9	Remote (Note 1)
10	Reserved, always 0
11	Internal limit active (Note 1)
12	Drive follows the command value, see table below
13	Following error, see table below
14 - 15	Manufacturer specific, reserved, always 0

Note 1: see the description in object 0x6041 *Statusword*

Usage of bits 12 and 13 is demonstrated in the Table 3:

Bit	Value	Definition
12	0	Drive does not follow the command value – Target position is ignored
	1	Drive follows the command value – Target position used as input to position control loop
13	0	No following error
	1	Following error. See chapter " Position Control Function Objects" and the description of the objects 0x6065 "Following error window", 0x6066 "Following error time out"

Table 19: Cyclic Synchronous Position Mode *Statusword*, bits 12,13

4.10 Cyclic Synchronous Velocity Mode

4.10.1 General

This chapter describes Cyclic Synchronous Velocity mode functionality.

The objects related to the mode are presented in the following table. Refer to chapter "The Object Dictionary" for detailed descriptions of the objects.

Object	Definition
0x60B1	Velocity Offset
0x60B2	Torque Offset
0x60FF	Target velocity
0x6064	Position actual value
0x606C	Velocity actual value
0x6077	Torque actual value

This mode is only available for ECAT. The overall structure for this mode is shown in Figure 17. In this mode, the trajectory generator is located in the control device, not in the ELMO drive, providing a target velocity to the ELMO drive, which performs velocity control and torque control. If desired, the position control loop may be closed over the communication system. Optionally, additive velocity and torque values may be provided by the control system in order to allow a second source for velocity and/or a torque feed forward. Measured by sensors, the ELMO drive may provide actual values for position, velocity and torque to the control device.

The cyclic synchronous velocity mode covers the following sub-functions:

- Demand value input
- Velocity capture using position sensor or velocity sensor
- Velocity control function with appropriate input and output signals
- Limitation of torque demand

Various sensors may be used for velocity capture. In particular, the aim is that costs are reduced and the drive power system is simplified by evaluating position and velocity using a common sensor. This option is achieved by using a resolver or an encoder. The behavior of the control function is influenced by control parameters such as limit functions that are externally applicable.

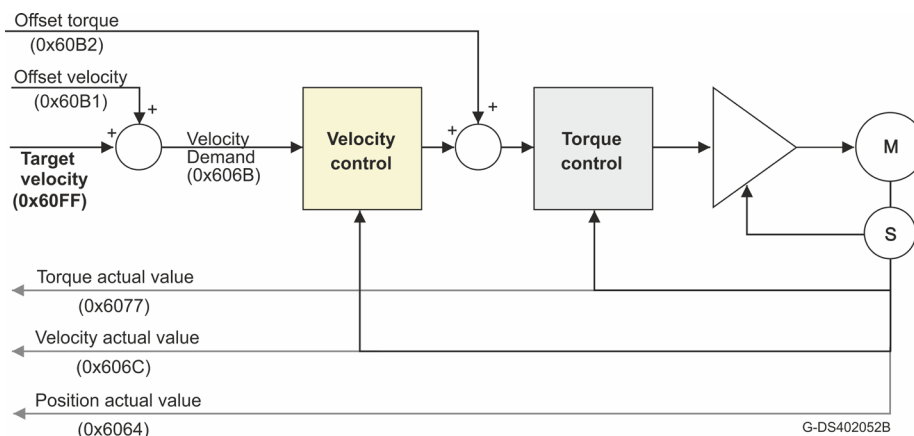


Table 20: Cyclic synchronous velocity mode

4.10.2 Functional Description

Figure 18 demonstrates the inputs and outputs of the drive control function. The input (from the control device point of view) is the target velocity and optionally, a velocity offset (to be added to the target velocity to allow two instances to set up the velocity) as well as a torque offset. Especially in cascaded control structures, where a velocity control is followed by a torque control, the output of the velocity control loop is used as an input for a further calculation in the ELMO drive device.

The drive device may support limitation of motor speed and a quick stop function for emergency reasons. The torque may be limited as well.

The interpolation time period defines the time period between two updates of the target velocity and/or additive velocity and is used for inter-cycle interpolation.

The velocity actual value is used as mandatory output to the control device. Further outputs may be the torque actual value and the velocity sensor actual value.

All values are transformed, if necessary, from user-defined units to internal units such as increments with the functions described in the **Factors** Chapter.

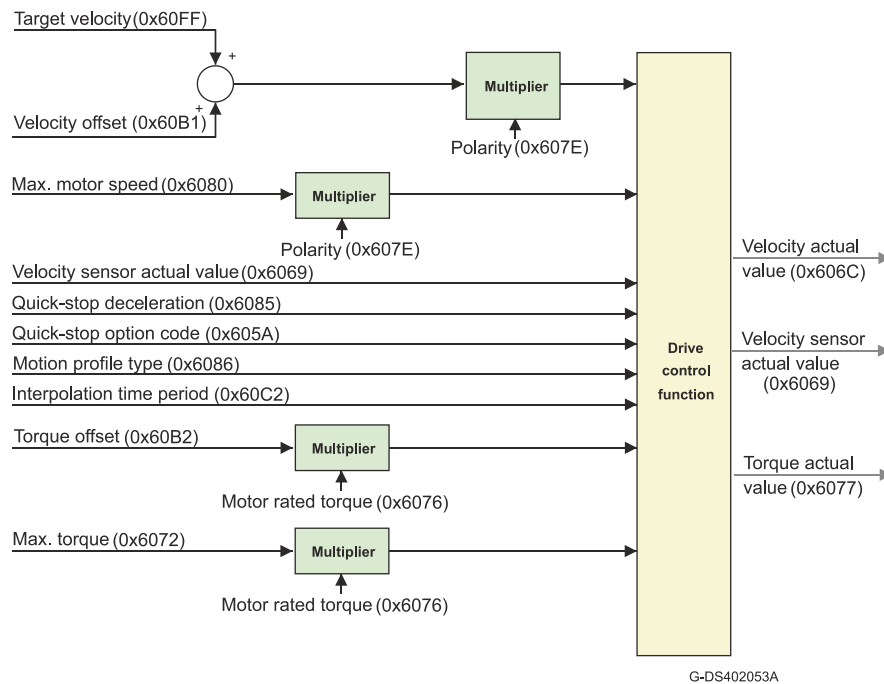


Table 21: Cyclic synchronous velocity control function

4.10.3 Controlword of Cyclic Synchronous Velocity Mode

Bit	Function
0	Switch on (Note 1)
1	Enable voltage (Note 1)
2	Quick stop (Note 1)
3	Enable operation (set motor on) (Note 1)
4 - 6	Reserved, 0
7	Fault reset (Note 1)

Bit	Function
8	Halt
9 - 15	Reserved, 0

Note 1 : see the description in object 0x6040 *Controlword*

In case of a Halt, the drive stops the interpolation and stops the motor according to *halt options code*, see the description in object 0x605D.

In case the motor is stopped due to an internal fault or *Controlword* command, the interpolation is disabled. Interpolation can be enabled again only after the device enters the OPERATION_ENABLE.

If the motor is stopped due to an internal fault or *Controlword* command, the interpolation is disabled. Interpolation can be enabled again only after the device enters the OPERATION_ENABLE mode.

If no target 0x60FF is received within the extrapolation timeout (Object 0x3675), the motor stops according to *quick stop options code*, see the description in object 0x605A.

4.10.4 Statusword of Cyclic Synchronous Velocity Mode

Bit	Function
0	Ready to switch on (Note 1)
1	Switched on (Note 1)
2	Operation enabled (Note 1)
3	Fault (Note 1)
4	Voltage enabled (Note 1)
5	Quick stop (Note 1)
6	Switch on disabled (Note 1)
7	Warning (Note 1)
8	Manufacturer specific, reserved, always 0
9	Remote (Note 1)
10	Reserved, set to 0
11	Internal limit active (Note 1)
12	Drive follows the command value (see table below)
13	Reserved, set to 0
14 - 15	Manufacturer specific, reserved, always 0

Note 1: see the description in object 0x6041 *Statusword*

Table 4 demonstrates the usage of Bit 12:

Bit	Value	Definition
	0	Drive does not follow the command value – Target velocity ignored

Bit	Value	Definition
12	1	Drive follows the command value – Target velocity used as input to velocity control loop

Table 22: Cyclic Synchronous Velocity Mode. *Statusword*, using bit 12

4.11 Cyclic Synchronous Torque Mode

4.11.1 General

This mode is only available for ECAT. Figure 19 demonstrates the overall structure for this mode. With this mode, the trajectory generator is located in the control device, not in the drive device. In cyclic synchronous mode, it provides a target torque to the ELMO drive device, which performs torque control. Optionally, additive torque value may be provided by the control system in order to allow a second source to set up the torque. Measured by sensors, the ELMO drive device may provide actual values for position, velocity and torque to the control device.

The cyclic synchronous torque mode covers the following sub-functions:

- Demand value input
- Torque capture.
- Torque control function with appropriate input and output signals
- Limitation of torque demand

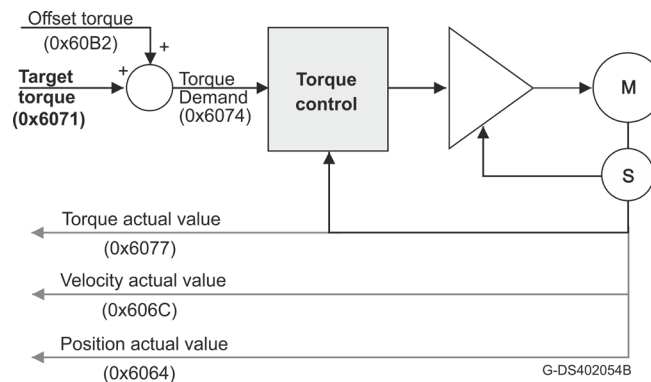


Table 23: Cyclic synchronous torque mode

4.11.2 Functional Description

Figure 20 demonstrates the inputs and outputs of the torque control function. The input (from the control device point of view) is the target torque and optionally, a torque offset (to be added to the target torque to allow two instances to set up the torque) as well as a torque offset.

The drive device supports limitation of motor speed and torque. The interpolation time period defines the time period between two updates of the target

Torque. The outputs may be the torque actual value and the velocity sensor actual value. All values are transformed, if necessary, from user-defined units to internal units with the functions described in the **Factors** Chapter.

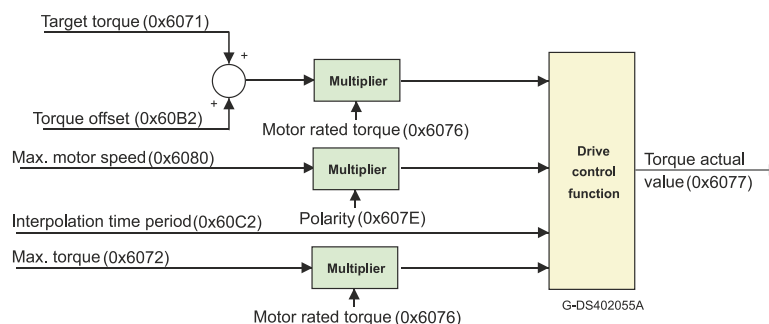


Table 24: Cyclic synchronous torque control function

4.11.3 Controlword of Cyclic Synchronous Torque Mode

Bit	Function
0	Switch on (Note 1)
1	Enable voltage (Note 1)
2	Quick stop (Note 1)
3	Enable operation (set motor on) (Note 1)
4 - 6	Reserved, 0
7	Fault reset (Note 1)
8	Halt
9 - 15	Reserved, 0

Note 1: see the description in object 0x6040 *Controlword*

In case of a Halt, the drive stops the interpolation and stops the motor according to *halt options code*, see the description in object 0x605D.

If the motor is stopped due to an internal fault or *Controlword* command, the interpolation is disabled. Interpolation can be enabled again only after the device enters the OPERATION_ENABLE mode.

If no target 0x6071 is received within the extrapolation timeout (Object 0x3675), the motor stops according to *quick stop options code*, see the description in object 0x605A.

4.11.4 Statusword of Cyclic Synchronous Torque Mode

Bit	Function
0	Ready to switch on (Note 1)
1	Switched on (Note 1)
2	Operation enabled (Note 1)
3	Fault (Note 1)
4	Voltage enabled (Note 1)
5	Quick stop (Note 1)
6	Switch on disabled (Note 1)
7	Warning (Note 1)
8	Manufacturer specific, reserved, always 0
9	Remote (Note 1)
10	Reserved, always 0
11	Internal limit active (Note 1)
12	Drive follows the command value, see table below
13 - 15	Manufacturer specific, reserved, always 0

Note 1: see the description in object 0x6041 *Statusword*

The usage of bit 12 is presented in Table 5.

Bit	Value	Definition
12	0	Drive does not follow the command value – Target torque ignored
	1	Drive follows the command value – Target torque used as input to torque control loop

Table 25: Cyclic Synchronous Torque Mode. *Statusword*, using bit 12

4.12 Profile Velocity

4.12.1 General

Object	Definition
0x6069	Velocity sensor actual value
0x 6060	Modes of operation
0x 606A	Sensor selection code
0x 606B	Velocity demand value
0x 606C	Velocity actual value
0x 606D	Velocity window
0x 606E	Velocity window time
0x 606F	Velocity threshold
0x 6070	Velocity threshold time
0x 60FF	Target velocity

Profile Velocity mode includes the following sub-functions:

- Demand value input via trajectory generator
- Velocity capture using the position sensor or velocity sensor
- Velocity control function with the appropriate input and output signals
- Monitoring of the *profile velocity* using a window function
- Monitoring of the *velocity actual value* using a threshold

The input parameters of the reference value generator are:

- Profile velocity
- Profile acceleration
- Profile deceleration
- Emergency stop
- Motion profile type

The velocity controller calculates a torque variable. When a different *target position* arrives, it is executed immediately.

A target velocity can be executed only in OPERATION ENABLED state. Otherwise, the process aborts with an emergency message.

The velocity, acceleration and deceleration are submitted to the limits according to the relevant limit range.

4.12.2 Functional Description

Figure 21 demonstrates the defined structure of the profile velocity mode. The actual velocity may be obtained through differentiation from the position encoder and is represented in position encoder increments.

The target reached bit -bit 10- is set to 1 in the *Statusword* when the difference between the target velocity and the velocity actual value is within the velocity window longer than the velocity window time.

As soon as the velocity actual value exceeds the velocity threshold longer than the velocity threshold time, then bit 12 in *Statusword* is set to 0. Below this threshold the bit is set to 1 and indicates that the axis is stationary.

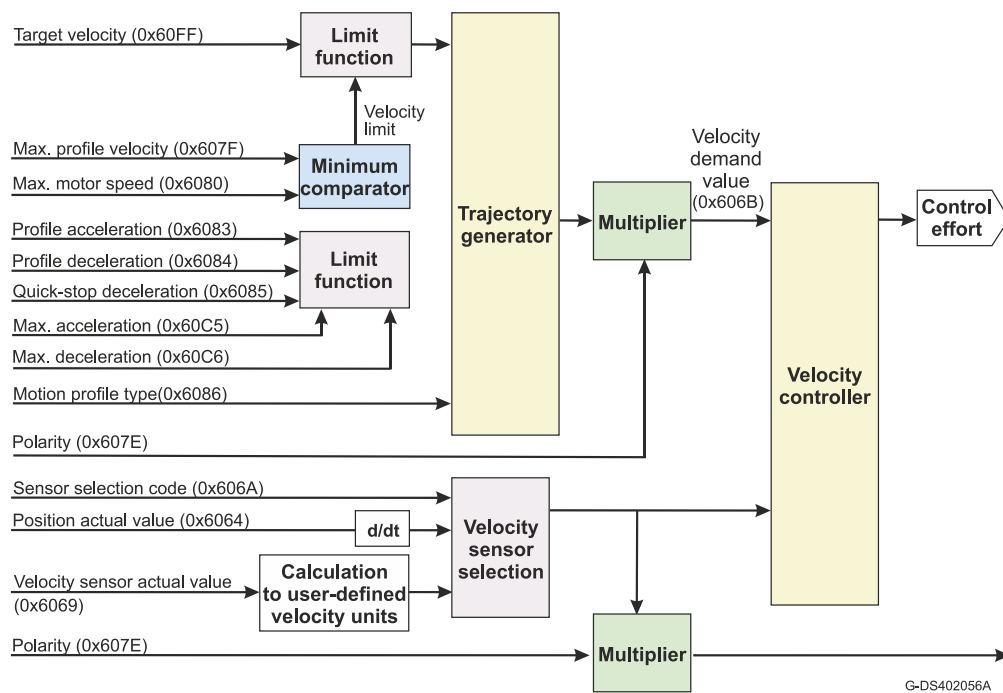


Table 26: Profile velocity mode

4.12.3 Controlword of the Profile velocity mode

Bit	Function
0	Switch on (Note 1)
1	Enable voltage (Note 1)
2	Quick stop (Note 1)
3	Enable operation, set motor on, (Note 1)
4 - 6	Reserved, always 0
7	Fault reset (Note 1)
8	Halt
9 - 15	Reserved, always 0

Note 1: see the description in object 0x6040 *Controlword*

Name	Value	Description
Halt	0	Execute the motion
	1	Stop axle

4.12.4 Statusword of the Profile velocity mode

Bit	Function
0	Ready to switch on (Note 1)
1	Switched on (Note 1)
2	Operation enabled (Note 1)
3	Fault (Note 1)
4	Voltage enabled (Note 1)
5	Quick stop (Note 1)
6	Switch on disabled (Note 1)
7	Warning (Note 1)
8	Manufacturer specific, reserved, always set to 0
9	Remote (Note 1)
10	Target reached
11	Internal limit active (Note 1)
12	Speed is zero
13	Reserved, always set to 0
14...15	Manufacturer specific, reserved, always set to 0

Note 1: see the description in object 0x6041 *Statusword*

Name	Value	Description
Target reached Bit 10	0	Halt = 0: <i>Target velocity</i> not (yet) reached. Halt = 1: Axle decelerates
	1	Halt = 0: <i>Target velocity</i> reached (Note 1) Halt = 1: Velocity of axle is 0
Speed is zero, bit 12	0	Speed not equal to 0
	1	Speed equals 0

Note 1: See chapter " Object dictionary" and the description of the objects 0x606D " Velocity window ", 0x606E " Velocity window time".

4.13 Profile Torque Mode

4.13.1 General

Object	Definition
0x6071	Target torque
0x6072	Max torque
0x6073	Max current
0x6074	Torque demand value
0x6075	Motor rated current
0x6076	Motor rated torque
0x6077	Torque actual value
0x6078	Current actual value
0x6087	Torque slope

This chapter describes the profile torque mode. The profile torque mode allows a host (external) control system (i.e. closed-loop speed controller, open-loop transmission force controller) to transmit the target torque value, which is processed via the trajectory generator. In profile torque mode, torque slope parameters are required.

All definitions within this document refer to rotating motors. Linear motors require that all *torque* objects refer to a *force* instead. For the sake of simplicity, the objects are not duplicated and their names should not be modified. As an example, the linear motor target force must be transmitted using the target torque object. Refer to the object descriptions for additional information.

The torque control parameters, power stage parameters and motor parameters are defined as objects so that they can be handled (i.e. downloaded) in a standard way.

The torque demand, torque actual value, current actual value may be available to the user as parameters, if they are monitored.

4.13.2 Functional Description

Figure 22 demonstrates the defined structure of the profile torque mode. The actual velocity may be obtained through differentiation from the position encoder and is represented in position encoder increments.

If the host control system switches the *Controlword* bit 8 (halt) from 0 to 1, then the trajectory generator ramps its control output down to zero. However, if the host control system switches the *Controlword* bit 8 (halt) from 1 to 0, then the trajectory generator ramps its control output up to the target torque. In both cases the trajectory generator takes the Torque slope (Object 0x6087) into consideration.

The target reached bit -bit 10- is set to 1 in the *Statusword* when the difference between the target torque and the torque actual value is within the Torque window (object 0x3621) longer than the Torque window time (object 0x3622).

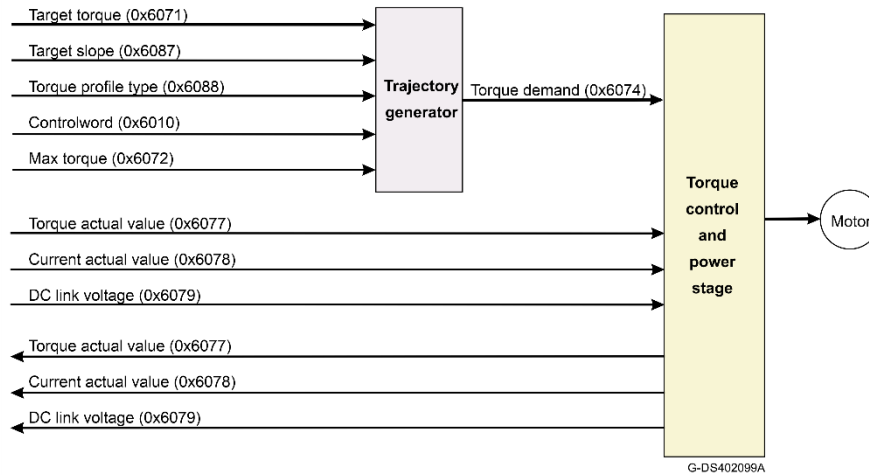


Table 27: Profile torque mode

Elmo's Platinum drives support Profile Torque mode when selected. When Profile Torque objects are set, some internal commands are affected. These internal commands remain even after another operating mode is chosen. The following list shows the objects that are affected, additional information about these commands is available in the Platinum Command Reference manual:

Objects	Command Reference commands
Max Torque [0x6072]	Does not change PL[1]
Max Current [0x6073]	Does not change PL[1]
Torque Demand Value [0x6074]	Same as DV[1]
Motor Rated Current [0x6075]	Change in command CL[1] is reflected in object 0x6075 and object 0x6076, but the reverse does not hold
Motor Rated torque [0x6076]	Change in command CL[1] is reflected in object 0x6075 and object 0x6076, but the reverse does not hold
Torque Actual Value [0x6077]	Same as IQ
Current Actual Value [0x6078]	Same as IQ

The following should also be noted:

Because DS402 defines all relevant torque and current as relative to rate values, motor current and motor torque are, herein, considered to be the same.

The following objects imitate each other and the last value entered is valid:

- 0x6072, 0x6073 for reference
- 0x6077, 0x6078 for feedback

4.13.3 Controlword of the Profile Torque Mode

Bit	Function
0	Switch on (Note 1)

Bit	Function
1	Enable voltage (Note 1)
2	Quick stop (Note 1)
3	Enable operation, set motor on (Note 1)
4 - 6	Reserved, 0
7	Fault reset (Note 1)
8	Halt
9 - 15	Reserved, 0

Note 1: see the description in **Object 0x6040 Controlword**

Name	Value	Description
Halt	0	Execute the motion
	1	Stop axle

4.13.4 Statusword of the Profiled Torque Mode

Bit	Function
0	Ready to switch on (Note 1)
1	Switched on (Note 1)
2	Operation enabled (Note 1)
3	Fault (Note 1)
4	Voltage enabled (Note 1)
5	Quick stop (Note 1)
6	Switch on disabled (Note 1)
7	Warning (Note 1)
8	Manufacturer specific, reserved, always 0
9	Remote (Note 1)
10	Target Reached
11	Internal limit active (Note 1)
12 -13	Reserved, 0
14 - 15	Manufacturer specific, reserved, always 0

Note 1: see the description in **Object 0x6041 Statusword**

Name	Value	Description
Target reached	0	Target torque not (yet) reached.
	1	Target torque reached.

4.14 Profile Torque Mode with Commutation Angle

TBD

Chapter 5: The Object Dictionary

The object dictionary is essentially a grouping of objects that are accessible via receive and transmit SDOs. Part of the object can be mapped to transmit and receive PDOs (tPDO and rPDO, respectively) in a predefined manner.

The following layout (Table 6: Object Dictionary Layout

) is used with the objects in the object dictionary:

Index (Hex)	Object
0x0000	Padding object
0x1000 - 0x13FF	Device specific and communication objects. 1003, 1006, 100b, 1016, 1017, reserved in ECAT
0x1400 - 0x14FF	rPDO communication parameter per-axis (some of the entries are not relevant for ECAT)
0x1500 - 0x15FF	reserved
0x1600 - 0x1603 (axis 1) 0x1610 – 0x1613 (axis 2) 0x1620 – 0x1623 (axis 3) 0x1630 – 0x1633 (axis 4) 0x1640 – 0x1643 (axis 5) 0x1650 - 0x1653 (axis 6)	rPDO for modular device. 4 per axis (up to 6 axes). 1600-1603 used in CAN for compatibility.
0x1700	Safety rPDO. Device level only, FSoE Control container
0x1701 - 0x1704	rPDO for general purpose
0x1800 - 0x18FF	tPDO communication parameters per-axis
0x1A00 - 0x1A03 (axis 1) 0x1A10 - 0x1A13 (axis 2) 0x1A20 - 0x1A23 (axis 3) 0x1A30 - 0x1A33 (axis 4) 0x1A40 - 0x1A43 (axis 5) 0x1A50 - 0x1A53 (axis 6)	tPDO for modular device. 4 per axis (up to 6 axes: 0x10 offset). 1A00-1A03 used in CAN for compatibility.
0x1B00	Safety tPDO. Device level only, FSoE Status container
0x1B01 - 0x1B04	tPDO for general purpose
0x1B10	EtherCAT multi sample recorder container object
0x1C00 - 0x1C4F	ECAT communication and sync manager objects
0x1D00 - 0x1DFF	Reserved

Index (Hex)	Object
0x1E00 - 0x1E10	Reserved for SRA structure
0x1F00 - 0x1FFF	MDP objects
0x2000 - 0x2DFF	Manufacturer general object – device
0x2E00 - 0x2EFF	Manufacturer safety related objects
0x2F00 - 0x2FFF	Manufacture interface objects - device
0x3000 - 0x35FF	Elmo commands alias objects: axis related
0x3600 - 0x376F	Elmo manufacturer specific objects: axis related
0x3770 - 0x37FF	Elmo safety specific objects
0x3800 - 0x5FFF	same as 0x3000 - 0x37FF per axis with offset 0x800: 6 axes total
0x6000 - 0x65FF	DS402 objects axis related, offset 0x800
0x6600 - 0x67FF	FSOE: Safety related object
0x6800 - 0x9FFF	Same as 0x6000 to 0x67FF for other axes offset 0x800: 6 axes
0xE000 - 0xEFFF	MDP safety objects not used for servo applications
0xF000 - 0xFEFF	MDP general connection object (ETG5003)

Table 28: Object Dictionary Layout

5.1 Objects Properties

The following lists the object in ascending order

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
1000/0	Device type	UINT32	RO	No	
1001/0	Error Register	UINT8	RO	No	
1002/0	Manufacturer Status Register	UINT32	RO	Tx	CAN only
1003/16	Pre-Defined Error Field	UINT32	RO	No	CAN only
1006/0	Communication Cycle Period	UINT32	RW	No	CAN only
1008/0	Manufacturer device name	VISIBLE STRING	CONST	No	
1009/0	Manufacturer hardware version	VISIBLE STRING	CONST	No	
100A/0	Manufacturer software version	VISIBLE STRING	CONST	No	
100B/0	Manufacturer software version	VISIBLE STRING	CONST	No	
1010/1	Store parameters	UINT32	RW	No	
1011/1	Restore default parameters	UINT32	RW	No	
1013/0	High resolution time stamp	UINT32	RW	Rx	CAN only
1016/2	Consumer heartbeat time	UINT32	RW	No	CAN only
1017/0	Producer heartbeat time	UINT16	RW	No	CAN only
1018/4	Identity object	RECORD	RO	No	
1023/3	OS command	RECORD	Depending on sub-index	No	
1024/0	OS command mode	UINT8	WO	No	
1029/1	Error behavior	UINT8	RW	No	CAN only
10F1/2	Error Settings	Depending on sub-index	RW	No	ECAT only
1400/2 - 1403/2	rPDO communication parameter	Depending on sub-index	RW	No	CAN only

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
1600/8 – 1603/8, 1701/8 - 1704/8	rPDO mapping parameter	UINT32	RW	No	0x1600 – 0x1603: ECAT, CAN 0x1701 - 0x1704: ECAT
1800 - 1803	Transmit PDO parameter	RECORD	RW	No	CAN only
0x1A00/8- 0x1A03/8, 0x1B01/8-0x1B0/8	Transmit PDO mapping	UINT32	RW	No	0x1A00-0x1A03: CAN, ECAT 0x1B01-0x1B04: ECAT
1C00/4	Sync Manager Communication Type	UINT8	RO	No	ECAT only
1C12/20	rPDO assign (Sync Manager channel 2)	UINT16	RW	No	ECAT only
1C13/20	tPDO assign (Sync Manager channel 3)	UINT16	RW	No	ECAT only
1C32/32, 1C33/32	Sync Manager 2, 3 parameter	Depending on sub-index	Depending on sub-index	No	ECAT only
200A/11	Manufacturer software version	VISIBLE STRING	RO	No	
2012/0	Transmit Binary interpreter	Binary interpreter query (object 0x42)	WO	Tx	CAN only
2013/0	Receive Binary interpreter	Binary interpreter command (object 0x43)	RO	Yes	CAN only
2030/16	Bring recorded data	UINT8	RW	No	CAN only
2031/1	Recorder header	OCTET STRING	RO	No	CAN only
2035/0	Upload data parameters	UINT32	RW	No	CAN only
2036/0	Upload data	OCTET STRING	RW	No	CAN only
2045/0	SDO block upload delay	UINT16	RW	No	CAN only
2046/0	ECAT DC Inhibit Time	UINT16	RW	No	CAN only
2051/0	CAN Download data	UINT64	WO	No	CAN only

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
2065/10	Vector mode configuration	Depending on sub-index	Depending on sub-index	Depending on sub-index	ECAT only
2082/0	CAN controller status	UINT32	RO	Tx	CAN only
2090/0	CAN FW download	UINT32	WO	No	CAN only
2202/2	Extended inputs	UINT32	RW	Tx	
2205/2	Analog Input Object	INT16	RO	Tx	
2206/0	5 V DC Supply	INT16	RO	Tx	
2207/0	Shunt count	INT32	RO	Tx	
2210/4	Counter object	Depending on sub-index	RO	Tx	ECAT only
22A1/3	Extended Outputs	UINT32	RW	Rx	
22F0/48	User Integer array	INT32	RW	Rx	ECAT only
22F1/48	User float array	REAL32	RW	Rx	ECAT only
22F3/254	SIL R1 Array	INT32	RW	Rx,Tx	ECAT only
22F4/128	SIL R2 Array	REAL64	RW	Rx,Tx	ECAT only
2300/8,2301/8	Error Logger Safety Primary/Secondary CPU	UINT32	RO	No	
2303/64	Safety Monitoring Error Object	UINT16	RO	No	
2304/0	ErrAck logger clear	UINT8	WO	No	ECAT only
2E00/0	Safety mode	UINT32	RW	No	
2E01/5	Safety mode	RECORD	Depending on sub-index	No	
2E02/0	Drive safe parameters CRC	UINT32	RW	No	
2E03/0	Extended safety options	UINT32	RW	No	
2E10/80,2E11/80	Safety I/O parameters	UINT32	RW	No	
2E12/8,2E13/8	Safety input/output status	BOOLEAN	2E12 – RO 2E13 - WO	2E12 – Tx 2e13 - Rx	
2E20/0	Safety options	UINT16	RW	No	
2E30/0	Safety address	UINT16	RW	No	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
2F00/48	User Integer Array	INT32	RW	ECAT: TxMap CAN: RxMap, TxMap	
2F01/48	User float array	REAL32	RW	ECAT: TxMap CAN: RxMap, TxMap	
2F20/4	PDO events	UINT32	RW	No	CAN only
2F21/0	Emergency event mask	UINT32	RW	No	CAN only
2F30/8	Socket's sensor internal temperature	INT16	RO	Tx	
2F31/8	Socket's sensor external temperature	INT16	RO	Tx	
2F32/8	Time capture tag	UINT32	RO	Tx	
2FE3/4	Time event counter	UINT32	RO	Tx	
2FE4/8- 2FE6/8	Socket position 1 - Socket position 3	REAL64	RO	Tx	
2FE8/8 – 2FEA/8	Socket velocity 1 - Socket velocity 3	REAL32	RO	Tx	
2FEC/8	Analog sensor amplitude	REAL32	RO	Tx	
3000/254	Elmo Commands	INT32	RW	No	
3605/0	General Reference	INT32	RW	Rx	
3606/0	Home On Touch Probe	UINT16	RW	No	
3607/0	Elmo Status Register Array	UINT32	RO	Tx	
360D/3	Vector mode table	UINT16	RO	Tx	
3610/0	Drive Temperature in C	REAL32	RO	Tx	
3617/0	Power calculation	REAL32	RO	Tx	
3618/0	Percentage of max thermal limit	UINT8	RO	Tx	
3620/5	Home on Block Limit Parameters	RECORD	RW	No	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
3625/0	Gantry yaw offset	INT32	RW	Tx	
3630/0	Safety position calibration start	UINT8	RW	No	
3640/0	Sample VQ	REAL32	RW	Tx	
3675/0	Extrapolation timeout	INT32	RW	No	
36E0/0	Gain Scheduling Manual Index	UINT16	RW	Rx	
36E4/28	Additional position actual value	INT32	RO	No	
36E5/28	Additional velocity actual value	INT32	RO	No	
36FD/0	Digital Inputs	UINT32	RW	No	
3771/2	Pole/Sensor Resolution Factor	REAL32	RW	No	
3772/0	SLP digital input/s for position absolute sensing	UINT16	RW	No	
3773/0	Absolute reference position	INT32	RW	No	
3774/0	Absolute reference position window	INT32	RW	No	
377B/0	Safety Output number used to operate brake	UINT16	RW	No	
377C/0	Safety Input number for brake manual release	UINT16	RW	No	
3780/25	General sensor 1 params	INT32	RW	No	
3781/25	General sensor 2 params	INT32	RW	No	
3783/0	Torque sensor configuration options	UINT32	RW	No	
3784/0	Torque sensor general parameters	INT32	RW	No	
3785/8	Primary torque sensor parameters	INT32	RW	No	
3787/8	Secondary torque sensor parameters	INT32	RW	No	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
3788/8	SLT fault reaction parameters	REAL32	RW	No	
3790/0	Feedbacks Options	UINT16	RW	No	
3791/0	Position comparison tolerance	UINT32	RW	No	
3795/2	Safe sensor Position value.	INT32	RO	Tx	
379A/3	Time to SW STO	UINT32	RW	No	
37A0/8	Request for operational function	UINT16	RW	No	
37B0/0	SBT command	BOOLEAN	RO	Tx	
37B1/0	SBT SOS service	UINT32	RW	No	
37B2/0	SBT ramp time delay	UINT16	RW	No	
37B3/0	SBT test duration time	UINT16	RW	No	
37B4/0	SBT torque limit	UINT16	RW	No	
37B5/0	SXT status	BOOLEAN	RO	Tx	
37B6/0	SXT torque maximum positive limit	UINT16	RW	No	
37B7/0	SXT torque maximum negative limit	UINT16	RW	No	
37B8/0	SXT error reaction	UINT32	RW	No	
37B9/8	SAT command	BOOLEAN	RO	Tx,Rx	
37BA/8	SAT torque limit	UINT16	RW	No	
37BB/8	SAT error reaction	UINT32	RW	No	
37BC/8	SAT status	BOOLEAN	RO	Tx	
6007/0	Abort connection option code	INT16	RW	No	
603F/0	Error code	UINT16	RO	Tx	
6040/0	ControlWord	UINT16	RW	Rx	
6041/0	StatusWord	UINT16	RO	Tx	
605A/0	Quick stop option code	INT16	RW	No	
605B/0	Shutdown option code	INT16	RW	No	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
605C/0	Disable operation option code	INT16	RW	No	
605D/0	Halt option code	INT16	RW	No	
605E/0	Fault reaction option code	INT16	RW	No	
6060/0	Modes of operation	INT8	RW	Rx	
6061/0	Modes of operation display	INT8	RO	Tx	
6062/0	Position demand value	INT32	RO	Tx	
6063/0	Position actual value - increments	INT32	RO	Tx	
6064/0	Position actual value	INT32	RO	Tx	
6065/0	Following error window	UINT32	RW	Rx	
6066/0	Following error time out	UINT16	RW	Rx	
6067/0	Position window	UINT32	RW	Rx	
6068/0	Position window time	UINT16	RW	Rx	
6069/0	Velocity sensor actual value	INT32	RO	Tx	
606A/0	Sensor selection code	INT16	RW	Rx	
606B/0	Velocity demand value	INT32	RO	Tx	
606C/0	Velocity actual value	INT32	RO	Tx	
606D/0	Velocity window	UINT16	RW	Rx	
606E/0	Velocity window time	UINT16	RW	Rx	
606F/0	Velocity threshold	UINT16	RW	Rx	
6070/0	Velocity threshold time	UINT16	RW	Rx	
6071/0	Target torque	INT16	RW	Rx	
6072/0	Max torque	UINT16	RW	Rx	
6073/0	Max current	UINT16	RW	Rx	
6074/0	Torque demand value	INT16	RO	Tx	
6075/0	Motor Rate Current	UINT32	RW	No	
6076/0	Motor Rate Torque	UINT32	RW	No	
6077/0	Torque Actual value	INT16	RO	Tx	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
6078/0	Current Actual value	INT16	RO	Tx	
6079/0	DC link circuit voltage	UINT32	RO	Tx	
607A/0	Profile target position	INT32	RW	Rx	
607B/2	Position range limit	INT32	RW	Rx	
607C/0	Home offset	INT32	RW	No	
607D/2	Software position limit	INT32	RW	Rx	
607E/0	Polarity	UINT8	RW	Rx	
607F/0	Max profile velocity	UINT32	RW	No	
6080/0	Max motor speed	UINT32	RW	Rx	
6081/0	Profile velocity	UINT32	RW	Rx	
6082/0	End velocity	UINT32	RW	Rx	
6083/0	Profile acceleration	UINT32	RW	Rx	
6084/0	Profile deceleration	UINT32	RW	Rx	
6085/0	Quick stop deceleration	UINT32	RW	Rx	
6086/0	Motion profile type	INT16	RW	Rx	
6087/0	Torque Slope	UINT32	RW	Rx	
608F/2	Position encoder resolution	UINT32	RW	Rx	
6090/2	Velocity encoder resolution	UINT32	RW	Rx	
6091/2	Gear ratio	UINT32	RW	Rx	
6092/2	Feed constant	UINT32	RW	Rx	
6096/2	Velocity factor	UINT32	RW	No	
6097/2	Acceleration factor	UINT32	RW	No	
6098/0	Homing method	INT18	RW	No	
6099/2	Homing speeds	UINT32	RW	No	
609A/0	Homing acceleration	UINT32	RW	No	
60B0/0	Position offset	INT32	RW	Rx	
60B1/0	Profile velocity	UINT32	RW	Rx	
60B2/0	Torque offset	INT16	RW	Rx	
60B8/0	Touch probe function	UINT16	RW	Rx	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
60B9/0	Touch probe status	UINT16	RO	Tx	
60BA/0	Touch probe 1 positive edge	INT32	RO	Tx	
60BB/0	Touch probe 1 negative edge	INT32	RO	Tx	
60BC/0	Touch probe 2 positive edge	INT32	RO	Tx	
60BD/0	Touch probe 2 negative edge	INT32	RO	Tx	
60C5/0	Max acceleration	UINT32	RW	Rx	
60C6/0	Max deceleration	UINT32	RW	Rx	
60D0/2	Touch probe source	INT16	RO	No	
60D1/0	Touch Probe Time Stamp 1 positive value	UINT32	RO	Tx	
60D2/0	Touch Probe Time Stamp 1 negative value	UINT32	RO	Tx	
60D3/0	Touch Probe Time Stamp 2 positive value	UINT32	RO	Tx	
60D4/0	Touch Probe Time Stamp 2 negative value	UINT32	RO	Tx	
60D5/0	Touch Probe 1 positive edge counter	UINT16	RO	Tx	
60D6/0	Touch Probe 1 negative edge counter	UINT16	RO	Tx	
60D7/0	Touch Probe 2 positive edge counter	UINT16	RO	Tx	
60D8/0	Touch Probe 2 negative edge counter	UINT16	RO	Tx	
60E0/0	Positive torque limit value	UINT16	RW	Rx	
60E1/0	Negative torque limit value	UINT16	RW	Rx	
60E3/36	Supported homing methods	INT8	RO	No	
60EA/0	Commutation angle	UINT16	RW	Rx	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
60F2/0	Position Option Code	UINT16	RW	Rx	
60F4/0	Following error actual value	INT32	RO	Tx	
60FA/0	Control effort	INT32	RO	Tx	
60FC/0	Position demand internal value	INT32	RO	Tx	
60FD/0	Digital Inputs	UINT32	RO	Tx	
60FE/0	Digital outputs	UINT32	RW	Rx	
60FF/0	Target velocity	INT32	RW	Rx	
6502/0	Supported drive modes	UINT32	RO	Tx	
6611/0	Safe position	INT32	RO	Tx	
6613/0	Safe velocity	INT32	RO	Tx	
6616/0	Safe torque	INT16	RO	Tx	
6630/0	Restart acknowledge command	BOOLEAN	RO	Tx,Rx	
6631/0	Restart acknowledge status	BOOLEAN	RO	Tx	
6632/0	Error acknowledge command	BOOLEAN	RO	Tx,Rx	
6633/0	Error acknowledge status	BOOLEAN	RO	Tx	
6640/0	STO command	BOOLEAN	RO	Tx,Rx	
6641/0	STO restart acknowledge behavior	BOOLEAN	RW	No	
6643/0	Safety STO activate SBC	UINT32	RW	No	
6644/0	STO status	BOOLEAN	RO	Tx	
6650/8	SS1 command	BOOLEAN	RO	Tx,Rx	
6651/8	Safety SS1 time to STO	UINT16	RW	No	
6653/8	Safety SS1 time to STO	UINT32	RW	No	
6654/8	Safety SS1 time for velocity zero.	UINT32	RW	No	
6656/8	Safety SS1 deceleration limit window	UINT32	RW	No	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
6657/8	Safety SS1 time delay for deceleration monitoring.	UINT32	RW	No	
6658/8	Safety SS1 activate SBC	UINT32	RW	No	
665F/8	SS1 status	BOOLEAN	RO	Tx	
6660/0	SBC command	BOOLEAN	RO	Tx,Rx	
6661/0	SBC brake time delay	UINT16	RW	No	
6667/0	SBC status	BOOLEAN	RO	Tx	
6668/8	SOS command	BOOLEAN	RO	Tx,Rx	
666A/8	Safety SOS position zero window	UINT32	RW	No	
666C/8	Safety SOS velocity zero window	UINT32	RW	No	
666F/8	SOS status	BOOLEAN	RO	Tx	
6670/8	SS2 command	BOOLEAN	RO	Tx,Rx	
6671/8	Safety SS2 time to SOS	UINT16	RW	No	
6672/8	Safety SS2 time for velocity zero	UINT16	RW	No	
6674/8	Safety SS2 deceleration limit	UINT32	RW	No	
6675/8	Safety SS2 time delay for deceleration monitoring.	UINT16	RW	No	
6676/8	SS2 restart acknowledge behavior	BOOLEAN	RW	No	
6677/8	SS2 error reaction	UINT32	RW	No	
667F/8	SS2 command	BOOLEAN	RO	Tx	
6680/8	SSR command	BOOLEAN	RO	Tx,Rx	
6681/8	Safety SSR time for velocity monitoring	UINT16	RW	No	
6683/8	SSR velocity upper limit	INT32	RW	No	
6685/8	SSR velocity lower limit	INT32	RW	No	
6686/8	SSR time for velocity in limit	UINT16	RW	No	
6688/8	SSR deceleration limit	UINT32	RW	No	

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
6689/8	Safety SSR time delay for deceleration monitoring.	UINT16	RW	No	
668A/8	SSR error reaction	UINT32	RW	No	
668F/8	SSR status	BOOLEAN	RO	Tx	
6690/8	SLS command	BOOLEAN	RO	Tx,Rx	
6691/8	Safety SLS time for velocity monitoring	UINT16	RW	No	
6693/8	SSR velocity limit	UINT32	RW	No	
6694/8	SLS time for velocity in limit	UINT16	RW	No	
6695/8	SLS time delay for deceleration monitoring.	UINT16	RW	No	
6697/8	SLS deceleration limit	UINT32	RW	No	
6698/8	SLS error reaction	UINT32	RW	No	
669F/8	SLS status	BOOLEAN	RO	Tx	
66A0/8	SLP command	BOOLEAN	RO	Tx,Rx	
66A2/8	SLP position upper limit	INT32	RW	No	
66A4/8	SLP position lower limit	INT32	RW	No	
66A5/8	SLP error reaction	UINT32	RW	No	
66A7/8	SSR status	BOOLEAN	RO	Tx	
66A8/8	SMS status	BOOLEAN	RO	Tx	
66AA/8	SMS velocity maximum positive	UINT32	RW	No	
66AC/8	SMS velocity maximum negative	UINT32	RW	No	
66AD/8	SMS error reaction	UINT32	RW	No	
66B0/8	SLT command	BOOLEAN	RO	Tx,Rx	
66B1/8	SLT torque limit positive direction	UINT16	RW	No	
66B2/8	SLT torque limit negative direction	UINT16	RW	No	
66B3/8	SLT error reaction	UINT32	RW	No	
66B7/8	SLT status				

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
66B8/8	SLI command	BOOLEAN	RO	Tx,Rx	
66BA/8	SLI position relative upper limit	UINT32	RW	No	
66BC/8	SLI position relative lower limit	UINT32	RW	No	
66BD/8	SLI error reaction	UINT32	RW	No	
66BF/8	SLI status	BOOLEAN	RO	Tx	
66D0/8	SDI positive direction command	BOOLEAN	RO	Tx,Rx	
66D1/8	SDI negative direction command	BOOLEAN	RO	Tx,Rx	
66D3/8	SDI position zero window	UINT32	RW	No	
66D5/8	SDI velocity zero window	UINT32	RW	No	
66DE/8	SDI positive direction Status	BOOLEAN	RO	Tx	
66DF/8	SDI negative direction Status	BOOLEAN	RO	Tx	
66E0/8	SSM status	BOOLEAN	RO	Tx	
66E2/8	SSM velocity upper limit	INT32	RW	No	
66E4/8	SSM velocity lower limit	INT32	RW	No	
66E8/8	SCA status	BOOLEAN	RO	Tx	
66EA/8	SCA position upper limit	INT32	RW	No	
66EC/8	SCA position lower limit	INT32	RW	No	
6760/9	FSOE slave frame elements	RECORD	RW	No	ECAT only
6770/4	FSOE master frame elements	RECORD	RW	No	ECAT only
6790/8	FSOE CiA402-Slave Connection Module Information	RECORD	RW	No	ECAT only
6791/8	FSOE CiA402-Slave Connection Communication Parameter	RECORD	RW	No	ECAT only

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
67A0/8	FSOE CiA402-Slave Connection Diagnosis	RECORD	RW	No	ECAT only
F000/2	Modular device profile object	RECORD	RW	No	ECAT only
F030/8	Configured module list object	UDINT	RW	No	ECAT only
F050/8	Detected module list object	UDINT	RO	No	ECAT only
F5FF/1	Configuration management	RECORD	RO	No	ECAT only
F6F0/1	Input latch local time stamp	UDINT	RO	Tx	ECAT only
F6F2/1	Sync0 latch ESC time stamp	UINT64	RO	Tx	ECAT only
F980/1	Device FSOE Slave Address	RECORD	RO	No	ECAT only
F9F0/0	Manufacturer Serial Number	VISIBLE_STRING	RW	No	ECAT only
F9F1/1	CDP Functional generation number	UINT32	RO	No	ECAT only
F9F2/1	SDP Functional generation number	UINT32	RO	No	ECAT only
F9F3/0	Vendor name	VISIBLE_STRING	RO	No	ECAT only
F9F4/1	Semiconductor SDP Device Name	VISIBLE_STRING	RO	No	ECAT only
F9F5/1	Output Identifier	UINT32	RO	No	ECAT only
F9F6/0	Time since power-on	UINT32	RO	No	ECAT only
F9F8/0	Firmware Update Functional Generation Number	UINT32	RO	No	ECAT only
FBF0/3	Device Reset Command	RECORD	Depending on sub-index	No	ECAT only
FBF2/3	Store Parameters Command	RECORD	Depending on sub-index	No	ECAT only
FBF3/3	Calculate Checksum Command	RECORD	Depending on sub-index	No	ECAT only

Object (Hex) Subindex (Dec)	Name	Data Type	Access	Map	Comment
FBF4/3	Load Parameters Command	RECORD	Depending on sub-index	No	ECAT only

5.2 Product Code & Revision Number

The product code & revision number are read by the EtherCAT master during the scanning of the network as part of the drive unique identification number. The number denoted via object **0x1018.3** to **0x1018.4** is then used by the host prior to the configuration of the device.

Each of the version files listed above include a different product code and revision number. The ESI (EtherCAT Slave Identification) file includes both the product code and revision number.

The Platinum product code:

Bit number	Description
0 - 15	Internal product code version, starting from 0x0002
16-23	Product code type: 0x01: Reserved – compatibility only 0x10: Platinum drive revision C and above 0x12: Reserved – compatibility only 0x13: Quartet family STO only 0x14: Reserve for Quartet family
24	EtherCAT ID switches: '0': Switches are not included '1': ID switches are included
25-27	Safety option: 0x0: Does not support functional safety 0x1: Functional Safety are supported
28-31	Reserved

The Platinum drive revision number:

Bit number	Description
0-15	Internal revision number, starting from 0x0002
16-19	Number of supported axes – including virtual axis. The drive may include less axes than defined in the revision number.
20-23	Number of safety axis (0 for no safety)
24-31	Reserved

5.3 Interpreter Prefixes

Interpreter prefixes are directly executed without requiring them previously to have been compiled. Towards this intention, Elmo has designed for example, the **OJ** and **OH** methods to access objects (including objects within the Object Dictionary) without using SDO handlers.

Refer to the Platinum Command Reference document for Interpreter Prefixed commands.

5.4 Communication Objects

5.4.1 Object 0x1000: Device type

This object contains information about the device type and functionality. It is comprised of a 16-bit field that describes the device profile used, and a second 16-bit field that gives additional information about optional functionality of the device.

MSB				LSB
Byte 4	Byte 3	Byte 2	Byte 1	
Additional information		Device profile number		

- Object description:

Index	0x1000
Name	Device type
Object code	VAR
Data type	UNSIGNED32
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0x0A0192

5.4.2 Object 0x100A: Manufacturer software version

This object contains the version identification of the manufacturer's software, similar to **VR[1]** command, but without date and drive name. For example, if **VR[1]** command returns value =Bell 02.00.00.07 B00 23Apr2019, the object returns 02.00.00.07 B00.

- Object description:

Index	0x100A
Name	Manufacturer software version
Object code	VAR
Data type	VISIBLE STRING
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	No
Default value	No

The object can be read by segmented SDO. See example of segmented SDO upload transfer in chapter 4.6 Uploading Data Using SDPO.

5.4.3 Object 0x100B: Bootloader version

This object contains the version identification of the bootloader software, similar to **VR[5]** command, but without date and drive name. For example, if the **VR** command returns value =Titan-Boot 02.00.01.00 B12 11Dec2018, the object returns 02.00.01.00 B12.

- Object description:

Index	0x100B
Name	Manufacturer software version
Object code	VAR
Data type	VISIBLE STRING
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	CONST
PDO mapping	No
Non-volatile	yes
Value range	No
Default value	No

The object can be read by segmented SDO. See example of segmented SDO upload transfer in chapter 4.6 Uploading Data Using SDPO.

5.4.4 Object 0x1001: Error register

This object is an error register for the device.

Object description:

Index	0x1001
Name	Error register

Index	0x1001
Object code	VAR
Data type	UNSIGNED8
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED8
Default value	0

- Data description:

Bit	Description
0	Generic error
1	Current
2	Voltage
3	Temperature
4	Communication error (overflow, error state)
5	Device profile specific
6	Reserved (always 0)
7	Manufacturer specific

If a bit is set to 1, the specified error has occurred.

5.4.5 Object 0x1002: Manufacturer status register

This object is a common status register for manufacturer-specific purposes. It returns the status similar to the SR command. The object returns Status Register for axis 1 only.

- Object description:

Index	0x1002
Name	Manufacturer status register
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.4.6 Object 0x1003: Pre-defined error field

The object contains the last 16 errors that were reported by the drive via the EMCY message.

For details about the EMCY Error Code refer to the section.

Sub-index 0 contains the number of actual errors recorded in the array, starting at sub-index 1. Up to 16 errors can be retrieved.

If there are no errors, the value of sub-index 0x00 is 0x00 and a read access of any of the other allowed sub-indexes is responded by an SDO abort message (abort code: 0x08000024).

Writing a 0 to sub-index 0 empties the array. An Abort message (error code: 0x06090030) is transmitted on attempt to write values higher than 0. Writing to object 0x2F21 also empties the array.

Every new error is stored in sub-index 0x01, older errors are moved to the next higher sub-index.

If an error is present in sub index 1...16, the read SDO returns with a data field containing the predefined error field that has the following structure:

MSB		LSB	
Bits 31...24	Bits 23...16	Bits 15...0	
Elmo error code	Emergency error register	EMCY error code	

- Object description:

Index	0x1003
Name	Pre-defined error field
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of actual errors
Access	RW
PDO mapping	No
Non-volatile	No

Value range	0...16
Default value	0

Sub-index	1 - 16
Description	Standard error field
Access	RO
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.4.7 Object 0x1006: Communication cycle period

The object provides the communication cycle period, defining SYNC interval.

The value is given in multiple of microseconds. If the value is set to 0, the transmission of SYNC messages is disabled. By changing the value from 0, and the synchronous counter overflow value is greater than 0 the first SYNC message starts with the counter value reset to 1. The transmission the SYNC messages starts within one communication cycle period as given by the value after it is set to the new value.

- Object description:

Index	0x1006
Name	Communication cycle period
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.4.8 Object 0x1008: Manufacturer device name

This object contains the manufacturer device name, such as *Guitar*, *Trombone* and *Whistle*.

- Object description:

Index	0x1008
Name	Manufacturer device name
Object code	VAR
Data type	VISIBLE STRING
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	None
Default value	None

5.4.9 Object 0x1009: Manufacturer hardware version

This object contains the version number of the manufacturer's hardware. Refer to the Product code described in 5.2 Product Code & Revision Number.

- Object description:

Index	0x1009
Name	Manufacturer hardware version
Object code	VAR
Data type	VISIBLE STRING
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	None
Default value	None

5.4.10 Object 0x100A: Manufacturer software version

This object contains the version identification of the manufacturer's software, for example: 02.00.03.00.B07. Note that the **VR[1]** returns a string that includes the drive name and the released dated of the version.

- Object description:

Index	0x100A
Name	Manufacturer software version

Index	0x100A
Object code	VAR
Data type	VISIBLE STRING
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	No
Default value	No

5.4.11 Object 0x100B: Bootloader version

This object contains the version identification of the bootloader software, for example: 02.00.01.01.B02. Note that **VR[5]** retrieves the bootloader version as well and includes the prefix “Titan Boot” with the released dated as well.

- Object description:

Index	0x100B
Name	Manufacturer software version
Object code	VAR
Data type	VISIBLE STRING
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	CONST
PDO mapping	No
Non-volatile	yes
Value range	No
Default value	No

5.4.12 Object 0x1010: Store parameters

This object is used to save parameters in non-volatile memory. Through read access, the drive provides information about its save capabilities, using:

- Sub-index 0: Largest supported sub-index
- Sub-index 1: Save all parameters

- Read access to sub index 1 returns 1

Write access is protected. To avoid accidental storage, it is only executed when a specific signature save (Lower case only can be used) is written to the sub index 1. The signature is: "save". See example.

On reception of the correct signature in sub-index 1 the drive stores the parameters and confirms the SDO transmission (SDO download initiate response). If the storing failed, the drive responds with the SDO abort transfer with abort code 0x06060000.

If the wrong signature is received, the drive refuse to store with abort code 0x08000020

- Object description:

Index	0x1010
Name	Store parameters
Object code	ARRAY
Data type	UNSIGNED32
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	1
Default value	1

Sub-index	1
Description	Save all parameters
Access	RW
Non-volatile	No
PDO mapping	No
Value range	None
Default value	None

5.4.12.1 Example of write SDO to object 0x1010 sub index 1

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
601	22	10	10	01	73	61	76	65

5.4.13 Object 0x1011: Restore default parameters

This object is used to restore parameters from non-volatile memory. Through read access, the drive provides information about its restore capabilities, using:

- Sub-index 0: Highest sub-index supported
- Sub-index 1: Restore all default parameters
- Read access to sub index 1 returns 1

Write access is protected. To avoid accidental, restore, it is only executed when a specific signature load (Lower case only can be used) is written to the appropriate sub index 1. The signature is "load". See example.

On reception of the correct signature in sub-index 1 the drive restores the parameters saved in FLASH and then confirms the SDO transmission (SDO download initiate response). If the restoring failed, the drive responds with the SDO abort transfer with abort code 0x06060000. If a wrong signature is written, the drive responds with SDO abort transfer with abort code 0x08000020.

- Object description:

Index	0x1011
Name	Restore default parameters
Object code	ARRAY
Data type	UNSIGNED32
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	1
Default value	1

Sub-index	1
Description	Restore all default parameters
Access	RW
PDO mapping	No
Non-volatile	No
Value range	None
Default value	None

5.4.13.1 Example of write SDO to object 0x1011 sub index 1

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
601	22	11	10	01	6c	6f	61	64

5.4.14 Object 0x1013: High Resolution Time Stamp

This object defines the High-resolution time stamp, in 1 microsecond. It is 0 if not used.

- Object description:

Index	0x1013
Name	High resolution time stamp
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Access	RW
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.4.15 Object 0x1016: Consumer heartbeat time

The consumer heartbeat time defines the expected heartbeat cycle time and thus has to be higher than the corresponding producer heartbeat time configured on the device producing this heartbeat. Monitoring starts after the reception of the first heartbeat message from producer. If the consumer heartbeat time is 0, the corresponding entry is not used. The time period must be a multiple of 1 ms.

The bit placement of the object entries is displayed in the table.

UNSIGNED32			
MSB			LSB
Bits	31...24	23...16	15...0
Value	0	Node-ID	Heartbeat time, in milliseconds
Encoding	-	Unsigned8	Unsigned16

If more than one consumer heartbeat entry is configured for the same Node ID, the device aborts the SDO download with abort code 0604 0043h.

- Object description:

Index	0x1016
Name	Consumer heartbeat time
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	2
Default value	2

Sub-index	1, 2
Description	Consumer heartbeat time
Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32

5.4.16 Object 0x1017: Producer heartbeat time

This object defines the cycle time of the heartbeat, which must be a multiple of 1 millisecond. It is 0 if not used.

- Object description:

Index	0x1017
Name	Producer heartbeat time
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN
Multi-axes	No

- Entry description:

Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	0

5.4.17 Object 0x1018: Identity Object

This object stores the Vendor ID, Product code, Revision number and Serial number.

- Object description:

Index	0x1018
Name	Identity object
Object code	RECORD
Data type	Identity (object 0x23)
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	4
Default value	4

Sub-index	1
Description	Vendor ID
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0x0000009A

Sub-index	2
Description	Product code
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	None

Sub-index	3
Description	Revision number
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	None

Sub-index	4
Description	Serial number
Entry category	Mandatory
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	None

Refer to the Product code described in 5.2 Product Code & Revision Number.

5.4.18 Object 0x1023: OS Interpreter Command

This object is used by the **OS** interpreter.

- Object description:

Index	0x1023
Name	OS command
Object code	RECORD
Data type	OS command record (object 0x25)
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Data type	UNSIGNED8
Non-volatile	Yes
Value range	3
Default value	3

Sub-index	1
Description	Command
Access	WO
PDO mapping	No
Non-volatile	No
Value range	None
Default value	None

Sub-index	2
Description	Status: 1: Last command completed, reply ready 3: Last command rejected, error in reply 255: Executing
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1, 3, 255
Default value	None

Sub-index	3
Description	Reply
Access	RO
PDO mapping	No
Data type	OCTET STRING
Non-volatile	No
Value range	None
Default value	None

5.4.19 Object 0x1024: OS command mode

This object is used with the **OS** interpreter (see [Chapter 10](#)).

- Object description:

Index	0x1024
Name	OS command mode
Object code	VAR
Data type	UNSIGNED8
Protocol	ECAT, CAN
Multi-axes	No

- Entry description:

Access	WO
PDO mapping	No
Non-volatile	No
Value range	0...3 0: Execute next command immediately 1 - 2: Not supported 3: Abort execution
Default value	0

5.4.20 Object 0x1029: Error behavior object

The object is non-axis related, CAN only

This object reports the CAN communication state after a heartbeat failure. The value of the object asserts that after such a failure, the CAN communication state is:

- 0: If current state is NMT Operational then go to NMT Pre-operational
- 1: No state change
- 2: Go to NMT state Stopped

- Object description:

Index	0x1029
Name	Error behavior object
Object code	ARRAY
Data type	UNSIGNED8
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
-----------	---

Description	Number of error classes
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	1
Default value	1

Sub-index	1
Description	Communication error
Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED8
Default value	1

5.4.21 Object 0x10F1: Error Settings Object

The Ether CAT Error settings object defines the slave error reaction behavior and sync error counter limit.

- Sub-index 1 sets slave Local Error Reaction for Ether CAT communication errors.
- Sub index 2 sets the value of Sync Error Counter Limit.
The only permitted value of sub index 1 is 2 "Device specific state". Download SDO with setting value of sub index 1 different from 2 shall return SDO abort message with error code 0x06090030 "Parameter is out of range".
- The value 2 "Device specific state" defines the following behavior:
Internal Sync Error Counter incremented by 3 in case of a missed SM2 Event and decremented by 1 in case of a received SM2 Event. When Sync Error Counter reached or exceeded the value of Sync Error Counter Limit (object 0x10F1 sub-index 2) the drive shall perform the following actions:
 - Switch to Safe Operational state with AL error indicating (AL Status Code 0x1A),
 - Set to 1 Sync Error (object 0x1c32 sub index 32),
 - Perform action defined by object 0x605E "Fault Reaction option Code",
 - The Sync Error Counter is reset:
 - When AL error is acknowledged by Host
 - When the drive is synchronized again and switched to Operational state.



Note: No reset is performed when the Sync error Counter Limit is set to 0.

- The following table describes the object:

Index	0x10F1
Name	Error Settings
Object code	RECORD
Data type	Depending on sub-index
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Largest supported sub-index
Access	RO
PDO mapping	No
Data type	UNSIGNED8
Non-volatile	No
Value range	1, 2
Default value	2

Sub-index	1
Description	Local error reaction
Access	RW
PDO mapping	No
Data type	UNSIGNED32
Non-volatile	No
Value range	Only value 2 is permitted
Default value	2

Sub-index	2
Description	Sync Error Counter limit
Access	RW
PDO mapping	No
Data type	UNSIGNED16
Non-volatile	No
Entry category	Optional
Value range	UNSIGNED16
Default value	0

The Figure 23 example shows the use of the Sync Error Counter.

SM Event	1	0	1	0	1	0	1	0	1	0	1
Internal Sync Error Counter (incremented by 3) Error Counter Limit = 9	0	3	2	5	4	7	6	9	9	9	9

Table 29: Example for weighted Error Counter

In the example the Internal Sync Error Counter is incremented by 3 in case of a missed SM2 Event and decremented by 1 in case of a received SM2 Event. After 8 bus cycles the Sync Error Counter Limit is reached, and the multiple error is indicated to the master by an EtherCAT state change to SAFEOP with AL Status Code 0x1A.

5.5 rPDO

5.5.1 Objects 0x1600 – 0x1603, 0x1701 - 0x1704: rPDO mapping parameter

These objects contain the mapping for the PDOs that the Elmo drive is able to receive. The sub-index 0 contains the number of valid entries within the mapping record. This number of entries is also the number of the application variables that are received with the corresponding PDO. Maximum 8 objects can be mapped to single PDO, so the length of a PDO data can be 8X8=64 bits.

- Note that ECAT Sync Manager 2 size maximum is 128 Bytes.
- The sub-indexes from 1 to number of entries contain the information about the mapped application variables. These entries describe the PDO contents by their index, sub-index, and length. All three values are hexadecimal coded.
- Write access to objects for mapping procedure is permitted:
 - in PREOP and OPERATIONAL NMT states in CAN
 - in PREOP ECAT state in ECAT

The PDO mapping element is a 32-bit field (object 0x21), divided as follows:

MSB	LSB
Index: 16-bit	Sub-index: 8-bit Object length: 8 bits

If the change in PDO mapping cannot be executed (for example, the PDO length is exceeded or the SDO client attempts to map an object that cannot be mapped), the device responds with an Abort SDO Transfer Service.

Sub-index 0 determines the valid number of objects that have been mapped. To change the PDO mapping, sub-index 0 must be set to 0 (mapping is deactivated). Only then can the objects be remapped.

When a new object is mapped by writing a sub-index between 1 and 8, if the object does not exist or it cannot be mapped, the SDO transfer is aborted with the Abort SDO Transfer Service.

After all objects are mapped, sub-index 0 is set to the valid number of mapped objects. Then the drive rechecks the mapping integrity. Finally, the PDO is created by writing to its communication parameter COB-ID.

When sub-index 0 is set to a value greater than 0, the device may validate the new PDO mapping before transmitting the response of the SDO service. If an error is detected, the Elmo drive transmits the Abort SDO Transfer Service with abort codes 0x06020000, 0x06040041 or 0x06040042.

For details of PDO mapping procedure in CAN refer to chapter "CAN PDO Mapping", for ECAT PDO Mapping – to "ECAT Manual"

When the sub-index 0 is read, the actual number of valid mapped objects is returned.

- Object description:

Index	0x1600 – 0x1603, 0x1701 - 0x1704
Name	rPDO mapping parameter
Object code	RECORD
Data type	PDO Mapping (object 0x21),
Protocol	0x1600 – 0x1603: ECAT, CAN 0x1701 - 0x1704: ECAT

Index	0x1600 – 0x1603, 0x1701 - 0x1704
Multi-axes	0x1600 – 0x1603: Yes for ECAT, No for CAN 0x1701 - 0x1704: No

- Entry description:

Sub-index	0
Description	Number of mapped application objects in PDO
Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED8
Default value	For ECAT - None For CAN: 0x1600, 0x1601: 1 0x1602, 0x1603: 0

Sub-index	1-8
Description	PDO mapping for application object to be mapped
Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	For ECAT - None For CAN: 0x1600 sub-index 1: 0x60400010 0x1601 sub-index 1: 0x20120040 0x1600, 0x1601 sub-indices 2-8: 0 0x1602, 0x1603 sub-indices 1-8: 0

Up to 8 objects can be mapped to a single rPDO.

Dummy entries are supported by the Elmo drive.

The SDO client is responsible for data consistency.

5.5.2 Objects 0x1400 - 0x1403: rPDO communication parameter

Object description:	These objects contain the details of the rPDO communication method and data.
Value definition:	Sub-index 0x00 contains the number of valid object entries within the record.

Sub-index 0x01 contains the COB-ID of the rPDO, see table below.

MSB				LSB	
Bits	31	30	29	28 - 11	10 - 0
meaning	valid=0, Not valid=1	Reserved	Frame, always 0	0	11 bit CAN-ID

The bit valid (bit 31) enables rPDOs.

There may be PDOs configured (e.g. by default) but not used, and therefore set to *not valid* (deleted).

Attempt to set bit 29 (frame) to 1_b is responded with the SDO Abort Transfer service (abort code: 0x06090030).

It is not allowed to change bit 0 to 30 while the PDO exists and is valid (bit 31 = 0). In this case, when attempting to change the values from bit 0 to bit 30, the device responds with the SDO abort transfer service (abort code: 0x06090030).

Predefined CAN-ID values are presented in following table

Object index	rPDO number	Predefined CAN-ID
0x1400	rPDO1	0x200 + NODE-ID
0x1401	rPDO2	0x300 + NODE-ID
0x1402	rPDO3	0x400 + NODE-ID
0x1403	rPDO4	0x500 + NODE-ID

- Object description:

Index	0x1400 – 0x1403
Name	rPDO communication parameter
Object code	RECORD
Data type	PDO communication parameter record (object 0x20)
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	2
Default value	2

Sub-index	1
-----------	---

Description	COB-ID used by PDO
Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0x1400: 0x27F 0x1401: 0x37F 0x1402: 0x8000047F 0x1403: 0x8000057F

Sub-index	2
Description	Transmission type
Access	RW
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED8
Default value	255 for 0x1400, 0x1402, 0x1403 254 for 0x1401

Sub-index 2 defines the transmission type of the rPDO (see Table below). Transmission type may be 255, 254 or 1...240. An attempt to change the value of the transmission type to any unsupported value is responded with the SDO abort transfer service (abort code: 0x06090030). Description of rPDO transmission type, sub-index 2.

Value	Description
0x00	Synchronous, acyclic
0x01...0xF0	Synchronous, cyclic, every 1 ...240 SYNCs
0xF1...0xFD	Reserved
0xFE	Event driven (manufacturer specific)
0xFF	Event driven (device profile and application profile specific)

The Elmo drive treats type 254 and 255 similarly.

Synchronous type means that the device will actuate the received data with the reception of the next 1...240 SYNC. Consider when PDO is defined as synchronous it may be sent by producer in time more than Sync period.

Event-driven means PDO may be received at any time the device will actuate the data immediately. See description of object 0x2F20 for detailed information related to defined events.

5.6 tPDO

5.6.1 Objects 0x1A00-0x1A03, 0x1B01-0x1B04: Transmit PDO mapping parameter

These objects contain the mapping parameter for the PDOs that the Elmo drive is able to transmit. Sub-index 0 contains the number of valid entries within the mapping record. This number of entries is also the number of the application variables that are transmitted with the corresponding tPDO.

- Object description:

Index	0x1A00- 0x1A03, 0x1B01-0x1B04														
Name	Transmit PDO mapping														
Object code	RECORD														
Data type	PDO mapping parameter record (object 0x21): Sub-index 0: UNSIGNED8 Sub-index 1-8: UNSIGNED32														
Protocol	0x1A00-0x1A03: CAN, ECAT 0x1B01-0x1B04: ECAT														
Multi-axes	<table> <tr> <td>0x1A00- 0x1A03:</td><td>Axis related, CAN and ECAT</td></tr> <tr> <td>0x1A10-0x1A13:</td><td>Axis related, ECAT only</td></tr> <tr> <td>0x1A20-0x1A23:</td><td>Axis related, ECAT only</td></tr> <tr> <td>0x1A30-0x1A33:</td><td>Axis related, ECAT only</td></tr> <tr> <td>0x1A40-0x1A43:</td><td>Axis related, ECAT only</td></tr> <tr> <td>0x1A50-0x1A53:</td><td>Axis related, ECAT only</td></tr> <tr> <td>0x1B01-0x1B04</td><td>Non-axis, ECAT only</td></tr> </table>	0x1A00- 0x1A03:	Axis related, CAN and ECAT	0x1A10-0x1A13:	Axis related, ECAT only	0x1A20-0x1A23:	Axis related, ECAT only	0x1A30-0x1A33:	Axis related, ECAT only	0x1A40-0x1A43:	Axis related, ECAT only	0x1A50-0x1A53:	Axis related, ECAT only	0x1B01-0x1B04	Non-axis, ECAT only
0x1A00- 0x1A03:	Axis related, CAN and ECAT														
0x1A10-0x1A13:	Axis related, ECAT only														
0x1A20-0x1A23:	Axis related, ECAT only														
0x1A30-0x1A33:	Axis related, ECAT only														
0x1A40-0x1A43:	Axis related, ECAT only														
0x1A50-0x1A53:	Axis related, ECAT only														
0x1B01-0x1B04	Non-axis, ECAT only														

- Entry description:

Sub-index	0				
Description	Number of mapped application objects in PDO				
Access	Read/Write				
PDO mapping	No				
Non-volatile	No				
Value range	0...8				
Default value	For ECAT: none; For CAN: <table> <tr> <td>0x1A00, 0x1A01:</td><td>1</td></tr> <tr> <td>0x1A02, 0x1A03:</td><td>0</td></tr> </table>	0x1A00, 0x1A01:	1	0x1A02, 0x1A03:	0
0x1A00, 0x1A01:	1				
0x1A02, 0x1A03:	0				

Sub-index	1
Description	PDO mapping for nth application object to be mapped
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	$0 \dots (2^{32}) - 1$
Default value	For ECAT: none; For CAN: 0x1A00: 0x60410010 0x1A01: 0x20130040 0x1A02, 0x1A03: 0

Sub-index	2 - 8
Description	PDO mapping for nth application object to be mapped
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	0...(2 ³²)-1
Default value	For ECAT: none; For CAN: 0

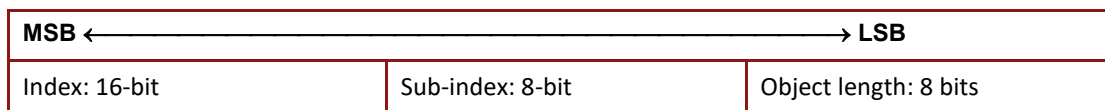
Up to eight objects can be mapped to a single tPDO.

Dummy entries are supported by the Elmo drive.

- Write access to objects for mapping procedure is permitted:
 - in PREOP and OPERATIONAL NMT states in CAN
 - in PREOP ECAT state in ECAT

Sub-index from 1 to 8 contains the information of the mapped application objects. The object describes the content of the PDO by their index, sub-index and length (see Figure below).

The length contains the length of the application object in bit. This may be used to verify the mapping.



An attempt to change the value of an object entry to any value that is not supported is responded with the SDO abort transfer service. The cause for a not supported value could be the mapping (index and sub-index) of a non-existing application object, a wrong length for the mapped application object, or a wrong length for the PDO.

5.6.2 Objects 0x1800 - 0x1803: Transmit PDO communication parameter

This object contains the communication parameters for the PDOs that the Elmo drive is able to transmit.

Object is relevant to CAN only.

Value definition:

- **Sub-index 0** contains the number of valid object entries within the record. Its value is at least 5.
- **Sub-index 0x1** contains the COB-ID of the tPDO (see table below).

MSB	Sub-index 1	LSB			
Bits	31	30	29	28 - 11	10 - 0
meaning	valid=0, Not valid=1	RTR 0 – RTR allowed on this PDO 1 – no RTR allowed	Frame, always 0	0	11 bit CAN-ID

The bit valid (bit 31) enables rPDOs. There may be PDOs configured (e.g. by default) but not used, and therefore set to *not valid*.

Attempting to set bit 29 (frame) to 1_b or bit 30 (RTR) to 0_b is responded with the SDO abort transfer service (abort code: 0x06090030).

It is not allowed to change bit 0 to 30 while the PDO exists and is valid (bit 31 = 0_b). In this case, when attempting to change the values from bit 0 to bit 30, the device responds with the SDO abort transfer service (abort code: 0x06090030).

Predefined CAN-ID values are presented in following table.

Object index	tPDO number	Predefined CAN-ID
0x1800	tPDO1	0x180 + NODE-ID
0x1801	tPDO2	0x280 + NODE-ID
0x1802	tPDO3	0x380 + NODE-ID
0x1803	tPDO4	0x480 + NODE-ID

- Object description:

Index	0x1800 – 0x1803
Name	Transmit PDO parameter
Object code	RECORD
Data type	PDO communication parameter record (object 0x20)
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries

Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	6
Default value	6

Sub-index	1
Description	COB-ID used by PDO
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	Not applicable
Default value	0x1800: 0x400001FF 0x1801: 0x400002FF 0x1802: 0xC00003FF 0x1803: 0xC00004FF

Sub-index	2
Description	Transmission type
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	0...240, 254,255
Default value	0x1800: 0xFF 0x1801: 0xFE 0x1802, 0x1803 0

Sub-index	3
Description	Inhibit time
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	0...65536
Default value	0 (no inhibit time between messages)

Sub-index	4
------------------	----------

Description	Reserved
Access	
PDO mapping	
Value range	
Default value	

Sub-index	5
Description	Event timer
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	0...65536
Default value	0

Sub-index	6
Description	SYNC start value
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	0...65536
Default value	0

- **COB ID used by PDO**
Only the predefined COB ID can be written to the drive. An attempt to write another COB ID will result in an abort (Abort Code 0x06090030).
- RTR is not supported
- Sub-index 2 defines the transmission type of the tPDO (see Table). An attempt to change the value of the transmission type to any not supported value shall be responded with the SDO abort transfer service (abort code: 0x06090030).

5.6.2.1 Description of tPDO transmission type, sub index 2

Value	Description
0x00	Synchronous, acyclic
0x01	Synchronous, cyclic, every 1 SYNC
--	--
0xF0	Synchronous, cyclic, every 240 SYNC
0xF1	Reserved

Value	Description
--	--
0xFD	Reserved
0xFE	Event driven (manufacturer specific)
0xFF	Event driven (device profile and application profile specific)

The Elmo drive treats type 254 and 255 similarly

Synchronous cyclic (sub index 2 = 1...240) means that the tPDO is transmitted after every N number of SYNCs are received, where N is the value of sub index 2.

Synchronous acyclic transmission (sub index 2 = 0) means that the tPDO is transmitted once only after receiving the first sync after tPDO is enabled.

Event-driven means that the PDO may be transmitted at any time based on the occurrence of drive internal event. See the description of object **0x2F20** for detailed information related to defined events.

5.6.2.2 Inhibit Time

Inhibit time (sub index 3) defines the minimum time interval to transmit a tPDO. The resolution of the inhibit time is 100 microseconds. If sub index 3 is set to 0, then the minimum time interval to transmit tPDO is disabled.

5.6.2.3 Event Timer

When a tPDO transmission type is 254 or 255, an event timer defined by sub index 5 can be used. The timer defines the maximum interval for tPDO transmission. The event occurs when the time has elapsed. The event timer resolution is 1ms. The event causes the transmission of this tPDO in addition to other asynchronous events. The occurrence of an event sets the timer again. A value of 0 disables this function.

5.7 ECAT Related Objects

5.7.1 Objects 0x1C00: Sync manager communication type

Sync manager communication type object 0x1C00 describes the use of the Sync Manager channels, see table below.

- Object description:

Index	0x1C00
Name	Sync Manager Communication Type
Object code	ARRAY
Data type	UNSIGNED8
Protocol	ECAT
Multi-axes	None

- Entry description:

Sub-index	0
Description	Number of used Sync Manager channels
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	4
Default value	4

5.7.2 Objects 0x1C12: rPDO assign (Sync Manager Channel 2)

rPDO assign (Sync Manager Channel 2) object 0x1C12 is described in following tables.

- Object description:

Index	0x1C12
Name	rPDO assign (Sync Manager channel 2)
Object code	ARRAY
Data type	UNSIGNED16
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of assigned rPDOs
Access	Read /write

PDO mapping	No
Non-volatile	No
Value range	0 - 20
Default value	none

Sub-index	1-20
Description	Mapped object index of assigned rPDO
Access	Read /Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	None

5.7.3 Objects 0x1C13: tPDO assign (Sync Manager Channel 3)

tPDO assign (Sync Manager Channel 3) object 0x1C13 is described in following tables.

- Object description:

Index	0x1C13
Name	tPDO assign (Sync Manager channel 3)
Object code	ARRAY
Data type	UNSIGNED16
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of assigned tPDOs
Access	Read /write
PDO mapping	No
Non-volatile	No
Value range	0 - 20
Default value	none

Sub-index	1-20
Description	Mapped object index of assigned tPDO
Access	Read /Write

PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	None

5.7.4 Objects 0x1C32, 0x1C33: Sync Manager 2, 3 parameters

Sync Manager 2 parameter object 0x1C32 and Sync Manager 3 parameter object 0x1C33 contain information about the synchronization behavior of the device. Following picture presents timing in the case of DC sync0 mode. Refer to ECAT description in Platinum Administrative Guide for details.

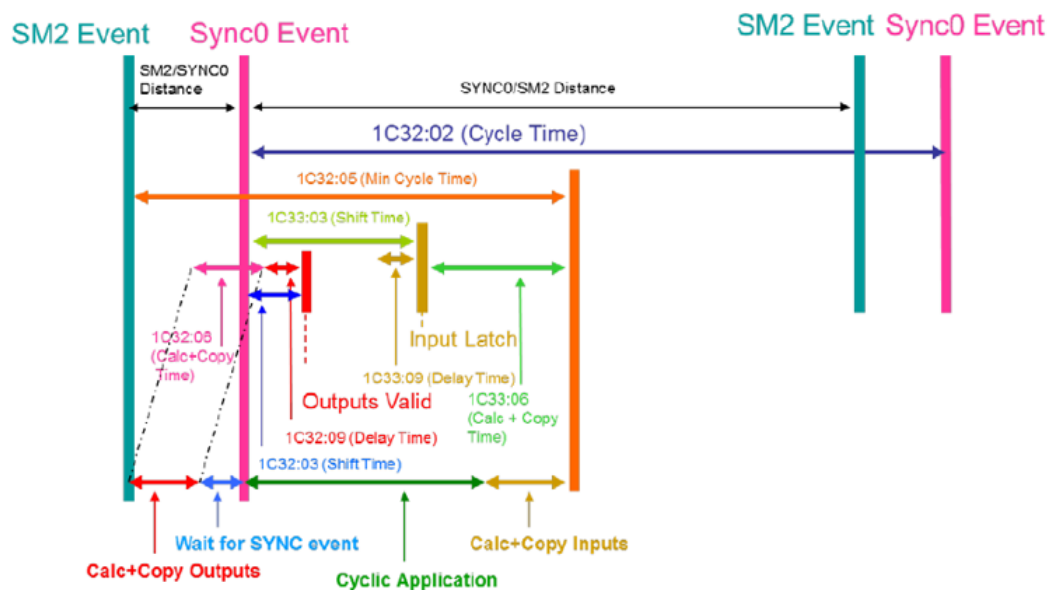


Table 30: Timing in the case of DC sync0 mode

- Object description:

Index	0x1C32,0x1C33
Name	Sync Manager 2,3 parameter
Object code	RECORD
Data type	See entry description
Protocol	ECAT
Multi-axes	None

- Entry description:

Sub-index	0
Description	Number of sub-indices
Access	Read only

PDO mapping	No
Data types	UNSIGNED8
Non-volatile	Yes
Value range	32
Default value	32

Sub-index	1
Description	Synchronization type: 0x00: Free run (not synchronized) 0x01: Synchronous, synchronized with SM event: For 0x1C32: synchronized with SM2 event For 0x1C33: synchronized with SM3 event 0x02: DC Mode For 0x1C32 and 0x1C33: Synchronized with Sync0 0x03: DC mode For 0x1C32 and 0x1C33: Synchronized with Sync1 For 0x1C33 only: DC Shift of Input Latch with Sync1 0x22: SM event mode For 0x1C33 only: SM2 event mode, SM2 shift of Input Latch
Access	Read/Write
PDO mapping	No
Data types	UNSIGNED16
Non-volatile	No
Value range	0...0x22
Default value	none

Sub-index	2
Description	Cycle time: - Free run mode (Sub-index 1 = 0): the value is the time between two local timer events in ns - SM Event mode (Sub-index 1 = 1 or 0x22): the value is minimum time between two SM2 events in ns

	DC Sync0 (Sub-index 1 = 2): the value is Sync0 Cycle time, i.e. the time between two Sync0 pulses (ESC Register 0x9A3-0x9A0) in ns Note: Both 0x1C32.2 and 0x1C33.2 must have the same value
Access	Read only
PDO mapping	No
Data types	UNSIGNED32
Non-volatile	No
Value range	UNSIGNED32
Default value	None

Sub-index	3
Description	Shift time: Time between related event, depending on Synchronization type (SM2/SM3 event or DC Sync0) and the associated action (Outputs valid/Input Latch, output data available at the process) in ns. Is equal or greater than "delay time" sub-index 9
Access	Read only
PDO mapping	No
Data types	UNSIGNED32
Non-volatile	No
Value range	UNSIGNED32
Default value	None

Sub-index	4
Description	Synchronization types supported: Bit 0: Free Run supported Bit 1: Synchronous supported Bit 4-2: DC type supported: 000: No DC 001: DC Sync0 010: Reserved 100: reserved Bit 6-5: Shift settings: 00: No output shift supported 01: Output shift with local timer (Shift Time)

	10: Output shift with Sync1 Bit 9-6: Reserved Bit 10: Delay Times should be measured (is dependent on configuration) Bit 11: Delay Time is fixed (synchronization is done by hardware) Bit 13-12: Reserved Bit 14: Dynamic Cycle Times: The bit is set, indicating that the drive cannot calculate the cycle time. Bit 15: Reserved
Access	Read only
PDO mapping	No
Data types	UNSIGNED16
Non-volatile	Yes
Value range	UNSIGNED16
Default value	None

Sub-index	5
Description	Minimum cycle time supported: Both 0x1C32.5 and 0x1C33.5 must have the same value Used in Synchronous Mode (SM2/SM3) and DC mode (Sync0) The value done in ns
Access	Read only
PDO mapping	No
Data types	UNSIGNED32
Non-volatile	Yes
Value range	UNSIGNED32
Default value	None

Sub-index	6
Description	Calculate and copy time:

	• For 0x1C32: Minimum time for SM2 event to Sync0 event in ns. The time needed by the drive to copy the process data from the Sync Manager to the local memory and perform calculation if necessary before the data is sent to process. • For 0x1C33: Minimum time for inputs after Input Latch in ns. The time needed by the drive to perform calculation on the input values and the copy to the SM3 before the data is available in ECAT. Used in DC mode
Access	Read only
PDO mapping	No
Data types	UNSIGNED32
Non-volatile	Yes
Value range	UNSIGNED32
Default value	none

Sub-index	7
Description	Minimum delay time Used only in DC Sync0/Sync1 mode (Sub-index 1 = 2/3)
Access	Read only
PDO mapping	No
Data types	UNSIGNED32
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0

Sub-index	8
Description	Get cycle time: Bit 0: 0: Measurement of local; cycle time stopped 1: Measurement of local; cycle time started Bit 1: 0: do nothing 1: Reset the error counters
Access	Read / Write
PDO mapping	No

Data types	UNSIGNED16
Non-volatile	No
Value range	0...3
Default value	None

Sub-index	9
Description	Delay time: HW delay time of the slave in ns from Sync0 to Outputs valid. If Sub-index 7 value is greater than 0, this Delay time is maximum delay time Used only in DC Sync0/Sync1 mode (Sub-index 1 = 2/3)
Access	Read only
PDO mapping	No
Data types	UNSIGNED32
Non-volatile	Yes
Value range	UNSIGNED32
Default value	None

Sub-index	10
Description	reserved
Access	-
PDO mapping	-
Data types	-
Non-volatile	-
Value range	-
Default value	-

Sub-index	11
Description	SM event missed counter: This error counter is incremented when the drive expects a SM event, but does not receive it in time. Used in DC Sync0/Sync1 mode
Access	Read only
PDO mapping	No
Data types	UNSIGNED16
Non-volatile	Yes

Value range	UNSIGNED16
Default value	None

Sub-index	12
Description	Cycle time too small counter: The counter is incremented when the cycle time is too small so that the local cycle cannot be completed and input data cannot be provided before the next cycle. Used in Synchronous SM2/SM3 and DC Sync0/Sync1 modes
Access	Read only
PDO mapping	No
Data types	UNSIGNED16
Non-volatile	Yes
Value range	UNSIGNED16
Default value	None

Sub-index	13
Description	Shift time too short counter: The counter is incremented when the time distance between Sync0 and the Outputs Valid is too short due to short shift time. Used in Synchronous SM2/SM3 and DC Sync0/Sync1 modes
Entry category	Mandatory
Access	Read only
PDO mapping	No
Data types	UNSIGNED16
Non-volatile	Yes
Value range	UNSIGNED16
Default value	none

Sub-index	14-31
Description	Reserved
Access	-
PDO mapping	-

Data types	-
Non-volatile	-
Value range	-
Default value	-

Sub-index	32
Description	Sync error
Access	Read only
PDO mapping	No
Data types	BOOLEAN
Non-volatile	
Value range	TRUE, FALSE
Default value	None

5.8 Elmo Specific Objects

5.8.1 Object 0x200A: Version object

This object contains the version identification of the drive, like the **VR** command.

- Object description:

Index	0x200A
Name	Manufacturer software version
Object code	RECORD
Data type	VISIBLE STRING
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	See following table
Description	See following table
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	No
Default value	No

The object can be read by segmented SDO

Sub-indices description:

Sub-index	Description	Format	Example
0	Maximum sub-index	-	11
1	Drive FW version	<Product Name><Drive SW Version> <Build Number><Date>	02.00.00.07 B00 23Apr2019
2	Safety SW version	<SafetySw><Safety SW Version> <Build Number><Date>	SafetySw 02.00.00.07 B10 23Apr2019
3	Safety HW version	<SafetyHw><HW Version> <Build Number><Date>	SafetyHw 01.00.00.00 B00 10Dec2014
4	STO version	<STO><Software Version> <Build Number><Date>	01.00.00.00 B18 14Nov2018
5	Boot version	<Boot><Software Version><Build Number><Date>	Titan-Boot 02.01.01 B00 20Jan2019
6	SIL version	<Anan><Anan Software Version><Build Number><Date>	SIL not configured

Sub-index	Description	Format	Example
7	Drive name	<Drive name>	Bell
8	ManufacturerHwVersion	<Drive HW version>	0x4009
9	EtherCAT Version	<EtherCAT PRU Rev:> <revision number><Build:> <build number>	EtherCAT PRU Rev: 2 Build: 4EE
10	Drive user name	<Drive user name>	Elmo Drive
11	Drive barcode	<Barcode>	PBEL-SF-5/100FES

5.8.2 Object 0x2012: Binary interpreter input

This object is a binary interpreter object (refer to 5.1 Objects Properties).
concerning the byte stream).

- Object description:

Index	0x2012
Name	Transmit Binary interpreter
Object code	VAR
Data type	Binary interpreter query (object 0x42)
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Write only
PDO mapping	tPDO
Non-volatile	No
Value range	No
Default value	No

5.8.3 Object 0x2013: Binary interpreter Receive

This object is a binary interpreter object.

- Object description:

Index	0x2013
Name	Receive Binary interpreter
Object code	VAR
Data type	Binary interpreter command (object 0x43)
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Read only
PDO mapping	Yes
Non-volatile	No
Value range	No
Default value	No

5.8.4 Object 0x2030: Recorder data

This object is used to retrieve recorder parameters according to **RC** and the sub-index field. If N is sub index, the 0x2030.N fetches the parameter, recorded in RC = (1 << (N-1)) The actual recorded signal is according to the **RV[]** mapping.

- Object description:

Index	0x2030
Name	Bring recorded data
Object code	RECORD
Data type	Depending on sub-index
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of supported elements
Access	Read only
PDO mapping	No
Data type	UNSIGNED8
Non-volatile	Yes
Value range	16
Default value	16

Sub-index	1...16
Description	Per the entry of the RV[]
Access	RW
PDO mapping	No
Data type	REAL64
Non-volatile	No

Value range	-1.0e37 ... 1.0e37
Default value	No

5.8.4.1 Uploading Recorded Data

- The recorder data is uploaded in according with the SDO Upload Segmented protocol.
- The sub-index in the **Object 0x2030** is a value of the **BH** command, means upload **Object 0x2030** sub index 2 is the same as to send **BH=2** command. Refer to the Platinum Drive Command Reference guide.
- If the recorder variables are changed during upload, the process will be aborted.

5.8.4.2 Example of uploading recorded data

- Move axis with speed 32000:
MO=1;
JV=32000
- Set **RR=0**; to stop previous recording, if active
- Set **RL=2**; to get 2 samples
- Set **RG=2**; Recorder gap, record every 2 TS
- Set **RC=2**; Choose recoding signal Index 2 (Velocity command)
- Set **RR=2**; Start recorder
- When recording is completed (**RR==0**) we can upload recorded data using object 0x2030.
Note that the "Velocity command" record signal is presented in internal units, so 32000 will be presented as $32000 * TS = 32000 * 50e-6 = 1.6$.
- Value 1.6 in floating points is presented as 0x3FCCCCC

5.8.4.3 Uploading the Velocity Command

To upload Velocity command via **Object 0x2030** sub index 2:

- SDO client sends upload initiate SDO request

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
0x602	0x40	0x30	0x20	0x02	0x00	0x00	0x00	0x00

- Server responses

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
0x582	0x41	0x30	0x20	0x02	0x0C	0x00	0x00	0x00

This Means: Segmented, there are 12 bytes to be uploaded: 4 bytes are BH command header, 4 bytes are 1st sample of Velocity command, and 4 bytes are 2nd sample of Velocity command. Note, Velocity command is presented in accordance with its data type (REAL32)

- SDO client sends

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
602	60	00	00	00	00	00	00	00

- Server responses:

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
-------	--------	--------	--------	--------	--------	--------	--------	--------

582	00	08	00	00	00	CC	CC	CC
-----	----	----	----	----	----	----	----	----

Where (header starts from Byte 1):

bytes 1-4 = 08, means "data contains 8 bytes";

bytes 5-7 the part of the 1st sample data, see explanation later

5. SDO client sends

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
602	70	00	00	00	00	00	00	00

6. Server responses:

COBID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
582	15	3F	CC	CC	CC	3F	00	00

Where:

Bytes 5,4,3,2 are 0x3FCCCCC, that is the floating-point representation in internal units of Velocity command = 32000, the 2nd sample point. The same are Byte 1 of the second SDO transfer and Bytes 7-5 of the first SDO transfer, that present the 1st sample of Velocity command.

5.8.5 Object 0x2031: Recorder header

This object is used to retrieve recorder header (similar to **RH=1** command).

- Object description:

Index	0x2031
Name	Recorder header
Object code	RECORD
Data type	Depending on sub-index
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of supported elements
Access	Read only
PDO mapping	No
Data type	UNSIGNED8
None-volatile	Yes
Value range	1
Default value	1

Sub-index	1
-----------	---

Description	Recorder header
Access	Read only
PDO mapping	No
Data type	OCTET STRING
None-volatile	No
Value range	No
Default value	No

- The recorder header is uploaded in according with the SDO Upload Segmented protocol.
- If the recorder variables are changed during upload, the process will be aborted.

5.8.6 Object 0x2035: Upload data parameters

This object is for ELMO internal use only, not for user. To upload data (personality, parameters, user program...) via CAN the SDO Upload Segmented protocol **Object 0x2036** is used. The **Object 0x2035** shall be sent before SDO segmented to define upload procedure parameter.

- Object description:

Index	0x2035
Name	Upload data parameters
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Read / Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	No

5.8.7 Object 0x2036: Upload data

This object is for ELMO internal use only, not for user.

To upload data (personality, parameters, user program...) via CAN the SDO Upload Segmented protocol **Object 0x2036** is used. The **Object 0x2035** should be sent before SDO segmented to define upload procedure parameter.

- Object description:

Index	0x2036
Name	Upload data
Object code	VAR
Data type	OCTET STRING
Protocol	CAN

- Entry description:

Access	Read / Write
PDO mapping	No
Non-volatile	No
Value range	No
Default value	No

5.8.8 Object 0x2045: SDO block upload delay

This object is for ELMO internal use only, not for user.

The object contains the delay to be inserted between uploaded blocks in SDO block upload protocol in ms.

- Object description:

Index	0x2045
Name	SDO block upload delay
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Read / Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	0

5.8.9 Object 0x2046: ECAT DC Inhibit Time

This object presents EtherCAT Distributed clock inhibit time in ms. It supports additional delay time between initialization of DC mode and start of DC monitoring.

- Object description:

Index	0x2046
Name	ECAT DC Inhibit Time
Object code	VAR
Data type	UNSIGNED16
Protocol	ECAT
Multi-axes	No

- Entry description:

Access	Read / Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	0

5.8.10 Object 0x2051: CAN download data

The object is for ELMO internal use only, not for user

- Object description:

Index	0x2051
Name	CAN Download data
Object code	VAR
Data type	UNSIGNED64
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Write only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED64
Default value	No

5.8.11 Object 0x2065 Vector Mode Configuration Object

The vector mode is a method allowing to update real time data to the drive via PDO. The data is added to the relevant table per feature used with the mechanism of Auto Increment and Auto roll-over which allows an endless feeding of the table by the host.

This feature is useful for example in Output Compare mode where the number of desired compared positions are much more than the capacity of the Output compare table in the drive. The host, in this example, can fill up the table and continue feeding it as the points which were already used can be overridden by the new points downloaded via PDO to the drive.

**Note:**

For the ECAM and the Output compare table the “Next available index” is mapped to object 0x360D

The tables and modes that can be configured are:

Feature	Table	Object
ECAM	ET	0x30AD
UI	UI	0x32E2
Output compare 1	GV	0x30F7
Output compare 2	GV	0x30F7
Error Mapping	ET	0x30AD

Index	0x2065
Name	Vector mode configuration
Object code	RECORD
Data type	NA
Protocol	ECAT

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	10
Default value	10

Sub-index	1
Description	Vector Start Index
Access	Read/Write

PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	No

Sub-index	2
Description	Vector Last Index
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	No

Sub-index	3
Description	Vector Object number
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	No

Sub-index	4
Description	Vector Control
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED8
Default value	No

Sub-index	5
Description	Vector 8 Bits
Access	Read/Write
PDO mapping	RxMap

Non-volatile	No
Value range	UNSIGNED8
Default value	No

Sub-index	6
Description	Vector 16 Bits
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	No

Sub-index	7
Description	Vector 32 Bits
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	No

Sub-index	8
Description	Vector 64 Bits
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED64
Default value	No

Sub-index	9
Description	Vector Status
Access	Read only

PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED8
Default value	No

Sub-index	10
Description	Vector Next Index
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	No

The detailed description as follows

SI	Name	Details
1	Vector Start Index	The first index in the table
2	Vector End Index	The last index. Will roll to 'start' when override
3	Vector object number	The object used per the mode. See table above
4	Vector control	Control the mode via start, stop and pause option.
5	Vector 8 bits	For 8 bits data
6	Vector 16 bits	For 16 bits data
7	Vector 32 bits	For 32 bits data
8	Vector 64 bits	For 64 bits data
9	Vector Status	Information about the status of the feeding
10	Vector next index	Available index including roll-over.

5.8.12 Object 0x2082: CAN controller status

The object presents CAN controller status: Receive error counter, Transmit error counter, Error bits (TBD)

- Object description:

Index	0x2082
Name	CAN controller status
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	No

5.8.13 Object 0x2090: CAN FW download

The object is for ELMO internal use only, not for user

- Object description:

Index	0x2090
Name	CAN FW download
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Write only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	No

5.8.14 Object 0x20FE: Output Modification Mask

The object sets Output Modification Mask for Digital Outputs 1 – 16 and Extended Outputs 1 - 32.

Functional description

In ECAT communication the "Event" is considered as "ECAT Abort connection or ECAT switch from Operational to another state or from Safe Operational to Initialization state".

In CAN communication the "Event" is considered as "Heartbeat cycle failed or NMT Stop command".

In the case of "Event" the following action should be applied:

- The Digital outputs that masked by the 0x20FE.2 are set to logic 0 (**OP** bit n =0 for all axes)
- The Extended outputs that masked by the 0x20FE.3 are set to logic 0 (**XO**[1] bit n =0).
- The actions is applied one time only on "Event". To restore the High state of Digital Output or Extended output, Master can set **OP** or **XO** [1] bit to 1 again.
- If a bit of **OP** or **XO** was masked and set to 0 on "Event", the change of the Mask after the action will not set the bit to 1 again.
- Object description:

Attributes	0x20FE
Name	Output Modification Mask
Object code	ARRAY
Data type	UNSIGNED32
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub index
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	3
Default value	3

Sub-index	1
Description	Reserved
Access	RW
PDO mapping	No

Non-volatile	No
Value range	UNSIGNED32
Default value	0

Sub-index	2
Description	Digital Output Mask
Access	RW
PDO mapping	No
Non-volatile	Yes
Value range	0 – 0xFFFF
Default value	0

- The sub-index 2 bit placement:

Bits 31 – 16	Bit 15	...	Bit 1	Bit 0
0, reserved	DOUT16 mask	...	DOUT2 mask	DOUT1 mask

Sub-index	3
Description	Extended Output Mask
Access	RW
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0

- The sub-index 3 bit placement:

Bit 31	...	Bit 1	Bit 0
Extended output 32 mask	...	Extended output 2 mask	Extended output 1 mask

- The value of sub-indexes 2 and 3 description:

Value	Description
1	Modify output value to 0
0	Do not modify output value

5.8.15 Object 0x2202: Extended Inputs

The object presents extended inputs similar to **XI[n]** command.

- Object description:

Index	0x2202
Name	Extended inputs
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of supported elements
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	2
Default value	2

Sub-index	1
Description	Extended input value bits 0...31 Sub-index 1 value is dependent on status of drive inputs and sub-index 2: 0x2202.1 = 0x2202.2 XOR Inputs
Access	Read/Write (Write access has no effect, but does not return error)
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	No

Sub-index	2
Description	Logic/Polarity
Access	Read/Write
PDO mapping	TxMap
Non-volatile	Yes

Value range	UNSIGNED32
Default value	0

5.8.16 Object 0x2205: Analog Input Object

This object returns the value of the analog input in A2D bits. The conversion from A2D bits to voltage (V) is:

$$V_{IN}[x] = 0x2205[x] * factor$$

where:

x is the number of the input

factor is as follows:

- Quartet drive inputs 1-4: 0.000373
- Platinum drive input 1: 0.000373
- Platinum drive input 2: 0.006

Typically, Platinum drives include 2 analog inputs and Quartets include 4 analog inputs. However, in some hardware configurations there might be a single analog input. If an analog input is not supported the reply is 0.

- Object description:

Index	0x2205
Name	Analog Input Object
Object code	ARRAY
Data type	INTEGER16
Protocol	CAN, ECAT

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	1 – 4 depends on drive
Default value	Same as “Value range”

Sub-index	1 for Platinum drive 1 – 4 for Quartet
Description	Analog Input 1
Access	Read only

PDO mapping	TxMap
Non-volatile	No
Value range	+/- 28000
Default value	None

Sub-index	2 for Platinum Drive
Description	Analog input 2
Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	-1800 to +1800 A2D ticks
Default value	None



Note:

The value received via object 0x2205 include the analog offset defined via **AS[]** command and the analog dead band defined vis **AD[]** command.

5.8.17 Object 0x2206: 5V DC supply

The object returns the value of 5V DC supply in mVolts.

- Object description:

Index	0x2206
Name	5 V DC Supply
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Access	Read only
PDO mapping	Yes
Non-volatile	No
Value range	INTEGER16
Default value	No

5.8.18 Object 0x2207: Shunt Count

This Object indicates Shunt count value.

- Object description:

Attributes	0x2207
Name	Shunt count
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-Axes	No

- Entry description:

Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	0...2500000
Default value	None

5.8.19 Object 0x2210: Counter Object

The counter object is mainly for internal use and not recommended to be used for any application. The content of this object may be modified without further notice.

Index	0x2210
Name	Counter object
Object code	RECORD
Data type	See sub index
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of supported elements
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	4
Default value	4

Sub-index	1
Description	RT counter
Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	No

Sub-index	2
Description	HS Counter
Access	Read only
PDO mapping	TxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0

Sub-index	3
-----------	---

Description	SF Counter
Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED64
Default value	No

Sub-index	4
Description	SIL Counter
Access	Read only
PDO mapping	TxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0

5.8.20 Object 0x22A1: Extended Outputs

The object presents extended outputs like **XO[n]** command. The Extended Outputs are presented in special hardware configuration only.

- Object description:

Index	0x22A1
Name	Extended Outputs
Object code	Record
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of supported elements
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	3
Default value	3

Sub-index	1
Description	Extended output value bits 0...31 The drive extended outputs status is dependent on values of sub-indexes 1,2 and 3: Outputs = (0x22A1.1 XOR 0x22A1.2) & 0x22A1.3
Access	Read/Write
PDO mapping	RxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0

Sub-index	2
Description	Logic/Polarity, 0: Active low, 1: active high
Access	Read/Write
PDO mapping	RxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0

Sub-index	3
Description	Extended Output Mask, 0: input disabled, 1: input enabled
Access	Read/Write
PDO mapping	RxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	0xFFFFFFFF

Extended output bit (sub index 1) cannot be set if relevant bit in extended output mask is 0.

5.8.21 Object 0x22F0: User Integer Array

This object provides an array of 48 integer numbers for general-purpose. The object is alias of **UI[N]** command.

- Object description:

Index	0x22F0
Name	User Integer array (ECAT)
Object code	ARRAY
Data type	INTEGER32
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	48
Default value	48

Sub-index	1-48
Description	User Integer Array (ECAT)
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.8.22 Object 0x22F1: User Float Array

This object provides an array of 48 integer numbers for general-purpose. The object is alias of **UF[N]** command.

- Object description:

Index	0x22F1
Name	User float array (ECAT)
Object code	ARRAY
Data type	REAL32
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	48
Default value	48

Sub-index	1-48
Description	User Float Array (ECAT)
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	REAL32
Default value	0

5.8.23 Object 0x22F3: SIL R1 Array

This object provides an array of 128 entries of 32 bits integer numbers for general-purpose use. The object is alias of **R1[N]** command.

- Object description:

Index	0x22F3
Name	SIL R1 Array
Object code	ARRAY
Data type	INTEGER32
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	128
Default value	128

Sub-index	1-128
Description	General purpose SIL array
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.8.24 Object 0x22F4: SIL R2 Array

This object provides an array of 64 entries of 64 bits double (floating) numbers for general-purpose use. The object is alias of **R2[N]** command.

- Object description:

Index	0x22F4
Name	SIL R2 Array
Object code	ARRAY
Data type	REAL64
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	64
Default value	64

Sub-index	1-64
Description	General purpose SIL array
Access	Read/Write
PDO mapping	RxMap
Non-volatile	No
Value range	REAL64
Default value	0

5.8.25 Object 0x2300/0x2301: Error Logger Primary/Secondary Safety CPU

Objects 0x2300 & 0x2301 read the Safety Diagnostic Error of the primary and secondary CPUs respectively.

The objects include 8 sub-indices which the errors are logged as detected during the safety process.

Sub-index 0 retrieves the number of Safety Diagnostic Errors detected. The diagnostics logger is reset when the errors are clear via power up or modification of the EtherCAT from PreOP to SafeOP.

Diagnostics errors are divided into groups per the monitored function. The groups are determined according to bits 24-31 and described in the table below.

- Object description:

Attributes	0x2300/1
Name	Error Logger Safety Primary/Secondary CPU
Object code	Array
Data type	UNSIGNED32
Category	Mandatory

- Entry description:

Access	Read Only
PDO mapping	No
Value range	As detailed below
Default value	0

- Data description:

Value	Description
0	Indicate the Number of Safety Diagnostic Error, Max of 8 Errors.
1-8	Each sub-index Indicate the Error accord as detailed below.

5.8.25.1 Error Representation

Each sub-index Error has a 32-bit representation where:

Bit 31 - 24	Bit 23-16	Bit 15 - 0
Error Module Group	Additional group info	Error code

5.8.25.2 The Error Module Groups

The following table describe the module groups and the error.

Value	Description	Recovery
0x00	Safety Infrastructure	General errors related to the flow of the safety tasks
0x01	Safety STO	Failures in the hardware STO path
0x02	Safety Functions	Diagnostics error related to the safety functions
0x03	Safety FSoE state machine	Error due to failure in the FSoE state machine

Value	Description	Recovery
0x04	Safety FSoE Infrastructure	Errors due to FSoE general flow test
0x05	Safety Sensors	Diagnostics faults resulted from the error in safety feedbacks
0x06	Torque Sensors	Diagnostics error from the current sensor feedback
0x07	Safe IOs	Diagnostics errors from the safety inputs \ outputs (including safe brake output)
0x08	General errors	Errors that have no distinctive group

5.8.26 Object 0x2303: Safety Monitoring Error Object

Object 0x2303 retrieves the **monitoring error** code per the safety function that is currently active. The object also retrieves monitoring errors which are related to other functionalities e.g., Enabling device (dead man switch).

Monitoring Errors are errors occurred due to a violation of safety function boundaries e.g., speed threshold violated during SLS monitoring. Monitoring errors are always indicated by the Error Acknowledge bit as well. The reason for the Error Acknowledge is indicated via the 0x2303 object.

The 0x2203 is activated in conjunction with **CV[2]**. **CV[2]** indicates which of the mapped functions in object 0x1B00 was violated. Based on this the relevant index of 0x2203 includes the error detail – up to 32 entries available where each for the relevant function mapped in 0x1B00.

The Object 0x2303 also informs the violation of general functions. These errors are indicated from sub index 33 to 64. When a general function is violated the relevant bit in **CV[8]** is set. Based on the bit that was set, the index of 0x2303 pointed by this bit, informs the relevant error of the violated general function e.g., enabling device time out is informed via 0x2303 sub index 33.

The object is non-axis related.

- Object description:

Attributes	0x2303
Name	Err Ack monitoring description
Object code	Array
Data type	dt_unsigned16
Category	Mandatory

- Entry description:

Access	Read Only
PDO mapping	No
Value range	As detailed below
Default value	0

- Data description:

Value	Description
0	Number of sub-indexes - 32
1 -32	Retrieves the Monitoring Error Code of the corresponding place in the 0x1B00 object (mapped safety functions)
33-64	General logic monitoring errors according to fixed sub-indexes.

5.8.26.1 Example for Safe Function

- The following is mapped to object 0x1B00:

Sub-index	Safety function	Hex view	Description
0x1B00[0]	-	0x20	Number of sub-indexes
0x1B00[1]	-	0x67600108	Safe command word
0x1B00[2]	STO	0x66400001	Safe Torque Off
0x1B00[3]	SS1	0x66500101	Safe Stop 1, instance 1
0x1B00[4]	SOS	0x66680101	Safe Operation Stop, instance 1
⋮	⋮	⋮	⋮

- The SOS Safety function has two Safety Objects:
 - 0x666A: Position window, if motor position is out of this range a monitoring error will be detected.
 - 0x666c: Velocity window, if motor Velocity is out of this range a monitoring error will be detected.
- Assume that a monitoring error has accrued in SOS function due to violation of the position, as a result:
 - STO will be deployed – motor disabled.
 - ErrorAck will be set; (retrieve via **CV[1]**).
 - CV[2]** = 0x00000005. (according to mapping **0x1B00**)
 - The error is now indicated in 2203[5]: **0x0006: "SOS - Position limit violated"**

5.8.26.2 Example for Logic Function Violation

- Enabling logic is enabled via SRA object **0x2E20**, object value is 8.
- If timeout had occurred, the following is indicated (assume both errors are set)
- CV[8]** bit 0 is set to 1
- 0x2303 sub index 33 indicates error 1 and the whole object is as follows:

Sub-index	Safety function/General logic	Monitoring Error Code	description
0x2303[0]	-	-	Number of sub-indexes
0x2303[1]	STO	0x0000	No violation
0x2303[2]	SS1	0x0000	No violation

Sub-index	Safety function/General logic	Monitoring Error Code	description
0x2303[3]	SOS	0x0006	SOS Position limit violated
:	:	:	:
0x2303[33]	Enabling Logic	0x0001	Safe Enabling inputs, Activation override function timer elapse

5.8.27 Object 0x2304: ErrAck Logger Clear

- Object description:

Index	0x2304
Name	ErrAck logger clear
Object code	VAR
Data type	UNSIGNED8
Protocol	ECAT
Multi-axes	No

- Entry description:

Access	Write only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED8
Default value	No

Clear the error acknowledge internal bit and the monitoring error list. Used when the ErrAck bit is not mapped.

Value of 1 is to be used – other values will also clear the error but are reserved for future use and not recommended to use.

5.8.28 Object 0x2F00: User Integer Array

This object provides an array of 48 integer numbers for general-purpose. The object is alias of **UI[N]** command.

- Object description:

Index	0x2F00
Name	User Integer array
Object code	ARRAY
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	48
Default value	48

Sub-index	1-48
Description	User Integer Array
Access	Read/Write
PDO mapping	ECAT: TxMap CAN: RxMap, TxMap
Non-volatile	Yes
Value range	INTEGER32
Default value	0

5.8.29 Object 0x2F01: User Float Array

This object provides an array of 48 integer numbers for general-purpose. The object is alias of **UF[N]** command. The object is non-axis related.

- Object description:

Index	0x2F01
Name	User float array
Object code	ARRAY
Data type	REAL32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	48
Default value	48

Sub-index	1-48
Description	User Float Array
Access	Read/Write
PDO mapping	ECAT: TxMap CAN: RxMap, TxMap
Non-volatile	Yes
Value range	REAL32
Default value	0

5.8.30 Object 0x2F20: tPDO asynchronous events

This object is used to select the events that cause asynchronous tPDOs to be transmitted, with transmission type 254, 255. PDOs with other transmission types are ignored.

Sub-indices 1 to 4 define events for transmitting tPDO1, tPDO2, tPDO3 and tPDO4, respectively. The event definition for the PDO transmission is a bit field, as follows:

Bit	Event
0	Motion complete: MS = 0

Bit	Event
1	Main homing complete: HM[1] = 0
2	Auxiliary homing complete: HY[1] = 0
3	Motor shut down by exception: MO = 0
4	Motor started: MO = 1
5	User program <i>emit</i> command
6	OS interpreter execution complete
7	Reserve
8	Motion Started event
9	External Digital Input Event (for internal use only, not for user)
10...23	Reserved
24	PDO data changed
25	Timer event (for internal use only, not for user)
26	Digital input event in according to 0x60FD
27	Status word event
30	Reserved
31	Binary interpreter command processing complete

Asynchronous tPDOs obey the inhibit time restrictions, as explained in section 5.6.2 Objects 0x1800 - 0x1803: Transmit PDO communication parameter.

- Object description:

Index	0x2F20
Name	PDO events
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indices
Access	Read only
PDO mapping	No
Non-volatile	Yes

Value range	4
Default value	4

Sub-index	1
Description	Events for PDO1 trigger
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0x01000000

Sub-index	2
Description	Events for PDO2 trigger
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0x80000000

Sub-index	3
Description	Events for PDO3 trigger
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0

Sub-index	4
Description	Events for PDO4 trigger
Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32

Default value	0
---------------	---

5.8.31 Object 0x2F21: Emergency Event mask

This object selects events as the cause for transmitting emergency objects.

Updating the object **0x2F21** resets the object **0x1003** sub index 0 and object **0x603F** sub index 0. The following tables define not all possible emergencies, which can be masked by this object

- Object description:

Index	0x2F21
Name	Emergency event mask
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN
Multi-axes	No

- Entry description:

Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED16
Default value	0xFFFF (all emergencies enabled)

The value of the object, as presented in following table, is a bit field, where every bit is related to defined group of events as the cause for transmitting emergency objects, meaning 0 – emergency is masked, 1- emergency can be transmitted (enabled).

Refer also to section 16.1 Emergency Codes

0x2F21 Bit	Mask name	Emergency messages EMCY Error Code (Hex) below enabled	Details and Resolution
0	Reserved for CAN message lost	8110	HW or SW buffer is overflowed. Check the rate of the messages for Sync, NMT or rPDO.
1	Reserved		
2	Reserved for Attempt to access non configured rPDO	8210	rPDO received is not mapped or contains incorrect number of bytes. Check the rPDO mapping and the rPDO sent by the host.

0x2F21 Bit	Mask name	Emergency messages EMCY Error Code (Hex) below enabled	Details and Resolution
			Check if number of rPDO bytes equals to the bytes that were mapped
3	Heartbeat event	8130	Consumer heartbeat time elapsed. Check if producer message was sent in time. Check the consumer time in object 0x1016.
4-6	reserved		
7	Motor shut down by fault	All emergencies related to motor fault	See 16.1 Emergency Codes
8	rPDO failed	6300	Object mapped to an rPDO returned an error during interpretation. If the rPDO is used as a motion set-point and the motion failed to perform. Check if data of rPDO is correct with respect to the mapped object. Check the reason for the failure as indicated in "Elmo Error Code" (byte 3 aliases to the EC command). Optional reasons are that parameter is out of range, operating mode is not supported, error mapping in a progress, motor failed to start, etc.
9	DS 402 IP Emergency	FF02	IP mode failure: Underflow: Interpolated periods defined via object 0x2F75 is elapsed Interpolation queue full (overflow): the new set point is received before previous set point was fully processed. Bad sub-index Wrong set-points order in case of IP sub mode is -1. In case of underflow: Check that a new set-point (object 0x60C1) arrived prior to the time defined by 0x2F75. Check master Sync or rPDO message rate: it might be lower than the period time (0x60C2). In case of overflow:

0x2F21 Bit	Mask name	Emergency messages EMCY Error Code (Hex) below enabled	Details and Resolution
			Check master Sync or rPDO message rate: it might be higher than the period time (0x60C2). In case of bad index: Check that the mapped set-point object (sub index of 0x60C1) is relevant to the IP sub-mode. In case of wrong set-point order: Check that the order of set points 0x60C1.1 and 0x60C1.2 are according to the IP sub-mode definition.
10	Recovered from bus off	8140	EMCY is sent after the drive recovered from bus-off state Check the physical connection of the CAN Check that the baud rate of all nodes is the same as the master's Check the CAN 120 Ω terminators
11	Cannot enable motor	FF10	An attempted to start the motor via Binary Interpreter or DS-402 control word (0x6040) failed. See ?????? for indication and possible resolution.
12	Motor disabled local	FF30	Motor disabled immediately by interpreter (MO=0) or user program (MO=0).

5.8.32 Object 0x2F30: Socket's sensor internal temperature

Index	0x2F30
Name	Socket's sensor internal temperature
Object code	ARRAY
Data type	INT16
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
-----------	---

Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Value range	8
Default value	8

Sub-index	1-8
Description	Sensor internal temperature
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	INT16
Default value	0

The Sub indexes are per the socket that the serial sensor is connect.

Temperature readout is the raw data read from the sensor and is based on the sensor's manufacturer data.

Supports:

- Mitsubishi sensors
- Heidenhain EnDAT sensors.

The object is alias to **TI[5]** command.

5.8.33 Object 0x2F31: Socket's sensor external temperature

Index	0x2F31
Name	Socket's sensor external temperature
Object code	ARRAY
Data type	INT16
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Value range	8

Default value	8
---------------	---

Sub-index	1-8
Description	Sensor external temperature
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	INT16
Default value	0

The Sub indexes are per the socket that the serial sensor is connect.

Temperature readout is the raw data read from the sensor and is based on the sensor's manufacturer data.

Supports:

- Heidenhain EnDAT sensors.

Object is alias to **TI[6]** command.

5.8.34 Object 0x2FE2: Time Capture Tag

The object provides the time in microseconds from the Time Capture event to the last TS interrupt. The object is calculated every high-speed cycle (**HS**). Each sub index of the object (1 to 4) presents a different Time Capture module. The optional source signals for the time capture event are Port A Index, Port B Index, Port C Index, digital input1 to 16 are defined via **GT[1] – GT[4]** command. Note that the source for the event differs between Platinum drive and Quartet drive.

- Object description:

Index	0x2FE2
Name	Time tag
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	CONST
PDO mapping	No
Non-volatile	Yes
Value range	4
Default value	4

Sub-index	1-4
Description	Time tag in microseconds for Time Capture 1 - 4
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.8.35 Object 0x2FE3: Time Event Counter

This object provides time capture event counter for Time Capture modules 1 – 4. Source signal for event (Port A Index, Port B Index, Port C Index, DIN 1 – DIN 6) is defined by **GT[1] – GT[4]** command. Refer to Platinum Administrative Guide for Time Capture configuration.

Object description:

Index	0x2FE3
Name	Time event counter
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Value range	4
Default value	4

Sub-index	1-4
Description	Time event counter for Time Capture 1 - 4
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.8.36 Object 0x2FE4 – 0x2FE6: Socket position 1 - Socket position 3

Index	0x2FE4 – 0x2FE6
Name	0x2FE4: Socket position 1 (e.g. Main Position), 0x2FE5: Socket position 2 (e.g. Aux Position 1), 0x2FE6: Socket position 3 (e.g. Aux position 2)
Object code	ARRAY
Data type	REAL64
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported

Access	CONST
PDO mapping	No
Value range	8
Default value	8

Sub-index	1-8
Description	Socket number which the position is read from
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	REAL64
Default value	0

Each socket can handle up to 3 positions. The position indication can also be named as Main Position and Auxiliary position1 & Auxiliary position 2.

5.8.37 Object 0x2FE8 – 0x2FEA: Socket velocity 1 - Socket velocity 3

Index	0x2FE8 – 0x2FEA
Name	0x2FE8: Socket velocity 1 (e.g., Main sensor velocity), 0x2FE9: Socket position 2 (e.g., Aux velocity 1), 0x2FEA: Socket position 3 (e.g., Aux velocity 2)
Object code	ARRAY
Data type	REAL32
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Value range	8
Default value	8

Sub-index	1-8
Description	Socket number which the velocity is read from
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	REAL32
Default value	0

Each socket can handle up to 3 velocities readout. The velocity indication can also be named as Main velocity and Auxiliary velocity & Auxiliary velocity 2.

5.8.38 Object 0x2FEC: Analog Sensor Amplitude

Index	0x2FEC
Name	Analog Sensor amplitude
Object code	ARRAY
Data type	UNSIGNED32
Protocol	ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	CONST
PDO mapping	No
Value range	8
Default value	8

Sub-index	1-8
Description	Analog sensor amplitude
Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	REAL32
Default value	0

Indicate the amplitude of the analog sensor read out.

The sensor amplitude is calculated as: $\sqrt{\sin^2 + \cos^2}$

Amplitude depends on the sensor and the A2D used for the sensing.

Max and minimum amplitude can be monitored via **WT[1]** (defined via **S#[13]** & **S#[14]**).

The drive can report an error if the amplitude is below the **S#[11]** and above the **S#[12]**.

5.8.39 Object 0x3000: Elmo Commands

The Elmo Command object is a method that gives the user access to (almost) any Elmo '2 letters' command. - refer to the Command Reference (*MAN-P-CommandRef*). The '2 letters' command is a list of commands from 'A0 to ZZ', where the 0x3000 object is **A0** (**A1** is 0x3001, **A2** 0x3002 etc.) and 0x33A7 is **ZZ**. The Command Reference manual includes the alias objects of each command.

The 0x3000 commands are axis dependent and an offset of 0x800 should be used to address the other axes. For example, axis 2 starts with 0x3800 which is the alias to AX2.**A0** and 0x4000 is AX3.**A0** etc.

Commands that have array are represented by the sub index of the relevant 0x3000 object. For example, **UI[]** command is alias of 0x32E2, to access **UI[16]** the 0x32E2 sub index 16 should be used.

Note that not all commands can be addressed via the 0x3000. In general, visible string commands and commands that include uploading and downloading procedure are not accessible. Refer to the Command reference manual (*MAN-P-CommandRef*).

Index	0x3000
Name	Elmo Command
Object code	ARRAY
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Attribute	Value
Access	Read/write
PDO mapping	No
Non-volatile	No
Value range	NA
Default value	No

The attribute of the 0x3000 is as described in the alias command.

5.8.40 Object 0x3605: General Reference

The reference general purpose object can be used for various tasks as defined in the table below.

- Object description:

Index	0x3605
Name	General Reference
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Attribute	Value
Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	$-2^{31} \dots (2^{31}-1)$
Default value	No

The object functionality can be defined by **GS[12]**:

GS[12] Value	Details
0	Position/velocity reference, when connected to socket (sensor ID=29). The reference object allows a direct access to a socket. Sockets are virtual entities which allow access to feedback functions. These functions include for example speed calculations, commutation calculations, position, filters etc. The socket can also be used as a reference for follower \ ECAM mode or virtual profiler which can be used as emulation to other drives
1	Velocity integral drain
2	Reserved
3	Asymmetric torque/current limits. The Low 16 bits for minimum limit, and the high 16 bits for the maximum limit. Measured in 1/1000 of the rated torque.

5.8.41 Object 0x3606: Home On Touch Probe

When the object is set, the last position captured via touch probe 1 will be set as the “Home” position of the main position feedback. The Touch Probe 1 captured position indicated in Object 0x607C.0 will be used as the home offset value.

Position adjustment completion is indicated by success returning of the object via SDO. This object is not map-able.

- Object description:

Index	0x3606
Name	Home on touch probe
Object code	Variable
Data type	UNSIGNED16
Protocol	CAN, ECAT
Category	Optional

- Entry description:

Attribute	Value
Sub-index	0
Access	RW
PDO mapping	No
Value range	0,1
Default value	Not defined

The value definition is:

Value	Details
0	Adjust the position of the touch probe 1 socket according to the touch probe 1 rising edge value and the homing offset (object 0x607C)
1	Adjust the position of the touch probe 1 socket according to the touch probe 1 falling edge value and the homing offset (object 0x607C)

5.8.42 Object 0x3607: Elmo Status Register Array

The object is the alias for the Elmo Status Register and Extended Status Register. Sub-index 1 is alias of **SR[1]** command and **Object 0x1002**, sub-index 2 is alias of **SR[2]** command.

- Object description:

Index	0x3607
Name	Elmo Status Register Array
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of supported elements
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED8
Default value	2

Sub-index	1
Description	Elmo Status Register
Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	None

Sub-index	2
Description	Extended Status Register
Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32

Default value	None
---------------	------

5.8.43 Object 0x360D: Vector Mode Table

The vector mode object is used in conjunction with the auto increment table access done using object 0x2065.

The vector mode allows a direct access to tables via PDO with auto increment mode and auto roll-over. The 0x360D indicates the next available index in the tables and modes that are used per the following table. In these modes the drive uses the next index according to the mode and the host that feeds the table via the PDO can know when the next index is available.

Index	0x3620D
Name	Vector mode table
Object code	RECORD
Data type	UNSIGNED16
Protocol	CAN, ECAT
Category	Optional

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Entry category	Mandatory
Access	Read only
PDO mapping	No
Value range	3
Default value	3

Sub-index	1
Description	Output Compare 1 – next index
Entry category	Mandatory
Access	Read only
PDO mapping	TxMap
Value range	0-1023
Default value	0

Sub-index	2
Description	Output Compare 2 – next index
Entry category	Mandatory

Access	Read only
PDO mapping	TxMap
Value range	0-1023
Default value	0

Sub-index	3
Description	ECAM table – next index
Entry category	Mandatory
Access	Read only
PDO mapping	TxMap
Value range	0-2047
Default value	0

5.8.44 Object 0x3610: Drive Temperature in °C

The object returns the value of drive temperature in degree of Celsius.

- Object description:

Index	0x3610
Name	Drive Temperature in C
Object code	VAR
Data type	REAL32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	-40.0 ... 120.0
Default value	none

5.8.45 Object 0x3617: Power calculation

The object returns the instantaneous calculated power supplied by the drive. The power is calculated every HS cycle (refer to HS command). The value is in Watts.

Object alias: **WF[108]** command.



Note:

Accumulation of the power does not guarantee accurate total power consumption. It is recommended to use the EAS recorder under “Analysis \ Power Output [Watt]”.

- Object description:

Index	0x3617
Name	Power calculation
Object code	VAR
Data type	REAL32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	0 – 65,000
Default value	none

5.8.46 Object 0x3618: Max Thermal Limit - Percentage

The object calculates the I2T thermal state in %. When the drive is under current saturation the limit is 100%.

- Object description:

Index	0x3618
Name	Percentage of max thermal limit
Object code	VAR
Data type	UNIT8
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	current saturation in % (Ic/Ip)
Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	0-100
Default value	0

5.8.47 Object 0x3620: Home On Block Limit Parameters

The “Home on block” is Elmo proprietary homing methods similar to PLCopen Step block mode, allowing to determine home position “against the wall” where the motor is blocked by a physical limit. The activated via the DS402 Homing mode (0x6060.0 = 6) with parameters that required this special mode.

Detailed description of the sub-indices is in the following paragraph.

- Object description:

Index	0x3620
Name	Home on Block Limit Parameters
Object code	RECORD
Data type	Home On Block Limits, Object 0x45
Protocol	CAN, ECAT
Category	Optional

- Entry description:

Sub-index	0
Description	Number of supported elements

Entry category	Mandatory
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED8
Default value	5

Sub-index	1
Description	Torque limit - Maximum torque
Entry category	Mandatory
Access	Read/Write
PDO mapping	No
Value range	0...50000
Default value	0

Sub-index	2
Description	Time Limit
Entry category	Mandatory
Access	Read/Write
PDO mapping	No
Value range	0...107374182
Default value	0

Sub-index	3
Description	Distance limit
Entry category	Mandatory
Access	Read/Write
PDO mapping	No
Value range	0...2147483647
Default value	0

Sub-index	4
Description	Detection Velocity Limit
Entry category	Mandatory
Access	Read/Write

PDO mapping	No
Value range	0...2147483647
Default value	0

Sub-index	5
Description	Detection Velocity Time Limit
Entry category	Mandatory
Access	Read/Write
PDO mapping	No
Value range	0...107374182
Default value	0

Sub index 1, Torque limit - Maximum torque.

The torque limit value set in object 0x3620 sub index 1 is relative to the rated torque and presented in thousands of rated torque. When set to 1000, it means that the full rated torque is applied. Rated torque is defined by object 0x6076 and reflects the drive torque in mNm. Since the relation between torque and current is linear, we will consider the torque units as mA in the following descriptions.

Relations between Torque limit 0x3620.1 and torque limit in mA are:

$$\text{Torque Limit (mA)} = (\text{Torque Limit 0x3620.1} * \text{Rated Torque 0x6076}) / 1000$$

$$\text{Torque Limit 0x3620.1} = (\text{Torque Limit (mA)} * 1000) / \text{Rated Torque 0x6076}$$

Example for conversion from mA to rated torque:

Suppose, the rated torque is set 2000 mA and the user wants to set the limit to 600mA. The Torque Limit object shall be set:

$$\text{Torque Limit 0x3620.1} = 600 * 1000 / 2000 = 300$$

Setting 0x3620 sub index 1 to zero disables its functionality, i.e. actual torque is considered as exceeding the torque limit at any time.

Sub index 2, Time limit

if the Step Block condition is not met in the Time Limit, an error is issued. Time limit is set in msec.

The drive operates with resolution of 250 microseconds. Max allowed value is: 107374182 msec.

Setting 0x3620 sub index 2 to zero disables time limit functionality, meaning: "do not check the time limit and do not issue an error".

Sub index 3, Distance limit

if the Step Block conditions are not met within a Distance Limit travel, an error is issued.

Distance limit is defined in user units. The accuracy of the distance limit is dependent on the velocity of the homing. Drive calculates the distance in TS*HS uSec resolution.

Setting 0x3620 sub index 3 to zero disables distance limit functionality, meaning "do not check the distance limit and do not issue an error".

Sub index 4, Detection Velocity Limit

If the torque limit is reached and the actual velocity falls below the value of the Detection Velocity limit 0x3620 sub index 4 for at least the Detection Time Limit 0x3620 sub index 5, then homing is considered as attained (drive sets bit 12 of the SW), and when the velocity falls to zero, the drive will set bit 10 (TR) of the SW.

Detection Velocity Limit is defined in user units. Max allowed value is 2147483647

Sub index 5, Detection Time Limit

Detection time limit is presented in milliseconds. Max allowed value is 107374182.

If the Detection Time Limit is set to zero, the home attained bit will be set immediately when the torque limit (0x3620.1) is exceeded and the velocity is lower than Detection Velocity Limit

5.8.48 Object 0x3625: Gantry Yaw Offset

The Gantry Yaw offset allows to set a constant offset to overcome non-linear differences between the master and slave such as backlash gaps. The value is given in counts.

Index	0x3625
Name	Gantry yaw offset
Object code	VAR
Data type	INT32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Sets offset to the gantry Yaw position
Access	Read / Write
PDO mapping	TxMap
Non-volatile	No
Value range	INT32
Default value	0

5.8.49 Object 0x3630: Safety Position Calibration Start

Safety absolute position requires to calibrate the safety position to a predefined position which is determined by a digital input – reference signal. The calibration is a non-safe procedure that is initiated when bit 0 of **Object 0x3630** is set to '1'. This non-safe object triggers the initiation of the safety calibration procedure. The safety procedure will then detect the rising edge of the selected reference signals and once sensed will set the predefined absolute position (**Object 0x3773**) to the safety position (**Object 0x6611**). After the calibration is completed, the host can reset the bit to 0.

Index	0x3630
Name	Safety position calibration start
Object code	VAR
Data type	UINT8
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Indicates that an absolute safety position calibration is active.
Access	Read / Write
PDO mapping	No
Non-volatile	No
Value range	0,1
Default value	0

5.8.50 Object 0x3640: Sample VQ

The object retrieves the actual active voltage in PWM units.

The max value can be retrieved via **WF[4]** (object 0x3327.4) and depends on the sampling time (**TS**) and PWM multiplication (**XP[2]**)

Index	0x3640
Name	Sample VQ
Object code	VAR
Data type	REAL32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
-----------	---

Description	
Access	Read / Write
PDO mapping	TxMap
Non-volatile	No
Value range	REAL32
Default value	0

5.8.51 Object 0x3675 – Extrapolation Timeout

The object defines the number of extrapolation cycles that will be performed by the drive in situations that the motion set point from the host fails to arrive in time. The object is relevant to all time dependent motion modes including IP mode, Cyclic synchronous position, velocity & torque *CSP, CSV * CST) and cyclic synchronous torque with commutation angle (CSTCA). When the extrapolation period elapses, the drive stops under Quick Stop conditions. The extrapolation time is based on the interpolation time or SYNC period in which the motion is interpolated.

- Object description:

Index	0x3675
Name	Extrapolation timeout
Object code	VAR
Data type	INT16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Extrapolation cycles for time dependent motion modes
Access	Read / Write
PDO mapping	No
Non-volatile	No
Value range	1-32767
Default value	0

5.8.52 Object 0x36E0: Gain Scheduling Manual Index

This object allows manual selection of the gain scheduling index. Two types of scheduling can be selected simultaneously. The object's 8 low bits (LSB) are used as an index for one of the selected gain scheduling tables, while the high bits (MSB) are used to select an index from another gain scheduling table.

By setting the relevant **GS[]**, according to the desired scheduling, to either 67 or 68, the Low or the High bytes are used respectively. The optional **GS[]** can be:

GS[2] for control loop (**KI[2]**, **KP[2]**, and **KP[3]**) scheduling

GS[16] and **GS[17]** for speed advance filter 1 and filter 2 scheduling

GS[18] for position advanced filter scheduling

- Object description:

Index	0x36E0
Name	Gain Scheduling Manual Index
Object code	Variable
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Attribute	Value
Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	0...65535
Default value	0

5.8.53 Object 0x36E4: Additional Position Actual value

This object provides the additional position actual values. The position value is given in counts.

- Object description:

Attributes	0x36E4
Name	Additional position actual value
Object code	ARRAY
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	28
Default value	28

Sub-index	1...8
Description	Socket 1-8 main HW position
Access	RO
PDO mapping	No
Non-volatile	No
Value range	INTEGER32
Default value	No

Sub-index	11...18
Description	Socket 1-8 Auxiliary 1 HW position
Access	RO
PDO mapping	No
Non-volatile	No
Value range	INTEGER32
Default value	No

Sub-index	21...28
Description	Socket 1-8 Auxiliary 2 HW position
Entry category	Mandatory
Access	RO
PDO mapping	No
Non-volatile	No
Value range	INTEGER32
Default value	No

5.8.54 Object 0x36E5: Additional Velocity Actual Value

This object provides the additional velocity actual values. The velocity value is given in counts/sec.

- Object description:

Attributes	0x60E5
Name	Additional velocity actual value
Object code	ARRAY
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of entries
Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	28
Default value	28

Sub-index	1...8
Description	Socket 1-8 main sensor velocity
Access	RO
PDO mapping	No
Non-volatile	No
Value range	INTEGER32

Default value	None
---------------	------

Sub-index	11...18
Description	Socket 1-8 Auxiliary 1 sensor velocity
Access	RO
PDO mapping	No
Non-volatile	No
Value range	INTEGER32
Default value	None

Sub-index	21...28
Description	Socket 1-8 Auxiliary 2 sensor velocity
Access	RO
PDO mapping	No
Non-volatile	No
Value range	INTEGER32
Default value	None

5.8.55 Object 0x36FD Digital Inputs

The object presents digital inputs logical state

- Object description:

Index	0x36FD
Name	Digital Inputs
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/Write
PDO mapping	No
Non-volatile	No
Value range	Bit field, range is not applicable
Default value	None

The object indicates the digital inputs status on read access. Bit placement is presented in the tab below (1: means logical active state, 0: logical not active):

31...16	15...0
Logical state of digital inputs 1 to 16	Reserve, 0

The object is also indicated by **OJ.36FD[0]** command.

If any digital input was defined as sticky general purpose digital input (see **IL[]** command description) writing '1' to this input clears the bit.

The digital inputs can also be read via **IP** command.

Bits 16...31 of the object are aliases to bits 16...31 of 0x60FD with optionally write access.

5.9 DS-402

5.9.1 Object 0x6007: Abort Connection Option Code

This object defines the motor control behavior when the following events occurred:

- CAN bus-off
- CAN heartbeat
- CAN NMT stopped state entered via NMT command
- ECAT Watchdog event
- ECAT Abort connection or ECAT switch from Operational to another state or from Safe Operational to Initialization state
- Object description:

Index	0x6007
Name	Abort connection option code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Access	Read/Write
PDO mapping	No
Non-volatile	Yes
Value range	INTEGER16
Default value	1

- Data description:
Command details are found in the Platinum Command Reference Manual.

Option Code	Description	Details
0	No action	
1	Set Fault Signal	The drive behavior is defined by <i>Fault reaction option code</i> , object 0x605E: Any way motor is off (MO=0) Motor failure code is 0x800 (MF command) Emergency message will be sent (if not masked, see Object 0x2F21) with error code 0x8130 The failure is reported and possibly activates an AUTOERR routine similar to other failures (MF command)

Option Code	Description	Details
2	Device control command Disable voltage	Is not supported
3	Device control command <i>Quick stop</i>	The drive behavior is defined by <i>Quick stop option code</i>, object 0x605A: Motor stops with quick stop deceleration, object 0x6085 or with profiler deceleration, Object 0x6084 according The motor switched off or remains on the drive enters SWITCH ON DISABLE state or QUICK STOP state



Note:

Regardless of actual 0x6007 value, the drive performs action defined by **Object 0x20FE**

5.9.2 Object 0x603F: Error code

This object captures the code of the last fault that occurred in the drive and initiates EMCY message transmission. For CAN drive it corresponds to the value of the lower 16 bits of object 0x1003 sub index 1, pre-defined error field.

The optional Error codes are described in the Emergency section.

- Object description:

Attributes	0x603F
Name	Error code
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	None

5.9.3 Object 0x6040: Controlword

Controlword contains bits for:

- Controlling the servo state
- Controlling operating modes
- Manufacturer-specific options
- Object description:

Index	0x6040
Name	Controlword
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	None

Data description:

15	-	10	9	8	7	6	-	4	3	2	1	0
Reserved		Operation mode specific (Profile position mode)		Halt	Fault reset	Operation mode specific (Profile position mode)			Enable operation	Quick stop	Enable voltage	Switch on
MSB												LSB

5.9.3.1 Bits 0 – 3 and 7

Device control commands are triggered by the following bit patterns in the ControlWord:

Command	Bits of the ControlWord					Transitions
	7	3	2	1	0	
	Fault Reset	Enable Operation	Quick Stop	Enable Voltage	Switch On	
Shutdown	0	X	1	1	0	2, 6, 8
Switch ON	0	0	1	1	1	3

Command	Bits of the ControlWord					Transitions
Switch ON and Enable Operation	0	1	1	1	1	3+4 Automatic transition to Enable operation state after executing SWITCHED ON state functionality
Disable Voltage	0	X	X	0	X	7, 9, 10, 12
Quick Stop	0	X	0	1	X	7, 10, 11
Disable Operation	0	0	1	1	1	5
Enable Operation	0	1	1	1	1	4, 16
Fault Reset		X	X	X	X	15

Bits marked with X are not relevant.

5.9.3.2 Bits 4, 5, 6 and 9

These bits are operation-mode specific. Their description is found in the chapter about the special mode.
(CSTCA mode is like CST mode)

Operation Mode	Bits of the ControlWord			
	4	5	6	9
Profile Position Mode	New set-point	Change set immediately	relative	Blending mode
Profile Velocity Mode	Reserved	Reserved	Reserved	Reserved
Profile Torque Mode	Reserved	Reserved	Reserved	Reserved
Homing Mode	Homing operation start	Reserved	Reserved	Reserved
Interpolation Position Mode	Enable Interpolation	Reserved	Reserved	Reserved
CSP	Reserved	Reserved	Reserved	Reserved
CSV	Reserved	Reserved	Reserved	Reserved
CST	Reserved	Reserved	Reserved	Reserved

5.9.3.3 Bit 8

This bit enables the Halt function in all modes: pp, pv, tq, hm, ip, csp, csv, cst, and cstca.

5.9.3.4 Bits 10 - 15

These bits are reserved.

5.9.4 Object 0x6041: Statusword

The Statusword indicates the present state of the drive. No bits are latched. The Statusword contains bits for:

- The current drive state
- The operating state of the mode
- Manufacturer-specific options
- Object description:

Index	0x6041
Name	Statusword
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	No

- Data description:

Bit	Description
0	Ready to switch on
1	Switched on
2	Operation enabled
3	Fault
4	Voltage enabled
5	Quick stop
6	Switch on disabled
7	Warning
8	Manufacturer specific, reserved, always set to 0
9	Remote
10	Target reached
11	Internal limit active
12 - 13	Operation mode specific

Bit	Description
14 - 15	Manufacturer specific, reserved, always set to 0

5.9.4.1 Bits 0 - 3, 5 and 6

The following bits indicate the status of the device. Note that x may be of any digit value:

Value (binary)	State
xxxx xxxx x0xx 0000	Not ready to switch on
xxxx xxxx x1xx 0000	Switch on disabled
xxxx xxxx x01x 0001	Ready to switch on
xxxx xxxx x01x 0011	Switch on
xxxx xxxx x01x 0111	Operation enabled
xxxx xxxx x00x 0111	Quick stop active
xxxx xxxx x0xx 1111	Fault reaction active
xxxx xxxx x0xx 1000	Fault

5.9.4.2 Bit 4: Voltage Enabled

Voltage higher than the "under voltage" threshold is applied to the drive when this bit is set to 1. Under voltage is denoted via the **WS[26]** command.

5.9.4.3 Bit 5: Quick Stop

When reset to 0, this bit indicates that the drive is reacting to a Quick Stop request or already in Quick Stop state. Bits 0, 1 and 2 of the Statusword must be set to 1.

5.9.4.1 Bit 7: Warning

A Warning bit set in Statusword in the following two cases:

- Any predefined parameter threshold is violated (refer to the **WT** command description in the Platinum Drive Command Reference):
 - WT[1]** bit 2 is set: Under voltage threshold is exceeded.
 - WT[1]** bit 3 is set: Over voltage threshold is exceeded.
 - WT[1]** bit 4 is set: Analog sensor low voltage threshold is exceeded.
 - WT[1]** bit 5 is set: Temperature threshold is exceeded.
 - WT[1]** bit 9 is set: Analog sensor high voltage threshold is exceeded.
- Any bit in **WE** (refer to the **WE** command description in the Platinum Drive Command Reference) is set to 1 when the related **AE[4]** bit is set to 1 (meaning: treat fault as warning) and related fault event occurs. In this case related **MF** bit should not be set.

5.9.4.2 Bit 8

This bit is reserved for the manufacturer. It is not used and is set to 0.

5.9.4.3 Bit 9: Remote

If bit 9 is set to 1, the drive is in Remote mode, if it is set to 0 – in local mode. Refer to section "Remote and Local modes" ##

5.9.4.4 Bit 10: Target Reached

Bit 10 is set by the drive to indicate that a set-point has been reached. The set-point is dependent on the operating mode. Refer to the relevant chapter per the required operation mode. The change of a target value by software alters this bit.

If the quick stop option code is 5, 6 or 7, this bit is set when the quick stop operation is finished, and the drive is halted.

If a Halt occurs and the drive has halted, this bit is also set.

If the motor axis was stopped by FLS, RLS or STOP switch the bit is reset.

5.9.4.5 Bit 11: Internal Limit Active

The drive sets this bit to indicate that an internal limitation is active. Internal limit includes the following: software position limit, FLS, RLS, and STOP switch.

5.9.4.6 Bits 12 and 13

These bits are operation-mode specific. For detailed information refer to the specific mode.

5.9.4.7 Bits 14 and 15

These bits are reserved. They are not used and are set to 0.

5.9.5 Object 0x605A: Quick Stop Option Code

This object determines the action to take if the Quick Stop function is executed.

- Object description:

Index	0x605A
Name	Quick stop option code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	yes
Value range	see data description
Default value	2

- Data description:

Value	Description
-32,768...-1	Manufacturer specific, Not supported
0	Disable drive function, go to SWITCH ON DISABLED state
1	Slow down on slow-down ramp and then disable the drive function, go to SWITCH ON DISABLED state
2	Slow down on quick-stop ramp and then disable the drive function, go to SWITCH ON DISABLED state
3	Slow down on current limit and transit into SWITCH ON DISABLED state
4	Not supported
5	Slow down on slow-down ramp and stay in QUICK STOP state
6	Slow down on quick stop ramp and stay in QUICK STOP state
7	Slow down on current limit and stay in QUICK STOP state
8...32,767	Not supported

An attempt to set an unsupported value causes the transmission of abort code 0609 0030: Value exceeded.

5.9.6 Object 0x605B: Shutdown Option Code

This object determines which action to take in the following transition:

From OPERATION ENABLED to READY TO SWITCH ON.

- Object description:

Index	0x605B
Name	Shutdown option code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes
Value range	See data description
Default value	0

- Data description:

Value	Description
-32,768...-1	Manufacturer specific, not supported
0	Disable drive function
1	Slow down on slow-down ramp; disable drive function
2...32,767	Not supported



Note:

An attempt to set an unsupported value causes the transmission of abort code 0609 0030, value exceeded.

5.9.7 Object 0x605C: Disable operation option code

This object determines which action to take in the following transition:
from OPERATION ENABLED to SWITCHED ON.

- Object description:

Index	0x605C
Name	Disable operation option code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes
Value range	See data description
Default value	0

- Data description:

Value	Description
-32,768...-1	Manufacturer specific, not supported
0	Disable drive function
1	Slow down on slow-down ramp and then disable drive function
2...32,767	Reserved, not supported



Note:

An attempt to set an unsupported value causes the transmission of abort code 0609 0030: Value exceeded.

5.9.8 Object 0x605D: Halt option code

This object determines which action to take if bit 8 (halt) in the ControlWord is active.

- Object description:

Index	0x605D
Name	Halt option code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes
Value range	See data description
Default value	1

- Data description:

Value	Description
-32,768...-1	Manufacturer specific, not supported
0	Reserved, not supported
1	Slow down on slow down ramp and stay in Operation Enabled state
2	Slow down on quick stop ramp and stay in Operation Enabled state
3	Slow down on current limit and stay in Operation Enabled state

5.9.9 Object 0x605E: Fault Reaction Option Code

This object indicates what motion will be performed in case of a drive fault.

The drive enters a fault state when a critical violation occurs during Operation Enabled state (servo is enabled). In any situation of a fault state, the drive will transmit an EMCY message and will inform the Fault state in Object 0x6041.

The following violations are unique:

Physical violation:

The drive disables the servo immediately.

Typical cases: Over Voltage, Temperature, Loss of feedback (or any feedback fault), Safe Torque Off switch was set etc.

Operational violation:

The drive performs the reaction as requested via object 0x605E. The optional reactions are described in the table below. Typical cases: Speed tracking error, Feedback Position is out of position limits etc.

For more details refer to Emergency chapter in this Guide and MF command in the Command reference manual.

- Object description:

Index	0x605E
Name	Fault reaction option code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes
Value range	See data description
Default value	2

- Data description:

Value	Description
-32,768...-1	Reserved
0	Disable drive, motor free to rotate
1	Slow down on slow-down-ramp to velocity 0, and then disable-motor
2	Slow down on quick-stop ramp to velocity 0, and then disable motor
3	Slow down on current limit to velocity 0, and then disable motor

Value	Description
4...32,767	Reserved

The following should be noted:

An attempt to set an unsupported value causes the transmission of abort code 0609 0030: Value exceeded.

All amplifier faults are considered fatal. When a fatal fault occurs, the drive is no longer able to control the motor, requiring that the drive be switched-off immediately.

Slow down ramp is defined by object 0x6084: Profile Deceleration (DC command) is the alias command for this object.

5.9.10 Object 0x6060: Modes of operation

This Object indicates the next requested operation mode. The actual operation mode is denoted by the Object 0x6061.

An attempt to access an unsupported mode results-in abort code 0609 0030: Parameter value is out of range.

- Object description:

Attributes	0x6060
Name	Modes of operation
Object code	VAR
Data type	INTEGER8
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	As detailed below
Default value	0

- Data description:

Value	Description
*All values indicated with a * are not supported modes and cannot be written by the user	
-128...-4	Reserved, not supported*
-3	CAN encoder mode. Read access of 0x6060 and 0x6061 returns 7
-2, -1	Reserved, not supported*
0	No profiler is supported, motor cannot be enabled
1	Profile position mode

Value	Description
2	Velocity, not supported*
3	Profiled velocity mode
4	Torque profiled mode
5	Reserved, not supported*
6	Homing mode
7	Interpolated position mode (available in CANopen only)
8	Cyclic synchronous position
9	Cyclic synchronous velocity
10	Cyclic synchronous torque
11	Cyclic synchronous torque with commutation angle
12...127	Reserved, not supported*

5.9.11 Object 0x6061: Modes of Operation Display

This Object shows the current mode of operation. The meaning of the returned value corresponds to that of the modes of operation option code (**Object 0x6060**).

- Object description:

Attributes	0x6061
Name	Modes of operation display
Object code	VAR
Data type	INTEGER8
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	refer to table in Object 0x6060
Default value	0

- Data description:

Similar to **Object 0x6060**, modes of operation.

The actual mode is reflected in the modes of operation display (**Object 0x6061**), and not in modes of operation (**Object 0x6060**).

5.9.12 Object 0x6062: Position Demand Value

The value of this Object is taken from the internal position command and is given in position units after being converted by position factor.

- Object description:

Attributes	0x6062
Name	Position demand value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.13 Object 0x6063: Position Actual Internal Value

The actual value of the position measurement device is one of the two input values of the closed loop position control. The data unit is defined as increments.

- Object description:

Attributes	0x6063
Name	Position actual value - increments
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.14 Object 0x6064: Position Actual Value

This Object represents the actual value of the position sensor, in user-defined units. It returns the motor position feedback (PU command) value.

- Object description:

Attributes	0x6064
Name	Position actual value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.15 Object 0x6065: Following Error Window

This Object defines a range of tolerated position values symmetrical to the position demand value. If the position actual value is outside the following error window during a defined following-error-time-out, a following error bit is set in Statusword in operating modes csp, ip, pp.

If bit 8 of the command **AE[4]** is set to 1 (meaning treat the fault event as a warning), then Warning bit 7 is set in Statusword, and the drive does not change its DS402 state.

If bit 8 of the command **AE[4]** is set to 0, then the drive enters the Fault Reaction state.

The drive behavior is defined by the Fault Reaction option code, object 0x605E. Motor failure code is 0x100 (**MF** command). An Emergency message will be sent (if not masked, see Object 0x2F21) with the error code 0x8611.

The following error indications may occur:

- When a drive is blocked
- When the profile velocity is unreachable
- Due to wrong closed loop coefficients

The value of the object is reflected in **ER[3]**.

- Object description:

Attributes	0x6065
Name	Following error window
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	1e9

5.9.16 Object 0x6066: Following Error Time Out

When a following error occurs and continues equal to or longer than the defined value of the timeout, the corresponding bit 13 following error in the Statusword is set to 1 indicating a following error threshold violation. This indication is valid in csp, ip, pp modes. Setting of the following error bit causes a Fault state. For details refer to descriptions of the object 0x6065.

- Object description:

Attributes	0x6066
Name	Following error time out
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	1 - 65535
Default value	20

5.9.17 Object 0x6067: Position Window

This Object defines a symmetrical range of accepted positions relative to the target position. If the actual value of the position encoder is within the position window during position window time, this target position is regarded as reached. The Target reached bit (bit 10) in the Statusword will be set to 1 in profile position mode. The object has no role in IP & CSP motion modes.

- Object description:

Attributes	0x6067
Name	Position window
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
--------	------------

PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	100

5.9.18 Object 0x6068: Position Window Time

When the actual position is within the position window during the defined position window time, given in milliseconds, the corresponding bit 10 target reached in the Statusword is set to 1 in pp mode. The object has no role in csp and ip modes. Refer to the description in the position window object.

- Object description:

Attributes	0x6068
Name	Position window time
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	20

5.9.19 Object 0x6069: Velocity Sensor Actual Value

This object defines the value read from a velocity encoder, in increments/second. If the velocity actual value is reflected by object 0x606C, it is scaled by velocity factor 0x6096.

- Object description:

Attributes	0x6069
Name	Velocity sensor actual value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	RO
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.20 Object 0x606A: Sensor Selection Code

This object is also used to determine the source of the velocity actual value (object 0x606C), which determines whether a differentiated position signal or the signal from a separate velocity sensor needs to be evaluated.

- Object description:

Attributes	0x606A
Name	Sensor selection code
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No

Value range	0,1
Default value	1

- Data description:

Value	Description
0x0000	Actual velocity value from position encoder
0x0001	Actual velocity value from velocity encoder
0x0002 – 0xFFFF	Reserved

5.9.21 Object 0x606B: Velocity Demand Value

This object defines the value of the velocity command as reflected by the trajectory generator. This value is in user defined units.

- Object description:

Attributes	0x606B
Name	Velocity demand value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.22 Object 0x606C: Velocity Actual Value

This object is represented in velocity units and is coupled with the velocity used as input to the velocity controller. The object is taken from either the position sensor or the velocity sensor. In **UM=5** (single position loop), this object reflects the load and the motor value; in **UM=4** (dual loop), object 0x606A determines which sensor is reflected.

- Object description:

Attributes	0x606C
Name	Velocity actual value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	None

5.9.23 Object 0x606D: Velocity Window

This object monitors whether the required process velocity has been achieved after an eventual acceleration or deceleration (braking) phase. It is measured in velocity units.

- Object description:

Attributes	0x606D
Name	Velocity window
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No

Value range	UNSIGNED16
Default value	100

5.9.24 Object 0x606E: Velocity Window Time

The corresponding bit 10 target reached is set in the Statusword when the difference between the target velocity and the velocity actual value is within the velocity window longer than the velocity window time. The value of the velocity window time is given in multiples of milliseconds.

- Object description:

Attributes	0x606E
Name	Velocity window time
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	20

5.9.25 Object 0x606F: Velocity Threshold

When the velocity actual value is less than or equal to the velocity threshold during or after the velocity threshold time, bit 12, “speed is zero”, is set to 1 in the Statusword, indicating that the axle is stationary. Otherwise, this bit is set to 0. The object value is given in velocity units.

- Object description:

Attributes	0x606F
Name	Velocity threshold
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	100

5.9.26 Object 0x6070: Velocity Threshold Time

The velocity threshold time is given in multiples of milliseconds. See the description in Object 0x606F.

- Object description:

Attributes	0x6070
Name	Velocity threshold time
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED16
Default value	20

5.9.27 Object 0x6071: Target Torque

This object is the input value for the torque controller in profile torque mode and the value is given per thousand of rated torque.

- Object description:

Attributes	0x6071
Name	Target torque
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER16
Default value	0

5.9.27.1 Example

If a torque that is relative to current of 2 Newton-meter is needed, and object 0x6076 (Motor Rate Torque) is 3200 mNm, then:

$$[0x6071] = 2000 \text{ mNm} \times 1000 / 3200 \text{ mNm} = 625$$

This number means 62.5 % of Motor Rate Torque

5.9.28 Object 0x6072: Maximum Torque

This value represents the maximum permissible torque in the motor and is given per thousand of rated torque (Object 0x6076 that is given in mNm).

- Object description:

Attributes	0x6072
Name	Max torque
Object code	VAR
Data type	UINT16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	1...50000
Default value	50000

This object defines max torque command in mNm for current controller.

For example:

If we want max torque command to be 4000 mNm,

And in [0x6076] is set to 3200 mNm

Then $[0x6072] = 4000 * 1000 / 3200 = 1250$

Setting any of the indices of **CL[]** or **PL[]** will override the value of this object to the default value.

5.9.29 Object 0x6073: Maximum Current

This value represents the maximum permissible current in the motor and is given per thousand of rated current.

- Object description:

Attributes	0x6073
Name	Max current
Object code	VAR
Data type	UINT16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	1...50000
Default value	50000

This object defines max current command in mA for current controller.

For example:

If we want max current command to be 4000 mA,

And in [0x6075] is set to 3200 mA

Then $[0x6073] = 4000 * 1000 / 3200 = 1250$

Setting any of the indices of **CL[]** or **PL[]** will override the value of this object to the default value.

5.9.30 Object 0x6074: Torque Demand Value

This value represents the torque output value of profiler and is given in units of per thousand of rated torque.

- Object description:

Attributes	0x6074
Name	Torque demand value
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER16
Default value	0

The object sets **DV[1]** after its conversion to Ampere

5.9.31 Object 0x6075: Motor Rated Current

This value is taken from the motor nameplate and is entered in milliamp. Depending on the motor, this current may be either DC, peak or rms (root-mean-square) current. All relative current data refers to this value.

- Object description:

Attributes	0x6075
Name	Motor Rate Current
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes

Value range	1...MC*1000
Default value	0

Refer to **Motor Torque Constant – Kt** for more details.

5.9.32 Object 0x6076: Motor Rate Torque

This value is taken from the motor name plate and is entered as a multiple of mNm (milli Newton meter). All relative torque data refer to this value.

For linear motors, the object name is not changed, but the motor rated force value must be entered as a multiple of mN (milli Newton).

All relative torque data refers to this value. Refer to **Motor Torque Constant – Kt**.

- Object description:

Attributes	0x6076
Name	Motor Rate Torque
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes
Value range	1...2 ³² (1....232)
Default value	0

5.9.32.1 Motor Torque Constant – KT

Motor torque constant is the torque that is produced divided by the armature current. In other words, how much torque the motor produces in a given current.

This constant is defined by the Motor rated torque divided by the motor rated current and respectively object 0x6076 divided by object 0x6075.

The torque is presented by milli Newton meter (or mN for linear motors) and the current is presented by milli amperes (mAmps).

The **Kt** is considered when torque command is given to the drive and when the torque is introduced as feedback. The following objects / set points / commands are related by the **Kt**:

Command/Object	Notes
TC	Elmo legacy torque command
0x6071	Target torque
0x6072	Max Torque
0x6074	Torque demand value
0x6077	Torque actual value
0x6087	Torque demand value
0x60B2	Torque Offset
0x60E0, 0x60E1	Positive, Negative torque limit
0x3061	Torque following error
0x6616	Safety Torque actual value



Note:

If the drive is configured for safety torque (object 0x3783.0 = 1) a modification of the rated torque (0x6076) or rated current (0x6075) results in a safety diagnostics failure.

The 0x6075 & 0x6076 objects are not affected by the **CL[1]** command.

If either one of 0x6075 & 0x6076 is set to 0, than during power up the objects will be updated to **CL[1]*1000**



Note:.

The **CL[1]** command effects the I2T calculations only and has no effect on the **Kt**.

5.9.33 Object 0x6077: Torque Actual Value

The torque actual value corresponds to the instantaneous torque in the drive motor. The value is given in units of per thousand of rated torque (object 0x6076).

- Object description:

Attributes	0x6077
Name	Torque Actual value
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER16
Default value	0

This Object is the same as the Actual Current (IQ) after conversion to Amperes.

5.9.34 Object 0x6078: Current Actual Value

This object is the current actual value. The value is given in units of per thousand of rated current.

- Object description:

Attributes	0x6078
Name	Current Actual value
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER16
Default value	0

This Object is the same as the Actual Current (IQ) after conversion to Amperes

5.9.35 Object 0x6079: DC Link Circuit Voltage

This object provides the instantaneous DC link voltage at the drive. The value is given in mV.

- Object description:

Attributes	0x6079
Name	DC link circuit voltage
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	No

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	none

5.9.36 Object 0x607A: Target Position

The target position is the set-point position to which the drive should move in profile position mode, using the current settings of motion control parameters such as velocity, acceleration, deceleration, and motion profile type. The target position is given in user-defined position units. It is converted to position increments using the position factor. The target position is interpreted as absolute or relative, depending on the Relative flag in the ControlWord.

The object is also relevant for CSP mode.

- Object description:

Attributes	0x607A
Name	Profile target position
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.37 Object 0x607B: Position Range Limit

This object contains two sub-indices that limit the numerical range of the motion: Min Position Range Limit and Max Position Range Limit. On reaching or exceeding these limits, the feedback value automatically wraps to the other end of the range (modulo motion). Wrap-around can be prevented by setting Software Position Limits (object 0x607D.1 and 0x607D.2).

The value of the position range limit is reflected in the **XM[1]** and **XM[2]** commands, to which the range and restrictions are ultimately submitted, see the Platinum drive Command Reference Manual.

When the drive is powered up the object gets **XM[1]** and **XM[2]** to sub index 1 and 2 respectively.

If both values are set to 0 then Software Position Limits limit the motion if at least one of them is set to non-zero value.

If both Position Range Limits and both Software Position Limits are set to 0, drive shall wrap around the maximum range of the position register (231). This motion is called Modulo 32 motion.

The Object 0x607B cannot be set while in OPERATION ENABLED or QUICK STOP state.

- Object description:

Attributes	0x607B
Name	Position range limit
Object code	ARRAY
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	RxMap
Non-volatile	Yes
Value range	2
Default value	2

Sub-index	1
Description	Min position range limit
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes

Value range	INTEGER32
Default value	0

Sub-index	2
Description	Max position range limit
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	INTEGER32
Default value	0

5.9.38 Object 0x607C: Home Offset

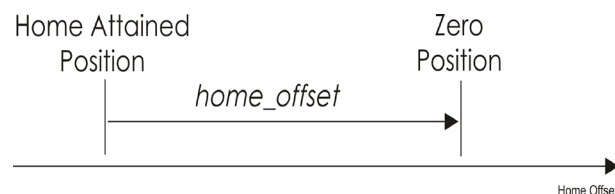
This Object is the difference between the zero position for the application and the machine home position (found during homing), measured in position units, or

Home Offset = Zero Position – Home Attained Position

So, zero position is calculated:

Zero position = Home Attained Position + Home Offset

During homing, the machine home position is found. Once homing is completed, the zero position is offset from the home position by adding the home offset to the home position. All subsequent absolute moves are taken relative to this new zero position, as illustrated in the following diagram.



For example:

Before homing, the zero position is 1000. Homing offset is set by the user equals 100. The Homing method defines the index pulse as home attained event. When moving and this event occurred, the position was 900. The Platinum drive changed actual position at the index pulse (Home Attained Position) from 900 to (-) home offset value, setting it -100. As a result of this, the zero position is changed from 1000 to 0.

Note, in method 37, contrary to all other methods, Platinum drive changed actual position at the Home Attained Position to home offset value.

- Object description:

Attributes	0x607C
Name	Home offset

Attributes	0x607C
Object code	VAR
Data type	INTEGER32
Category	Optional

- Entry description:

Access	Read/write
PDO mapping	No
Value range	-231...(231)-1
Default value	0

5.9.39 Object 0x607D: Software Position Limit

This object contains the sub-parameters min position limit and max position limit, which define the absolute position limits for the position demand value and the position actual value. Every new target position is checked against these limits. The position limits are specified in position units (same as target position). If the feedback position detects the Software Position limit, the drive shall stop and the Internal Limit Active bit in the Status word is set to 1 (bit 11).

If the drive is enabled via the Control word while the actual position is outside of the position limit boundaries (e.g. Object 0x607D.1= -10,000 and 0x6064 = -12,000), then the motion is permitted toward the boundaries (e.g. 0x607A = -9000) but is restricted to the other side (e.g. 0x607A = -13,000).

When the drive is powered up **VL[3]** and **VH[3]** are set to sub index 1 and 2 respectively.

The value of the software position limit is reflected in the **VH[3]** and **VL[3]** commands, to which the range and restrictions are submitted. See the Platinum drive Command Reference Manual.

If both values are set to 0 while at least one of Position Range Limits (0x607B) is set to non-zero value, servo cannot be enabled, motor enable procedure fails with error 74 "DS-402 profiler problem" (**EE[5]=74**, **EE[2]=0x607D**).

If both Position Range Limits and both Software Position Limits are set to 0, drive shall wrap around the maximum range of the position register (231). This motion is called Modulo 32 motion.

- Object description:

Attributes	0x607D
Name	Software position limit
Object code	ARRAY
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
-----------	---

Description	Number of entries
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	2
Default value	2

Sub-index	1
Description	Min position limit
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	INTEGER32
Default value	0

Sub-index	2
Description	Max position limit
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	INTEGER32
Default value	0

5.9.40 Object 0x607E: Polarity

The Polarity object allows the master to inverse the direction of the motion in a single command.

The Polarity can be set to either Position or Velocity reference. By setting the relevant bit (according to table below) the motion is reversed without the need to reverse the reference command.

Bit 6 of the Polarity Object is relevant in Profile Velocity mode and Cyclic Synchronous Velocity mode.

Bit 7 of the Polarity Object is relevant in Profile Position mode and in Cyclic Synchronous Position mode.

- Data description:

7	6	5	0
Position polarity value	Velocity polarity value	Reserved, is set to 0	

- Object description:

Attributes	0x607E
Name	Polarity
Object code	VAR
Data type	UNSIGNED8
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	0x00, 0x40, 0x80, 0xc0
Default value	0

- Description values:

Bit 6/Bit 7 value	Description
0	Multiply by 1
1	Multiply by -1

5.9.41 Object 0x607F: Max Profile Velocity

The max profile velocity is the maximum speed allowed in either direction during a profiled move. It is given in the same units as profile velocity. The object value is reflected in **VH[2]**.

Speed command to the speed controller is internally saturated to the value of the object.

- Object description:

Attributes	0x607F
Name	Max profile velocity
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	No
Non-volatile	Yes
Value range	UNSIGNED32
Default value	2e9

5.9.42 Object 0x6080: Maximum Motor Speed

The max motor speed is the maximum speed allowed for the motor. The RPM value from the user is converted to counts per second according to the motor resolution (**CA[18]**). The value is used to saturate the speed command to the speed controller. Note that the motor actual speed can overshoot this value.

- Object description:

Attributes	0x6080
Name	Max profile velocity
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap

Non-volatile	No
Value range	UNSIGNED32
Default value	0xFFFFFFFF

5.9.43 Object 0x6081: Profile Velocity

This object is the velocity normally attained at the end of the acceleration ramp during a profiled position move and is valid for both directions of motion. The profile velocity is given in user-defined speed units. It is converted to position increments per second using the velocity encoder factor.

The value and default value of the profile velocity is reflected in the **SP** command, to which the range and restrictions are submitted, see the Platinum Drive Command Reference Manual.

Note that if the distance is too short, the actual velocity will be lower than the Profile velocity. In such cases a triangle profile is performed.

- Object description:

Attributes	0x6081
Name	Profile velocity
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	100000

5.9.44 Object 0x6082: End Velocity

The end velocity defines the velocity that the drive will have upon reaching the target position in pp mode. Normally, the drive stops at the target position; that is, the end velocity = 0. The end velocity is given in the same units as Profile Velocity 0x6081.

The value and default value of the end velocity is reflected in the **FS** command, to which the range and restrictions are submitted, see the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6082
Name	End velocity
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.9.45 Object 0x6083: Profile Acceleration

The profile acceleration defines the acceleration limits for the pp and pv modes. The object is given in user-defined acceleration units. It is converted to position increments per second² using the normalizing factors.

The value of the profile acceleration is reflected in the **AC** command, to which the range and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6083
Name	Profile acceleration
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	yes
Value range	UNSIGNED32
Default value	1e9

5.9.46 Object 0x6084: Profile Deceleration

The profile deceleration defines the deceleration limits for the profile position and profile velocity modes. The object is given in the same units as profile acceleration. If the end velocity (Object 0x6082) is different than 0, which means that the drive will not decelerate to full stop, this object is not valid, and the profiled deceleration is similar to the profiled acceleration.

The value of the profile deceleration is reflected in the **DC** command, to which the range and restrictions are submitted. Refer to the Platinum Command Reference Manual.

- Object description:

Attributes	0x6084
Name	Profile deceleration
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	1e9

5.9.47 Object 0x6085: Quick Stop Deceleration

The quick stop deceleration is the deceleration used to stop the motor if the Quick Stop command is given and the quick stop option code (0x605A), is set to 2 or 6. The quick stop deceleration is reflected also in the **QS** command.

The quick stop deceleration is given in the same units as the profile acceleration.

- Object description:

Attributes	0x6085
Name	Quick stop deceleration
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	UNSIGNED32
Default value	1e9

5.9.48 Object 0x6086: Motion Profile Type

This object is used to select the type of motion profile used to perform a profile move.



Note:

In Profile position and Profile Velocity modes the **SF** command can be used to smooth the motion.

- Object description:

Attributes	0x6086
Name	Motion profile type
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	For values refer to Data description table below
Default value	0

- Data description:

Value	Description
-32,768...-1	Manufacturer specific, reserved
0	Linear ramp (trapezoidal profile)
1	Not supported
2	Not supported
3	Supported for compatibility
4...32,767	Reserved

5.9.49 Object 0x6087: Torque Slope

This object describes the rate of change of torque in units of per thousand of rated torque per second.

- Object description:

Attributes	0x6087
Name	Torque Slope
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	10000000

The drive will except any unsigned32 bit value, but the actual slope of the torque will be limited to the default value.

5.9.50 Object 0x608F: Position Encoder Resolution

The object defines the configured encoder increments and number of motor revolutions.

The position encoder resolution shall be calculated by the following formula:

$$\text{position encoder resolution} = \frac{\text{encoder increments}}{\text{motor revolutions}}$$

The values of the Position Encoder Resolution are reflected in the **FC[1]/FC[2]** commands, to which the ranges and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x608F
Name	Position encoder resolution
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Non-volatile	Yes
Default value	2

Sub-index	1
Description	Encoder increments
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	0.....(232-1)
Default value	1

Sub-index	2
Description	Motor revolutions
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	0.....(232-1)
Default value	1



Note:

If one of the values 0x608F.1 or 0x608F.2 is set to 0, the resulting position encoder resolution will be equal to 0.

5.9.51 Object 0x6090: Velocity Encoder Resolution

The object defines the configured encoder increments per second and number of motor revolutions per second.

The velocity encoder resolution shall be calculated by the following formula:

$$velocity\ encoder\ resolution = \frac{encoder\ \frac{increments}{second}}{motor\ \frac{revolutions}{second}}$$

The values of the Position Encoder Resolution are reflected in the **FC[3]/FC[4]** commands, to which the ranges and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6090
Name	Velocity encoder resolution
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Non-volatile	Yes
Default value	2

Sub-index	1
Description	Encoder increments per second
Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	0.....(232-1)
Default value	1

Sub-index	2
Description	Motor revolutions per second
Access	Read/write

PDO mapping	RxMap
Non-volatile	Yes
Value range	0.....(232-1)
Default value	1



Note:

If one of the values 0x6090.1 or 0x6090.2 is set to 0, the resulting velocity encoder resolution will be equal to 0.

5.9.52 Object 0x6091: Gear Ratio

The object defines the configured motor shaft revolutions and number of driving shaft revolutions.

The gear ration shall be calculated by the following formula:

$$\text{gear ratio} = \frac{\text{motor shaft revolutions}}{\text{driving shaft revolutions}}$$

The values of the Gear Ratio are reflected in the **FC[5]/FC[6]** commands, to which the ranges and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6091
Name	Gear ratio
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Non-volatile	Yes
Default value	2

Sub-index	1
Description	Motor shaft revolutions
Access	Read/write
PDO mapping	RxMap
Value range	0.....(232-1)
Non-volatile	Yes
Default value	1

Sub-index	2
Description	Driving shaft revolutions
Access	Read/write
PDO mapping	RxMap
Value range	0.....(232-1)
Non-volatile	Yes
Default value	1



Note:

If one of the values 0x6091.1 or 0x6091.2 is set to 0, the resulting gear ratio will be equal to 0.

5.9.53 Object 0x6092: Feed Constant

The object defines the configured feed constant, which is the distance per one revolution of the driving shaft of the gear box.

The gear ration shall be calculated by the following formula:

$$\text{gear ratio} = \frac{\text{motor shaft revolutions}}{\text{driving shaft revolutions}}$$

The values of the Feed Constant are reflected in the **FC[7]/FC[8]** commands, to which the ranges and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6092
Name	Feed constant
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Non-volatile	Yes
Default value	2

Sub-index	1
Description	Feed
Access	Read/write
PDO mapping	RxMap
Value range	0.....(232-1)
Non-volatile	Yes
Default value	1

Sub-index	2
Description	Driving shaft revolutions
Access	Read/write
PDO mapping	RxMap
Value range	0.....(232-1)
Non-volatile	Yes
Default value	1



Note:

If one of the values 0x6092.1 or 0x6092.2 is set to 0, the resulting feed constant will be equal to 0.

5.9.54 Object 0x6096: Velocity Factor

The velocity factor can be used to match the velocity units to the user defined velocity units.

The velocity factor shall be used as follows:

$$\text{velocity value} = \frac{\text{position value}}{s} \cdot \text{velocity factor}$$

The values of the Velocity Factor are reflected in the **FC[9]/FC[10]** commands, to which the ranges and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6096
Name	Velocity factor
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Non-volatile	Yes
Default value	2

Sub-index	1
Description	Numerator
Access	Read/write
PDO mapping	RxMap
Value range	1.....(232-1)
Non-volatile	Yes
Default value	1

Sub-index	2
Description	Denominator
Access	Read/write
PDO mapping	No
Value range	1.....(232-1)
Non-volatile	Yes
Default value	1

5.9.55 Object 0x6097: Acceleration Factor

The acceleration factor can be used to match the acceleration units to the user defined acceleration units.

The acceleration factor shall be used as follows:

$$\text{acceleration value} = \frac{\text{velocity value}}{s} \cdot \text{acceleration factor}$$

The values of the Acceleration Factor are reflected in the **FC[11]/FC[12]** commands, to which the ranges and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

- Object description:

Attributes	0x6097
Name	Acceleration factor
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Non-volatile	Yes
Default value	2

Sub-index	1
Description	Numerator
Access	Read/write
PDO mapping	RxMap
Value range	1.....(232-1)
Non-volatile	Yes
Default value	1

Sub-index	2
-----------	----------

Description	Denominator
Access	Read/write
PDO mapping	RxMap
Value range	1.....(232-1)
Non-volatile	Yes
Default value	1

5.9.56 Object 0x6098: Homing Method

This Object determines the method used during homing.

- Object description:

Attributes	0x6098
Name	Homing method
Object code	VAR
Data type	INTEGER8
Protocol	CAN, ECAT
Category	Mandatory

- Entry description:

Access	Read/write
PDO mapping	No
Value range	See below for values
Default value	1

- Data description:

Value	Description
-4,-3,-2, -1	Manufacturer specific
0	Not Supported
1...14, 17...30, 33...35, 37	Refer to the functional description
38...127	Reserved

5.9.57 Object 0x6099: Homing Speeds

This Object defines the speeds used during homing, in velocity units. The value is normalized to increments by velocity code factor. Typically, a high speed is used when searching for a home switch and the slow speed is used when searching for the index.

- Object description:

Attributes	0x6099
Name	Homing speeds
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Category	Mandatory

- Entry description:

Sub-index	0
Description	Number of entries
Entry category	Mandatory
Access	Read only
PDO mapping	No
Non-volatile	Yes
Value range	2
Default value	2

Sub-index	1
Description	Speed during search for switch High homing speed
Entry category	Mandatory
Access	Read/write
PDO mapping	No
Value range	1...2147483647
Default value	1000

Sub-index	2
Description	Speed during search for zero Low Homing Speed
Entry category	Mandatory
Access	Read/write
PDO mapping	No

Value range	1...2147483647
Default value	1000

The speed is submitted to the maximum speed limit given by the user during setup. Otherwise, an abort message with abort code 0609 0030, Value range of parameter exceeded, is activated.

5.9.58 Object 0x609A: Homing Acceleration

This Object establishes the acceleration to be used for all accelerations and decelerations with the standard homing modes and is given in acceleration units. Note, when homing procedure is interrupted due to Halt, STOP command, Motor Fault, Quick Stop command... etc, deceleration value is defined by relevant option code.

- Object description:

Attributes	0x609A
Name	Homing acceleration
Object code	VAR
Data type	UNSIGNED32
Category	Optional

- Entry description:

Access	Read/write
PDO mapping	No
Value range	1... 2147483647
Default value	No

Home deceleration is performed according to the same value.

5.9.59 Object 0x60B0: Position Offset

This object defines the offset of the target position. The offset is provided in user defined position units. The value itself is absolute and independent of how often it is transmitted over the communication system, for example, transmitted twice does not mean double value. Since the additive position value represents an offset to the target position it can also be used to control the drive with relative values regarding the target position.

- Object description:

Attributes	0x60B0
Name	Position offset
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub Index	0
Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.60 Object 0x60B1: Velocity Offset

This object provides the offset for the velocity value. The offset is given in user defined velocity units. In cyclic synchronous position mode. This object contains the input value for velocity feed forward. In cyclic synchronous velocity mode it contains the commanded offset of the ELMO drive. The value itself is absolute and independent of how often it is transmitted over the communication system, for example transmitted twice does not mean double value. Since the additive velocity value represents an offset to the target velocity, it can be also used to control the drive with relative values with regard to the target velocity.

- Object description:

Attributes	0x60B1
Name	Velocity offset
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub Index	0
Access	Read/write
PDO mapping	RxMap
Value range	INTEGER32
Default value	0

5.9.61 Object 0x60B2: Torque Offset

This object provides the offset for the torque value. The offset is given in per thousand rated torque. In cyclic synchronous position mode and cyclic synchronous velocity mode. This object contains the input value for torque feed forward. In cyclic synchronous torque mode, it contains the commanded additive torque of the drive, which is added to the target torque value. The value itself is absolute and thus independent of how often it is transmitted over the communication system, for example transmitted twice does not mean double value.

- Object description:

Attributes	0x60B2
Name	Torque offset
Object code	VAR
Data type	INTEGER16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub Index	0
Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER16
Default value	0

5.9.62 Object 0x60B8: Touch Probe Function

Object **0x60B8** indicates the configured function of the touch probe.

For continuous touch probe mode (60B8h, bit 1 = 1b or 60B8h, bit 9 = 1b) a counter per touch probe channel is incremented on each touch probe event. Thus, the control device may check how many touch probe events happen between the control cycles. Per touch probe and per edge a counter objects (Objects 60D5h, 60D6h, 60D7h and 60D8h) are defined.

- Object description:

Attributes	0x60B8
Name	Touch probe function
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Attribute	Value
Sub-index	0x00
Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	Bit field, range is not applicable
Default value	Manufacturer-specific

Value definitions are shown in Table 7.

Bit	Binary Value	Definition
0	0	Switch off touch probe 1
	1	Enable touch probe 1
1	0	Trigger first event
	1	Continuous
2, 3	00	Trigger with touch probe 1 input as defined in bits 0,1,4,5 Source signal defined by GI[1]
	01	Trigger with zero impulse signal (Index positive edge) of position encoder Setting PAL is like GI[1] = 0,1,2 (Index of port A,B,C.
	10	Touch probe source as defined in Object 0x60D0, sub-index 0x01: 1 – like 0x60B8 bits 2, 3 = 00

Bit	Binary Value	Definition
		5 – HW zero, GI[1] shall be exchanged by values 0,1,2 (Index Port A,B,C) in accordance with used Port 6 – SW zero, GI[1] should be exchanged by values 3 (DIN1, TBD, what DIN shall be used?)
	11	Reserved
4	0	Switch off sampling at the positive edge of touch probe 1
	1	Enable sampling at the positive edge of touch probe 1
5	0	Switch off sampling at the negative edge of touch probe 1
	1	Enable sampling at the negative edge of touch probe 1
6, 7 (Reserved)	-	User defined (for example, for testing)
8	0	Switch off touch probe 2
	1	Enable touch probe 2
9	0	Trigger first event
	1	Continuous
10, 11	00	Trigger with touch probe 2 input as defined in bits 8,9,12,13 Source signal defined by GI[2]
	01	Trigger with zero impulse signal (Index positive edge) of position encoder Setting PAL is like GI[2] = 0,1,2 (Index, in accordance with used Port).
	10	Touch probe source as defined in object 0x60D0, sub-index 0x02: 2 – like 0x60B8 bits 10, 11 = 00 5 – HW zero, GI[2] should be exchanged by values 0,1,2 (Index Port A,B,C) in accordance with used Port 6 – SW zero, GI[2] should be exchanged by values 4 (DIN2, TBD, what DIN shall be used?)
	11	Reserved
12	0	Switch off sampling at the positive edge of touch probe 2
	1	Enable sampling at the positive edge of touch probe 2
13	0	Switch off sampling at the negative edge of touch probe 2
	1	Enable sampling at the negative edge of touch probe 2
14, 15 (Reserved)	-	User defined (for example, for testing)

Table 31: Touch Probe value definitions

5.9.63 Object 0x60B9: Touch Probe Status

Object **0x60B9** indicates the status of the touch probe.

- Object description:

Attributes	0x60B9
Name	Touch probe status
Object code	Variable
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Attribute	Value
Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Non-volatile	No
Value range	See table 11-2
Default value	Not defined

Value definitions are shown in Table 8.

Bit	Binary Value	Definition
0	0	Touch probe 1 is switched off
	1	Touch probe 1 is enabled
1	0	Touch probe 1 does not have a positive edge value stored
	1	Touch probe 1 has a positive edge value stored
2	0	Touch probe 1 does not have a negative edge value stored
	1	Touch probe 1 has a negative edge value stored
3 to 5	0	Reserved
6, 7	-	User defined (for example, for testing) (Reserved)
8	0	Touch probe 2 is switched off
	1	Touch probe 2 is enabled
9	0	Touch probe 2 does not have a positive edge value stored

Bit	Binary Value	Definition
	1	Touch probe 2 has a positive edge value stored
10	0	Touch probe 2 does not have a negative edge value stored
	1	Touch probe 2 has a negative edge value stored
11 to 13	0	Reserved
14, 15	-	User defined (for example, for testing) (Reserved)

Table 32: Touch Probe Status definitions

5.9.64 Object 0x60BA: Touch Probe 1 Positive Edge

This object provides the position value of the touch probe 1 at positive edge. The value is given in user-defined position units.

- Object description:

Attributes	0x60BA
Name	Touch probe 1 positive edge
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	None

5.9.65 Object 0x60BB: Touch Probe 1 Negative Edge

This object provides the position value of the touch probe 1 at negative edge. The value is given in user-defined position units.

- Object description:

Attributes	0x60BB
Name	Touch probe 1 negative edge
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	None

5.9.66 Object 0x60BC: Touch Probe 2 Positive Edge

This object provides the position value of the touch probe 2 at positive edge. The value is given in user-defined position units.

- Object description:

Attributes	0x60BC
Name	Touch probe 2 positive edge
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Non-volatile	No

Sub-index	0x00
Value range	INTEGER32
Default value	None

5.9.67 Object 0x60BD: Touch Probe 2 Negative Edge

This object provides the position value of the touch probe 2 at negative edge. The value is given in user-defined position units.

- Object description:

Attributes	0x60BD
Name	Touch probe 2 negative edge
Object code	VAR
Data type	INTEGER32
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	INTEGER32
Default value	None

5.9.68 Object 0x60C5: Max Acceleration

This object defines the configured maximal acceleration. It is used to limit the profile acceleration that was set as part of the profile position mode set point. The value is measured in user-defined acceleration physical units. The object is reflected in the **VH[1]** command.

The actual acceleration limit is the minimum value between the max acceleration and the **SD** command.

- Object description:

Attributes	0x60C5
Name	Max acceleration
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	1...(2 ³²)-1
Default value	(2 ³²)-1

5.9.69 Object 0x60C6: Max Deceleration

This object defines the configured maximal deceleration. It is used to limit the profile deceleration that was set as part of the profile position mode set point. The value is measured in user-defined acceleration physical units. The object value is reflected in the **VL[1]** command.

The actual deceleration limit is the minimum value between the max deceleration and the **SD** command.

- Object description:

Attributes	0x60C6
Name	Max deceleration
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
--------	------------

PDO mapping	RxMap
Non-volatile	Yes
Value range	1...(2 ³²)-1
Default value	(2 ³²)-1

5.9.70 Object 0x60D0: Touch Probe Source

This object provides the source of the touch probe functions.

- Object description:

Attributes	0x60D0
Name	Touch probe source
Object code	ARRAY
Data type	INTEGER16
Category	Optional

- Entry description:

Sub-index	0x00
Description	Highest sub-index supported
Entry category	Mandatory
Access	Read-only
PDO mapping	No
Value range	0x01, 0x02
Default value	Not defined

Sub-index	0x01
Description	Touch probe 1 source
Entry category	Mandatory
Access	Read-only
PDO mapping	No
Value range	0x01, 0x05, 0x06
Default value	0x01

Sub-index	0x02
Description	Touch probe 2 source

Sub-index	0x02
Entry category	Optional
Access	Read-only
PDO mapping	No
Value range	0x01, 0x02, 0x06
Default value	0x02

- Value definition for sub index 1:

Value	Definition
-32768...1	Manufacturer specific
0	Reserved
+1	Touch probe 1 input (Capture 1 or 3) is like 0x60B8 bits 2, 3 = 00
+2	Reserved
+3	Reserved
+4	Reserved
+5	HW zero impulse signal (Index pulse positive edge) of position encoder. GI[1] shall be exchanged by values 0,1,2 (Index Port A,B,C)
+6	SW zero impulse signal (Index input positive edge). GI[1] shall be exchanged by values 3-8 (DIN1-6, TBD, what DIN shall be used?)
+7...+32767	Reserved

- Value definition for sub index 2:

Value	Definition
-32768...1	Manufacturer specific
0	Reserved
+1	Reserved
+2	Touch probe 2 input (Capture 2 or 4) is like 0x60B8 bits 10, 11 = 00
+3	Reserved
+4	Reserved
+5	HW zero impulse signal (Index pulse positive edge) of position encoder. GI[2] shall be exchanged by values 0,1,2 (Index Port A,B,C)
+6	SW zero impulse signal (Index input positive edge). GI[2] shall be exchanged by values 3-8 (DIN1-6, TBD, what DIN shall be used?)
+7...+32767	Reserved

5.9.71 Object 0x60D1: Touch Probe Time Stamp 1 Positive Value

This object shall provide the time stamp value of the touch probe 1 at positive edge. The value should be given in nanoseconds with resolution of 1 microsecond.

- Object description:

Attributes	0x60D1
Name	Touch Probe Time Stamp 1 positive value
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned32
Default value	Not defined

5.9.72 Object 0x60D2: Touch Probe Time Stamp 1 Negative Value

This object shall provide the time stamp value of the touch probe 1 at negative edge. The value should be given in nanoseconds with resolution of 1 microsecond.

- Object description:

Attributes	0x60D2
Name	Touch Probe Time Stamp 1 negative value
Object code	VAR
Data type	UNSIGNED32
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned32
Default value	Not defined

5.9.73 Object 0x60D3: Touch Probe Time Stamp 2 Positive Value

This object shall provide the time stamp value of the touch probe 1 at positive edge. The value should be given in nanoseconds with resolution of 1 microsecond.

- Object description:

Attributes	0x60D3
Name	Touch Probe Time Stamp 1 positive value
Object code	VAR
Data type	UNSIGNED32
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned32
Default value	Not defined

5.9.74 Object 0x60D4: Touch Probe Time Stamp 2 Negative Value

This object shall provide the time stamp value of the touch probe 2 at negative edge. The value should be given in nanoseconds with resolution of 1 microsecond.

- Object description:

Attributes	0x60D4
Name	Touch Probe Time Stamp 1 negative value
Object code	VAR
Data type	UNSIGNED32
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned32
Default value	Not defined

5.9.75 Object 0x60D5: Touch Probe 1 Positive Edge Counter

This object shall provide a continuous counter that is incremented with each positive edge at touch probe 1. The counter is only valid if the touch probe input is enabled (60B8h, bit 0 = 1b).

For single event measuring only the value of bit 0 shall be evaluated. For continuous measuring the value is an unsigned 16-bit value with overflow.

- Object description:

Attributes	0x60D5
Name	Touch Probe 1 positive edge counter
Object code	VAR
Data type	UNSIGNED16
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned16
Default value	Not defined

5.9.76 Object 0x60D6: Touch Probe 1 Negative Edge Counter

This object shall provide a continuous counter that is incremented with each negative edge at touch probe 1. The counter is only valid if the touch probe input is enabled (60B8h, bit 0 = 1b).

For single event measuring only the value of bit 0 shall be evaluated. For continuous measuring the value is an unsigned 16-bit value with overflow.

- Object description:

Attributes	0x60D6
Name	Touch Probe 1 negative edge counter
Object code	VAR
Data type	UNSIGNED16
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned16

Sub-index	0x00
Default value	Not defined

5.9.77 Object 0x60D7: Touch Probe 2 Positive Edge Counter

This object shall provide a continuous counter that is incremented with each positive edge at touch probe 2. The counter is only valid if the touch probe input is enabled (60B8h, bit 0 = 1b).

For single event measuring only the value of bit 0 shall be evaluated. For continuous measuring the value is an unsigned 16-bit value with overflow.

- Object description:

Attributes	0x60D7
Name	Touch Probe 2 positive edge counter
Object code	VAR
Data type	UNSIGNED16
Category	Optional
Multi-axes	Yes

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned16
Default value	Not defined

5.9.78 Object 0x60D8: Touch Probe 2 Negative Edge Counter

This object shall provide a continuous counter that is incremented with each negative edge at Touch probe 2. The counter is only valid if the touch probe input is enabled (60B8h, bit 0 = 1b).

For single event measuring only the value of bit 0 shall be evaluated. For continuous measuring the value is an unsigned 16-bit value with overflow.

- Object description:

Attributes	0x60D8
Name	Touch Probe 2 negative edge counter
Object code	VAR
Data type	UNSIGNED16
Category	Optional

- Entry description:

Sub-index	0x00
Access	Read-only
PDO mapping	TxMap
Value range	Unsigned16
Default value	Not defined

5.9.79 Object 0x60E0: Positive Torque Limit Value

This object shall indicate the configured maximum positive torque in the motor. The value should be given per thousand of rated torque. Positive torque takes effect in the case of motion in positive velocity or regenerative operation is negative velocity.

- Object description:

Attributes	0x60E0
Name	Positive torque limit value
Object code	VAR
Data type	UMSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No

Value range	1...50000
Default value	1

5.9.80 Object 0x60E1: Negative Torque Limit Value

This object shall indicate the configured maximum negative torque in the motor. The value should be given per thousand of rated torque. Negative torque takes effect in the case of motion in negative velocity or regenerative operation is positive velocity.

- Object description:

Attributes	0x60E1
Name	Negative torque limit value
Object code	VAR
Data type	UMSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	1...50000
Default value	1

5.9.81 Object 0x60E3: Supported Homing Methods

This Object indicates supported homing methods.

- Object description:

Attributes	0x60E3
Name	Supported homing methods
Object code	ARRAY
Data type	INTEGER8
Category	Optional

- Entry description:

Sub-index	0
Description	Number of entries
Entry category	Mandatory
Access	RO
PDO mapping	No
Value range	33
Default value	33

Sub-index	1...36
Description	Supported homing methods. Relations between sub-indexes and methods see table below
Entry category	Mandatory
Access	RO
PDO mapping	No
Value range	No
Default value	No

- Value description:

Sub-indexes	Related supported methods
1...14	1...14
15...28	17...30
29...31	33...35
32	37
33...36	-4,-3,-2,-1

5.9.82 Object 0x60EA: Commutation Angle

This object indicates the electrical commutation (field) angle reference in cyclic synchronous torque mode with commutation angle (CSTCA). The value should be given by multiplying of commutation angle in electrical degree by (216/3600).

- Object description:

Index	0x60EA
Name	Commutation angle
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Attribute	Value
Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	0...65535
Default value	0

5.9.83 Object 0x60F2: Position Option Code

This object defines the configured positioning behavior in the profile positioning mode.

The value of the Position Option Code is reflected in the **PO[1]** command, to which the range and restrictions are submitted. Refer to the Platinum Drive Command Reference Manual.

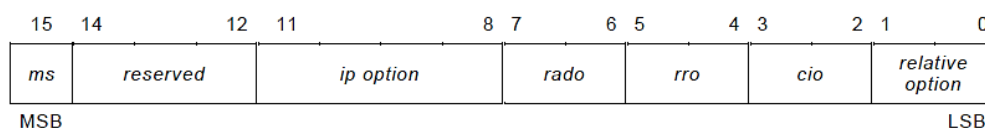
- Object description:

Attributes	0x60F2
Name	Position Option Code
Object code	VAR
Data type	UNSIGNED16
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	Yes
Value range	0...65535
Default value	0

The following diagram shows the bit-field structure of a **0x60F2** value:



LEGEND

ms = manufacturer-specific
 rro = request-response option
 cio = change immediately option
 rado = rotary axis direction option

The *relative option* bits control the behavior of the relative motion command as detailed in the following table:

Value of Bits 0 and 1	Remarks
0x0 (default)	Positioning moves are performed relative to the preceding (internal absolute) target position or relative to the present location if there is no preceding target position (after Motor On).
0x1	Positioning moves are performed relative to the actual position demand value (object 0x60FC), that is, the output of the trajectory generator.
0x2	Positioning moves are performed relative to the position actual value (object 0x6064).

Value of Bits 0 and 1	Remarks
0x3	Reserved.

The *change immediately option (cio)* bits are described in the following table:

Value of Bits 2 and 3	Remarks
0x0 (default)	The drive device readapts the actual motion to the new target position immediately.
0x1	The actually performed positioning task will be continued and blended with the newly commanded task when the target position is reached.
0x2	Reserved.
0x3	Reserved.

The *rotary axis direction options (rado)* bits define the behavior of the position modulo:

Mode	Value of Bits 6 and 7	Remarks
Normal positioning	0x0 (default)	Normal rotary positioning is similar to linear axis positioning. If the position range limits (object 0x607B tag=Object_0x607B_Position_range_limit) are achieved or exceeded, the input value wraps automatically to the other end of the range. Movement greater than the modulo is possible only with this bit combination.
Negative movement	0x1	Positioning only in the negative direction. If the target position is higher than the actual position, the axis moves over the minimum limit of the position range (Object 0x607B.1) to the target position.
Positive movement	0x2	Positioning only in positive direction. If the target position is lower than the actual position, the axis moves over the maximum limit of the position range (Object 0x607B.2) to the target position.
Positioning with shortest way	0x3	Positioning with the shortest way to the target position. NOTE: If the difference between the actual value and the target position in a 360° system is 180°, the axis moves in the positive direction.

5.9.84 Object 0x60F4: Following Error Actual Value

The object defines the following error actual value. The following error is the difference between the position demand and the position feedback values and is given in position units.

- Object description:

Attributes	0x60F4
Name	Following error actual value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.85 Object 0x60FA: Control Effort

The output of the Position control loop in velocity user units.

- Object description:

Attributes	0x60FA
Name	Control effort
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.86 Object 0x60FC: Position Demand Internal Value - Increments

The position output of the trajectory generator in increments.

- Object description:

Attributes	0x60FC
Name	Position demand internal value
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.87 Object 0x60FD Digital Inputs

This object presents the digital input's status on read access. Bit placement is presented in the following table (1: means logical active state, 0: logical not active).

For more details about the digital input functionality refer to **IL[]** command, **IP** command, and **IB[]** command.

- Object description:

Attributes	0x60FD
Name	Digital inputs
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	No
Value range	UNSIGNED32

Default value	No
---------------	----

- Data description:

31	16	15	4	3	2	1	0
Logical state of digital inputs 1...16 equals to IP command bits 16...31		Reserve, 0		Interlock	Main Home Switch	FLS	RLS

Interlock bit is set to '1' when one (or both) of the STO inputs is disabled and the drive is in safety state.



Note:

Object 0x36FD can be used for the same purpose as 0x60FD but with a Write Access. The Write Access is required when the digital input is defined as "sticky-bit", after the input is set to '1'. The host is required to confirm it by writing '1' to this input which is then reset back to 0. See **IL[]** command for more details about "sticky-bits".

5.9.88 Object 0x60FE: Digital Outputs

The object sets or resets general purpose digital outputs.



Note:

Bit 0 is defined as "Brake". For the Elmo drive, the brake function is performed automatically when the servo is on or off. For more information about the digital output refer to the **OL[]**, **OB[]**, **OP**, and **BP** commands in the Platinum Command reference.

The bit placement is:

31.....24	23	22	21	20	19	18	17	16	15...1	0
Reserved	D08	D07	D06	D05	D04	D03	D02	D01	Reserve	Brake not used

- Object description:

Attributes	0x60FE
Name	Digital outputs
Object code	ARRAY
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Sub-index	0
Description	Highest sub index
Entry category	Mandatory
Access	RO

PDO mapping	No
Non-volatile	Yes
Value range	1
Default value	1

Sub-index	1
Description	Physical Outputs
Entry category	Mandatory
Access	RW
PDO mapping	RxMap
Non-volatile	No
Value range	UNSIGNED32
Default value	0

5.9.89 Object 0x60FF: Target Velocity

The *target velocity* is the input for the trajectory generator. The value is given in velocity units. The value is reflected in the **JV** command.

- Object description:

Attributes	60FFh
Name	Target velocity
Object code	VAR
Data type	INTEGER32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read/write
PDO mapping	RxMap
Non-volatile	No
Value range	INTEGER32
Default value	0

5.9.90 Object 0x6502: Supported Drive Modes

- Object description:

Attributes	0x6502
Name	Supported drive modes
Object code	VAR
Data type	UNSIGNED32
Protocol	CAN, ECAT
Multi-axes	Yes

- Entry description:

Access	Read only
PDO mapping	TxMap
Non-volatile	Yes
Value range	0x3ED
Default value	0x3ED

- Data description, if bit is set – the mode is supported:

31	10	9	8	7	6	5	4	3	2	1	0
Reserve		Cst	csv	csp	ip	hm	Reserve	tq	pv	Reserve	pp

5.10 SRA – Safety Related Application Objects

The following chapter describes the Safety Related Application objects including the ETG-6100 “Safety Drive Profile” and the Elmo manufacturer specific objects. The SRA is a general term for the CRC protected objects that include:

- Safe function parameters for the safety sensor monitoring
- Control/status objects to activate and read the safety function status
- Informative safe data that is required for the diagnostics of the relevant sensors used to monitor the safety functions

Any modification of an SRA object while the drive is in safe state will result in safety failure and STO.

Most of the SRA objects are up to 8 instances per Axis, excluding STO and SBC, allowing to define several different thresholds for the safety limits monitoring.

Note: “Write” and “Read” access of all Safety objects, can be written only during “Pre-Op”, “Safe-Op” of the EtherCAT state machine and can be at any state.

Comprehensive information about the operation of the safety function and objects can be found in “**Platinum Safety Drive Manual**”.

5.11 Elmo Manufacturer Specific SRA Objects

5.11.1 Object 0x2E00: Safety Mode

The object defines the driver safety mode. If value is 0x10203040 the driver in standalone mode, otherwise FSoE is selected.

- Object description:

Attributes	0x2E00
Name	Safety mode
Object code	VAR
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	Safety mode selection, standalone or FSoE
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.11.2 Object 0x2E01: Safety Manager Parameters

The object is used to protect the safety configuration and authorize prevent unauthorized changes during the commissioning stage. It includes the parameter CRC for standalone mode and the password that is required in order to modify the CRC.

- Object description:

Attributes	0x2E01
Name	Safety mode
Object code	RECORD
Data type	UNSIGNED32

- Entry description:

Sub-index	1
Description	Drive password – Password to protect the authenticity of the safety CRC update. Password can't be reveled

	and should be known to safety engineer. If value is zero no authentication needed and CRC of SRA is updated during "Apply SRA"
Access	Read \ Write
PDO mapping	No
Value range	UNSIGNED32
Default value	0

Sub-index	2
Description	SRA crc- For Standalone safety mode, the value is compared to driver calculated CRC. The procedure is done every power up, if CRC is not equals the drive is in STO state. The CRC is calculated and automatically downloaded by the EAS during the safety configuration before the commissioning of the SRA. The CRC can be updated only if the password in sub index 1 is correct.
Access	Read \ Write
PDO mapping	No
Value range	UNSIGNED32
Default value	0

Sub-index	3
Description	New password – Setting of new drive password/modification of existing password, is possible only after drive reset (RS[2]) or if the password was correctly set to sub index 1.
Access	Write only
PDO mapping	No
Value range	UNSIGNED32
Default value	0

Sub-index	4
Description	Save counter –

	Counter is increased each save.
Access	Read only
PDO mapping	No
Value range	UINT16
Default value	0

Sub-index	5
Description	SRA save counter – Counter is increased each time a new SRA crc value is saved via SV command. In order to prevent abuse of the SRA parameters of unauthorized modification, this value cannot be reset in no circumstances. Value is 0 if the access to the SRA sector in the flash is prevented.
Access	Read only
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.11.3 Object 0x2E02: Drive safe parameters CRC

In FSoE safety mode the object holds the SRA parameters CRC value set during the parameter state by the FSoE master.

- Object description:

Attributes	0x2E02
Name	Drive safe parameters CRC
Object code	VAR
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	The CRC received from the FSoE host during the FSoE "Parameter" state.
Access	RW
PDO mapping	No

Value range	UNSIGNED32
Default value	0

5.11.4 Object 0x2E03: Extended Safety Options

Configuration of extended safety options are initialized according to object 0x2E03.

- Object description:

Attributes	0x2E03
Name	Extended safety options
Object code	VAR
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	Extended safety options, see description bits below
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

0x2E03 configuration bits:

Bit	Description
0 - 7	5V HALL monitoring
8-31	Reserved

5V_HALL monitoring is enabled/disabled by following conditions:

- If 0x2E03 LSB value is 0xA5 then, monitoring is disabled.
- If 0x2E03 LSB value isn't equal 0xA5 then, monitoring is enable.

5.11.5 Objects 0x2E10/0x2E11 Safety I/O Parameters

Each of the objects is an array that includes all the relevant data for the safe input and output. These objects are part of the Safety relevant parameter (SRA). These objects are configured for safety systems with safety I/Os in both Standalone safety drive or under safety fieldbus (FSOE).

Attributes

Attribute	Description
Type	Unsigned long 32
Object entry	Array
Access	Read - Write
Source	RS232, USB, TCP, EoE, CoE
Restrictions	Write until SafeOp in EtherCAT
Range	None
Index Range	1-8
Unit modes	All
Non-volatile	No
Attribute	None

5.11.5.1 Object 0x2E10[i]

This object includes all safety-inputs parameters:



Note:

Test pulse duration is defined in safe outputs parameters.

Sub-Index	Description	Safe Input #	Default value
0	Number of sub-index's	0	80
1	Safe IO ID (must be in range of 0-9)	1	0
2	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0
3	Test pulse width		5 (500usec)
4	Input filter (for all safe inputs)		5 (500uSec)
5	Max time between test pulses (in case external generated test pulses) [msec]		8
6	Invert test pulse (in case self-generated test pulses)		0

Sub-Index	Description	Safe Input #	Default value
	0 – SRC, 1- SINK		
7-10	Reserved		0
11	Safe IO ID (must be in range of 0, 10-19)	2	0
12	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0
13	Test pulse width		5 (500usec)
14	Reserved		0
15	Max time between test pulses (in case external generated test pulses) [msec]		8
16	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
17-20	Reserved		0
21	Safe IO ID (must be in range of 0, 20-29)	3	0
22	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0
23	Test pulse width		5 (500usec)
24	Reserved		0
25	Max time between test pulses (in case external generated test pulses) [msec]		8
26	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
27-30	Reserved		0
31	Safe IO ID (must be in range of 0, 30-39)	4	0
32	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0
33	Test pulse width		5 (500usec)
34	Reserved		0

Sub-Index	Description	Safe Input #	Default value
35	Max time between test pulses (in case external generated test pulses) [msec]		8
36	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
37-40	Reserved		0
41	Safe IO ID (must be in range of 0, 40-49)		0
42	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.	5	0
43	Test pulse width		5 (500usec)
44	Reserved		0
45	Max time between test pulses (in case external generated test pulses) [msec]		8
46	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
47-50	Reserved	6	0
51	Safe IO ID (must be in range of 0, 50-59)		0
52	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0
53	Test pulse width		5 (500usec)
54	Reserved		0
55	Max time between test pulses (in case external generated test pulses) [msec]		8
56	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
57-60	Reserved	7	0
61	Safe IO ID (must be in range of 0, 60-69)		0
62	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0

Sub-Index	Description	Safe Input #	Default value
63	Test pulse width		5 (500usec)
64	Reserved		0
65	Max time between test pulses (in case external generated test pulses) [msec]		8
66	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
67-70	Reserved		0
71	Safe IO ID (must be in range of 0, 70-79)	8	0
72	Input mask, 32bit mask that presents the value to be OR'd with safety function control word, in case the safety input is Hi. In this way more than one function can be executed with the same input.		0
73	Test pulse width		5 (500usec)
74	Reserved		0
75	Max time between test pulses (in case external generated test pulses) [msec]		8
76	Invert test pulse (in case self-generated test pulses) 0 – SRC, 1- SINK		0
77-80	Reserved		0

5.11.5.2 Object 0x2E11[i]

This object includes all safety-outputs parameters:

Sub-Index	Description	Safe Output #	Default value
0	Number of sub-index's	0	80
1	Safe IO ID (must be in range of 0-9)	1	0
2	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[2] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
3	Test pulse duration (for all safe outputs).		4 (500uSec)
4	Invert test pulse 0 – SRC, 1- SINK		0
5-10	Reserved		0

Sub-Index	Description	Safe Output #	Default value
11	Safe IO ID (must be in range of 0, 10-19)	2	0
12	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[12] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
13	Reserved		0
14	Invert test pulse 0 – SRC, 1- SINK		0
15-20	Reserved		0
21	Safe IO ID (must be in range of 0, 20-29)	3	0
22	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[22] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
23	Reserved		0
24	Invert test pulse 0 – SRC, 1- SINK		0
25-30	Reserved		0
31	Safe IO ID (must be in range of 0, 30-39)	4	0
32	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[32] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
33	Reserved		0
34	Invert test pulse 0 – SRC, 1- SINK		0
35-40	Reserved		0
41	Safe IO ID (must be in range of 0, 40-49)	5	0
42	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[42] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
43	Reserved		0
44	Invert test pulse 0 – SRC, 1- SINK		0

Sub-Index	Description	Safe Output #	Default value
45-50	Reserved		0
51	Safe IO ID (must be in range of 0, 50-59)	6	0
52	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[52] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
53	Reserved		0
54	Invert test pulse 0 – SRC, 1- SINK		0
55-60	Reserved		0
61	Safe IO ID (must be in range of 0, 60-69)	7	0
62	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[62] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
63	Reserved		0
64	Invert test pulse 0 – SRC, 1- SINK		0
65-70	Reserved		0
71	Safe IO ID (must be in range of 0, 70-79)	8	0
72	Output mask, 32bit mask that presents the value to be OR'd with safety function status word, in case the (safety status) 0x2E11[72] is not 0, the safe output is active, else non active. In this way more than one function can be operate the same output.		0
73	Reserved		0
74	Invert test pulse 0 – SRC, 1- SINK		0
75-80	Reserved		0

5.11.6 Objects 0x2E12/0x2E13: Safety Input Indication (Preliminary)

These objects are the input and output objects that can be mapped together with the Safety functions in the 0x1700 and 0x1B00 (corresponding) in the safety system. A mapped 0x2E12 object in the 0x1700 object indicates on a mapped RPDO and a mapped 0x2E13 object in the 0x1B00 object indicates on a mapped TPDO

Object 0x2E12: Safety input status – each of the 8 sub-indexes of the object represents a corresponding input. When mapped to the 0x1700 object, it will show the following logic:

- When input is in logic level '1' the corresponding mapped bit in the 0x1700 object is one.
- When input is in logic level '0' the corresponding mapped bit in the 0x1700 object is zero.

Object 0x2E13: safety out control/status corresponding - each of the 8 sub-indexes of the object represents a corresponding output. When mapped to the 0x1B00 object, it will show the following logic:

- When output is in logic level '1' the corresponding mapped bit in the 0x1B00 object is one.
- When output is in logic level '0' the corresponding mapped bit in the 0x1B00 object is zero.

Attributes

Attribute	Description
Type	Bool 1
Object entry	Array
Access	0x2E12 – read, 0x2E13 - write
Source	RS232, USB, TCP, EoE, CoE
Restrictions	None
Range	None
Index Range	1-8
Unit modes	All
Non-volatile	No
Attribute	None

5.11.7 Object 0x2E20: Safety Option

General safety configuration and functionality.

Attributes	0x2E20
Name	Safety options
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	Functionality configuration Bit field – see description in the following table.
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

Objects 0x2E20 bits description:

Bit	Description
0	STO HW inputs enabling bit, set to 1 to disable monitoring
1	12V encoder enabling bit, set to 1 to enable monitoring
2	Safety enabling input logic enabling bit, set to 1 to enable monitoring
3	VDD diagnostic enabling bit, set to 1 to enable monitoring
4	Reserved
5	Reserved
6	Reserved
7	Reserved
8	Reserved
9	Reserved
10	Reserved
11	Reserved
12	STO fault request due to diagnostic errors.
13	STO fault request due to SOS monitoring errors.
14	STO fault request due to SDI monitoring errors.
15	Reserved

5.11.8 Object 0x2E30: Safety Address

FSoE address set by the user. Value will be compared to the value received by the FSoE master during the FSoE state machine.

Attributes	0x2E30
Name	Safety address
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	FSoE ID
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.9 Object 0x3771: Pole/Sensor Resolution Factor

The object defines the conversion factors from position sensors (1, 2) units to safety units.

- Object description:

Attributes	0x3771
Name	Pole/Sensor Resolution Factor
Object code	ARRAY
Data type	FLOAT

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	2
Default value	2

Sub-index	1
Description	Pole/Sensor 1 Resolution Factor
Access	Read/write
PDO mapping	No
Value range	0 < value < 1.0e37
Default value	1

Sub-index	2
Description	Pole/Sensor 2 Resolution Factor
Access	Read/write
PDO mapping	No
Value range	0 < value < 1.0e37
Default value	1

5.11.10 Object 0x3772: SLP Digital Input/S For Position Absolute Sensing

The object defines the reference input selected for absolute safe position calibration.

The safe position calibration is required to determine the safe absolute position value used during the SLP and the SCA safety functions. For more details about the procedure address the *Platinum safety Manual*.

- Object description:

Attributes	0x3772
Name	SLP digital input/s for position absolute sensing
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	The digital input that is used for the position calibration.
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.11 Object 0x3773: Absolute Reference Position

The object holds the absolute position value that will be used during the SLP and SCA safety functions.

The value of the 0x3773 object will be set to the 0x6611 object when the digital input defined via object 0x3772 is set to high.

- Object description:

Attributes	0x3773
Name	Absolute reference position
Object code	VAR
Data type	SIGNED32

- Entry description:

Sub-index	0
Description	Predefined absolute position to be set to safety position object 0x6611 during position calibration.
Access	RW
PDO mapping	No
Value range	SIGNED32
Default value	0

5.11.12 Object 0x3774: Absolute Reference Position Window

The object is maintained for compatibility reasons.

- Object description:

Attributes	0x3774
Name	Absolute reference position window
Object code	VAR
Data type	SIGNED32

- Entry description:

Sub-index	0
Description	TBD
Access	RW
PDO mapping	No
Value range	SIGNED32
Default value	0

5.11.13 Object 0x377B: Safety Output Number used to Operate Brake

Objects holds the safe output index to engage/release the safe brake that is used by the SBC function.

The initialization procedure of the SBC verifies that the value is 7.

- Object description:

Attributes	0x377B
Name	Safety Output number used to operate brake
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	Safety Output number used to operate brake
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.14 Object 0x377C: Safety Input Number for Brake Manual Release

Safe input index for brake release. The brake release is allowed when the safe brake was engaged during a fault, in these situations a safety input can be used to release the brake and move the motor.

- Object description:

Attributes	0x377C
Name	Safety Input number for brake manual release
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	Safety Input number for brake manual release
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.15 Object 0x3780: General Sensor 1 Parameters

The object defines the specific position sensor 1 parameters. The sensor designation (whether it's position sensor 1, or 2) is according to object 3790.

- Object description:

Attributes	0x3780
Name	General sensor 1 params
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	25
Default value	25

Sub-index	1-25
Description	General Sensor 1 Parameter
Access	Read/write
PDO mapping	No
Value range	-2147483647 ... 2147483647
Default value	0

The sensor specific parameters (1-25) are defined according to the following tables:

5.11.15.1 SSI Sensor

Parameter sub-index	Parameter name	Remarks
1	SERIAL SAFE RAW POS SHIFT	Shift to right align the position
2	SERIAL SAFE MASK ST POS	Mask the lower word of the raw position bits
3	SERIAL SAFE MASK ST POS HIGH WORD	Mask the upper word of the raw position bits
4	SERIAL SAFE POS LEADING BIT MASK	Masks out the MSB of the safe position
5	SERIAL CLOCK NOMINAL VALUE	Nominal value of clock in measurement units

Parameter sub-index	Parameter name	Remarks
7	SERIAL ST POSITION DATA LENGTH	Sensor ST position length in bytes rounded up (ceiling).
9	SERIAL SAFE ERROR BIT NUM	Stores the error bit number.
12	SERIAL MODE REG	Serial mode (in bits 8-12)
13	SERIAL FEEDBACK REG	Number of transfers inside Serial Feedback/Control register (in bits 0-7)
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.15.2 BISS Sensor

Parameter sub-index	Parameter name	Remarks
1	SERIAL SAFE RAW POS SHIFT	Shift to right align the position, after initial shift to right align the CRC
2	SERIAL SAFE MASK ST POS	Extract lower word of ST raw sensor position data
3	SERIAL SAFE MASK ST POS HIGH WORD	Extract higher word of ST raw sensor position data
4	SERIAL SAFE POS LEADING BIT MASK	Masks out the MSB of the safe position
5	SERIAL CLOCK NOMINAL VALUE	Nominal value of clock in measurement units
6	SERIAL TOTAL DATA BYTES WITHOUT CRC	The total amount of data expected (not including CRC)
7	SERIAL ST POSITION DATA LENGTH	Sensor ST position length in bytes rounded up (ceiling).
8	SERIAL CRC DATA SHIFT	Shift to right align the CRC
9	SERIAL SAFE ERROR BIT NUM	Stores the error bit number.
10	BISS SAFE MASK CRC RELEVANT DATA	Masks the relevant data for the CRC calculation.
11	BISS SAFE MASK CRC RELEVANT DATA HIGH WORD	Masks the relevant data for the CRC calculation – high word.
12	SERIAL MODE REG	Serial mode (in bits 8-12)
13	SERIAL FEEDBACK REG	Number of transfers inside Serial Feedback/Control register (in bits 0-7)
18	SAFE SENSORS INVERT DIRECTION	Invert sensor-based position calculation

5.11.15.3 ENDAT Sensor

Parameter sub-index	Parameter name	Remarks
1	SERIAL SAFE RAW POS SHIFT	Shift to right align the position
2	SERIAL SAFE MASK ST POS	Mask the lower word of the raw position bits. Also used as safe position mask.
3	SERIAL SAFE MASK ST POS HIGH WORD	Mask the upper word of the raw position bits
4	SERIAL SAFE POS LEADING BIT MASK	Masks out the MSB of the safe position
5	SERIAL CLOCK NOMINAL VALUE	Nominal value of clock in measurement units
6	SERIAL TOTAL DATA BYTES WITHOUT CRC	The total amount of data expected (not including CRC)
7	SERIAL ST POSITION DATA LENGTH	Sensor ST position length in bytes rounded up (ceiling).
8	SERIAL CRC DATA SHIFT	Shift to right align the CRC. (EnDat - used after mirroring the data)
10	ENDAT CRC INITIAL REMAINDER	Modified initial CRC remainder value
11	ENDAT TOTAL DATA BYTES	Used for mirroring the data
12	SERIAL MODE REG	Serial mode (in bits 8-12)
13	SERIAL FEEDBACK REG	Number of transfers inside Serial Feedback/Control register (in bits 0-7)
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.15.4 SIN COS Sensor

Parameter sub-index	Parameter name	Remarks
1	SinOffset1	Offset of the sine signal 1 ADC
2	CosOffset1	Offset of the cosine signal 1 ADC
3	AmplitudeScalingFactor1	Compensates for difference in gain between the sine and cosine 1 signals by multiplying the sine by the parameter and dividing by 215
4	PhaseOffsetCorrectionFactor1	Phase shift compensation between sine and cosine 1 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
5	AmplitudeLowerBound1	Lowest allowable value for the sum of squares of sin and cos

Parameter sub-index	Parameter name	Remarks
6	AmplitudeUpperBound1	Highest allowable value for the sum of squares of sin and cos
7	SinOffset2	Offset of the sine signal 2 ADC
8	CosOffset2	Offset of the cosine signal 2 ADC
9	AmplitudeScalingFactor2	Compensates for difference in gain between the sine and cosine 2 signals by multiplying the sine by the parameter and dividing by 215
10	PhaseOffsetCorrectionFactor2	Phase shift compensation between sine and cosine 2 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
11	AmplitudeLowerBound2	Lowest allowable value for the sum of squares of sin and cos
12	AmplitudeUpperBound2	Highest allowable value for the sum of squares of sin and cos
13	QuadControlReg	QUAD control register
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.15.5 Resolver Sensor

Parameter sub-index	Parameter name	Remarks
1	SinOffset1	Offset of the sine signal 1 ADC
2	CosOffset1	Offset of the cosine signal 1 ADC
3	AmplitudeScalingFactor1	Compensates for difference in gain between the sine and cosine 1 signals by multiplying the sine by the parameter and dividing by 215
4	PhaseOffsetCorrectionFactor1	Phase shift compensation between sine and cosine 1 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
5	AmplitudeLowerBound1	Lowest allowable value for the sum of squares of sin and cos
6	AmplitudeUpperBound1	Highest allowable value for the sum of squares of sin and cos
7	SinOffset2	Offset of the sine signal 2 ADC
8	CosOffset2	Offset of the cosine signal 2 ADC
9	AmplitudeScalingFactor2	Compensates for difference in gain between the sine and cosine 2 signals by multiplying the sine by the parameter and dividing by 215

Parameter sub-index	Parameter name	Remarks
10	PhaseOffsetCorrectionFactor2	Phase shift compensation between sine and cosine 2 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
11	AmplitudeLowerBound2	Lowest allowable value for the sum of squares of sin and cos
12	AmplitudeUpperBound2	Highest allowable value for the sum of squares of sin and cos
13	RESOLVER EXCIT CYCLE TIME REG	Number of real time cycles * 2.
14	RESOLVER EXCIT AMP REG	Amplitude of the excitation signal.
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.15.6 Quad Sensor

Parameter sub-index	Parameter name	Remarks
13	QuadControlReg	QUAD control register
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.16 Object 0x3781: General Sensor 2 Params

The object defines the specific position sensor 2 parameters. The sensor designation (whether it's position sensor 1, or 2) is according to object 3790.

- Object description:

Attributes	0x3781
Name	General sensor 2 params
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	25
Default value	25

Sub-index	1-25
Description	General Sensor 2 Parameter
Access	Read/write
PDO mapping	No
Value range	-2147483647 ... 2147483647
Default value	0

The sensor specific parameters (1-25) are defined according to the following tables:

5.11.16.1 SSI Sensor

Parameter sub-index	Parameter name	Remarks
1	SERIAL SAFE RAW POS SHIFT	Shift to right align the position
2	SERIAL SAFE MASK ST POS	Mask the lower word of the raw position bits
3	SERIAL SAFE MASK ST POS HIGH WORD	Mask the upper word of the raw position bits
4	SERIAL SAFE POS LEADING BIT MASK	Masks out the MSB of the safe position
5	SERIAL CLOCK NOMINAL VALUE	Nominal value of clock in measurement units

Parameter sub-index	Parameter name	Remarks
7	SERIAL ST POSITION DATA LENGTH	Sensor ST position length in bytes rounded up (ceiling).
9	SERIAL SAFE ERROR BIT NUM	Stores the error bit number.
12	SERIAL MODE REG	Serial mode (in bits 8-12)
13	SERIAL FEEDBACK REG	Number of transfers inside Serial Feedback/Control register (in bits 0-7)
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.16.2 Biss Sensor

Parameter sub-index	Parameter name	Remarks
1	SERIAL SAFE RAW POS SHIFT	Shift to right align the position, after initial shift to right align the CRC
2	SERIAL SAFE MASK ST POS	Extract lower word of ST raw sensor position data
3	SERIAL SAFE MASK ST POS HIGH WORD	Extract higher word of ST raw sensor position data
4	SERIAL SAFE POS LEADING BIT MASK	Masks out the MSB of the safe position
5	SERIAL CLOCK NOMINAL VALUE	Nominal value of clock in measurement units
6	SERIAL TOTAL DATA BYTES WITHOUT CRC	The total amount of data expected (not including CRC)
7	SERIAL ST POSITION DATA LENGTH	Sensor ST position length in bytes rounded up (ceiling).
8	SERIAL CRC DATA SHIFT	Shift to right align the CRC
9	SERIAL SAFE ERROR BIT NUM	Stores the error bit number.
10	BISS SAFE MASK CRC RELEVANT DATA	Masks the relevant data for the CRC calculation.
11	BISS SAFE MASK CRC RELEVANT DATA HIGH WORD	Masks the relevant data for the CRC calculation – high word.
12	SERIAL MODE REG	Serial mode (in bits 8-12)
13	SERIAL FEEDBACK REG	Number of transfers inside Serial Feedback/Control register (in bits 0-7)
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.16.3 ENDAT Sensor

Parameter sub-index	Parameter name	Remarks
1	SERIAL SAFE RAW POS SHIFT	Shift to right align the position
2	SERIAL SAFE MASK ST POS	Mask the lower word of the raw position bits. Also used as safe position mask.
3	SERIAL SAFE MASK ST POS HIGH WORD	Mask the upper word of the raw position bits
4	SERIAL SAFE POS LEADING BIT MASK	Masks out the MSB of the safe position
5	SERIAL CLOCK NOMINAL VALUE	Nominal value of clock in measurement units
6	SERIAL TOTAL DATA BYTES WITHOUT CRC	The total amount of data expected (not including CRC)
7	SERIAL ST POSITION DATA LENGTH	Sensor ST position length in bytes rounded up (ceiling).
8	SERIAL CRC DATA SHIFT	Shift to right align the CRC. (EnDat - used after mirroring the data)
10	ENDAT CRC INITIAL REMAINDER	Modified initial CRC remainder value
11	ENDAT TOTAL DATA BYTES	Used for mirroring the data
12	SERIAL MODE REG	Serial mode (in bits 8-12)
13	SERIAL FEEDBACK REG	Number of transfers inside Serial Feedback/Control register (in bits 0-7)
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.16.4 SINCOS Sensor

Parameter sub-index	Parameter name	Remarks
1	SinOffset1	Offset of the sine signal 1 ADC
2	CosOffset1	Offset of the cosine signal 1 ADC
3	AmplitudeScalingFactor1	Compensates for difference in gain between the sine and cosine 1 signals by multiplying the sine by the parameter and dividing by 215
4	PhaseOffsetCorrectionFactor1	Phase shift compensation between sine and cosine 1 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
5	AmplitudeLowerBound1	Lowest allowable value for the sum of squares of sin and cos

Parameter sub-index	Parameter name	Remarks
6	AmplitudeUpperBound1	Highest allowable value for the sum of squares of sin and cos
7	SinOffset2	Offset of the sine signal 2 ADC
8	CosOffset2	Offset of the cosine signal 2 ADC
9	AmplitudeScalingFactor2	Compensates for difference in gain between the sine and cosine 2 signals by multiplying the sine by the parameter and dividing by 215
10	PhaseOffsetCorrectionFactor2	Phase shift compensation between sine and cosine 2 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
11	AmplitudeLowerBound2	Lowest allowable value for the sum of squares of sin and cos
12	AmplitudeUpperBound2	Highest allowable value for the sum of squares of sin and cos
13	QuadControlReg	QUAD control register
18	SAFE SENSORS INVERT DIRECTION	Invert sensor-based position calculation

5.11.16.5 Resolver Sensor

Parameter sub-index	Parameter name	Remarks
1	SinOffset1	Offset of the sine signal 1 ADC
2	CosOffset1	Offset of the cosine signal 1 ADC
3	AmplitudeScalingFactor1	Compensates for difference in gain between the sine and cosine 1 signals by multiplying the sine by the parameter and dividing by 215
4	PhaseOffsetCorrectionFactor1	Phase shift compensation between sine and cosine 1 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
5	AmplitudeLowerBound1	Lowest allowable value for the sum of squares of sin and cos
6	AmplitudeUpperBound1	Highest allowable value for the sum of squares of sin and cos
7	SinOffset2	Offset of the sine signal 2 ADC
8	CosOffset2	Offset of the cosine signal 2 ADC
9	AmplitudeScalingFactor2	Compensates for difference in gain between the sine and cosine 2 signals by multiplying the sine by the parameter and dividing by 215

Parameter sub-index	Parameter name	Remarks
10	PhaseOffsetCorrectionFactor2	Phase shift compensation between sine and cosine 2 signals to the cosine by adding the sine multiplied by the parameter and divided by 215
11	AmplitudeLowerBound2	Lowest allowable value for the sum of squares of sin and cos
12	AmplitudeUpperBound2	Highest allowable value for the sum of squares of sin and cos
13	RESOLVER EXCIT CYCLE TIME REG	Number of real time cycles * 2.
14	RESOLVER EXCIT AMP REG	Amplitude of the excitation signal.
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.16.6 Quad Sensor

Parameter sub-index	Parameter name	Remarks
13	QuadControlReg	QUAD control register
18	SAFE SENSORS INVERT DIRECTION	Invert sensor based position calculation

5.11.17 Object 0x3783: Torque Sensor Configuration Options

This object provides safety torque sensor configuration options.

- Object description:

Index	0x3783
Name	Torque sensor configuration options
Object code	Variable
Data type	UNSIGNED32

- Entry description:

Attribute	Value
Sub-index	0
Access	Read/write
PDO mapping	No
Value range	See table below
Default value	0

The following should be noted.

Bits	Value	Description
7...0	0	No torque sensor is used
	1	Analog current (torque) sensors (ADC)
	2	Digital current (torque) sensors ($\Sigma\Delta$)
15...8	0	Motor type: brushless 3-phase
	1	Motor type: brushless 2-phase (stepper)
23...16	0	Reserved
31...24	0	Reserved

5.11.18 Object 0x3784: Torque Sensor General Parameters

The object defines the safety torque sensor parameters common for primary and secondary CPUs.

- Object description:

Attributes	0x3784
Name	Torque sensor general parameters
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	8
Default value	8

Sub-index	1
Description	Rated torque factor
Access	Read/write
PDO mapping	No
Value range	0.....(231-1)
Default value	0

Sub-index	2....8
Description	Reserved
Access	Read/write
PDO mapping	No
Value range	-2 ³¹(231-1)
Default value	0

5.11.19 Object 0x3785: Primary Torque Sensor Parameters

The object defines the primary safety torque sensor parameters.

- Object description:

Attributes	0x3785
Name	Primary torque sensor parameters
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	8
Default value	8

Sub-index	1
Description	Phase A offset
Access	Read/write
PDO mapping	No
Value range	$-2^{15} \dots (2^{15}-1)$
Default value	2048

Sub-index	2
Description	Phase B offset
Access	Read/write
PDO mapping	No
Value range	$-2^{15} \dots (2^{15}-1)$
Default value	0

Sub-index	3
Description	Phase C offset

Access	Read/write
PDO mapping	No
Value range	-2 ¹⁵(215-1)
Default value	2048

Sub-index	4
Description	Phase D offset
Access	Read/write
PDO mapping	No
Value range	-2 ¹⁵(215-1)
Default value	2048

Sub-index	5
Description	Phase A factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144
Default value	131072

Sub-index	6
Description	Phase B factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144
Default value	131072

Sub-index	7
Description	Phase C factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144

Default value	131072
---------------	--------

Sub-index	8
Description	Phase D factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144
Default value	131072

5.11.20 Object 0x3787: Secondary Torque Sensor Parameters

The object defines the secondary safety torque sensor parameters.

- Object description:

Attributes	0x3787
Name	Secondary torque sensor parameters
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	8
Default value	8

Sub-index	1
Description	Phase A offset
Access	Read/write
PDO mapping	No
Value range	-2 ¹⁵(215-1)
Default value	0

Sub-index	2
-----------	---

Description	Phase B offset
Access	Read/write
PDO mapping	No
Value range	$-2^{15} \dots (215-1)$
Default value	0

Sub-index	3
Description	Phase C offset
Access	Read/write
PDO mapping	No
Value range	$-2^{15} \dots (215-1)$
Default value	0

Sub-index	4
Description	Phase D offset
Access	Read/write
PDO mapping	No
Value range	$-2^{15} \dots (215-1)$
Default value	0

Sub-index	5
Description	Phase A factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144
Default value	131072

Sub-index	6
Description	Phase B factor
Access	Read/write
PDO mapping	No

Value range	4096..... 262144
Default value	131072

Sub-index	7
Description	Phase C factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144
Default value	131072

Sub-index	8
Description	Phase D factor
Access	Read/write
PDO mapping	No
Value range	4096..... 262144
Default value	131072

5.11.21 Object 0x3788: SLT Fault Reaction Parameters

The object defines SLT fault reaction parameters, defines motor “move back” motion in case enabled by object 0x37A0.

- Object description:

Attributes	0x3788
Name	SLT fault reaction parameters
Object code	ARRAY
Data type	Float

- Entry description:

Sub-index	0
Description	Number of entries
Access	Read only
PDO mapping	No
Value range	8
Default value	8

Sub-index	1
Description	Relative position [UU]
Access	Read/write
PDO mapping	No
Value range	0.....4.5036E+15
Default value	0

Sub-index	2
Description	velocity [UU/sec]
Access	Read/write
PDO mapping	No
Value range	0.....4.5036E+15
Default value	0

Sub-index	3
Description	Acceleration \ Deceleration [UU/sec/sec]
Access	Read/write
PDO mapping	No
Value range	0.....4.5036E+15
Default value	1E+6

Sub-index	4
Description	Torque command [1/1000 rated torque]
Access	Read/write
PDO mapping	No
Value range	0.....1000
Default value	0

Sub-index	5
Description	Torque slope [1/1000 rated torque/sec]
Access	Read/write

PDO mapping	No
Value range	0.....4.5036E+15
Default value	1E+6

Sub-index	6
Description	Torque timeout: motor disables after this time [Sec] (for torque command as reaction)
Access	Read/write
PDO mapping	No
Value range	0.....100
Default value	0

Sub-index	7
Description	Velocity time out: motor disables after this time [Sec] (for velocity command as reaction)
Access	Read/write
PDO mapping	No
Value range	0.....100
Default value	0

Sub-index	8
Description	Motion direction.
Access	Read/write
PDO mapping	No
Value range	-1,0,1,2
Default value	0
Value Description	-1 – Always negative.
	0 – Reversed to application (Electrical) speed
	1 – Always positive.
	2 – Reversed to SLT violation direction.

Bits	Value	Description
7...0	0	No torque sensor is used
	1	Analog current (torque) sensors (ADC)
	2	Digital current (torque) sensors ($\Sigma\Delta$)
15...8	0	Motor type: brushless 3-phase
	1	Motor type: brushless 2-phase (stepper)
23...16	0	Reserved
31...24	0	Reserved

5.11.22 Object 0x3790: Feedbacks Options

The object defines the safe sensors' combination and the safe position source. Lower byte – combination, Upper byte – safe position source (either safe sensor 1 or safe sensor 2).

- Object description:

Attributes	0x3790
Name	Feedbacks Options
Object code	VAR
Data type	UNSIGNED16

- Entry description:

Access	Read/write
PDO mapping	No
Value range	0...0xFFFF
Default value	0

The value is defined as follows:

5.11.22.1 Lower Byte

Safe sensor 1	Safe sensor 2	Lower byte value of 0x3790
SsiA	SinCosB	1
BissA	SinCosB	2
EndatA	SinCosB	3
SinCosB / Hiperface_A+B	-	4
SsiA	ResolverB	5
BissA	ResolverB	6
EndatA	ResolverB	7
ResolverB		8
QuadA	QuadB	12
BissA	BissC	24
EndatA	EndatC	31

5.11.22.2 Upper Byte

Safe position calculation source	Upper byte value of 0x3790
Position sensor 1	1
Position sensor 2	2

5.11.23 Object 0x3791: Position Comparison Tolerance

The object defines the safe sensors' deviation tolerance in safety units.

- Object description:

Attributes	0x3791
Name	Position comparison tolerance
Object code	Variable
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Access	Read/write
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.11.24 Object 0x3795: Safe Sensor Position Value

Safe sensor position read value.

- Object description:

Attributes	0x3795
Name	Safe sensor Position value.
Object code	ARRAY
Data type	SIGNED32

- Entry description:

Sub-index	1
Description	Safety sensor 1 position, units [Safety counts].
Access	Read
PDO mapping	yes
Value range	SIGNED32
Default value	0

Sub-index	2
Description	Safety sensor 2 position
Access	Read
PDO mapping	yes
Value range	SIGNED32
Default value	0

5.11.25 Object 0x379A: Time to Software STO

In case STO request enabled by object 0x2E20, the object holds time delay in mSec before initiating the STO request. 3 instances implemented, one for Software STO in case STO is an error reaction of safety function, second for monitoring error due to SOS monitoring error, one instance for all SOS instances, third the same for SDI monitoring error. Adding time delay to the STO function allows the host to stop the motion prior to the disabling of the motor by the STO

- Object description:

Attributes	0x379A
Name	Time to SW STO
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	STO Time to software STO
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

Sub-index	1
Description	SOS Time to software STO
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

Sub-index	2
Description	SDI Time to software STO
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.11.26 Object 0x37A0: Request for Operational Halt or Fault Stopping Profile

The object defines an operation that will be taken by the drive profiler before the safety fault reaction occurs. The operation is a non-safe stopping of the motion or an option to move back in case of monitoring faults due to SLT or SAT violation. The functions and the operation reaction are described in the table below.

Attributes	0x37A0
Name	Request for operational function
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	Request for operational halt or fault stopping profile.
Access	Read\Write
PDO mapping	No
Value range	UINT16
Default value	0

- Object's values:

Bit Value	Description
0x0001U	SS1 halt request
0x0002U	SS2 halt request
0x0004U	SLS halt request
0x0008U	SSR halt request
0x0800U	SLT fault break request

5.11.27 Object 0x37B0: SBT Command

Status of SBT function. The SBT is designed to test a safe brake against torque assuring that the brake can mechanically hold the required torque. Object description:

Attributes	0x37B0
Name	SBT command
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	SBT command
Access	Read only
PDO mapping	TxMap
Value range	0,1
Default value	FALSE

5.11.28 Object 0x37B1: SBT SOS Service

SOS instance to be used by SBT function to monitor motor movement. The SOS function is used to guarantee that the load which is held by the brake remains steady within the SOS tolerance.

- Object description:

Attributes	0x37B1
Name	SBT SOS service
Object code	VAR
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SBT SOS service
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.11.29 Object 0x37B2: SBT Ramp Time Delay

SBT delay before brake torque monitoring starts and once finished time delay before SBT can be deactivated.

- Object description:

Attributes	0x37B2
Name	SBT ramp time delay
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	SBT ramp time delay
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.30 Object 0x37B3: SBT Test Duration Time

Test duration time when safe brake is tested by the SBT.

- Object description:

Attributes	0x37B3
Name	SBT test duration time
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	SBT test duration time
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.31 Object 0x37B4: SBT Torque Limit

Torque minimum expected threshold during SBT test duration.

- Object description:

Attributes	0x37B4
Name	SBT torque limit
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	SBT torque limit
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.32 Object 0x37B5: SXT Status

Command and status of SXT function.

- Object description:

Attributes	0x7B5
Name	SXT status
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	SXT status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.11.33 Object 0x37B6: SXT Torque Maximum Positive Limit

SXT positive direction torque limit.

- Object description:

Attributes	0x37B6
Name	SXT torque maximum positive limit
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	SXT torque maximum positive limit
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.34 Object 0x37B7: SXT Torque Maximum Negative Limit

SXT negative direction torque limit.

- Object description:

Attributes	0x37B6
Name	SXT torque maximum negative limit
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	SXT torque maximum negative limit
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.35 Object 0x37B8: SXT Error Reaction

SXT error reaction for violation of torque limits.

- Object description:

Attributes	0x37B8
Name	SXT error reaction
Object code	VAR
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SXT error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.11.36 Object 0x37B9: SAT Command

Command and status of SAT function.

- Object description:

Attributes	0x37B9
Name	SAT command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1- 8
Description	SAT command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.11.37 Object 0x37BA: SAT Torque Limit

SAT absolute torque limit.

- Object description:

Attributes	0x37BA
Name	SAT torque limit
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	SAT torque limit
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.11.38 Object 0x37BB: SAT Error Reaction

SAT error reaction for violation of torque limit.

- Object description:

Attributes	0x37B8
Name	SAT error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	SAT error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.11.39 Object 0x37BC: SAT Status

- Object description:

Attributes	0x37BC
Name	SAT status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1- 8
Description	SAT status
Access	RO
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12 ETG-6100 Safety Drive Profile Objects

5.12.1 Object 0x6611: Safe Position

Safe position in counts.

- Object description:

Attributes	0x6611
Name	Safe position
Object code	VAR
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Safe position
Access	Read only
PDO mapping	tPDO
Value range	INTEGER32
Default value	0

5.12.2 Object 0x6613: Safe Velocity

Safe velocity from safety position readout in Counts/Sec.

- Object description:

Attributes	0x6613
Name	Safe velocity
Object code	VAR
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	Safe velocity
Access	Read only
PDO mapping	tPDO
Value range	INTEGER32
Default value	0

5.12.3 Object 0x6616: Safe Torque

Safe torque to be used by safe functions.

- Object description:

Attributes	0x6616
Name	Safe torque
Object code	VAR
Data type	INTEGER16

- Entry description:

Sub-index	0
Description	Safe torque
Access	Read only
PDO mapping	tPDO
Value range	INTEGER16
Default value	0

5.12.4 Object 0x6630: Restart Acknowledge Control

When relevant bit in 0x1B00 (tPDO) in high level, restart acknowledge request is pending.

When relevant bit in 0x1700 (rPDO) is set to high the pending request is cleared.

- Object description:

Attributes	0x6630
Name	Restart acknowledge command
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	Restart acknowledge command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.5 Object 0x6631: Restart Acknowledge Status

- Object description:

Attributes	0x6631
Name	Restart acknowledge status
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	Restart acknowledge status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.6 Object 0x6632: Error Acknowledge Control

When relevant bit in 0x1B00 (tPDO) in high level an error acknowledge request is pending.

When relevant bit in 0x1700 (rPDO) is set to high the pended request is cleared.

- Object description:

Attributes	0x6632
Name	Error acknowledge command
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	Error acknowledge command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.7 Object 0x6633: Error Acknowledge Status

- Object description:

Attributes	0x6632
Name	Error acknowledge command
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	Error acknowledge command
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.8 Object 0x6640: STO Command

Once activated (0x1700 relevant bit set to low level), the power provided to the motor is stopped.

- Object description:

Attributes	0x6640
Name	STO command
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	STO command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.9 Object 0x6641: STO Restart Acknowledge Behavior

For details about the STO restart behavior please refer to the **Platinum Safety Drive Manual**.

- Object description:

Attributes	0x6641
Name	STO restart acknowledge behavior
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	STO restart acknowledge behavior
Access	RW
PDO mapping	No
Value range	TRUE, FALSE
Default value	FALSE

5.12.10 Object 0x6643: Safety STO Activate SBC

SBC instance used by STO to engage brake when STO is activated.

- Object description:

Attributes	0x6643
Name	Safety STO activate SBC
Object code	VAR
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	Safety STO activate SBC
Access	RW
PDO mapping	No
Value range	0
Default value	0

5.12.11 Object 0x6644: STO Status

- Object description:

Attributes	0x6644
Name	STO status
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	STO status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.12 Object 0x6650: SS1 Command

- Object description:

Attributes	0x6650
Name	SS1 command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1- 8
Description	SS1 command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.13 Object 0x6651: Safety SS1 Time to STO

Maximum allowed time from SS1 activation until STO must be activated.

- Object description:

Attributes	0x6651
Name	Safety SS1 time to STO
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	SS1 time to STO
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.14 Object 0x6653: Safety SS1 Velocity Zero Window

Velocity window for earlier activation of STO.

- Object description:

Attributes	0x6653
Name	Safety SS1 time to STO
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SS1 velocity zero window
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.15 Object 0x6654: Safety SS1 Time for Velocity Zero

Time within velocity window for earlier activation of STO.

- Object description:

Attributes	0x6654
Name	Safety SS1 time for velocity zero.
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SS1 time for velocity zero.
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.16 Object 0x6656: Safety SS1 Deceleration Limit Window

Minimum deceleration ramp to validate motor deceleration.

- Object description:

Attributes	0x6656
Name	Safety SS1 deceleration limit window
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SS1 deceleration limit window
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.17 Object 0x6657: Safety SS1 Time Delay for Deceleration Monitoring

Time delay from SS1 activation, to start monitoring motor deceleration.

- Object description:

Attributes	0x6657
Name	Safety SS1 time delay for deceleration monitoring.
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SS1 time for velocity zero.
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.18 Object 0x6658: Safety SS1 Activate SBC

SBC instance to be used by SS1 to engage brake. SBC time delay is defined by object 0x6661.

- Object description:

Attributes	0x6658
Name	Safety SS1 activate SBC
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SS1 activate SBC
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.19 Object 0x665F: SS1 Status

Once activated (0x1700 relevant bit set to low level), the power provided to the motor is stopped.

- Object description:

Attributes	0x665F
Name	SS1 status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1- 8
Description	SS1 status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.20 Object 0x6660: SBC Command

Brake engaged when relevant bit in 0x1B00 (tPDO) is low level, otherwise brake is released.

- Object description:

Attributes	0x6660
Name	SBC command
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	SBC command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.21 Object 0x6661: SBC Brake Time Delay

STO activation delay time after brake is engaged.

- Object description:

Attributes	0x6661
Name	SBC brake time delay
Object code	VAR
Data type	UINT16

- Entry description:

Sub-index	0
Description	SBC brake time delay
Access	RW
PDO mapping	No
Value range	0
Default value	0

5.12.22 Object 0x6667: SBC Status

- Object description:

Attributes	0x6667
Name	SBC status
Object code	VAR
Data type	BOOLEAN

- Entry description:

Sub-index	0
Description	SBC status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.23 Object 0x6668: SOS Command

- Object description:

Attributes	0x6668
Name	SOS command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SOS command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.24 Object 0x666A: Safety SOS Position Zero Window

SOS position window from latched safe position.

- Object description:

Attributes	0x666A
Name	Safety SOS position zero window
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SOS position zero window
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.25 Object 0x666C: Safety SOS Velocity Zero Window

If object value isn't zero, the safe velocity will be monitored to be within SOS velocity window.

- Object description:

Attributes	0x666A
Name	Safety SOS velocity zero window
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	Safety SOS velocity zero window
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.26 Object 0x666F: SOS Status

- Object description:

Attributes	0x666F
Name	SOS status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SOS status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.27 Object 0x6670: SS2 Command

- Object description:

Attributes	0x6668
Name	SS2 command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SS2 command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.28 Object 0x6671: SS2 Time to SOS

Maximum allowed time from SS2 activation until SOS must be activated.

- Object description:

Attributes	0x6651
Name	Safety SS2 time to SOS
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1 - 8
Description	Safety SS2 time to SOS
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.29 Object 0x6672: SS2 Time for Velocity Zero

Time within velocity window for earlier activation of SOS.

- Object description:

Attributes	0x6672
Name	Safety SS2 time for velocity zero
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	Safety SS2 time for velocity zero
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.30 Object 0x6674: Safety SS2 Deceleration Limit

Minimum deceleration ramp to validate motor deceleration.

- Object description:

Attributes	0x6653
Name	Safety SS2 deceleration limit
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1- 8
Description	Safety SS2 deceleration limit
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.31 Object 0x6675: Safety SS2 Time Delay for Deceleration Monitoring

Time delay from SS2 activation, to start monitoring motor deceleration.

- Object description:

Attributes	0x6675
Name	Safety SS2 time delay for deceleration monitoring.
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1 - 8
Description	Safety SS2 time delay for deceleration monitoring.
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.32 Object 0x6676: Safety SS2 Restart Acknowledge

In case restart acknowledge behavior enabled (object set to TRUE), deactivation of SOS requires first clearing restarted acknowledge pending request, see object 0x6630. Until clearing restart acknowledge pending request, SOS stays activated.

- Object description:

Attributes	0x6676
Name	SS2 restart acknowledge behavior
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SS2 restart acknowledge behavior
Access	RW
PDO mapping	No
Value range	TRUE, FALSE
Default value	FALSE

5.12.33 Object 0x6677: SS2 Error Reaction

SS2 error reaction for violation of deceleration limit.

- Object description:

Attributes	0x6677
Name	SS2 error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SS2 error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.34 Object 0x667F: SS2 Status

- Object description:

Attributes	0x6668
Name	SS2 command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SS2 status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.35 Object 0x6680: SSR Command

- Object description:

Attributes	0x6680
Name	SSR command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SSR command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.36 Object 0x6681: SSR Time to Velocity Monitoring

Maximum allowed time from SSR activation until SSR must be activated.

- Object description:

Attributes	0x6681
Name	Safety SSR time for velocity monitoring
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	Safety SSR time for velocity monitoring
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.37 Object 0x6683: SSR Velocity Upper Limit

- Object description:

Attributes	0x6683
Name	SSR velocity upper limit
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	SSR velocity upper limit
Access	RW
PDO mapping	No
Value range	INTEGER32
Default value	0

5.12.38 Object 0x6685: SSR Velocity Lower Limit

- Object description:

Attributes	0x6683
Name	SSR velocity lower limit
Object code	ARRAY
Data type	INTEGER32

- Entry description:

Sub-index	0
Description	SSR velocity lower limit
Access	RW
PDO mapping	No
Value range	INTEGER32
Default value	0

5.12.39 Object 0x6686: SSR Time for Velocity In Limit

Time within velocity window for earlier activation of SSR.

- Object description:

Attributes	0x6686
Name	SSR time for velocity in limit
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	0
Description	SSR time for velocity in limit
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.40 Object 0x6688: SSR Deceleration Limit

Minimum deceleration ramp to validate motor deceleration.

- Object description:

Attributes	0x6686
Name	SSR deceleration limit
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SSR deceleration limit
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.41 Object 0x6689: SSR Time Delay for Deceleration Monitoring

Time delay from SSR activation, to start monitoring motor deceleration.

- Object description:

Attributes	0x6689
Name	Safety SSR time delay for deceleration monitoring.
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	Safety SSR time delay for deceleration monitoring.
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.42 Object 0x668A: SSR Error Reaction

SS2 error reaction for violation of deceleration limit or velocity outside velocity window.

- Object description:

Attributes	0x668A
Name	SSR error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SSR error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.43 Object 0x668F: SSR Status

- Object description:

Attributes	0x6680
Name	SSR status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SSR status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.44 Object 0x6690: SLS Command

- Object description:

Attributes	0x6680
Name	SLS command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLS command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.45 Object 0x6691: SLS Time to Velocity Monitoring

Maximum allowed time from SLS activation until SLS must be activated.

- Object description:

Attributes	0x6691
Name	Safety SLS time for velocity monitoring
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	Safety SLS time for velocity monitoring
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.46 Object 0x6693: SLS Velocity Limit

- Object description:

Attributes	0x6683
Name	SSR velocity limit
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SSR velocity limit
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.47 Object 0x6694: SLS Time for Velocity In Limit

Time within velocity window for earlier activation of SLS.

- Object description:

Attributes	0x6694
Name	SLS time for velocity in limit
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	0
Description	SLS time for velocity in limit
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.48 Object 0x6695: SLS Time Delay for Deceleration Monitoring

Time delay from SLS activation, to start monitoring motor deceleration.

- Object description:

Attributes	0x6695
Name	SLS time delay for deceleration monitoring.
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	1- 8
Description	SLS time delay for deceleration monitoring.
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.49 Object 0x6697: SLS Deceleration Limit

Minimum deceleration ramp to validate motor deceleration.

- Object description:

Attributes	0x6697
Name	SLS deceleration limit
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SLS deceleration limit
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.50 Object 0x6698: SLS Error Reaction

SLS error reaction for violation of deceleration limit.

- Object description:

Attributes	0x6698
Name	SLS error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SLS error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.51 Object 0x669F: SLS Status

- Object description:

Attributes	0x6680
Name	SLS status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLS status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.52 Object 0x66A0: SLP Command

- Object description:

Attributes	0x66A0
Name	SLP command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLP command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.53 Object 0x66A2: SLP Position Upper Limit

- Object description:

Attributes	0x66A2
Name	SLP position upper limit
Object code	ARRAY
Data type	INT32

- Entry description:

Sub-index	0
Description	SLP position upper limit
Access	RW
PDO mapping	No
Value range	INT32
Default value	0

5.12.54 Object 0x66A4: SLP Position Lower Limit

- Object description:

Attributes	0x66A4
Name	SLP position lower limit
Object code	ARRAY
Data type	INT32

- Entry description:

Sub-index	0
Description	SLP position lower limit
Access	RW
PDO mapping	No
Value range	INT32
Default value	0

5.12.55 Object 0x66A5: SLP Error Reaction

SLP error reaction for violation of position limits.

- Object description:

Attributes	0x66A5
Name	SLP error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SLP error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.56 Object 0x66A7: SLP Status

- Object description:

Attributes	0x66A7
Name	SSR status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLP status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.57 Object 0x66A8: SMS Status

- Object description:

Attributes	0x66A8
Name	SSR status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SMS status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.58 Object 0x66AA: SMS Velocity Maximum Positive

SMS velocity range maximum positive.

- Object description:

Attributes	0x66AA
Name	SMS velocity maximum positive
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SMS velocity maximum positive
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.59 Object 0x66AC: SMS Velocity Maximum Negative

SMS velocity range maximum negative.

- Object description:

Attributes	0x66AC
Name	SMS velocity maximum negative
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SMS velocity maximum negative
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.60 Object 0x66AD: SMS Error Reaction

SMS error reaction for violation of velocity limits.

- Object description:

Attributes	0x66AD
Name	SMS error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SMS error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.61 Object 0x66B0: SLT Command

- Object description:

Attributes	0x66B0
Name	SLT command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLT command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.62 Object 0x66B1: SLT Torque Limit Positive Direction

- Object description:

Attributes	0x66B1
Name	SLT torque limit positive direction
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	0
Description	SLT torque limit positive direction
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.63 Object 0x66B2: SLT Torque Limit Negative Direction

- Object description:

Attributes	0x66B1
Name	SLT torque limit negative direction
Object code	ARRAY
Data type	UINT16

- Entry description:

Sub-index	0
Description	SLT torque limit negative direction
Access	RW
PDO mapping	No
Value range	UINT16
Default value	0

5.12.64 Object 0x66B3: SLT Error Reaction

SLT error reaction for violation of torque limits.

- Object description:

Attributes	0x6698
Name	SLT error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SLT error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.65 Object 0x66B7: SLT Status

- Object description:

Attributes	0x66A7
Name	SLT status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLT status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.66 Object 0x66B8: SLI Command

- Object description:

Attributes	0x66B8
Name	SLI command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLI command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.67 Object 0x66BA: SLI Position Relative Upper Limit

Upper relative limit from latched safe position.

- Object description:

Attributes	0x66B1
Name	SLI position relative upper limit
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SLI position relative upper limit
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.68 Object 0x66BC: SLI Position Relative Lower Limit

Lower relative limit from latched safe position.

- Object description:

Attributes	0x66B1
Name	SLI position relative upper limit
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SLI position relative upper limit
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.69 Object 0x66BD: SLI Error Reaction

SLI error reaction for violation of relative position limits.

- Object description:

Attributes	0x66BD
Name	SLI error reaction
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	1 - 8
Description	SLI error reaction
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	STO

5.12.70 Object 0x66BF: SLI Status

- Object description:

Attributes	0x66BF
Name	SLI status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SLI status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.71 Object 0x66D0: SDI Positive Direction Command

- Object description:

Attributes	0x66D0
Name	SDI positive direction command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SDI positive direction command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.72 Object 0x66D1: SDI Negative Direction Command

- Object description:

Attributes	0x66D1
Name	SDI negative direction command
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SDI negative direction command
Access	Read only
PDO mapping	tPDO, rPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.73 Object 0x66D3: SDI Position Zero Window

Position window limit from latched safe position.

- Object description:

Attributes	0x66D3
Name	SDI position zero window
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SDI position zero window
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.74 Object 0x66D5: SDI Velocity Zero Window

Velocity window limit to monitor motor direction.

- Object description:

Attributes	0x66D5
Name	SDI velocity zero window
Object code	ARRAY
Data type	UNSIGNED32

- Entry description:

Sub-index	0
Description	SDI velocity zero window
Access	RW
PDO mapping	No
Value range	UNSIGNED32
Default value	0

5.12.75 Object 0x66DE: SDI Positive Direction Status

- Object description:

Attributes	0x66BFa
Name	SDI positive direction Status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SDI positive direction Status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.76 Object 0x66DF: SDI Negative Direction Status

- Object description:

Attributes	0x66BFa
Name	SDI negative direction Status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SDI negative direction Status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.77 Object 0x66E0: SSM Status

- Object description:

Attributes	0x66E0
Name	SSM status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SSM status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.78 Object 0x66E2: SSM Velocity Upper Limit

- Object description:

Attributes	0x66E2
Name	SSM velocity upper limit
Object code	ARRAY
Data type	INT32

- Entry description:

Sub-index	0
Description	SSM velocity upper limit
Access	RW
PDO mapping	No
Value range	INT32
Default value	0

5.12.79 Object 0x66E4: SSM Velocity Lower Limit

- Object description:

Attributes	0x66E2
Name	SSM velocity lower limit
Object code	ARRAY
Data type	INT32

- Entry description:

Sub-index	0
Description	SSM velocity lower limit
Access	RW
PDO mapping	No
Value range	INT32
Default value	0

5.12.80 Object 0x66E8: SCA Status

- Object description:

Attributes	0x66E8
Name	SCA status
Object code	ARRAY
Data type	BOOLEAN

- Entry description:

Sub-index	1 - 8
Description	SCA status
Access	Read only
PDO mapping	tPDO
Value range	TRUE, FALSE
Default value	FALSE

5.12.81 Object 0x66EA: SCA Position Upper Limit

- Object description:

Attributes	0x66EA
Name	SCA position upper limit
Object code	ARRAY
Data type	INT32

- Entry description:

Sub-index	1 – 8
Description	SCA position upper limit
Access	RW
PDO mapping	No
Value range	INT32
Default value	0

5.12.82 Object 0x66EC: SCA Position Lower Limit

- Object description:

Attributes	0x66EC
Name	SCA position lower limit
Object code	ARRAY
Data type	INT32

- Entry description:

Sub-index	1 – 8
Description	SCA position lower limit
Access	RW
PDO mapping	No
Value range	INT32
Default value	0

5.12.83 Object 0x6760: FSoE Slave Frame Elements

The object contains the frame elements for the communication between the slave to the safety master via FSoE.

**Note:**

Sub-indices may vary according to the safety module used during the slave configuration.

Index	0x6760
Name	FSoE slave frame elements
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	Read only
PDO mapping	No
Value range	9
Default value	9

Sub-index	1
Description	FSoE Command The status from the host to the slave mainly for the FSoE state machine transitions
Data type	UINT8
Access	Read only
PDO mapping	tPDO
Non-volatile	No
Value range	UINT8
Default value	0

Sub-index	2
-----------	---

Description	FSoE connection ID The safety connection identification as set by the FSoE host during connection state
Data type	UINT16
Access	Read only
PDO mapping	tPDO
Non-volatile	No
Value range	UINT8
Default value	0

Sub-index	3-9
Description	CRC0 to CRC6 The CRC used according to the FSoE communication frame from slave to master.
Data type	UINT16
Access	Read only
PDO mapping	tPDO
Non-volatile	No
Value range	UINT16
Default value	0

5.12.84 Object 0x6770: FSoE Master Frame Elements

The object contains the frame elements for the FSoE communication between the master to the FSoE slave.

Index	0x6770
Name	FSoE master frame elements
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	Read only
PDO mapping	No

Value range	4
Default value	4

Sub-index	1
Description	FSoE Command The command from the host to the slave mainly for the FSoE state machine transitions
Data type	UINT8
Access	Write only
PDO mapping	rPDO
Non-volatile	No
Value range	UINT8
Default value	0

Sub-index	2
Description	FSoE connection ID The safety connection identification as set by the FSoE host during connection state
Data type	UINT16
Access	Write only
PDO mapping	rPDO
Non-volatile	No
Value range	UINT8
Default value	0

Sub-index	3-4
Description	CRC0 to CRC1 The CRC used according to the FSoE communication frame from master to slave.
Data type	UINT16
Access	Write only
PDO mapping	rPDO
Non-volatile	No
Value range	UINT16
Default value	0

5.12.85 Object 0x6790: Connection Module Information

The object contains information of the slave allowing the master to identify the slave in a safe way. The values are used during the FSoE state-machine.

Index	0x6790
Name	FSoE CiA402-Slave Connection Module Information
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	Read only
PDO mapping	No
Value range	8
Default value	8

Sub-index	1
Description	FSoE connection ID The safety connection identification as set by the FSoE host during connection state
Data type	UINT16
Access	Const
PDO mapping	RxPDO
Non-volatile	No
Value range	UINT8
Default value	0

Sub-index	2
Description	Type Get the device type.
Data type	VISIBLE_STRING

Access	Read only
PDO mapping	No
Non-volatile	No
Value range	modular device
Default value	modular device

Sub-index	3
Description	Name Get the device name.
Data type	VISIBLE_STRING
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	drive profile
Default value	drive profile

Sub-index	4
Description	Device Type Returns the value of object 0x1000
Data type	UNSIGNED32
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	0x0A0192

Sub-index	5
Description	Vendor ID Elmo vendor ID registered in Beckhoff / ETG Same as 0x1018.1
Data type	UNSIGNED32
Access	Read only
PDO mapping	No

Non-volatile	No
Value range	0x9A
Default value	0x9A

Sub-index	6
Description	Product code The same as 0x1018.2
Data type	UNSIGNED32
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	NA

Sub-index	7
Description	Revision number – The same as 0x1018.3
Data type	UNSIGNED32
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	NA

Sub-index	8
Description	Serial number The same as 0x1018.4
Data type	UNSIGNED32
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	UNSIGNED32
Default value	NA

5.12.86 Object 0x6791: Slave Connection Parameters

The object contains information of the slave used during the FSoE state machine.

Index	0x6791
Name	FSoE CiA402-Slave Connection Communication Parameter
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	Read only
PDO mapping	No
Value range	8
Default value	8

Sub-index	1
Description	Version Get the FSoE stack version
Data type	VISIBLE_STRING
Access	Const
PDO mapping	RxPDO
Non-volatile	No
Value range	NA
Default value	1

Sub-index	2
Description	Safety Address The safety unique address of the drive determined by the user during safety configuration. This is NOT the EtherCAT alias address.
Data type	UNSIGNED16

Access	Read only
PDO mapping	No
Non-volatile	No
Value range	0 - 65535
Default value	0

Sub-index	3
Description	FSoE connection ID The ID that the safety host sets to the drive during the connection state of the FSoE state machine.
Data type	UNSIGNED16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	0-65535
Default value	0

Sub-index	4
Description	Watchdog time The FSoE communication watchdog time where the drive must be addressed by the host to prevent watchdog. In mSec.
Data type	UNSIGNED16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	8 - 65534
Default value	64

Sub-index	5
Description	Unique device ID ID used by the host to identify the device
Data type	OCTET_STRING
Access	Read only

PDO mapping	No
Non-volatile	No
Value range	0
Default value	0

Sub-index	6
Description	Connection type Used by the host during the FSoE state machine
Data type	UNSIGNED16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	7
Description	Communication parameter length- Values read by the host during FSoE state machine state
Data type	UNSIGNED16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	2
Default value	2

Sub-index	8
Description	Application Parameter Length
Data type	UNSIGNED16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	10

Default value	10
---------------	----

5.12.87 Object 0x67A0: Slave Connection Diagnosis – FsoE

The object contains status and diagnostics of the FSoE communication between the drive and the FSoE master.

Index	0x67A0
Name	FSoE CiA402-Slave Connection Diagnosis
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Highest sub-index supported
Access	Read only
PDO mapping	No
Value range	2
Default value	2

Sub-index	1
Description	Connection State – The state of the connection (hex): Reset – 100 Session – 101 Connection – 102 Parameter – 103 Data – 104 Failsafe - 105
Data type	UINT16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	0x100 – 0x105
Default value	0x100

Sub-index	2
Description	Connection Diagnosis – See table below
Data type	UNSIGNED16
Access	Read only
PDO mapping	No
Non-volatile	No
Value range	0 - 65535
Default value	0

Connection Diagnoses table

Bits	Values
0 – 3:	0: No Error 1: Wrong command 2: Unknown command 3: Wrong connection ID 4: Wrong CRC 5: Watchdog expired 6: Wrong FSoE address 7: Wrong data 8: Wrong communication parameter length 9: Wrong communication parameter 10: Wrong application parameter length 11: Wrong application parameter 12: Unexpected session command 13: if failsafe data: communication error: failsafe data received
4	0: Error in FSoE master detected 1: Error in FSoE slave detected
5	1: FSoE slave sends no process data, slave in failsafe mode
6	1: Connection in startup
7	1: FSoE master sends no FSoE data (master in failsafe state)

5.13 MDP – Modular Device Profile

5.13.1 Object 0xF000: Modular Device Profile Object

This object contains information of how to interpret the objects in areas of the Modules.

- Object description:

Index	
Name	Modular device profile object
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	2
Default value	2

Sub-index	1
Description	Offset to add to object in the module according to the slot.
Data type	UINT
Access	RO
PDO mapping	No
Non-volatile	No
Value range	0x800
Default value	0x800

Sub-index	2
Description	Maximum number of modules. Same as maximum number of axes.
Data type	UINT

Access	RO
PDO mapping	No
Non-volatile	No
Value range	1-6
Default value	Depend on number of axes included in the product

5.13.2 Object 0xF030: Configuration Device Profile

The configured module list, includes indent numbers of configured module ID.

This object is configured by the master according to the configured modules in the order of the slots.

- Object description:

Index	
Name	Configured module list object
Object code	ARRAY
Data type	UDINT
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	8
Default value	8

Sub-index	1-8
Description	Configured module id number
Access	RW
PDO mapping	No
Non-volatile	No

Value range	- No module/Empty module 0x1-0xFFFFFFFF Configured module ID number
Default value	None

5.13.3 Object 0xF050: Detected Module List

The detected module ID list, configured by the master in object 0xF030.

- Object description:

Index	
Name	Detected module list object
Object code	ARRAY
Data type	UDINT
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	8
Default value	8

Sub-index	1-8
Description	Detected module id number
Access	RW
PDO mapping	No
Non-volatile	No
Value range	0 No module/Empty module 0x1-0xFFFFFFFF Detected module ID number
Default value	None

5.13.4 Object 0xF5FF: Configuration Management

FoE Upload/Download status indication.

- Object description:

Index	
Name	Configuration management
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	Configuration Transfer Status
Data type	UINT
Access	RO
PDO mapping	No
Non-volatile	No
Value range	0: No Upload/Download performed since power-up. Download status: 1: Download in progress. 2: Download completed OK. 0xFFFF: Download unknown error. 0x8000-0xFFFF: Download ended with error (0x8000 + 0x309E.11), check 0x309E.11 (EE[11]) for specific error. Upload status: 1: Upload in progress.

	2: Upload completed OK. 0xFFFF: Upload unknown error. 0x9000-0x9FFFF: Upload ended with error (0x9000 + 0x309E.7), check 0x309E.7 (EE[7]) for specific error.
Default value	0

5.13.5 Object 0xF6F0: Input Latch Local Timestamp

Time of writing Sync Manager 3 (Inputs).

- Object description:

Index	
Name	Input latch local time stamp
Object code	ARRAY
Data type	UDINT
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	Time of writing SM3 in μ Sec
Access	RO
PDO mapping	tPDO
Non-volatile	No
Value range	0-0xFFFFFFFF
Default value	None

5.13.5.1 Notes

Value valid only in case the object is mapped. Value is not valid using SDO.

5.13.6 Object 0xF6F2: Input Latch ESC Timestamp

ESC DC time in nanoseconds corresponding to the Sync0 latch time.

- Object description:

Index	
Name	Sync0 latch ESC time stamp
Object code	ARRAY
Data type	UNSIGNED64
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	Latch time of Sync0
Access	RO
PDO mapping	tPDO
Non-volatile	No
Value range	0-0xFFFFFFFFFFFFFFFF
Default value	None

5.13.6.1 Notes

Value valid only in DC synchronous mode.

Value valid only in case the object is mapped.

Value is not valid using SDO.

5.13.7 Object 0xF980: Device Safety Address

Device safety address. Reflect the value set by object 0x2E30.

- Object description:

Index	
Name	Device FSoE Slave Address
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	Safety Address
Data type	UINT
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1-0xFFFE
Default value	1

5.13.8 Object 0xF9F0: Manufacturer Serial Number

A string representing the drive serial number.

- Object description:

Index	
Name	Manufacturer Serial Number
Object code	VAR
Data type	VISIBLE_STRING
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	None
Default value	None

5.13.8.1 Notes

The object includes the same value as 0x1018:04

5.13.9 Object 0xF9F1: CDP Functional Generation Number

Common Device Profile Functional Generation.

- Object description:

Index	
Name	CDP Functional generation number
Object code	ARRAY
Data type	UINT32
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO

PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	CDP number
Access	RO
PDO mapping	No
Non-volatile	No
Value range	2
Default value	2

5.13.10 Object 0xF9F2: SDP Functional Generation Number

SDP functional generation number of drive module.

- Object description:

Index	
Name	SDP Functional generation number
Object code	ARRAY
Data type	UINT32
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	SDP number
Access	RO
PDO mapping	No
Non-volatile	No
Value range	0
Default value	0

5.13.11 Object 0xF9F3: Vendor Name

A string describing the vendor name: Elmo Motion Control.

- Object description:

Index	
Name	Vendor name
Object code	VAR
Data type	VISIBLE_STRING
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	Yes
Value range	None
Default value	Elmo Motion Control

5.13.12 Object 0xF9F4: Semiconductor SDP Device Name

String identifying the device type of drive module as defined by the SDP.

The Platinum returns: "N/A" indicating that no SDP is supported.

- Object description:

Index	
Name	Semiconductor SDP Device Name
Object code	ARRAY

Index	
Data type	VISIBLE_STRING
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	SDP device name
Access	RO
PDO mapping	No
Non-volatile	No
Value range	None
Default value	"N/A"

5.13.13 Object 0xF9F5: Output Identifier

Output identifier of drive module. This value can be mapped to both the RPRO (Outputs via SM2) and tPDO (Inputs via SM3). The master can write and read back the value through the tPDO to ensure the rPDO was properly received.

- Object description:

Index	
Name	Semiconductor SDP Device Name
Object code	ARRAY
Data type	BYTE
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	1
Default value	1

Sub-index	1
Description	Output identifier of drive module
Access	RW
PDO mapping	Yes
Non-volatile	No
Value range	0-255
Default value	0

5.13.14 Object 0xF9F6: Time Since Power On

Time in seconds since power on.

- Object description:

Index	
Name	Time since power-on
Object code	VAR
Data type	UNSIGNED32
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	No
Value range	0-0xFFFFFFFF
Default value	None

5.13.15 Object 0xF9F8: Firmware Update Functional Generation Number

Firmware Update Functional Generation Number supported by the drive.

- Object description:

Index	
Name	Firmware Update Functional Generation Number
Object code	VAR
Data type	UNSIGNED32
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Access	RO
PDO mapping	No
Non-volatile	No
Value range	0x00000001
Default value	0x00000001

5.13.16 Object 0xFBF0: Device Reset Command

Execution of this command causes the device to emulate a complete power cycle. This includes an ESC reset. There are two versions of this command:

- Device Reset:** Backup parameters will not change as a result of this reset. All setting parameters will revert to power-on defaults as a result of the reset.
- Factory Reset:** All backup parameters will be changed to their as-shipped defaults. All setting parameters will revert to power-on defaults as a result of the reset.
- Object description:

Index	
Name	Device Reset Command
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO

PDO mapping	No
Non-volatile	No
Value range	3
Default value	3

Sub-index	1
Description	Command
Data type	OCTET_STRING (6)
Access	RW
PDO mapping	No
Non-volatile	No
Value range	A device reset is initiated when the following byte sequence is sent: Byte 0: 0x74 Byte 1: 0x65 Byte 2: 0x73 Byte 3: 0x65 Byte 4: 0x72 Byte 5: Device Reset = 0x00 Factory Reset = 0x66
Default value	0

Sub-index	2
Description	Status
Data type	unsigned char
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Supported values: 0: Default value if the command has not been initiated. 2: last command completed, error, no response 255: command is executing
Default value	0

Sub-index	3
------------------	----------

Description	Response
Data type	OCTET_STRING (2)
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Byte 0: see Sub-index 2 Byte 1: Unused - Shall be zero
Default value	0

5.13.16.1 Notes

This command will cause all of the subsequent devices to be disconnected from the network.

5.13.17 Object 0xFB2: Store Parameters Command

Execution of this command will store all parameters to non-volatile memory.

- Object description:

Index	
Name	Store Parameters Command
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	3
Default value	3

Sub-index	1
Description	Command
Data type	OCTET_STRING (4)

Access	RW
PDO mapping	No
Non-volatile	No
Value range	Read: Bit 0 = 1: slave saves the non-volatile parameters when writing 0xFBF2:01 with 0x65766173 Bit 1-31: reserved, shall be 0 Write: All writable, non-volatile values will be stored in non-volatile memory when the following is sent: Byte 0: 0x73 Byte 1: 0x61 Byte 2: 0x76 Byte 3: 0x65 If other values are written the Abort Code "0x06040043 General parameter incompatibility reason" shall be returned.
Default value	Read: 0001

Sub-index	2
Description	Status
Data type	BYTE
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Supported values: 0: last command completed, no error, no response 2: last command completed, error, no response, see notes below 255: command is executing
Default value	0

Sub-index	3
Description	Response
Data type	OCTET_STRING (2)
Access	RO
PDO mapping	No
Non-volatile	No

Value range	Byte 0: see Sub-index 2 Byte 1: Unused - Shall be zero
Default value	0

5.13.17.1 Notes

In case of an error, sub-index 2 == 2, Object 0x3055 (**CD[2]** command) reports the reason for failure.

5.13.18 Object 0xFBF3: Calculate Checksum Command

Execution of this command will calculate a checksum for all non-volatile parameters as currently stored in non-volatile memory. The drive performs the calculation at the time of executing this command and returns this calculated value.

- Object description:

Index	
Name	Calculate Checksum Command
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	3
Default value	3

Sub-index	1
Description	Command
Data type	OCTET_STRING (4)
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Read: Returns information about the supported checksum type Bit 0 = 1: non-volatile parameters supported, at least 1 of the defined bits 1-7 shall be set Bit 1 = 1: CRC-32 Bit 2-31 = Reserved, shall be 0 Write: Only one bit can be written, bit-0 OR bit-1. If other values are written the Abort Code "0x06040043 Parameter is incompatible" shall be returned.

	Bit 0 = 1: Use default checksum algorithm of the slave Bit 1 = 1: CRC-32 Bit 2-31: Reserved, shall be 0
Default value	Read: 0003

Sub-index	2
Description	Status
Data type	BYTE
Access	RO
PDO mapping	No
Non-volatile	No
Value range	0: Default value if the command has not been initiated. 1: last command completed, no error, reply there 2: last command completed, error, no response, see note below. 255: command is executing
Default value	0

Sub-index	3
Description	Response
Data type	OCTET_STRING (6)
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Byte 0: see Sub-index 2 Byte 1: Unused - Shall be zero Byte 2-6: Checksum return value. See note below.
Default value	0

5.13.18.1 Notes

While checksum is calculated, value of sub-index 3, Bytes 2-6 may change.

Sub-index 3, Bytes 2-6 are valid only in case sub-index 3 Byte 0 = 1.

Possible errors for this command:

- More than one bit is set in sub-index 2.
- No parameters were saved to the non-volatile memory before.
- Incorrect parameters area detected on non-volatile memory.

CRC32 calculation includes polynomial: 0x814141AB and seed: 0xFFFFFFFF.

5.13.19 Object 0xFB4: Load Parameter Command

Execution of this command will load all parameters from non-volatile memory.

- Object description:

Index	
Name	Load Parameters Command
Object code	RECORD
Data type	None
Protocol	EtherCAT
Multi-axes	No

- Entry description:

Sub-index	0
Description	Number of sub-indexes
Access	RO
PDO mapping	No
Non-volatile	No
Value range	3
Default value	3

Sub-index	1
Description	Command
Data type	OCTET_STRING(4)
Access	RW
PDO mapping	No
Non-volatile	No
Value range	Read: Bit 0 = 1: slave loads the non-volatile parameters when writing 0xFB4:01 with 0x64616F6C Bit 1-31: reserved, shall be 0 Write: All writable, non-volatile parameters shall be loaded from non-volatile memory when the following is sent: Byte 0: 0x6C Byte 1: 0x6F Byte 2: 0x61 Byte 3: 0x64

	If other values are written the Abort Code "0x06040043 General parameter incompatibility reason" shall be returned.
Default value	Read: 0001

Sub-index	2
Description	Status
Data type	BYTE
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Supported values: 0: last command completed, no error, no response 2: last command completed, error, no response , see notes below 255: command is executing
Default value	0

Sub-index	3
Description	Response
Data type	OCTET_STRING (2)
Access	RO
PDO mapping	No
Non-volatile	No
Value range	Byte 0: see Sub-index 2 Byte 1: Unused - Shall be zero
Default value	0

5.13.19.1 Notes

In case of an error, sub-index 2 == 2, the reason for the error via Object 0x3055 (**CD[1]** command).

In case object 0x3055 is 0, the command failed due to one of the following reasons:

- One of the axis servos is ON (i.e. **SO == 1**)
- User program is running (i.e. **PS>0**)
- SIL is running (i.e. **SR[2]** bit 16-10 == 7)

Chapter 6: User Programming Language

6.1 Scope

This chapter covers the user program, compilation, sections (code, text), download, debugging timing.

6.2 General

A user program is high-level code, which the user can write and execute in a Platinum drive.

The code can include most of the interpreter commands (for example, **motion**, **status**) and functions (for example **sin()**) and allows auto routines, which provide a means to interrupt the user program code and jump to special event routines (for example, after digital input is set or in case of a fault).

Using the Elmo Application Studio (EAS) program editor, the user program can be edited, downloaded, debugged and even attached for debugging purposes during execution.

The user program runs in the background task of the drive. This means that it has the same precedence as the interpreter and allows the execution of user commands with no conflicts, regardless of the communication source.

Commands that are in the same user program line are completed before the next command line is processed.

The following sections describe the properties, usage abilities, execution and limits of user programs.

6.3 Structure and Timing

A user program is built up from three parts:

- Text code
- A symbol table
- Operation code

Using the EAS program editor, the user writes a text program. The program is then compiled by the EAS into a list of opcodes. Basically, each text line is converted into a list of opcodes, where the EOL (End-of-Line) opcode terminates a single execution line. The opcodes are executed by the drive, using the virtual machine process.

The virtual machine is called in every background cycle and runs the opcodes sequentially. To prevent starvation of the idle loop, each opcode cannot be executed for more than 3 msec (this is just a timeout to prevent background cycles that are too long).

The user program is downloaded to the non-volatile memory of the drive. After a successful download, the EAS sends a **CC** command, which loads the program into the RAM, from which the program is executed.

When the program is executed (by the **XQ##** command), each line in the program gets CPU time from the background process to be performed fully until the EOL (End Of Line) opcode .

The background is a non-deterministic loop. As a result, the execution of the user program is non-deterministic and can jitter between 100 µsec to 1 msec.

Note that background time is influenced by the drive real time process including control loop time (TS) profiler time (HS) and SIL (Software in the Loop) time. A higher sampling time means faster user program execution. Thus, a higher sampling time affects other drive procedures.

6.4 Keywords

User programs use the following keywords, which are protected by the compiler and cannot be declared as names of user variables. These words are colored by the EAS programming editor.

break	catch	case	continue	else	
elseif	end	exit	for	function	thread
global	goto	if	void	otherwise	reset
return	switch	try	until	wait	while
char	int	long	float	double	
unsigned char	unsigned int	unsigned long	#define	#undef	

6.5 How to Write a Program

A program is built from functions. A function must have a function call and a function definition.

Program may include labels as well. Labels are marked by **##** (e.g. **##start**) and are maintained for compatibility only. It is highly recommended **not** to use labels for new applications. Functions may also have an optional function declaration. A program can start directly with a function, as in the following example.

```
function main()
myfunc()
return

function myfunc()
int myval
myval = 1+2
return
```

In this example the function **main()** is used as the entry point, and it calls the function **myfunc()**, which sums 1+2 and returns nothing.

The user must set the **Entry Point** in the EAS editor to the required function in order to run it. In the above example, the **Entry Point** should be set to **main()**.

The function **myfunc()** can also be called using any interpreter communication terminal in the following manner.

XQ##myfunc

For more information about the **Entry Point** setting, refer to Elmo Application Studio (EAS) User Guide.

For compatibility reasons, labels, for example, **start** can also be used, as in the following example.

```
##start
myfunc()

function myfunc()      // begin the body of the function myfunc
...                   // lines in the body of the function main
return                // end the body of the function myfunc
```



Note: Elmo strongly recommends avoiding the use of labels and do recommend using functions. All the examples henceforth include functions only.

In this case, after the label (**##start**), a function call is written to run the function **myfunc**.

As can be seen in the example, all labels start with the characters **##** (e.g., **##start**).

The character pair **#@** is also used for declaring auto-routines (refer to the section 6.17 Auto-Routines).

6.6 User Functions

Simple functions start with the keyword **function** and end with the keyword **return**.

```
function function1()    //begin the body of the function function1
...
return                //end the definition of the function function1
```

The following is an example of a simple function, which uses the **MO** and **SO** commands.

```
function motor_on()
MO=1
until (SO==1)
PA=1000;

return
```

The following example shows how to create and call a function.

```
function main()    //begin the body of the function main
motor_on()        //call the function motor_on
return            //end the definition of the function main

function motor_on()
MO=1
until (SO==1)
return
```

In a function, that receives arguments (input arguments), the arguments are declared in the first line of the function.

In the example below, *arg1* is an integer type input argument, and *arg2* is a float type input argument.

```
function myfunction (int arg1, float arg2)
...
return
```

In functions that return results, the output argument (return value) is declared with its type within square brackets after the keyword **function**, as in the following example.

```
##start
main()
function main()
int a
[a] = myfunction()
return

function [ int x, ] = myfunction ( )
x = 7
return
```

The following is an example of a function with two input arguments and two return values:

```
##start
main()
```

```
function main()  
int a , b  
int c, d  
a = 5  
b = 6  
[c, d] = myfunction(a, b)  
return  
  
function [ int x, int y ] = myfunction ( int a, int b )  
x = b  
y = a  
return
```

6.7 Variables and Types

A variable must first be declared before it is used (in an expression or an assignment).

A variable definition line consists of type names (**int** or **float** etc.) and variable names. Variables may be scalar quantities for example, `int var1`, `float temp`. A global variable can also be one-dimensional arrays for example, `int arr[10]`, `float ftemp[4]`.



Note: Const floating point numbers are represented as double.

It is not recommended to use variables of two letters (XX, YY, **DG[2]** etc.) since this may be overwritten by the drive's native command. For example: `int AC` is forbidden and the compiler will alert. However, `int XX` might be allowed in some versions and might be overwritten by drive's `XX` command in other versions.

```
#start // a label. User can run this program from a terminal by entering:
```

```
// XQ##start
```

```
...
```

```
main() // call the function main
```

```
function main() // begin the body of the function main
```

```
int myvar
```

```
int myarray[5]
```

```
myarray[1] = 10
```

```
myvar = myarray[1] * 2
```

```
return
```

Two or more variables in the same definition line must be separated by commas. Alternatively, each variable may be declared in a separate line.

If a variable is a vector, it must be declared with its dimension in brackets after its name.

```
int z[10]
```

```
float w[2]
```

The vector dimension must be a positive constant number. If the dimension is defined as a floating-point number, it will be truncated to an integer.

```
int z[-10] // WRONG, not a valid dimension
```

```
float w[2.8] // dimension will be truncated to 2
```

A dimension of less than 1 is illegal.

```
int z[0] // Illegal dimension
```

```
float w[2]
```

6.7.1 Variable Names

The names of variables may include ASCII letters, digits (not leading) and underscores (not leading) only. Variable names are case-sensitive. The maximum variable name length is 11 characters. A variable name cannot be a keyword.

The following are examples of variable names:

unsigned short px1	// legal name
unsigned short pxa	// legal name
unsigned short px_	// legal name
unsigned short ab1234	// legal name
unsigned short 1px	// illegal name
unsigned short _px	// illegal name

6.7.2 Global Variables

Global variables are declared outside and before the function that uses them.

A function which uses a global variable must have the protective keyword **global** in a second declaration of the variable at the beginning of the function, as in the following example.

Global Variables cannot be addressed when the Binary Interpreter is used under CANopen communication. When the global variable contains two letters (e.g. **GG**, **HK[2]**, etc), the Elmo Application Studio (EAS) will not be able to address this variable. However, any global variable can be addressed via the CANopen OS interpreter.

```
int error          // first declaration of global variable
float yy[20]

main()             // calling the function main()

function main()
global int error   // global declaration for this function's use

error = 0          // set the global variable error
check()            // call the function check()
return

function check()
global int error   // global declaration for this function's use
int stepping       // local variable
...
error = 3          // set global variable "error"
...
return
```

Global variables can be addressed via the drive interpreter for example:

```
Drive02>errorerror=3
```

Global variables can be recorded via the EAS recorder. Recorded variables need to be mapped to the "Recorder Var" signal via the EAS "Map recording Variable" icon under the "Run and Debug".

6.8 Program Development and Execution

The process of program development for a Platinum drive includes the following steps:

- **Editing:** Writing and/or revising the program.
- **Compiling:** Using the compiler to process the program and find errors.
- **Loading:** Loading the program to the flash memory of the drive.
- **Debugging:** Observing the behavior of the program and correcting it where necessary.
- **Running** the program.

The Elmo Application Studio includes all the tools needed to perform this procedure. Using the Elmo Application Studio is fully explained in the EASII Inline Help.

6.8.1 Editing a Program

The drive program is written in simple text using any text editor. The Elmo Application Studio is recommended for program editing, because it provides several additional services, such as downloading the program to the drive, compiling the program and running it.

6.8.2 Compiling a Program

Each user program must be compiled after editing. Although the compiler does not reside in the DSP software, it is described here, because it is an integral part of the program development process. The compiler in the Elmo Application Studio is external, stand-alone software that can be accessed through the Composer software. The user can write and compile a program in off-line mode (without establishing communication with the drive) and then use the compiler to compile the program in order to produce address maps and run-time code.

If, in the course of compilation, the compiler finds syntax errors, it stops the compilation process and informs the user about the errors, presenting them in a convenient form.

The compiler is composed of a preprocessor and a code generator. The preprocessor evaluates pragmas and constant expressions. It is described in ????.

The compiler accepts the user program as a text file and files with targeted Platinum drive information. This information is required in order to ensure that the compiled code can run on the Platinum drive.

Although the compiler can locate syntax errors, it cannot find the following:

- Out-of-range command arguments
- Bad command contexts, such as an attempt to begin a motion when the motor is off.

These errors must be corrected in the debugging stage.

6.9 Error Codes

Various errors can be detected when a program is compiled, debugged or executed. The following table lists the applicable error codes, the accompanying error strings, expanded explanations of their meanings and examples.

Error Code	Error String	Meaning	Example
0	No errors	Successful compilation without errors.	0
1	Bad format	General error for bad syntax in a right- or left-hand expression.	for k = 1::10 Double colon between 1 and 10. b = a + () Empty expression in parentheses. k = 1:2:20 Colon expression used outside a for statement.
2	Empty expression	Expected right-hand expression is missing.	a = Right-hand expression is missing after the assignment operator.
3	Stack is full	Stack has overflowed its capacity.	
4	Bad index expression	An index expression of a variable is not evaluated to a single value.	a(2,3) The result of evaluating the expression in parentheses is two values, not a single value. a() The index expression is empty.
5	Bad variable type	The expected variable type is neither float nor int. This error may appear either after the global keyword or after input/output arguments in a function definition.	global float a
6	Parentheses mismatch	The number of opening parentheses does not match the number of closing parentheses. This applies to both parentheses and square brackets.	b = a(1)) There is an unneeded closing parenthesis.

Error Code	Error String	Meaning	Example
7	Value is expected	A right- or left-hand expression is not evaluated to a single value, or it has failed during evaluation due to a bad syntax expression.	$b = ^ a$ A value is expected before the ^ operator.
8	Operator is expected	During right-hand expression evaluation, there is no operator or terminator of a simple expression after successful value evaluation.	$b = a c$ After successful evaluation of a, an operator or expression terminator is expected, but c is not recognized as either an operator or terminator.
9	Out of memory in the data segment	During memory allocation for a global variable, there is segment.	
10	Bad colon expression	Error during evaluation of a colon expression. The colon expression may appear only in a "for"- statement. Bad syntax: more than three values or fewer than two values in the colon expression may also cause this error.	for k = 10:-1:5:9 Colon expression contains more than three values. for k = a Expected colon expression is missing in the "for"-statement after the assignment operator (=).
11	Name is too long	Variable or function name exceeds 12 characters.	int iuyuafdsf_876234
12	No such variable	Left-hand value is not recognized as either a variable or as a system command.	de = sin(0.5) Here "de" is neither a variable nor a function.
13	Too many Dimensions	Dimension of an array exceeds the maximum admissible number of dimensions (syntax allows only one-dimensional arrays).	int arr[12][2] An attempt was made to define a two-dimensional array.
14	Bad number of input arguments	The number of input arguments during a function call does not match the number of input arguments in the function definition.	function func(float a) ... func(a,2) The number of input arguments during the function call is two, while the function func is defined with only one input argument.

Error Code	Error String	Meaning	Example
15	Bad number of output arguments	Bad syntax in the left-hand expression: multiple output without brackets, or multiple output that exceeds the maximum admissible number of outputs (a maximum of 16 outputs is allowed).	a, b = func(1,2) Multiple outputs must be within brackets.
16	Out of memory	Compilation process ran out of memory. This error may occur if the user program is too large or too complex and there is not enough space in the code segment or in the symbol table.	
17	Too many arguments	The number of input or output arguments exceeds the maximum admissible number of input or output arguments (16).	function func (int a1, int a2, int a3, int a4, int a5, int a6, int a7, int a8, int a9, int a10, int a11, int a12, int a13, int a14, int a15, int a16, int a17, int a18, int a19, The number of input arguments exceeds 16.
18	Bad context	The compiler has found an error in the program context, such as mismatched brackets/parentheses or an improperly closed flow-control statement.	
19	Write file error	An error occurred while writing to a file.	
20	Read file error	An error occurred while reading from a file.	
21	Internal compiler error: bad database	A corrupted database has caused an internal compiler error.	
22	Function definition is inside another function or flow control block	An illegal function definition.	if a<0 a = 0 function func (int a) end Attempt to define a function inside an if block.

Error Code	Error String	Meaning	Example
23	Too many functions	The user program contains too many functions, and there is not enough space for them in the database.	
24	Name is keyword	A variable or function has the same name as a keyword. This error may occur if a variable name is identical to an auto-routine name.	int switch The word switch is a keyword, so its use as a variable name is illegal.
25	Name is not distinct	A variable or function name is not unique.	int func function func (int a) The function and the variable have the same name. function func (int a) int a The definition of the local variable a is illegal because this function already contains a local variable a as an input argument.
26	Variable name is invalid	This error occurs when: A variable or function name starts with a digit or underscore, not with a letter. A variable or function name is empty. On the variable definition line, a comma is used as a separator between variables, but the variable name after the comma is missing.	int _abc The variable name has a leading underscore. function (int a) The function name is missing after the function keyword. int a, b, The variable name is missing after the comma.
27	Bad separator between variables	The only legal separator between variables on the variable definition line is a comma. After a variable name, either a variable separator (comma) or an expression terminator is expected. Any other symbol causes this error.	int a b A comma as a variable separator is missing between a and b.
28	Illegal global variable Definition	A global variable must be declared inside a function with the global keyword, and it must	int a1 function func (int a) global float a1

Error Code	Error String	Meaning	Example
		be defined before the function. This error appears only if the global keyword is used in the wrong context: The global keyword was used outside the function. A variable declared as global inside a function is not defined previously. The type of variable declared in the definition outside the function differs from the type of the declaration inside the function.	The variable type of a1 in its definition is int, while inside the function, it is declared as float.
29	Bad variable definition	All local variables must be defined at the beginning of the function. Any variable definition coming after executable code in the function is illegal.	<pre>function func (int a) int b global int a1 b = a float c, d</pre> The definition of float variables c and d is illegal because it occurs after executable code. <pre>b = a</pre>
30	Variable is undefined	The iteration variable in a “for”-statement is not defined before it.	<pre>function func (int a) for k = 1:10 a = k: end return</pre> The iteration variable k is not defined before its use.
31	Bad separator between dimensions	Bad separator between dimensions (not a comma). This error is unused, because currently only one-dimensional arrays are allowed, so there is no need for a separator between dimensions.	

Error Code	Error String	Meaning	Example
32	Bad variable dimension	The legal variable dimension must be a positive number inside a square bracket. An expression inside square brackets that is not evaluated into a number or is evaluated as a number that is less than 1 (zero or negative) is illegal.	int arr [-12] Variable dimension is negative
33	Bad function format	Appears at function definition when: The function name is not unique or is empty. The function definition does not match its prototype. In addition, a function name with more than 12 characters will also cause this error.	function func (int a) function func (float a) Type of input argument in the function definition does not match the type in the prototype.
34	Illegal minus	A minus sign is illegal before a function call with multiple output arguments, and before parentheses that include multiple expressions.	[ab] = -func(c) Illegal minus sign before a function call with multiple outputs. -(2+3), c/5) Illegal minus before multiple expressions within parentheses.
35	Empty program	The user program is empty.	
36	Program is too long	The user program exceeds the maximum admissible length.	
37	Bad function call	An attempt has been made in a goto statement to jump to a function with a non-zero number of input or output arguments.	[a,b,c] - func(x,y) + 5 Illegal statement. The compiler checks whether there is an expression terminator immediately after the function call with multiple outputs. If there isn't, it issues this error.
38	Expression is expected	The expected expression — wait, until, while, if, elseif, switch or case — is missing.	if a = 0 end An expression expected after the if is missing.

Error Code	Error String	Meaning	Example
39	Code is too complex	The user program contains very complex code that includes too many nested levels (this expression actually contains more than 100 nested levels). A nested expressions means that there is one flow-control block inside another.	An if block inside a while loop
40	Line compilation is failed	A general error has occurred during an attempt to compile an expression.	
41	Case must follow switch	After a switch statement, the only legal statement is case ; otherwise, this error occurs.	switch a b = 0 case 1, b = 1 end The expression b=0 is illegal because switch must be followed by case .
42	Illegal case after otherwise	The otherwise statement must be the last statement of a switch block. Any case after otherwise is illegal.	switch a case 1, b = 1 otherwise b = 0 case 2, b = 1 end The statement case 2 is illegal because otherwise must be the last statement in a switch block.
43	Bad nesting	Flow-control block contradiction: else without if , mismatched end , flow-control block without end , etc.	
44	Code is not expected.	There is some unexpected code for evaluation after otherwise, end, else etc.	
45	Bad flow control expression	No assignment operator (=) sign after the iteration variable name in a for statement.	for k a = 0 end The assignment operator (=) is missing after the k.

Error Code	Error String	Meaning	Example
46	Too many errors	The compilation error buffer is full.	
47	Expression is out of Function	A function or label must contain executable code; otherwise, this error occurs.	<pre>int a1 a1 = 0 function func (int a) The executable code is illegal because it is outside the function.</pre>
48	Otherwise without any case	An otherwise appears immediately after a switch statement.	<pre>switch a otherwise b = 0 end After a switch statement, case is expected before otherwise.</pre>
49	Misplaced break	Break is legal only inside a switch , for , or while block; otherwise, this error occurs.	<pre>If a < 0 break end Using break in an if statement is illegal.</pre>
50	Too many outputs	The number of actual output arguments during a function call exceeds the number of output arguments in the function definition.	<pre>function [int b] = func (int a) ... return ... [c,d] = func(a) The function call is illegal because the number of outputs is two, while this function is defined with a single output argument.</pre>
51	Line is too long	The user program contains a line with more than 128 characters.	
52	Clause is too long	The user program contains an expression for evaluation that contains more than 512 characters. This expression may take several lines of user program text.	
53	Cannot find end of sentence	End of sentence not found within range.	

Error Code	Error String	Meaning	Example
54	Open file failure	There has been an attempt to open a non-existent file; the file name or path may be incorrect.	
55	Bad file name	The full file path is too long.	
56	No such function	An auto-routine, user function or label name must follow the keyword in a goto or reset statement; otherwise, this error occurs.	
57	Variable is array	An attempt has been made to assign an entire variable array, rather than a single member.	<pre>int arr[10] ##START arr[1] = 0 arr = 0</pre> The last expression is illegal because it tries to assign the entire array.
58	Variable is not array	An attempt has been made to assign a scalar variable according to an index, as an array.	<pre>int a1 // (a scalar) ##START a1[1] = 0</pre> The last expression is illegal because it tries to assign a scalar variable according to an index.
59	Mismatch between left and right-hand side expressions	The number of left-hand values does not match the number of values in the right-hand side of the expression.	<pre>[a,b] = 12 + c</pre> The number of values on the left is two, while the number of values after evaluation of the right-hand expression is one.
60	Illegal local array	Syntax does not allow definition of a local array. The array must be global.	<pre>function func (int a) int arr[10] ... return</pre> A local array is illegal.

Error Code	Error String	Meaning	Example
61	Function already has body	A function has more than one body.	<pre>function func (int a) wait 2000 return ... function func (int a) until a end</pre> The code is illegal because the function func has multiple bodies.
62	Opcode is not supported by Platinum drives	The specified version of the Platinum drive does not support a certain virtual command.	
63	Internal compiler error		
64	Expression is not finished	The user program contains an unfinished statement: an ellipsis (...) may be missing to indicate that the expression is continued on the next line.	The last line of user text may be: $a = b + 8 / 12 / (8^2 \cdot \sqrt{2}) - \sin(3.14/2))...$ After the ellipsis, another line should appear, but in this case it is missing.
65	Compiled code is too long	The compiled code exceeds the maximum space for the code segment in the Platinum drive's serial flash memory.	
66	Corrupted the Platinum drive setup files	The file containing the Platinum servo drive setup parameters is not in the defined format.	
67	Too many variables	The user program contains too many functions, and there is not enough space for them in the database.	
68	Variable name length mismatch to the Platinum drive setup	The allowed variable name length does not equal the length defined in the compiler.	

Error Code	Error String	Meaning	Example
69	Auto-routine has argument	The auto-routine cannot have any input or output argument; otherwise, this error occurs.	function AUTOEXEC (int a) A definition of an auto-routine that includes an input argument is illegal.
70	Label definition is inside flow control block	A label cannot be defined inside a flow-control block; otherwise, this error occurs.	
71	Function without return	The function does not end with the return keyword.	
72	Block comments is not finished	The comment block has no end.	Assume that the last line of user program text is: /* Just a comment The comment block is not closed.
73	Bad function after reset	A reset statement must contain an auto-routine, a global label or a user function without input arguments; otherwise, this error occurs.	reset func(12) A user function with an input argument comes after the reset keyword.
74	Bad jump to label	Occurs when an attempt is made to jump to: A global label from within a function A local label from with a global space A global label from within a global space at a goto statement A non-label at a goto label A local label at a reset statement	function func () ... return ... goto##func The last expression is illegal because func is not a label.
75	Illegal nargout	The nargout keyword is used outside a function.	##START if nargout > 2 The keyword nargout is used outside a function.
76	Function without body	An attempt has been made to call a function that has been defined but does not have a body.	
77	Bad goto statement	The goto keyword must be followed by ## or #@ before the name of a label; otherwise, this	goto START There is no ## or #@ between goto and the label name.

Error Code	Error String	Meaning	Example
		error occurs. This error may also occur if there is a goto statement within the body of a for loop.	for k = 1:10 ... goto##START ... end The goto statement in the for loop is illegal.
78	Auto routine is local	An auto-routine is defined as a local label.	function start (int a) ... #@AUTOEXEC ... return The AUTOEXEC auto-routine is defined inside a function as a local label.
79	Command has 'not program' flag	The program refers to a command that is not allowed to be used within a user program.	LS The program attempts to use the LS command, which has a "Not program" flag defined for it.
80	Image file too long	The Image file length exceeds the user code partition size.	
81	System function tdif is not supported by the Platinum drive	During evaluation of the wait flow-control, the tdif system function must be defined inside the Platinum drive; otherwise, this error occurs.	
82	Command has "not assign" flag	The program has assigned a value to a command with a "not assign" flag.	BG=10 The BG command has a "not assign" flag. Assigning a value to this command is illegal.
83	Keyword is not implemented - for future use	An attempt has been made to use an unimplemented keyword or feature.	

Error Code	Error String	Meaning	Example
84	Misplaced return	Illegal return, such as from a try...catch block	try ... if a > 6 return end catch ... end The return keyword is used inside the try block.
85	The try and catch keywords must be on a separate line	Any code on the same line with try or catch is illegal.	try, AC=10000000 ... The statement after the try keyword is illegal.
86	Division by zero	Division by zero.	
87	Identifier is missing	The #define directive does not contain an identifier.	#define The identifier is missing.
88	The name of identifier is not valid	The name of the identifier of the #define directive is not valid.	#define PI 6.24 #define VER+ The first name has a leading number. The second name contains the + character. A valid name may contain only letters, numbers, or underscores.
89	Identifier was defined before	An attempt has been made to use a #define directive with the same identifier.	#define NUM 3 ... #define NUM 4 The second statement is illegal because NUM has already been defined.
90	Condition is missing	The condition is missing in an #if or #elseif directive.	#if A condition must follow the #if directive.
91	Misplaced continue	The continue keyword is used outside a for or while loop, or inside an unclosed try...catch block.	
92	Misplaced #else or #elseif	An #else or #elseif directive has been used outside an #if-#endif block, or #else is used more than once in the same #if-#endif	#if NUM > 10 #elseif NUM > 5 #else

Error Code	Error String	Meaning	Example
		block, or #else is not the last directive before #endif.	#elseif NUM > 3 #endif The #else directive is not the last directive before the #endif.
93	Identifier wasn't declared	The identifier of an #undef directive was not defined previously.	#undef NUM This statement is illegal if it is not preceded by a #define NUM directive.
94	Mismatched type	The type of an evaluated constant expression is neither an integer nor a float.	
95	Too many identifiers have been defined	The user has defined more than 100 #define directives in the program.	
96	Misplaced #endif	An #endif directive is not preceded by #if.	
97	Cannot perform bit/logic operations on floating point type variables	Bad use of the bit operations	Example: #define aaa 5.5 #define bbb ~aaa

Table 33: Error Codes

6.10 The Preprocessor

Prior to code generation, the compiler preprocesses the user code, handling compiler directives as described in section 6.11 Compiler Directives. The preprocessor does the following:

- Searches for definitions of literal constants, such as **#define MYConst 5**, and replaces literal strings in the user code with their values.

6.11 Compiler Directives

6.11.1 #define

The **#define** directive may be used to assign a name to a literal string in a manner similar to that of the C language.

Syntax

#define identifier string

or

#define identifier

The **#define** directive substitutes the defined string for all subsequent occurrences of the identifier in the text code.

The identifier is replaced only when it forms a real occurrence. For instance, the identifier is not replaced if it appears in a comment, within a string or as part of a longer identifier. For example:

```
//#define UNIT_MOD1 3
```

The above is a comment. There will **not** be an identifier with the value 3.

Note: The #define directive without an identifier is illegal.

A **#define** directive without a string is not replaced. The identifier remains defined and can be tested using the **#ifdef** and **#ifndef** directives.

#ifdef and #ifndef must end with #endif.

The string argument consists of a series of tokens, such as keywords, constants, or complete statements. One or more space characters must separate a string from an identifier. A legal identifier name may consist of a maximum of 32 characters. A valid name consists only of letters, underscores, and numbers (but not leading a leading underscore or number, since a name must start with a letter). The name is case-sensitive.

The maximum admissible number of **#define** directives in a Platinum drive is 100.

When the string is evaluated (refer to the section 6.12 Evaluating Expressions Used in Compiler Directives), if the evaluation is successful, the identifier is replaced with the value obtained; otherwise, every appearance of the identifier is replaced with the string as a string.

The syntax that defines compiler directives in the Platinum language differs from the C language in a number of ways.

The redefinition of a **#define** directive with the same identifier is illegal unless the second definition with **#define** appears after the first definition has been removed by the **#undef** directive.

Examples

The following line defines the identifier `OPTION_A` without associating a constant value with it.

```
#define OPTION_A
```

The following line defines the identifier `MASK_AUTOEXEC` as the constant `0x0001`. Wherever the identifier appears, it is replaced by this value.

```
#define MASK_AUTOEXEC 0x0001
```

6.11.2 #undef

The **#undef** directive, as in C, removes the current definition of the specified identifier. All subsequent occurrences of the identifier name are processed without replacement.

Syntax

#undef identifier

The **#undef** directive must be paired with a **#define** directive in order to create a region in a source program in which an identifier has a special meaning.

Unlike the **#undef** directive in C, with a Platinum drive, you cannot apply **#undef** to an identifier that has not been previously defined. Repetition of the **#undef** directive with the same identifier is illegal.

Example

```
#define DEBUG_FLAG
...
#undef DEBUG_FLAG
```

In this example, the **#undef** directive removes the definition of `DEBUG_FLAG` previously created by the **#define** directive.

6.12 Evaluating Expressions Used in Compiler Directives

The **#define**, **#if** and **#elseif** directives may contain constant expressions for evaluation.

Such expressions – which may be either simple (a single number) or complex (a combination of operations) – must be evaluated to a single number.

A valid expression can operate only with:

- Numbers
- Values of the **#define** directive.

If the expression contains the identifier of a **#define** directive, the identifier must have a string that can be evaluated successfully.

Note: The syntax of the expression of a pre-compilation directive differs from the syntax of other expressions in that it has the following limitations:

- It cannot contain global or local variables; only constant values are valid.
- It cannot use any system or user functions, or system commands.
- Only integer and float data types are allowed. Arrays and array members are illegal.

An expression in a pre-compilation directive uses the same operators as other expressions in the user program.

The following table lists the valid operators according to their type.

Operator Type	Operator
Calculation operators	* / = - %
Logical operators	&&
Comparison operators	== != < > <= >=
Bitwise operators	& << >>
Unary logical operators	!
Unary bitwise operators	~

*The data type of the evaluation result depends on the operation and the type of operands. The result of logical, bitwise and comparison operations is always integer. With calculation operators, if both operands are integers, the type of the result is integer; otherwise, the type is float.

6.13 Program Flow Commands

6.13.1 if...elseif...else...end

Syntax

```
if ( expression )  
    statement1  
elseif ( expression 2 )  
    statement2  
else  
    statement3  
end
```

Conditional Expressions

The **if** keyword executes *statement1* if *expression1* is true (non-zero); if *expression1* is false, **elseif** is present, and *expression2* is true (non-zero), it executes *statement2*.

The **elseif** keyword may repeat scores of times within an **if** block. The statement following the **else** keyword, *statement3*, will be executed only if *expression1*, *expression2*, ... *expressionN* are all false (or zero).

In case of a comparison condition containing a float, for example, if (*I* == 0.001), the float is subject to precision, which is 4 digits after the decimal point.

Example

```
##start  
main()  
function main()  
    short a  
    if ( a==1 )  
        px=1000  
    elseif ( a==3 )  
        px=10000  
    else  
        px=2000  
    end  
return
```



Note: The statement cannot be on the same line as the **elseif** (or the **else**) keyword. The compiler will ignore such entries.

Using a float as a switch condition might have unexpected results.

6.13.2 goto

Syntax

```
goto ##LABEL
```

The program will continue its execution at the label specified by the **goto** command.

Example

```
if (PX>1000)
goto ##LABEL1
else
goto ##LABEL2
end
```

```
##LABEL1
...
##LABEL2
...
```



Note: It is highly recommended not to use labels. We strongly recommend using function calls instead.

6.13.3 switch...case...otherwise...end

Syntax

```
switch (expression)
case (case_expression1)
...
statement1
...
case (case_expression2)
...
statement2
...
otherwise
...
statement
...
end
```

Case Selection

The **switch** statement causes an unconditional jump to one of the statements in the **switch** body or to the last statement, depending on the value of the controlling expression, the values of the **case** expressions and the presence or absence of an **otherwise** label.

The **switch** body is normally a compound statement (although this is not a syntactic requirement).

Usually, some of the statements in the switch body are labeled with **case** labels or with the **otherwise** label.

In addition, labeled statements are not syntactic requirements, but the **switch** statement is meaningless without them.

The **otherwise** label can appear only once and must appear after at least one **case** label. In contrast to the **case** label, the **otherwise** label cannot be followed by an expression for evaluation. The **switch** and **case** expressions may be any logical and/or numerical expression. The **case** expression for each **case** label is compared for equality with the **switch** expression. If the **switch** expression and a **case** expression are equal, then that case is selected and the statements between the matching **case** expression and the next **case** or **otherwise** label are executed. After execution of the statements, a **break** keyword may appear.

It is not necessary for a **case** statement to finish with a **break**, because after executing the statements, an unconditional jump to the end of the **switch** command is performed automatically, and the next **case** statement is not executed.

If two or more **case** expressions match the **switch** expression, then the first matching **case** is selected.

Float

Using a float as a **switch** condition might have unexpected results. If the condition value and the different cases have a precision of a maximum of 4 digits after the decimal point, the comparison process will work fine. Precision of more than 4 digits after the decimal point might suffer from accuracy of less than 100%. In that case, the **switch...case** block will fail.

Examples

Example 1

```
##start
main()
function main()
short a, b, c
a=2
b=2
switch(a+b-2)  // start switch
case 1
    b=a+1
case 2
    c=a+b
case 3
    b=1
otherwise
    a=1
    b=2
```

```
end          // end switch
return
```

Example 2

```
switch(ui[1])

case (XM[1]+ 10000)    //using an expression as a case
    UI[1]=(UI[1]+100)

case XM[2]-30000
    UI[1]=0

end // switch
```

6.13.4 continue

Syntax

continue

The **continue** keyword transfers control to the next iteration of the smallest **for** or **while** loop in which it appears. It thereby enables a jump from the current position to the beginning of the **for** or **while** loop without executing statements through to the end of the loop.

A **continue** keyword outside a **for** or **while** loop is illegal.

The **continue** keyword may appear inside an **if...else** block or a **switch** block.

The **continue** keyword is not allowed within a **try...catch** block unless a **for** or **while** loop is completely enclosed within the **try...catch** block.

Example

```
##start
main()
function main()
int k
for k=1:9
    if (ui[k] == 0)
        continue
    end
    ui[5]=ui[5]+ui[2]
end
return
```

If **ui[k]** is equal to 0, the **continue** keyword transfers control to the next iteration of the **for** loop and skips the line **ui [5] = ui [5] + ui [2]**.

6.13.5 break

Syntax

break

The **break** statement terminates the execution of the nearest enclosing **for**, **switch**, or **while** statement in which it appears. Control passes to the statement that follows the terminated statement.

A **break** statement that is not within a **for**, **switch**, or **while** statement is illegal.

Example

```
##start
main()
function main()
int a, b
while ( a != 0 )
    if ( b == 1 )
        break // break the while loop
    end
end
return
```

6.13.6 exit

Syntax

exit

The **exit** command terminates program execution.

Example

```
##start
main()
function main()
int k
for k=1:9
    if (ui[k] == 0)
        exit // Terminate the program
    end
    ui[5]=ui[5]+ui[2]
end
return
```

If **ui[k]** is equal to 0, the **exit** keyword will terminate the program.

6.13.7 try...catch

A **try...catch** block is used to respond to an expected fault.

Syntax

try

statement; ...; statement

catch

statement; ...; statement

end

The Platinum drive stores the status (stack and base pointers) and executes the statements in the **try** block.

If successful, nothing else happens. If an error occurs, the status is restored, and the Platinum drive executes the **catch** block.

A failure in the **catch** block is treated as an unexpected failure.

Refer to the **MF** command in the Platinum Command Reference.

Example

```
##start
main()
function main()
try
    MO=1
    UM=2          // ACTION THAT WILL FAIL
catch
    MO=0
    wait(100)     // Wait for stabilization
    UM=2
    MO=1
    until (SO==1) // Wait until the "MO=1" process is really complete
end
return
```

6.14 Loops

6.14.1 while

Syntax

```
while (expression)
...
statement
...
end
```

The **while** keyword executes a *statement* repeatedly until *expression* is equal to 0. The expression can be logical and/or numerical.

It may be enclosed within parentheses or without parentheses.

Example

```
##start
main()
function main()
long i          // declare a counting variable
i = 0           // initialize the variable i
while (i < 10 ) // begin while loop body. The while
i = i + 1       // body of the while loop
end             // end of while loop
return
```

6.14.2 for

Syntax

```
for k=N1:N2:N3
...
end           // Iterates k from N1 to N3 with a step of N2.
```

or

```
k=N1:N2
...
end           // Iterates k from N1 to N2 with a step of 1.
```

In both cases, N1, N2, and N3 are numbers or simple expressions.

Remarks

If the iteration step is zero, the program is aborted, and the error code INFINITE_LOOP is issued.

If N1, N2, or N3 is a variable, it is evaluated once before the iteration begins. If the variable changes within the **for** loop, the iteration process is not affected.

The iteration variable **k** must be declared as a variable. The iteration variable must be scalar, not an array member. For example, the expression **for k[10]=1:10** is illegal, because **k** is an array. In that case, the compiler will report an error ("Variable is array" code error 57).

It is illegal to use iteration variables in a loop body. This may cause unexpected results.

Example

```
##start
main()
function main()
int i
for i=1:2:100 // Begin 'for' loop: 1 is the initial value of 'i'
    // 2 is the step. i = i + 2
    // 100 is the condition value.
    // When i == 100, -> finish the loop.
    ui[1] = ui[1] + 1 // Body of the loop
end
return
```

6.14.3 until

Syntax

until (expression)

The **until** keyword suspends execution of the program until the expression becomes true. The expression can be logical and/or numerical. Note that only the program is suspended and not any of the other drive's functions.

Example

```
until ((sr&1) == 0) // program will test the bit 0 of sr register,
                    // repeatedly, until it get a non zero value.
```


6.15 Program Delay Command

6.15.1 wait

The **wait** function delays execution of the program.

Syntax

```
wait(x)
```

The argument *x* is the delay time in milliseconds.

Example

```
wait ( 200 )    // delay the program for a 200 msec interval
```

Note: The delay occurs in the program. None of the other drive functions, interpreters, or communication is delayed due to it.

6.16 User Program Multi thread

User program multi thread is an option to run several instances at the same background cycle.

The parallel is created by a special function that called "thread".

Thread#(): initiates the tread execution and runs the first line of the created thread.

Thread function executes their "body code" in parallel with other thread functions.

The Main program (thread 0) must be active all times

Thread can be called from any running thread.

Thread functions are called thread1()/...thread5().

Maximum #define **USER_PROGRAM_NUM_THREADS** (6).

Syntax

```
thread thread1()  
/* Write code here*/  
return
```

Example

```
1. ##start  
2. function main()  
3. short a  
4. short i,j  
5. a = 5  
6. thread1()  
7. thread2()  
8. for i = 0:1:1000000  
9. j = i+1;  
10. wait(200);  
11. end  
12.  
13. return  
14.  
15.  
16. thread thread1()  
17. long a  
18. a =7  
19. return  
20.  
21. thread thread2()  
22. long a  
23. a = 8  
24. return
```

The flow is:

Background Cycles	1	2	3	4	5
Main code line	5	6	7	8	9
Thread1 line	-	18	19	-	-
Thread2 line	-	-	23	24	-

Lines order: 5,6,18,7,19,23,24,9

6.16.1 Debugging and breakpoints in multi thread

16 breakpoints are available for all threads.

Multiple threads can stop at the same breaking point.

In debug mode all the threads run in parallel.

Once thread is stopped in breakpoint – the other threads continue to run.

Debug commands such as: **continue**, **step Into**, **step Over**, and **Step Out** are called separately for each thread.

Example

```
##start
main()
function main()
thread1()
thread2()
thread3()
    wait(100);
end
return
thread thread1()
Banana()
return

thread thread2()
Banana()
return

thread thread3()
    wait(100);
    ui[1] = ui[1] +1
end
return

function Banana()
    ui[2] = ui[2] +1
return
```

thread 1 and thread 2 call to “banana” function. If there is a breakpoint in “banana” function – thread 1 will stop at this break-point then thread 2 will stop at this breakpoint.

6.17 Auto-Routines

An automatic routine (auto-routine) is a special type of routine that is executed automatically in response to a specific system event. These routines are executed only when the invocation condition of the auto-routine is satisfied.

For example, the AUTO_BG auto-routine will be called when a digital input, which is configured as the function **BEGIN**, becomes active.

In that case a **BG** command is also performed.

The user cannot expect that one event will precede the other one.

Auto-routines have no output and input arguments.

Auto-routines can be masked (running disabled) using the **MI** command.

Syntax

```
#@AUTO_name  
...  
return
```

The auto-routine syntax begins with characters #@ followed by one of the protected auto-routine names, for example, #@AUTO_I1.

The auto-routine body is ended by the keyword **return** (except in the case of the AUTOEXEC auto-routine).

The following table lists the protected names for auto-routines.

Auto-routine name	Precedence	Masking
BIT_AUTOEXEC	1 Highest after AUTO_PERR	MI = 1
BIT_AUTO_ER	2	MI = 2
BIT_AUTO_STOP	3	MI = 4
BIT_AUTO_BG	4	MI = 8
BIT_AUTO_RLS	5	MI = 16
BIT_AUTO_FLS	6	MI = 32
BIT_AUTO_ENA	7	MI = 64
BIT_AUTO_I1	8	MI = 128
BIT_AUTO_I2	9	MI = 256
BIT_AUTO_I3	10	MI = 512
BIT_AUTO_I4	11	MI = 1024
BIT_AUTO_I5	12	MI = 2048
BIT_AUTO_I6	13	MI = 4096
BIT_AUTO_HM	14	MI = 8192
BIT_AUTO_HY	15 Lowest	MI = 16536
BIT_AUTO_PERR	Highest precedence	Non-maskable

Table 34: Protected Auto-routine Names

6.17.1 AUTOEXEC

AUTOEXEC is a protected auto-routine name.

It is used to declare a user program as an auto-execution program.

The first time that a user program is run, the user must verify that the auto-execute program was downloaded successfully and that it runs as expected. After the user program is saved in the flash memory of the drive, it will start running automatically on power-up.

Masking

Either of the following lines sets masking of the execution of the AUTOEXEC auto-routine.

```
MI = 1  
MI = 0x1
```

Example

```
#@AUTOEXEC  
  
main()  
  
function main()  
...      // body of the program  
Return
```

Notice that the virtual machine refers to <#@AUTOEXEC> as an entry point label. In order to run the function main(), a function call to main() is required.

6.17.2 AUTO_ER

Syntax

```
#@AUTO_ER  
...  
return
```

AUTO_ER will run in response to a motor fault event.

For example, a motor fault event occurs if the motor is on (**MO**==1) and the bus voltage is disconnected (power-supply failure). In this case, the program will jump to the AUTO_ER auto-routine, where fault handling is performed.

The reason of auto error can be retrieved with **MF** command.

Masking

Either of the following lines sets masking of the execution of the AUTO_ER auto-routine.

```
MI = 2  
MI = 0x2
```

6.17.3 AUTO_STOP

Syntax

```
#@AUTO_STOP  
...  
return
```

AUTO_STOP is called when a digital input is configured for the Hard Stop function (**IL**[*n*] = 21, *n*=1 to 6) and it becomes active. During the time when the Hard Stop function is active (the STOP switch is held continuously on, for example), the motor motion is stopped and held (if it was performing a motion). The AUTO_STOP auto-routine can be used to indicate that a HARD_STOP occurred and to synchronize that event with other processes.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking

Either of the following lines sets masking of the execution of the AUTO_STOP auto-routine.

```
MI = 4  
MI = 0x4
```

6.17.4 AUTO_BG

Syntax

```
#@AUTO_BG  
...  
return
```

AUTO_BG is called when the digital input *n* is configured for the Begin function (**IL**[*n*]=13, *n*=1 to 6) and it becomes active.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking

Either of the following lines sets the masking of the execution of the AUTO_BG auto-routine.

```
MI = 8  
MI = 0x8
```

Example

```
function main()  
...  
UI[2]=1  
IL[2]=13    //Digital input 2 set to BG function  
UM=5
```

```
wait(200)
MO=1
until (SO==1)
PA=9000    //set the absolute target position. Wait for BG

while (UI[2]) // Wait for AUTO_BG
end

wait(2000) //delay
...
return

#@AUTO_BG
UI[2]=0    // indication that AUTO_BG is being executed
...
return
```

In this example digital input 2 (**IL[2]**) is set to be a **BG** input.

PA is set to position 9000.

When the digital input 2 becomes active, an **AUTO_BG** occurs, and the **BG** command is performed (the motor starts moving).

6.17.5 AUTO_RLS

Syntax

```
#@AUTO_RLS
...
return
```

AUTO_RLS is called when the digital input n is configured for the RLS function (**IL[n]**=9 or 8, $n=1$ to 6) and it becomes active.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking

Either of the following lines sets the masking of the execution of the **AUTO_RLS** auto-routine.

```
MI = 16
MI = 0x10
```

6.17.6 AUTO_FLS

Syntax

```
#@AUTO_FLS
...
return
```

AUTO_FLS is called when the digital input n is configured for the FLS function (**IL[n]**=11 or 10, $n=1$ to 6) and it becomes active.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking

Either of the following lines sets the masking of the execution of the AUTO_RLS auto-routine.

```
MI = 32
MI = 0x20
```

6.17.7 AUTO_ENA

Syntax

```
#@AUTO_ENA
...
return
```

AUTO_ENA is called when digital input n is configured for the INHIBIT function (**IL**[n]=1 or 0, n =1 to 6) and it is changed from an **active** to a **not active** state.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking

Either of the following lines sets the masking of the execution of the AUTO_RLS auto-routine.

```
MI = 64
MI = 0x40
```

6.17.8 AUTO_I1 to AUTO_I6

Syntax

```
#@AUTO_I1 // or #@AUTO_I2 to #@AUTO_I6
...
return
```

AUTO_I1 to AUTO_I6 are called when a digital input (1 to 6) is configured for a General Purpose Input function and it becomes active.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking (AUTO_I1 to AUTO_I6)

The following table lists the values to which the **MI** command is set in order to mask the execution of the AUTO_I1 to AUTO_I6 auto-routines for digital outputs 1 to 6.

Digital Input	Masking Value (MI Setting)
1	128 (0x80)
2	256 (0x100)
3	512 (0x200)
4	1024 (0x400)
5	2048 (0x800)
6	4096 (0x1000)

Example

The program example below uses general purpose digital inputs and auto-routines (AUTO_In).

```
/*  
    Switches (digital inputs) will command the motor and move it to  
    different positions.  
*/  
function main()  
    IL[2]=7      // set input as general purpose  
    IL[3]=7  
    IL[4]=7  
    IL[5]=7  
    MO=0  
    UM=5  
    UI[1]=0  
    while(1)  
        MI=0      // enable auto-routines  
        until(UI[1]!=0) // wait for user entry  
        MI=0x0F00 // disable auto-routines (IN2..IN5)  
        MO=1      // set motor ON  
        until(SO==1) // wait until motor is ready ON  
        PA=UI[1]  // set the absolute target position  
        BG        // begin  
        UI[1]=0   // prepare ui[1] for the next user entry  
    end  
    return  
  
#@AUTO_I2  
    UI[1]=9000    // set motor variable to new position  
    return  
  
#@AUTO_I3  
    UI[1]=-9000  
    return  
  
#@AUTO_I4  
    UI[1]=19000  
    return  
  
#@AUTO_I5  
    UI[1]=-19000  
    return
```

6.17.9 AUTO_HM

Syntax

```
#@AUTO_HM  
...  
return
```

AUTO_HM is called when the digital input 5 is configured for the MAIN HOME function (**IL**[5]=16 or 17) and becomes active.

Refer to the **IL** command section in the Command Reference for Platinum Line Drives.

Masking

Either of the following lines sets the masking of the execution of the AUTO_HM auto-routine.

```
MI = 8192  
MI = 0x2000
```

6.17.10 AUTO_PERR

Syntax

```
#@AUTO_PERR  
...  
return
```

AUTO_PERR is called when a run-time error occurs.

AUTO_PERR is used to handle a situation, which can cause a crash while the user program is running (see the example below).

The AUTO_PERR routine automatically blocks all other auto-routines; therefore, the **MI** command should be used to restore a response to them.

The reason for an auto program error can be retrieved with the **EC** command.

Masking

Either of the following lines sets the masking of the execution of the AUTO_HM auto-routine.

```
MI = 32768  
MI = 0x8000
```

Example

```
function main()  
while(1)  
  until(UI[1]!=0)  
  MO=1  
  until(SO==1)  
  PA=UI[1]  // set the absolute target position  
  BG  
  UI[1]=0  
end
```

```
return
```

```
#@AUTO_PERR
```

```
wait(200)
```

```
MO=0
```

```
UI[1]=0
```

```
reset main
```

```
return
```

In this example, the user entry values to **PA** use **UI[1]** (refer to the command in the Command Reference). The motor will move to the target set by **PA**.

An out of range value might terminate the program unless an **AUTO_PERR** exists and can handle this fault.

The **reset** command clears the stack and jumps back to the beginning line of function main.

6.18 User Program Stability Protection (AUTO_PERR, try...catch)

The user program system includes a run-time error protection mechanism, which consists of the **AUTO_PERR** auto-routine and the **try...catch** blocks.

The **AUTO_PERR** auto-routine is a mechanism that is used to handle errors after their occurrence. Refer to the section 6.17.10 AUTO_PERR.

The **try...catch** block is illustrated by the following example.

```
##start
main()
function main()
try
    MO=1          // action that might fail
catch
    until ( (SR&1) == 0 ) // correct action in case of a failure
    wait (100)
    MO=1
end
return
```

The **Try...catch** mechanism can catch a fault and handle the fault that occurred. If the **catch** part fails to handle the fault, the program may need to be terminated.

The **AUTO_PERR** mechanism can catch a program fault, solve it, and, usually, **reset** (clear the stack and jump) to a safe location in the program. At that location, generally, the program is initialized and run again without termination. Refer to the **reset** user program command.

In case the **try...catch** block fails in the **catch** part, **AUTO_PERR** can handle the failure.

A combination of the **try...catch** mechanism and **AUTO_PERR** is recommended for better program stability.

6.19 Reordering Global Variables

An 8-entry global user variable can be mapped to **RV[x]**, which is a recorder variable (with indices up to 8).

The **DB##RV[1]** to **DB##RV[8]** commands map specified global user variables to recorder variables 1 to 8.

The following procedure shows how to record a global user variable:

1. Using the EAS control panel, download a program to the drive.
2. Choose the global variable to be recorded, for example, MyGlobalRec.
3. Using the EAS terminal, enter a mapping command for the required variable, for example:
DB##RV[1] = MyGlobalRec
4. In the EAS Recording control panel, the map signal **General>Program Var1..8 to the Recorder** is displayed.



Note: The command **DB##RV[N]=varname** is needed after each user program download.

The user program variable *varname* can also be an array, for example, **arrname[4]**. The command **DB##RV** must be given the precise name of array and its index, for example: **DB##RV[3]=arrname[2]**.

6.20 User Program Restrictions and Limitations

Item	Limit	Remarks
Number of virtual machines (maximum active user programs)	1	Only one user program can be active (except "thread" that can run in parallel)
User program text maximum size	65536	bytes
User program compiled code maximum size	65536	bytes
Maximum length of the data segment— space for storing global variables	2048	Integer or float
Maximum number of functions	192	Maximum number of user functions, system functions, labels, auto routines
Maximum number of user functions	50	Maximum number of user functions
Maximum input arguments of user function	16	
Maximum output arguments of user function	16	
Maximum number of words in program line	128	
Maximum name length of function or variable name	12	Number of characters
Maximum expression size	256	In bytes
Stack memory size	2048	
Maximum number of break points	12	max number of break points in user program
Maximum long value	2147483647	
Maximum float value	3.402823466e+38	
Minimum float value	1.175494351e-38	

6.21 Examples

The program below demonstrates point-to-point absolute motion.

```
##START
P2P_Abs(100000,50000)

function P2P_Abs(int position, int Speed)
if (UM!=5) // Ensure motor is in position mode
    MO=0
end
if (MO==0)
    UM=5 // Set position mode
    MO=1 // Enable motor
end
PA= position // Set P2P motion target position
SP= Speed // Set P2P motion speed
BG // Begin motion
return
```

The program below demonstrates point-to-point relative motion:

```
##START
P2P_Rel(100000,50000)

function P2P_Rel(int Distance, int Speed)
if (UM!=5) // Ensure motor is in position mode
    MO=0
end
if (MO==0)
    UM=5 // Set position mode
    MO=1 // Enable motor
end
PR= Distance // Set P2P motion target position
SP= Speed // Set P2P motion speed
BG // Begin motion
return
```

The program below is an example of sinusoidal motion:

```
##START
VelocitySin (5000,20000)
return

function VelocitySin(int Duration,float amplitude)
    int InitTime
    int time
    float VelCoeff
    Duration= Duration*1000 // microseconds
    VelCoeff = 2*3.14159265/Duration
```

```

MO=1
InitTime=TM           // Get time in microseconds
while(1)
    time= TM-InitTime
    if time >Duration
        InitTime=InitTime+Duration
        time= TM-InitTime
    end
    JV=amplitude*sin(VelCoeff*time)  // Speed of motion
    BG
end
return

```

The program below is an example of current mode motion:

```

function main()

    MO=0    // motor off
    UM=1    // current unit mode
    RM=0    // disable the reference mode
    wait(100)
    MO=1    // motor on
    until(SO==1)
    TC=0.1  // software current command

    while(1)
    end
return

return

```

The program below is an example of current mode motion, with reference to sub-index 1, if commands, vectors are not strings:

```

function main()

    MO[1]=0    // motor off
    UM[1]=1    // current unit mode
    RM[1]=0    // disable the reference mode
    wait(100)
    MO[1]=1    // motor on
    until(SO[1]==1)
    TC[1]=0.1  // software current command

    while(1)
    end
return

```

```
function main()
    MO=0
    UM=5
    UI[1]=0
while(1)
    until(UI[1]!=0) // Wait for user entry
    MO=1           // Set motor ON
    until (SO==1) // Wait until motor is ready ON
    PA=UI[1]       // Set the absolute target position
    BG            // Begin
    Ui[1]=0        // Prepare UI[1] for the next user entry
end
return

// In case of any error, the below auto-routine will run automatically
#@AUTO_PERR
wait(200)
PA=0
MO=0
UI[1]=0
reset main      // Clear stack. Jump to main() function.
return
```

The above example allows control of the target position by setting **UI[1]**. This command is sent from an external port (RS232, USB....).

The program is protected against a wrong value in **UI[1]**. In this case, AUTO_PERR runs and can handle errors.

A reset command (reset main) clears the stack of any function information and then jumps back to main function.

The following example is the above example revised using **UI** instead of **UI[1]**:

```
function main()
    MO=0
    UM=5
    UI=0           // UI is equal to UI[1]
while(1)
    until(UI!=0)   // Wait for user entry
    MO=1           // Set motor ON
    until (SO==1) // Wait until motor is ready ON
    PA=UI          // Set the absolute target position
    BG            // Begin
    UI=0           // Prepare UI[1] for the next user entry
end
return

// In case of any error, the below auto-routine will run automatically
#@AUTO_PERR
wait(200)
```


PA=0

MO=0

UI=0

reset main // Clear stack. Jump to main() function.

return

A user program is high-level code, which the user can write and execute in a Platinum drive.

The code can include most of the interpreter commands (for example, **motion**, **status**) and functions (for example **sin()**) and allows auto routines, which provide a means to interrupt the user program code and jump to special event routines (for example, after digital input is set or in case of a fault).

Using the Elmo Application Studio (EAS) program editor, the user program can be edited, downloaded, debugged and even attached for debugging purposes during execution.

The user program runs in the background task of the drive. This means that it has the same precedence as the interpreter and allows the execution of user commands with no conflicts, regardless of the communication source.

Commands that are in the same user program line are completed before the next command line is processed.

The following chapter describes the properties, usage abilities, execution, and limits of user programs.

Chapter 7: Drive Errors and Warnings

The following chapter describes the Platinum Drive Sensor errors and warnings.

7.1 BIT – Built In Tests

At power-up, the drive performs built-in tests. The tests results are reported as errors, warnings and status via the **BZ[1]** command. **BZ[1]** includes safety diagnostics errors that are reported during the operation of the drive as well.

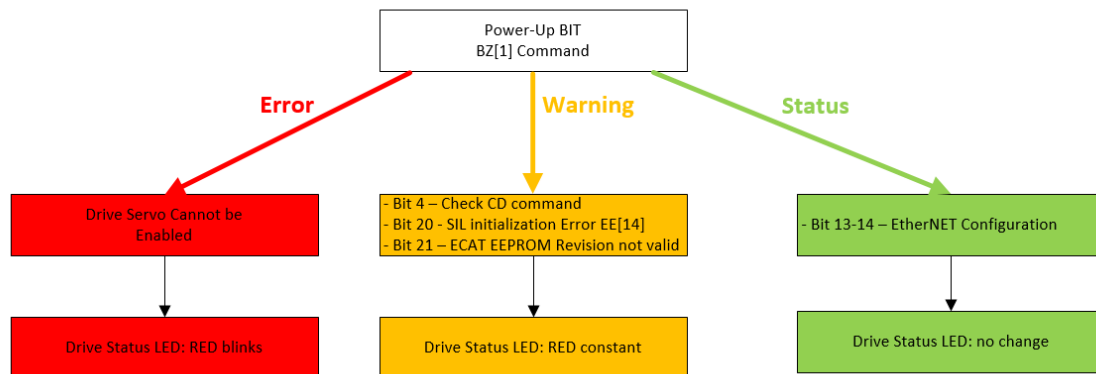


Table 35: BIT



Relevant commands

BZ[], EE[14]

7.2 Drive Continuing Status

Drive and axis continuing status is reported via **SR** command. Object **0x3607** is the alias object and can be mapped to a tPDO. **SR** is axis depended.

The SR reports errors, warnings, and status. The following bits of **SR** report errors and warnings:

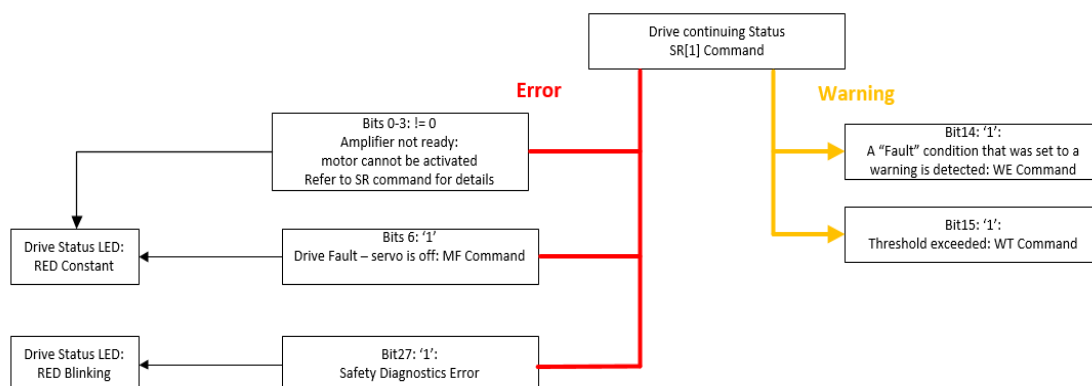


Table 36: Drive Continuing Status

7.2.1 Optional Status

SR[2] and **SR[3]** provides status information about the SIL and STO state.

SR[2] bit 19 indicates error in SIL operation

SR[3]: provides further information about the STO status



Note: Relevant commands

SR[]

7.3 Error Codes Originating from ELMO Commands

When ELMO command is sent to the drive (refer to Chapter 2: The Platinum Drive) from external host and the command cannot be processed or include an error, the drive will response with a specific indication and with the relevant error. The **EC command** indicates the error. For details of the error list refer to the “Drive General Error list” in the Platinum Command Reference manual.

Some errors have additional detailed that can be retrieved via the “Extended Error” command (**EE[]**).

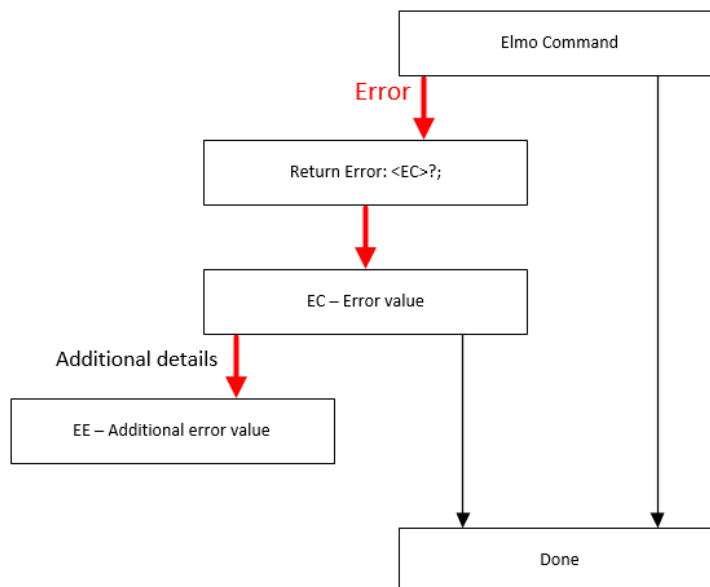


Table 37: EC - Error Codes

7.4 Emergency (EMCY)

Platinum servo drives issue an emergency code in response to an abnormal condition. All emergencies can be masked. For a description of emergency codes that can be masked, refer to [object 0x2F21](#).

The Emergency object COB-ID is 0x81 to 0xFF.

The structure of the manufacturer-specific emergency message is as follows:

Byte	Description
0,1	Emergency Error code
2	Emergency Error register
3	ELMO error code
4	Data byte 4 For ECAT: Axis number (0 – axis 1, 1 – axis 2...) For CAN: Axis number for axis related emergency or detailed data for non-axis related emergency. See table below
5-7	Data bytes 5-7, reserved



Note:

Unused/reserved bytes must be set to zero.

7.4.1 Emergency Codes

The following table (Table 45: Emergency Error Codes) lists the Emergency Error codes and their description with the relevant ELMO code where applicable.

Note, Axis number is 0 – for axis 1, 1- for axis 2..etc

“Elmo Error Code” is listed in the Command Reference manual (EC command & “Drive Error Code”)

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
Short protection (Amplifier Fault)	2340	03	0	Axis number	MF=32768 (0x8000) Motor disabled immediately due to short protection. Refer to MF command description in Command reference
Under voltage (Amplifier Fault)	3120	05	0	Axis number	MF= 4096 (0x1000) Motor disabled immediately due to under voltage protection. Refer to MF command description in Command reference

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
AC fail, loss of phase	3130	11	0	Axis number	MF= 32 (0x20) Motor disabled immediately due to loss of phase. Refer to MF command description in Command reference
Over voltage (Amplifier Fault)	3310	05	0	Axis number	MF= 8192 (0x2000) Motor disabled immediately due to overvoltage protection. Refer to MF command description in Command reference
Motor over-temperature (Amplifier Fault)	4210	09	0	Axis number	MF=65536 (0x10000) Motor disabled immediately due to motor over temperature protection. Refer to MF command description in Command reference
Drive over-temperature (Amplifier Fault)	4310	09	0	Axis number	MF=67108864 (0x4000000). Motor disabled immediately due to drive over-temperature protection. Refer to MF command description in Command reference
Gantry Yaw error limit exceeded	5280	81	0	Axis number	MF=1024 (0x400) Deceleration and servo off in accordance with fault reaction option code object 0x605E. Gantry Yaw error exceeded the value of ER[5] . Refer to MF command description in Command reference
External Inhibit was triggered (INH/ENB)	5441	21	0	Axis number	MF= 16 (0x10) Deceleration and servo off in accord with a fault reaction option code Object 0x605E tag=Object_0x605E_Fault_reaction_option_code due to Inhibit or Abort digital input. Refer to MF command description in Command reference
Additional Abort input is active	5442	81	0	Axis number	MF=33554432 (0x2000000) Deceleration and servo off in accord with a fault reaction option code object 0x605E tag=Object_0x605E_Fault_reaction_option_code due to additional abort input active

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
					state. Refer to MF command description in Command reference
Vector abort	6181	81	0	Axis number	MF=524288 (0x80000) All vector axes stopped immediately on fail of any axis
rPDO failed	6300	01	ELMO Error Code	Axis number	The action defined by received PDO failed. Refer to Command Reference, chapter "Drive General Error List"
Motor stuck	7121	21	0	Axis number	MF=2097152 (0x200000) Deceleration and servo off in accordance with fault reaction option code object 0x605E due to motor stack protection. Refer to MF command description in Command reference
Feedback error	7300	81	0	Axis number	MF=1 (0x1). Motor disabled immediately due to sensor error. Refer to MF command description in Command reference
Hall main feedback mismatch	7380	81	0	Axis number	MF=4 (0x4) Motor disabled immediately due to illegal Hall combination. Refer to MF command description in Command reference
Hall's sensor speed is too high or disconnected.	7381	81	0	Axis number	MF=64 (0x40) Deceleration and servo off in accordance with fault reaction option code object 0x605E. Refer to MF command description in Command reference
Commutation process fail	7382	81	0	Axis number	MF=2 (0x2). Motor disabled immediately due to commutation process fail. Refer to MF command description in Command reference
CAN message lost (overrun HW or SW buffer)	8110	11	ELMO Error Code	0x4000 (Sync lost) 0x2000 (rPDO lost)	Refer to Command Reference, chapter "Drive General Error List"

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
				0x200 (NMT lost) 0x100 (SDO lost)	
Communication failed, loss of synchronization or frame loss.	8130	11	ELMO Error Code	For CANopen drive: Node ID For ECAT: Axis number	MF=2048 (0x800). For ECAT drives: Deceleration and servo off in accordance with fault reaction option code object 0x605E. Motor is disabled due to loss of synchronization with ECAT Master. Refer to MF command description in Command reference For CANopen drives: Drive reaction according to object 0x1029 "Error behavior" and object 0x6007 "Abort Connection Option code". The possible reason is tag=Object_0x605E_Fault_reaction_option_code tag=Object_0x1016_Consumer_heartbeat_time a heartbeat violation, according to CANopen DS-301 object 0x1016, tag=Object_0x1016_Consumer_heartbeat_time bus off state or CANopen master NMT STOP command. Refer to Command Reference, chapter "Drive General Error List" Refer to MF command description in Command reference Refer to 0x1029, 0x6007 object description
Recovered from bus off	8140	11	ELMO Error Code	0	Relevant for CANopen only. Drive reaction according to object 0x1029 "Error behavior" and object 0x6007 "Abort Connection Option code" tag=Object_0x605E_Fault_reaction_option_code tag=Object_0x1016_Consumer_heartbeat_time tag=Object_0x1016_Consumer_heartbeat_time Refer to Command Reference, chapter "Drive General Error List" Refer to 0x1029, 0x6007 object description

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
Attempt to access a non-configured rPDO	8200	01	0	0	For CANopen drives only: Received rPDO is not valid or not configured
Incorrect rPDO length	8210	21	0	0	For CANopen drives only: Received rPDO has length different from defined by PDO configuration
Peak current has been exceeded.	8311	21	0	Axis number	MF=8 (0x8) Motor disabled immediately. The current has exceeded the value of MC but has not yet reached the level of a short. Refer to MF command description in Command reference
Speed tracking error exceeded speed error limit.	8480	81	0	Axis number	MF=128 (0x80) Deceleration and servo off in accordance with fault reaction option code object 0x605E. Speed error exceeded the value defined in ER[2] . Refer to MF command description in Command reference
Speed limit exceeded	8481	81	0	Axis number	MF=131072 (0x20000) Deceleration and servo off in accordance with fault reaction option code object 0x605E. Refer to MF command description in Command reference
Position tracking error exceeded position error limit	8611	21	0	Axis number	MF=256 (0x100) Deceleration and servo off in accordance with fault reaction option code object 0x605E. Position error exceeded the value defined in ER[3] . Refer to MF command description in Command reference
Feedback is out of position limits	8680	81	0	Axis number	MF=4194304 (0x400000) Deceleration and servo off in accordance with fault reaction option code object 0x605E due to position limit protection. Refer to MF command description in Command reference

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
					tag=Object_0x605E_Fault_reaction_option_code
CAN Interpolated mode emergency	FF02	81	ELMO Error Code	Axis number	For CANopen drives only: Either of the possibilities: IP mode underflow Interpolation queue full (overflow) Reference received in a wrong index. This EMCY is transmitted when operating in Profile Interpolated mode, sub mode 0 and rPDO with object 0x60C1 sub index 2 was received. Bad PVT send order Refer to Command Reference, chapter "Drive General Error List"
Failed to start motor	FF10	81	ELMO Error Code	Axis number	MF=536870912 (0x20000000) Cannot start motor due to internal problem. Refer to MF command description in Command reference. Refer to Command Reference, chapter "Drive General Error List"
Safe Torque Off (STO) switch are disabled. Safety state. (Amplifier Fault)	FF20	05	0	Axis number	MF=16384 (0x4000) Motor disabled immediately due to Safe Torque Off. STO state is reported via SR bits 25 and 26. Refer to MF command description in Command reference
Motor disabled by local command	FF30	81	0	Axis number	MF=1073741824 (0x40000000) MO=0 command received from interpreter or user program. Deceleration and servo off in accordance with AB[1] command "Motor OFF option code"
Motor disabled due to kinematics error	FF34	81	0	Axis number	MF=8388608 (0x800000) Motor immediately disabled due to kinematics error
Slave motor disabled due to	FF35	81	0	Axis number	MF=134217728 (0x8000000)

EMCY name	EMCY Error Code (Hex)	Err Reg (Hex)	ELMO Error Code	Data Byte 4	Related MF value/description
Gantry master error					Slave motor disabled immediately due to Gantry master error
Gantry Slave Disabled	FF40	81	0	Axis number	MF=16777216 (0x1000000) Deceleration and servo off in accordance with fault reaction option code object 0x605E. Gantry master disabled because Gantry slave is not enabled at current mode. Refer to MF command description in Command reference
Attached slave drive fault (Gantry)	FF50	81	0	Axis number	MF=268435456 (0x10000000) Motor disabled immediately due to failure in attached slave drive. Refer to MF command description in Command reference

Table 38: Emergency Error Codes

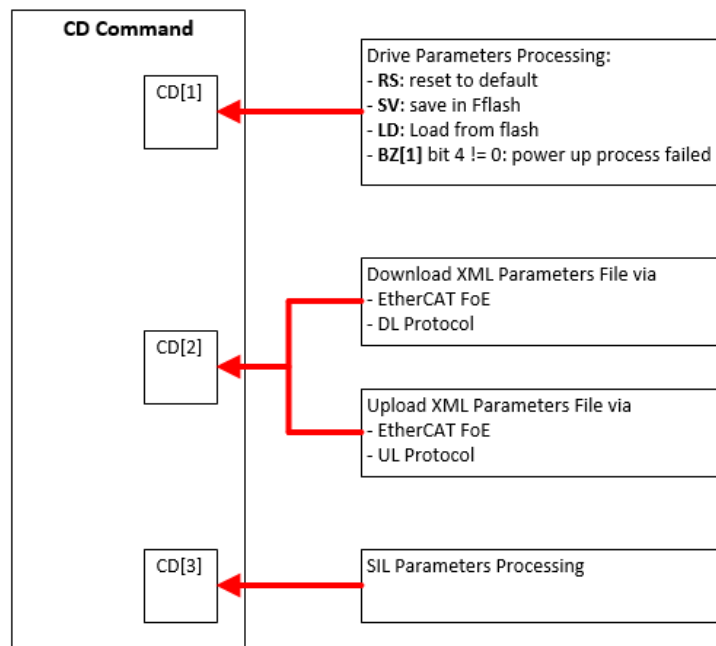
7.5 Drive Non-volatile Parameters

Drive non-volatile parameters are set of values that used for the drive configuration (e.g. control loop gains) and saved in a non-volatile flash memory. The parameters are loaded from the flash and processed during power up. The following procedures can be done by the user:

- Save parameters to the non-volatile memory
- Load parameters from the non-volatile memory
- Reset the parameters to factory default values.
- Download the parameter file to the drive memory using XML format
- Upload the parameters to the host file in XML format file

The upload and download procedures are done via ELMO proprietary UL & DL commands or via the standard EtherCAT FoE protocol (File Over EtherCAT).

In situations of an error in one of the above procedures, the error is reported via **CD** command (Object **0x3055**) as follows:



CD command format and attributes are detailed in the Platinum Command Reference.

7.6 Safety Errors

The Safety process includes two types of errors:

- **Diagnostics Errors** – Safety hardware components and safety software portions are tested periodically for failure detection
- **Monitoring Violation Errors** – when monitored threshold limit of a safety function is violated

The following sub-sections describe the processing flow of the safety errors:

7.6.1 Diagnostics Errors

Safety diagnostics errors are monitored as follows:

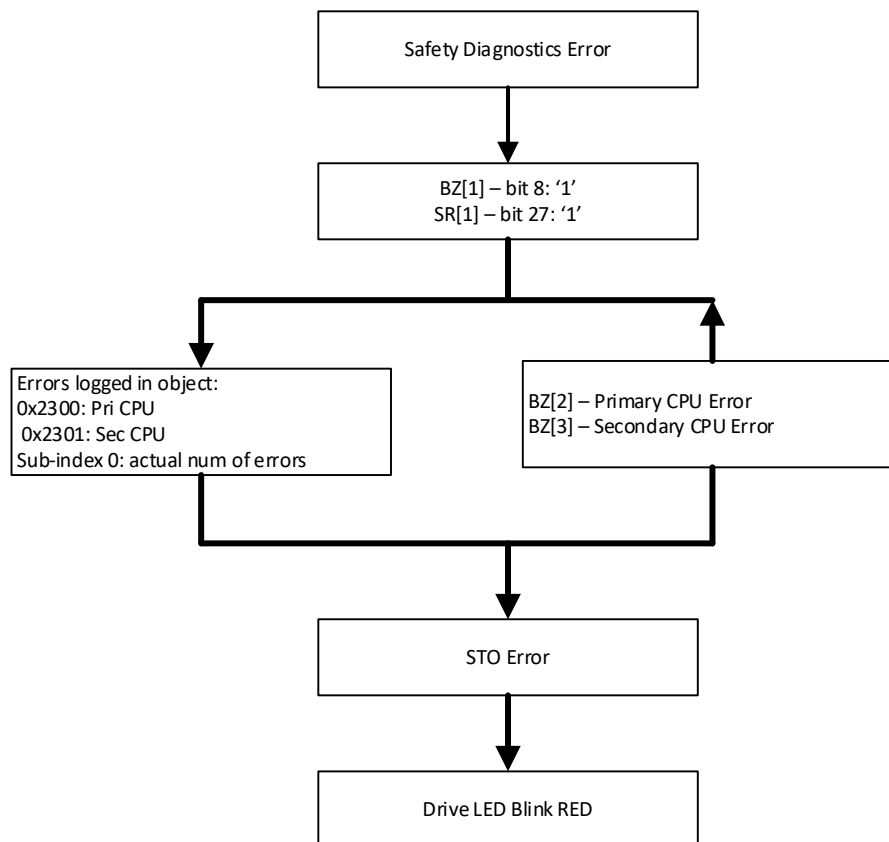


Table 40: Safety diagnostics errors

The EAS *Safety Monitoring* window details the diagnostics error in the relevant tabs:

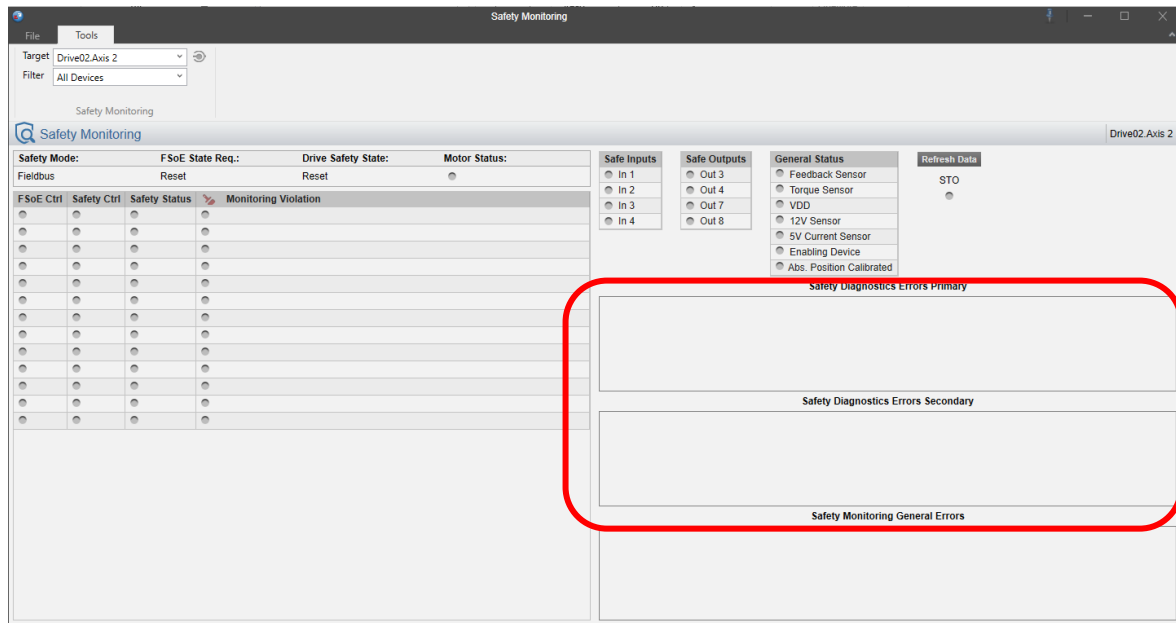


Table 41: EAS Safety Monitoring window

7.6.1.1 Diagnostic Error List

Error Prefix 0x0000 Error number	Error Details	Remedy comments
0x1	Memory Test Fail	Power up
0x2 – 0x6	CRC errors	
0x7 – 0x15	Watchdog, peripheral error	

Table 42 : Infrastructure diagnostics errors

Error Prefix 0x0100 Error number	Error Details	Remedy comments
0x1 – 0xE	STO 1 faults	Power up
0x11 – 0x28	STO 2 faults	
0x30 – 0x3B	STO general faults	
0x3C	Error discrepancy timeout between STO1 and STO2	
0x3D	STO auto periodic fail	

Table 43: STO diagnostics errors

Error Prefix 0x0200 Error number	Error Details	Remedy comments
0x01 – 0x06	Error Acknowledge and Restart Acknowledge state error	Power up
0x08 – 0x0E	SBC, Diagnostics state	
0x10 – 0x13	SLS, Diagnostics state	
0x14- 0x1D	SOS, Diagnostics state	
0x1A – 0x1E	STO, Diagnostics state	
0x20 – 0x25	SS1, Diagnostics state	
0x30 – 0x36	SS2, Diagnostics state	
0x40 – 0x42	SLT, Diagnostics state	
0x43	SLS, Diagnostics state	
0x50 – 0x52	SLP, Diagnostics state	
0x60 – 0x63	SDI, Diagnostics state	
0x64 – 0x65	SMS, Diagnostics state	
0x66 – 0x67	SLI, Diagnostics state	
0x68 – 0x69	SSM, Diagnostics state	
0x70 – 0x71	SCA, Diagnostics state	
0x72 – 0x73	SBT, Diagnostics state	
0x74 – 0x75	SXT, Diagnostics state	
0x76 - 0x80	SSR, Diagnostics state	
0x81 – 0x82	SAT, Diagnostics state	

Table 44: Safety function errors

Error Prefix 0x0300 Error number	Error Details	Remedy comments
0x030000000	Local reset or acknowledgement of RESET command	Power up
0x030000001	The received command is not allowed in the state	Power up
0x030000002	The received command is not defined	Power up
0x030000003	The connection does not match the connection ID transferred in the connection state	Power up
0x030000004	At least one of the received CRC_i does not match the calculated CRC_i	Power up
0x030000005	No valid Safety PDU was received within the FSoE watchdog time	Power up

Error Prefix 0x0300 Error number	Error Details	Remedy comments
0x030000006	The FSoE Slave Address transferred in the Connection state does not match the local address set on the FSoE Slave	Power up
0x030000007	The safety data sent back by the FSoE Slave in the Session, Connection and Parameter states do not match the expected values	Power up
0x030000008	The length of the communication parameter is wrong	Power up
0x030000009	The content of the communication parameter is wrong	Power up
0x03000000A	The length of the application parameter is wrong	Power up
0x03000000B	The content of application parameter is wrong	Power up
0x030000080	The FSoE stack got to an invalid internal state	Power up
0x030000081	The FSoE SRA CRC is not valid	Power up

Table 45: Safety FSoE State Machine errors

Error Prefix 0x0400 Error number	Error Details	Remedy comments
0x01 – 0x03	FSoE internal state error	Power up

Table 46: Safety FSoE Infrastructure errors

Error Prefix	Error	Error Details	Remedy comments
0x0501 0x0502	0x0001-0x0020	Quad sensor errors	Power up
0x0503 0x0504 0x0505 0x0506 0x0507 0x0508 0x050C 0x050D	0x0001 – 0x2000	Serial sensor errors	Power up
0x0509 0x050A	0x0001 – 0x0040	Analog sensor errors	Power up
0x050E	0x0001 – 0x4000	Safe EnDAT errors	Power up

Error Prefix	Error	Error Details	Remedy comments
0x050F 0x0510	0x0001 = 0x1000	Safe Acuro link \ Open link (Hengstler sensors)	Power up
0x0511	0x0001 – 0x0400	Safe BiSS Errors	Power up
0x0512	0x0001 – 0x8000	Safe DSL (Hiperface sensor)	Power up
0x0513	0x0001 – 0x0800	Safe EnDAT 3	Power up

Table 47: Safety Sensors errors

Error Prefix 0x0600 Error number	Error Details	Remedy comments
0x0002 – 0x0200	Safe Torque diagnostics error	Power up

Table 48: Safety Torque errors

Error Prefix 0x0700 Error number	Error Details	Remedy comments
0x0001- 0x0014	Safety IO State \ time error	Power up
0x0015 – 0x0021	Safety IO test pulse error	
0x0022 – 0x0028	safety IO voltage error	
0x0029 - 0x002A	Safety IO manager error	

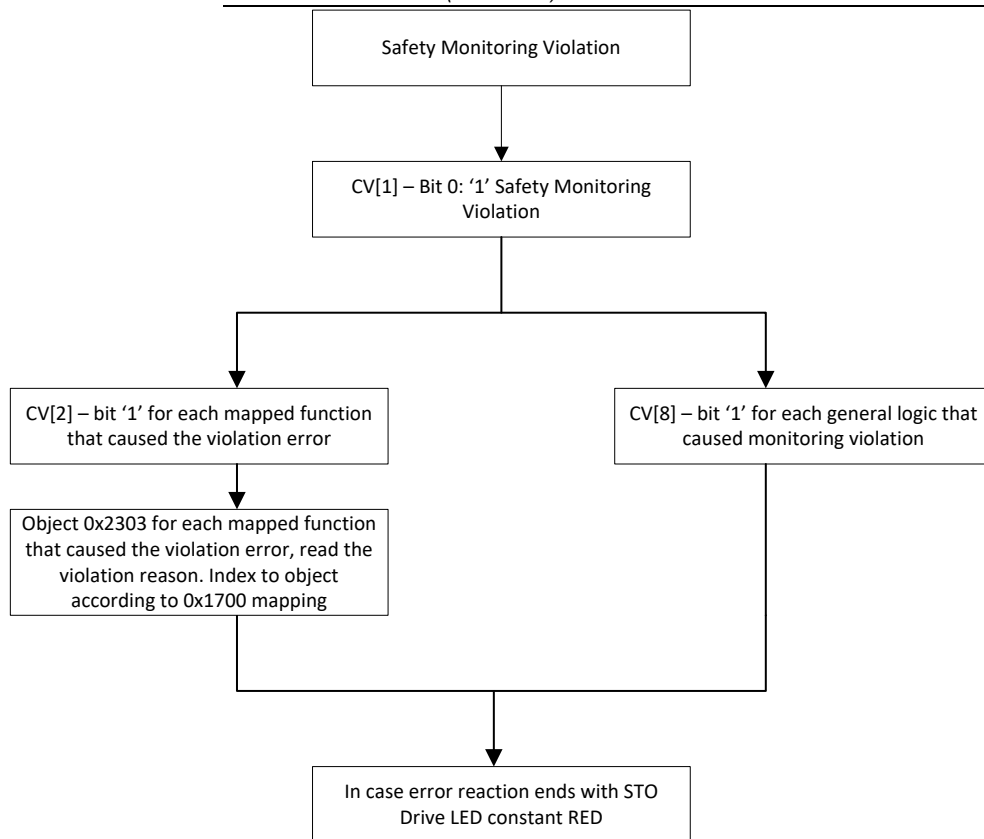
Table 49: Safety IO errors

Error Prefix 0x0800 Error number	Error Details	Remedy comments
0x0001	Error during safe position monitoring	Power up
0x0002	Error during safe torque monitoring	Power up
0x0003	Invalid Internal state error in absolute safety position process.	Power up
0x0004	Data Exchange error in absolute safety position process.	Power up
0x0005	Absolute safety position value crossed positive limit.	Power up
0x0006	SRA parameters CRC during run fail	Power up

Table 50: General safety errors

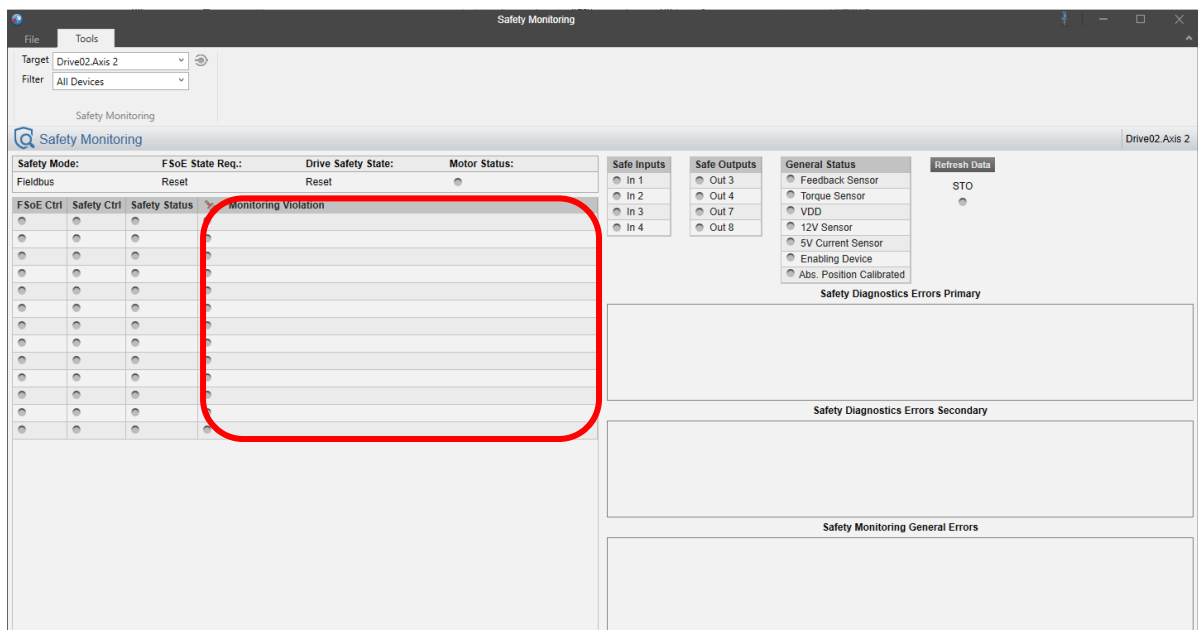
7.6.2 Monitoring Violation

Safety monitoring violation is detected as follows:

**Table 51: Safety monitoring violation**

The EAS Safety monitoring window informs the user when a safety parameter is violated.

All monitoring errors are described per the relevant safety function in the Platinum Safety manual.

**Table 52: Monitoring Violation**

7.6.2.1 Monitoring Error List

All monitoring errors can be cleared when the function is disabled and re-enabled. If the Error Acknowledge is used then the error ACK safety input (or bit that is defined in the FSoE control word) must be alternated. For more details about monitoring error remedy consult the latest Platinum Safety Manual.

Safe Function	Safe Object	Error Number	Error Details
SS1 function	Oj.6650	0x00000001	SS1 (service), velocity zero window not reached after SS1 time to STO
		0x00000002	SS1 (service), Deceleration limit violated
SLP function	Oj.66A0	0x00000003	Upper position limit violated
		0x00000004	Lower position limit violated
		0x00000005	Attempt to activate the SLP function before the axis was calibrated.
SOS function	Oj.6668	0x00000006	Position limit violated
		0x00000007	Velocity limit violated
SLT function	Oj.66B0	0x00000008	Positive Torque limit violated
		0x00000009	Negative Torque limit violated
SS2 function	Oj.6670	0x0000000A	Deceleration limit violated
SLS function	Oj.6690	0x0000000B	Velocity limit violated
		0x0000000D	Deceleration limit violated
SDIp function	Oj.66D0	0x00000010	Positive Velocity limit violated
		0x00000011	Positive position limit violated.
SDIn function	Oj.66D1	0x00000012	Negative Velocity limit violated
		0x00000013	Negative position limit violated.
SMS function	Oj.66A8	0x00000014	Positive Velocity limit violated
		0x00000015	Negative Velocity limit violated
SLI function	Oj.66B8	0x00000016	Upper Position limit violated
		0x00000017	Lower Position limit violated
SSR function	Oj.6680	0x00000020	Velocity upper limit violated
		0x00000021	Velocity lower limit violated
		0x00000022	Deceleration limit violated
General logic	Enabling device	0x00000001	Safe Enabling inputs, Activation override function timer elapse

Table 53: Monitoring Errors

Safe Function	Safe Object	Error Number	Error Details	Remedy comments
SS1 function	Oj.6650	0x00000001	SS1 (service), velocity zero window not reached after SS1 time to STO	Power up
		0x00000002	SS1 (service), Deceleration limit violated	Power up
SLP function	Oj.66A0	0x00000003	Upper position limit violated	Power up
		0x00000004	Lower position limit violated	Power up
		0x00000005	Attempt to activate the SLP function before the axis was calibrated.	Power up
SOS function	Oj.6668	0x00000006	Position limit violated	Power up
		0x00000007	Velocity limit violated	Power up
SLT function	Oj.66B0	0x00000008	Positive Torque limit violated	Power up
		0x00000009	Negative Torque limit violated	Power up
SS2 function	Oj.6670	0x0000000A	Deceleration limit violated	Power up
SLS function	Oj.6690	0x0000000B	Velocity limit violated	Power up
		0x0000000D	Deceleration limit violated	Power up
SDlp function	Oj.66D0	0x00000010	Positive Velocity limit violated	Power up
		0x00000011	Positive position limit violated.	Power up
SDln function	Oj.66D1	0x00000012	Negative Velocity limit violated	Power up
		0x00000013	Negative position limit violated.	Power up
SMS function	Oj.66A8	0x00000014	Positive Velocity limit violated	Power up
		0x00000015	Negative Velocity limit violated	Power up
SLI function	Oj.66B8	0x00000016	Upper Position limit violated	Power up
		0x00000017	Lower Position limit violated	Power up
SSR function	Oj.6680	0x00000020	Velocity upper limit violated	Power up
		0x00000021	Velocity lower limit violated	Power up
		0x00000022	Deceleration limit violated	Power up
General logic	Enabling device	0x00000001	Safe Enabling inputs, Activation override function timer elapse	Power up

Table 54. Monitoring Errors

7.6.2.2 General Details for Reading Monitoring Errors

Up to 32 safe functions can be mapped to Objects 0x1700 RPDO and 0x1B00 TPDO. The first 32 bits of **CV[2]** inform which safety function caused the monitoring error. The bits of **CV[8]** inform monitoring errors of

general logic. To read the specific monitoring errors use **Object 0x2303**, the logger of the safety monitoring errors.

The specific instance of **Object 0x2303** is found by reading the value of **CV[2]** and **CV[8]**, where **CV[2]** bit 0 correlate to 0x2303[1] and **CV[2]** bit 31 to 0x2303[32]. **CV[8]** bit 0 correlate to **CV[2]** bit 33.

0x2303[0] indicates the number of active monitoring errors.

For example, if the following is the mapping of safety functions in SRA **Objects 0x1700** RPDO and **0x1B00** TPDO.

Sub-index	Safety function	Hex view	Description
0x1B00[0]/0x1700[0]	-	0x20	Number of sub-indexes
0x1B00[1]/0x1700[1]	-	0xE7000108	Safe command word
0x1B00[2]/0x1700[2]	STO	0x66400001	Safe Torque Off
0x1B00[3]/0x1700[3]	SS1	0x66500101	Safe Stop 1, instance 1
0x1B00[4]/0x1700[4]	SOS	0x66680101	Safe Operation Stop, instance 1
0x1B00[5]/0x1700[5]	SLS	0x66900101	Safe Limited Speed, instance 1
0x1B00[6]/0x1700[6]	Safe Input 2	0x2E120201	Safe Input, instance 1
0x1B00[7]	Safe Output 1	0x2E130101	Safe Output, instance 1:

If **CV[1]** = 1, and **CV[2]**=0x8, then SLS has a monitoring error. To read an SLS specific error read 0x2303[4].

If **CV[8]**=1, then General logic of "Enabling device" has a monitoring error to read the specific error, read the value of 0x2303[33].

7.7 Sensors Errors and Warnings

It should be noted that all sensor errors can be converted to warnings by setting its bit in the error mask parameter **S#[4]**. In addition, for all sensors, bit 31 in the Error Word reports a Hall Sensor Error. The relevant Commands used to define the errors and warnings are:

W#[1]	Socket latched errors (cleared on motor on)
W#[2]	Socket warnings
W#[3]	Socket active errors

7.7.1 Sensor ID=1: Encoder Quad

Bit	Type	Title	Details	Resolution
0	Error	Pin A diagnostics	Quad pin A differential line error	Check encoder Physical connection
1	Error	Pin B diagnostics	Quad pin B differential line error	Check encoder Physical connection
2-6		Reserved		
7	Error	Quad states error	State machine – faulty transition	Check encoder Physical connection
8-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.2 Sensor ID=3: Analog Sin/Cos

Bit	Type	Title	Details	Resolution
0-4		Reserved		
5	Error	Amplitude Error	Analog signal amplitude exceeded allowed limits	Run sensor calibration again Check sensor
6		Reserved		
7	Error	Quad states error	State machine – faulty transition	Check encoder Physical connection
8-13		Reserved		
14	Warning	Amplitude low threshold warning	Analog signal amplitude is below threshold limit	Check sensor warning limits Check sensor
15	Warning	Amplitude high threshold warning	Analog signal amplitude is above threshold limit	Check sensor warning limits Check sensor
16-30		Reserved		

Bit	Type	Title	Details	Resolution
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.3 Sensor ID=4: Halls Only

Bit	Type	Title	Details	Resolution
0-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.4 Sensor ID=5: BiSS

Bit	Type	Title	Details	Resolution
0	Error	BiSS error bit	General error bit	Check error bit cause in the sensor datasheet
1-6		Reserved		
7	Warning	BiSS warning bit	General warning bit	Check warning bit cause in the sensor datasheet
8	Error	CRC Error	CRC Communication error	Check sensor configuration parameters Check sensor physical connection & shielding
9		Reserved		
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	Increase serial communication time Increase clock frequency Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: Check sensor and connection Check sensor configuration parameters
12-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.5 Sensor ID=6: Panasonic

Bit	Type	Title	Details	Resolution
0	Error	Over Speed	The encoder shaft revolves at a speed exceeding the value	Execute either "Reset errors" (T#[3] =1) or "Reset multiple

Bit	Type	Title	Details	Resolution
			specified for Power-OFF Mode.	revolution data and errors" (T#[2] =1) commands
1	Error	Full Absolute - Status	The main Power SW is turned ON while the encoder shaft is revolving at more than 100r/min.	Set encoder shaft revolution speed less than 100r/min and wait until the error is automatically reset.
2	Error	Count error	Single turn data is shifted due to operation error or failure when the main Power SW is ON	Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands. Power SW OFF/ON
3	Error	Counter overflow	The multiple revolution counter overflowed	Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands
4	-	Reserved		
5	Error	Multiple revolution error	Fault in multiple revolution data occurred when the main Power SW was turned ON	Return to origin. Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands.
6	Error	System down	The voltage of the encoder's built-in capacitor became less than 2.5V while the main Power SW was OFF	"Reset multiple revolution data and errors" (T#[2] =1). Check or replacement of external battery power supply is required.
7	Warning	Battery alarm	The external battery power supply voltage became less than 3.1V	Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands. Check or replacement of external battery power supply is required.
8	Error	CRC Error	CRC Communication error	Check sensor configuration parameters Check sensor physical connection & shielding
9	-	Encoder ID error	The encoder id doesn't exist or isn't consistent with the assigned resolution.	1. Set the correct sensor resolution according to the sensor type.

Bit	Type	Title	Details	Resolution
				2. Check if this sensor is supported.
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	Increase serial communication time Increase clock frequency Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: Check sensor and connection Check sensor configuration parameters
12-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.6 Sensor ID=7: Panasonic Incremental

Bit	Type	Title	Details	Resolution
0	Error	INIE error	Encoder internal initialization error	Execute either "Reset errors" (T#[3] =1) or Power SW OFF/ON
1	Error	Count error 1	Position data per revolution ABSA is shifted.	Execute either "Reset errors" (T#[3] =1) or Power SW OFF/ON
2		Count error 2	Edges Number of A and B is 13 or lower between CS signal edges.	
3	Error	Z signal error	Generation of internal Z signal is stopped. Applicable if PS logic turned from "1" to "0".	Execute either "Reset errors" (T#[3] =1) or Power SW OFF/ON
4	Error	CS Signal error	Logic value "1" if the logics of CS1,CS2 and CS3 are all "1" or all "0".	Execute either "Reset errors" (T#[3] =1) or Power SW OFF/ON
5, 6		Reserved		
8	Error	CRC Error	CRC Communication error	Check sensor configuration parameters Check sensor physical connection & shielding
9		Encoder ID error	The encoder id doesn't exist or isn't consistent with the assigned resolution.	1. Set the correct sensor resolution according to the sensor type. 2. Check if this sensor is supported.

Bit	Type	Title	Details	Resolution
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	Increase serial communication time Increase clock frequency Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: Check sensor and connection Check sensor configuration parameters
12-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.7 Sensor ID=8: Sine Virtual Sensor

Bit	Type	Title	Details	Resolution
0-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	Run commutation wizard again Check Halls connection

7.7.8 Sensor ID=9: EnDAT

Bit	Type	Title	Details	Resolution
0	Error	EnDAT error message 1	General sensor error	For details read out EnDAT sensor errors, use the following commands: 1. t#[7]=1 – read sensor errors 2. w#[4] – display errors * See detailed description below
1	Error	EnDAT error message 2	General sensor error	For details read out EnDAT sensor errors, use the following commands: 1. t#[7]=1 – read sensor errors 2. w#[4] – display errors * See detailed description below
2	Warning	EnDAT WRN bit	General sensor warning	For details read out EnDAT sensor warnings, use the following commands:

Bit	Type	Title	Details	Resolution
				t#[7]=2 – read sensor errors w#[5] – display warnings * See detailed description below
3	Warning	Additional info not supported	The additional info requested by the user isn't supported by this sensor.	
4	Error	Encoder doesn't support EnDAT 2.2	The sensor supports EnDAT 2.1 only	Select to use EnDAT 2.1 mode commands in sensor settings
5	Error	Incorrect resolution setting	The defined resolution doesn't match the setting in sensor	Set the correct sensor resolution parameters.
6	Warning	Additional Info Error	EnDat Additional Information 1 CRC	
7	Warning	Additional Info Error	EnDat Additional Information 2 CRC or bad command acknowledge	
8	Error	CRC Error	CRC Communication error	- Check sensor configuration parameters - Check sensor physical connection & shielding
9		Reserved		
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	- Increase serial communication time - Increase clock frequency - Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: - Check sensor and connection - Check sensor configuration parameters
12		Reserved		
13		Reserved		
14		Reserved		
15	Error	M battery		
16	Error	M Overflow /Underflow	The specified (multiturn) counting	Return the axis into the specified counting range of the encoder and reset the error.

Bit	Type	Title	Details	Resolution
			range of the encoder has been exceeded	
17	Error	M all power down	Both buffer battery voltage (UBAT) and main supply voltage (UP) has dropped below the specified limits	Check battery and connection New referencing of the axis has to take place
18	Error	M system	A multiturn related internal error occurred during the initialization phase of the encoder	Another power-up cycle should be tried. If this also fails, a hardware problem of the encoder is probable.
19	Error	M Pos 2		
20	Error	M Pos 1	Multiturn scanning or the multiturn position processing failed during the operation of the encoder	The consistency of the revolution counter value has to be verified. If this is not possible, referencing of the axis has to take place
21	Error	S all power down		
22	Error	S system	A single-turn related internal error occurred during the initialization phase of the encoder	Another power-up cycle should be tried. If this also fails, a hardware problem of the encoder is probable.
23	Error	S Pos 2		
24	Error	Temperature exceeded		
25	Error	Overcurrent	See 3.8.1 bit 1	See 3.8.1 bit 1
26	Error	Undervoltage	See 3.8.1 bit 2	See 3.8.1 bit 2
27	Error	Overvoltage	See 3.8.1 bit 3	See 3.8.1 bit 3
28	Error	S Pos 1		
29	Error	Signal Amplitude	See 3.8.1 bit 5	See 3.8.1 bit 5
30	Error	Light source	See 3.8.1 bit 6	See 3.8.1 bit 6
31	Error	Halls Error		

7.7.8.1 W#[4] Sensor Errors Detailed Description

The following table lists the possible EnDat sensor errors when a detailed error read out is performed by using: **t#[7]=1**, **w#[4]** commands.

Bit	Type	Title	Details	Resolution
9	Error	Battery	Change required	Replace the battery
10	Error	Overcurrent	Sensor overcurrent occurred	
11	Error	Undervoltage	Sensor undervoltage occurred	
12	Error	Overvoltage	Sensor overvoltage occurred	
13	Error	Position	Faulty position	
14	Error	Signal amplitude	Faulty signal amplitude	
15	Error	Light source	Faulty light source	

7.7.8.2 W#[5] sensor warnings detailed description

The following table lists the possible EnDat sensor warnings when a detailed error read out is performed by using: **t#[7]=2**, **w#[5]** commands.

Bit	Type	Title	Details	Resolution
7	Warning	Diagnostics		
8	Warning	Readiness		
9	Warning	Limit position		
10	Warning	Cyclic mode		
11	Warning	Reference points		
12	Warning	Battery charge		
13	Warning	Light source control reserve		
14	Warning	Temperature exceeded		
15	Warning	Frequency collision		

7.7.9 Sensor ID=11: Pulse & Direction

Bit	Type	Title	Details	Resolution
0	Error	Pin A diagnostics	Quad pin A differential line error	Check encoder Physical connection
1	Error	Pin B diagnostics	Quad pin B differential line error	Check encoder Physical connection
2-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.10 Sensor ID=15: Copy Main Profile

Bit	Type	Title	Details	Resolution
0-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.11 Sensor ID=16: Analog Input

Bit	Type	Title	Details	Resolution
0-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.12 Sensor ID=18: SSI

Bit	Type	Title	Details	Resolution
0	Error	SSI error bit	General error bit	Check error bit cause in the sensor datasheet
1-6		Reserved		
7	Warning	SSI warning bit	General warning bit	Check warning bit cause in the sensor datasheet
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	1. Increase serial communication time 2. Increase clock frequency Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: 1. Check sensor and connection 2. Check sensor configuration parameters
12-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.13 Sensor ID=19: Yaskawa

Bit	Type	Title	Details	Resolution
0	Error	Backup Alarm	Backup power is down, internal data has been lost	Reset command
1	Error	Encoder Error	Error in encoder	Reset command
2	Warning	Battery Warning	Battery voltage drop	
3	Error	Absolute Error	Possible error in position data.	Re-feed power
4	Error	Overspeed	Speed at power feed too high.	Re-feed power
5	Error	Overheat	Temperature inside encoder is too high	
6	Warning	Reset Complete	Reset the encoder with a RESET command	Re-feed power
8	Error	CRC Error	CRC Communication error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the errors persist: 1. Check sensor and connection 2. Check sensor configuration parameters

- An error message will display when one of the bit set with option to send reset command (T#[3]=1).

7.7.14 Sensor ID=20: Gantry Sensor

Bit	Type	Title	Details	Resolution
0-31		No Specific Errors		

7.7.15 Sensor ID=21: Serial Exclusive

7.7.15.1 Master In Three Slaves Configuration

(GG[1].5-7 == 0/1/5) && (GG[1].1 == 1)

Bit	Type	Title	Details	Resolution
0	Error	Serial Exclusive sensor error	Communication channel 1: cycle not completed	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 1 physical connection
1	Error	Serial Exclusive sensor error	Communication channel 1: CRC error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 1 physical connection
2	Error	Serial Exclusive sensor error	Communication channel 2: cycle not completed	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 2 physical connection
3	Error	Serial Exclusive sensor error	Communication channel 2: CRC error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 2 physical connection
4	Error	Serial Exclusive sensor error	Communication channel 3: cycle not completed	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal

Bit	Type	Title	Details	Resolution
				3. Check channel 3 physical connection
5	Error	Serial Exclusive sensor error	Communication channel 3: CRC error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 3 physical connection
6	Error	Serial Exclusive sensor error	Communication channel 1: sync error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 1 physical connection
7	Error	Serial Exclusive sensor error	Communication channel 2: sync error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 2 physical connection
8	Error	Serial Exclusive sensor error	Communication channel 3: sync error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check channel 3 physical connection
9-11		Reserved		
12		N/A		
13-331		Reserved		

7.7.15.2 Master in single slave configuration (Gantry)

(GG[1].5-7 == 2/3/4/6/7) && (GG[1].1 == 1)

Bit	Type	Title	Details	Resolution
0	Error	Serial Exclusive sensor error	Communication cycle not completed	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check physical connection
1	Error	Serial Exclusive sensor error	Communication CRC error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check physical connection
2-5		Reserved		
6	Error	Serial Exclusive sensor error	Communication sync error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check physical connection
7-11		Reserved		
12		N/A		
13-31		Reserved		

7.7.15.3 Slave

(GG[1].5-7 == 2/3/4/6/7) && (GG[1].1 == 0)

Bit	Type	Title	Details	Resolution
0	Error	Serial Exclusive sensor error	Communication cycle not completed	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check physical connection
1	Error	Serial Exclusive sensor error	Communication CRC error	1. Check master and slave Serial Exclusive sensors configuration

Bit	Type	Title	Details	Resolution
				2. Make sure that TS of master and slave are equal 3. Check physical connection
2-5		Reserved		
6	Error	Serial Exclusive sensor error	Communication sync error	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check physical connection
7-11		Reserved		
12		TS trigger sync error	Slave TS is not synchronized to master TS	1. Check master and slave Serial Exclusive sensors configuration 2. Make sure that TS of master and slave are equal 3. Check physical connection
13-31		Reserved		

7.7.16 Sensor ID=22: Resolver

Bit	Type	Title	Details	Resolution
0-4		Reserved		
5	Error	Amplitude Error	Analog signal amplitude exceeded allowed limits	1. Run sensor calibration again 2. Check sensor
6-13		Reserved		
14	Warning	Amplitude low threshold warning	Analog signal amplitude is below threshold limit	1. Check sensor warning limits 2. Check sensor
15	Warning	Amplitude high threshold warning	Analog signal amplitude is above threshold limit	1. Check sensor warning limits 2. Check sensor
16-30		Reserved		

Bit	Type	Title	Details	Resolution
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.17 Sensor ID=25: Nikon

Bit	Type	Title	Details	Resolution
8	Error	CRC Error	CRC Communication error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
9	-	Reserved		
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	1. Increase serial communication time 2. Increase clock frequency 3. Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: 1. Check sensor and connection 2. Check sensor configuration parameters
12	Error	Busy+ MemBusy	Logical sum output of the errors: (See bits #7 + #8 in 3.14.1)	Execute "Reset errors" (T#[3]=1) to read the detailed error status
13	Alarm	Batt	The external battery voltage is 3.0V (TYP) or less	Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands. Check or replacement of external battery power supply is required.
14	Error	OvSpd+MemErr+OvTemp+OvFlow	Logical sum output of the errors: (See bits #3 + #4 + #9 + #2 in 3.14.1)	Execute "Reset errors" (T#[3]=1) to read the detailed error status
15	Error	STErr+PSErr+MTErr+IncErr	Logical sum output of the errors: (See bits #5 + #6 + # in 3.14.1)	Execute "Reset errors" (T#[3]=1) to read the detailed error status

Bit	Type	Title	Details	Resolution
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.17.1 W#[4] sensor errors detailed description

The following table lists the possible Nikon sensor errors when a "record extended status" command – T#[7] is performed by the user. In order to display the recorded errors, the previous command should be followed by a W#[4] command.

Bit	Type	Title	Details	Resolution
0	Error	Batt	The external battery voltage is 3.0V (TYP) or less	Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands. Check or replacement of external battery power supply is required.
1	Error	MTErr	The voltage of the built-in backup capacitor or the external battery became 2.45V (TYP) or less	"Reset multiple revolution data and errors" (T#[2] =1) . Check or replacement of external battery power supply is required.
2	Error	OvFlow	The multiple revolution counter overflowed	Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands.
3	Error	OvSpd	Overspeed fault occurred	1. Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands. 2. Power SW OFF/ON.
4	Error	MemErr	EEPROM access error	1. Execute either "Reset errors" (T#[3] =1) or "Reset multiple revolution data and errors" (T#[2] =1) commands. 2. Power SW OFF/ON.
5	Error	STErr	Single turn position data error	Power SW OFF/ON

Bit	Type	Title	Details	Resolution
6	Error	PSErr	Position data error - Inconsistence in the position data acquired by the MT vs ST modules	Power SW OFF/ON
7	Warning	Busy	Calculation ST position value	Automatically turned off
8	Warning	MemBusy	Sensor's EEPROM is currently being accessed	Automatically turned off
9	Warning	OvTemp	Encoder's temperature has exceeded the threshold	Automatically turned off
10	Error	IncErr	An error in the incremental signal was detected	Power SW OFF/ON
11-15	-	Reserved		

7.7.18 Sensor ID=28: Hiperface

Bit	Type	Title	Details	Resolution
0-3		Reserved		
4	Error	Hiperface Init	Hiperface is in init procedure	1. Wait for power up procedure to finish 2. Check serial data line connection 3. Check sensor Power supply
5	Error	Amplitude Error	Analog signal amplitude exceeded allowed limits	1. Run sensor calibration again 2. Check sensor
6	Error	Hiperface communication error	Timeout or faulty checksum received during "get status" command	1. Check sensor 2. Check serial data line connection
7	Error	Hiperface general error	Hiperface sensor reports an error	1. See W#[4] for details (in order to interpret the read error status refer to the manufacturers manual)
8-13		Reserved		
14	Warning	Amplitude low threshold warning	Analog signal amplitude is below threshold limit	1. Check sensor warning limits 2. Check sensor

Bit	Type	Title	Details	Resolution
15	Warning	Amplitude high threshold warning	Analog signal amplitude is above threshold limit	1. Check sensor warning limits 2. Check sensor
16-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.19 Sensor ID=30: Velocity Observer IR

Bit	Type	Title	Details	Resolution
0-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.20 Sensor ID=34: Motor Simulation

Bit	Type	Title	Details	Resolution
0-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.21 Sensor ID=36: BEMF Observer

Bit	Type	Title	Details	Resolution
0	Error	BEMF Observer sensor error	Quality of estimated feedback is low	1. Check sensor parameters 2. Tune commutation parameter SC[15]
1-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.22 Sensor ID=38: Generic Serial IMU/Gyro

Bit	Type	Title	Details	Resolution
0	Error	Header value error	Received header value does not match expected value	1. Check header value 2. Check physical connection
1	Error	Length error	Received data length is different than expected	1. Check length configuration 2. Check baud rate configuration 3. Check physical connection
2	Error	Checksum error		1. Check checksum configuration 2. Check baud rate configuration 3. Check physical connection
3	Error	Data counter error		Check physical connection
4-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.23 Sensor ID=39: Mitsubishi

Bit	Type	Title	Details	Resolution
0	Error	CPU alarm	Encoder internal data damaged (alarm hold)	Stop operation
1	Error	Angle calculation alarm	Data per revolution calculation error	Stop operation
2	Error	Data alarm	Data per revolution error	Stop operation when received consecutively
3	Error	Encoder thermal alarm	Encoder section hot (115±5°C)	Stop operation
4	Warning	Encoder thermal warning	Encoder section hot (96±5°C)	
5	Warning	Multi-revolution alarm	Multi-revolution count data error	Note at next power ON, multi-revolution may be deviated
6	Error	ABS lost alarm	Multi-revolution backup data damaged (alarm hold)	Stop operation Clear with request

Bit	Type	Title	Details	Resolution
7		Reserved		
8	Error	CRC Error	CRC Communication error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
9	-	Encoder ID error	The encoder id doesn't exist or isn't consistent with the assigned resolution.	1. Set the correct sensor resolution according to the sensor type. 2. Check if this sensor is supported.
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	1. Increase serial communication time 2. Increase clock frequency 3. Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: 1. Check sensor and connection 2. Check sensor configuration parameters
12-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.24 Sensor ID=40: Acurolink

Bit	Type	Title	Details	Resolution
0	Error	Standard Position Error		
1	Error	OEM Read Error		
2	Error	OEM Write Error		
3	Error	Diagnose Services Error		
4	Error	Operational Mode Change Error		
5	Error	Encoder Reset Error		

Bit	Type	Title	Details	Resolution
6	Error	Invalid Operational Mode Error		
7	Error	Invalid Command Error		
8-9		Reserved		
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	1. Increase serial communication time 2. Increase clock frequency 3. Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: 1. Check sensor and connection 2. Check sensor configuration parameters
12-14		Reserved		
15	Error	Tx MRF to Rx DTF Timeout Error	Communication or sensor error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
16	Error	Invalid Rx DTF Type Error		
17	Error	Invalid Rx DTF Addr/Data Error		
18	Error	Invalid Rx DTF Encoder State Error	Communication or sensor error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
19	Error	Invalid Rx DTF CRC Error	CRC Communication error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
20-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection

7.7.25 Sensor ID=41: Openlink

Bit	Type	Title	Details	Resolution
0	Error	Standard Position Error		
1	Error	OEM Read Error		
2	Error	OEM Write Error		
3	Error	Diagnose Services Error		
4	Error	Operational Mode Change Error		
5	Error	Encoder Reset Error		
6	Error	Invalid Operational Mode Error		
7	Error	Invalid Command Error		
8-9		Reserved		
10	Error	Communication Error	Communication cycle didn't complete in the defined communication time	1. Increase serial communication time 2. Increase clock frequency 3. Check sensor and connection
11	Error	Sensor is being initialized	The error is indicating the sensor is being initialized.	If the error persists: 1. Check sensor and connection 2. Check sensor configuration parameters
12-14		Reserved		
15	Error	Tx MRF to Rx DTF Timeout Error	Communication or sensor error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
16	Error	Invalid Rx DTF Type Error		
17	Error	Invalid Rx DTF Addr/Data Error		

Bit	Type	Title	Details	Resolution
18	Error	Invalid Rx DTF Encoder State Error	Communication or sensor error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
19	Error	Invalid Rx DTF CRC Error	CRC Communication error	1. Check sensor configuration parameters 2. Check sensor physical connection & shielding
20-30		Reserved		
31	Error	Halls Error	Wrong Hall state or wrong transition	1. Run commutation wizard again 2. Check Halls connection